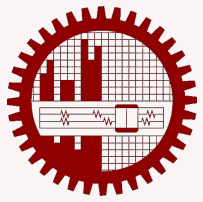


Bangladesh University of Engineering and Technology

Department of Computer Science & Engineering

OOP Question Bank With Answers

Object Oriented Programming (C++ & Java)

CSE 107 (L-1/T-2) & CSE 201 (L-2/T-1)

Year Coverage: 2007–08 to 2023–24 |
Topics: C++ OOP, Inheritance, Templates, Java OOP, Multithreading & more

Authors & Contributors

Md Ratul Hasan (2405009) — Question Curation & Organisation
— LaTeX Typesetting

Claude AI — LaTeX Typesetting

Table of Contents

Part I — C++ Object Oriented Programming

Part II — Java Object Oriented Programming

BUET · CSE 107 / CSE 201

Object Oriented Programming

Section S1

Philosophy & Fundamentals of OOP

S1 — Philosophy & Fundamentals of OOP

Q1. Explain the key features of object-oriented programming (OOP). How does a class differ from a structure in OOP? **(6+4 marks)** [CSE 107 | 2023–24]

Answer

Key Features of Object-Oriented Programming:

1. **Encapsulation:** Bundling data (attributes) and the functions that operate on that data into a single unit called a *class*, and restricting direct access to some of the object's components via access specifiers (`private`, `protected`, `public`).
2. **Abstraction:** Hiding implementation details and exposing only the necessary interface to the user. The user interacts with an object through its public methods without knowing the internal workings.
3. **Inheritance:** A mechanism by which a new class (*derived class*) acquires the properties and behaviours of an existing class (*base class*), promoting code reuse.
4. **Polymorphism:** The ability of a single interface to represent different underlying forms — achieved through function overloading (compile-time) and virtual functions (run-time).
5. **Message Passing:** Objects communicate by calling each other's methods, analogous to sending messages.

Class vs. Structure in OOP (C++):

Class	Structure (`struct`)
Members are `private` by default.	Members are `public` by default.
Supports all OOP features: inheritance, encapsulation, polymorphism, access control.	In C++ also supports OOP features, but by convention used for plain data aggregates.
Typically used to model objects with both data and behaviour.	Typically used for lightweight, passive data containers (no or minimal methods).
Inheritance default: `private`.	Inheritance default: `public`.

Q2. Explain the importance of using constructors and destructors in C++. Why and how does Java drop writing destructors in programs? **(5+5 marks)** [CSE 107 | 2023–24]

Answer

Importance of Constructors in C++:

- Guarantee that every object starts in a *valid, well-defined state* — no uninitialized memory or dangling pointers.
- Can be overloaded to support multiple ways of creating an object (default, parameterised, copy constructors).
- Enable the *RAII* (Resource Acquisition Is Initialisation) idiom: resources such as file handles or heap memory are acquired in the constructor.

Importance of Destructors in C++:

- Automatically release resources when an object's lifetime ends, preventing *memory leaks* and *resource leaks*.
- Essential for classes that manage raw heap memory (`new`/`delete`), file I/O, network sockets, etc.
- Complete the RAII contract: what the constructor acquires, the destructor releases.

Why Java drops destructors:
Java uses **automatic garbage collection (GC)**. The JVM tracks all object references and automatically reclaims heap memory when an object is no longer reachable — the programmer never calls `delete`. Since memory management is automated, an explicit destructor is unnecessary.
Instead, Java provides:

- **`finalize()` (deprecated):** Called by the GC before reclaiming an object, but its timing is non-deterministic and unreliable.
- **`try-with-resources` / `AutoCloseable`:** The recommended modern approach for deterministic release of non-memory resources (files, connections).

Q3. Describe why constructor overloading is possible while destructor overloading is not allowed. **(11 marks)** [CSE 107 | 2022–23]

Answer

Constructor Overloading is possible because:

- An object can be created in multiple ways — from no arguments, from specific values, from a copy of another object, etc.
- Each overloaded constructor has a **different parameter list**, so the compiler can unambiguously select the correct one based on how the object is created.
- This follows the same rules as regular function overloading.

```cpp
class Point {
public:
    Point() : x(0), y(0) {}                // default
    Point(int a, int b) : x(a), y(b) {}  // parameterised
    Point(const Point& p) : x(p.x), y(p.y) {} // copy
private:
    int x, y;
};
```

Destructor Overloading is NOT possible because:

1. A destructor **takes no parameters** — it is always called with *no arguments* by the compiler when the object's lifetime ends. Without parameters, there is no basis for distinguishing multiple versions.
2. A destructor **returns nothing** — there is no return type to differentiate on either.
3. There is **only one way** an object's lifetime can end — the compiler must know exactly which destructor to call without any ambiguity. Multiple destructors would make this impossible.
4. A class can have **at most one destructor**, named `~ClassName()`, with no parameters.

Q4. Discuss the difference between a class and an object. Describe how C++ supports inheritance. **(11 marks)** [CSE 107 | 2021–22]

Answer

Class vs. Object:

Class	Object
A *blueprint* or *template* that defines the structure and behaviour of a type.	A concrete *instance* of a class that occupies memory.
Defined once; no memory allocated for the class itself.	Each object has its own copy of the data members.
Exists at compile time as a type definition.	Created at run time via constructors.
Example: `class Car { …};`	Example: `Car myCar;`

How C++ supports Inheritance:
C++ supports inheritance using the colon (`:`) syntax in the class declaration, with an access specifier controlling how inherited members are exposed:

```cpp
class Animal {          // Base class
protected:
    string name;
public:
    Animal(string n) : name(n) {}
    virtual void speak() { cout << name << " makes a sound\n"; }
};

class Dog : public Animal {   // Derived class
public:
    Dog(string n) : Animal(n) {}
    void speak() override { cout << name << " barks\n"; }
};
```

Types of inheritance in C++:

- `public` inheritance: `public`/`protected` members remain `public`/`protected` in derived.
- `protected` inheritance: both become `protected`.
- `private` inheritance: both become `private`.
- C++ also uniquely supports **multiple inheritance**: `class C : public A, public B { };`

Q5. Explain abstraction with an appropriate example. Write down the properties of abstract class and abstract method. **(5+3+2 marks)** [CSE 107 | 2020–21]

Answer

Abstraction is the OOP principle of hiding internal implementation details and exposing only the essential features of an object to the outside world.
Example: A `Car` class exposes methods `start()`, `accelerate()`, and `brake()`. The driver does not need to know about fuel injection, engine timing, or brake hydraulics — those details are hidden.

```cpp
class Car {
    void fuelInjection() { /* hidden */ }   // private detail
public:
```

```
    void start()      { fuelInjection(); /* ... */ }
    void accelerate() { /* ... */ }
    void brake()      { /* ... */ }
};
```

Properties of an Abstract Class:

- Contains at least one *pure virtual function* (`= 0` in C++; declared `abstract` in Java).
- **Cannot be instantiated** — no objects can be created directly from an abstract class.
- Can have constructors, data members, and concrete (non-pure) methods.
- A derived class must override *all* pure virtual functions; otherwise the derived class is also abstract.

Properties of an Abstract Method (Pure Virtual Function):

- Has a declaration but **no definition** in the base class (e.g. `virtual void draw() = 0;`).
- Forces every concrete subclass to provide its own implementation.
- Provides a common interface across a class hierarchy.

Q6. What is the purpose of constructors and destructors? Write three special properties of a constructor that make it distinct from other member functions. **(10 marks)** [CSE 107 | 2016–17, 2020–21]

Answer

Purpose of Constructors: A constructor is a special member function that is automatically called when an object is created. Its purpose is to *initialise* the object's data members to valid starting values, allocate resources (e.g. heap memory), and set up any invariants the object must maintain.
Purpose of Destructors: A destructor is automatically called when an object goes out of scope or is explicitly deleted. Its purpose is to *release* resources held by the object (e.g. `delete` heap memory, close file handles, release locks) to prevent resource leaks.

Three special properties of a constructor that distinguish it from other member functions:

1. **Same name as the class:** A constructor must have exactly the same name as the class it belongs to. No other member function may share the class name.
2. **No return type (not even `void`):** A constructor does not declare or return any value. Specifying a return type is a compile error.
3. **Called automatically:** A constructor is invoked implicitly by the compiler whenever an object of the class is created — the programmer never calls it explicitly by name. (Contrast with ordinary member functions, which must be called explicitly.)

Q7. Differentiate between early binding and late binding with appropriate examples. **(8 marks)** [CSE 201 | 2014–15, 2013–14; CSE 107 | 2020–21]

↪ *Similar: CSE 201 | 2015–16, 2012–13, 2009–10, 2008–09; CSE 107 | 2017–18*

Answer

Early Binding (Static / Compile-Time Binding):
The compiler resolves the function call and links it to the correct function definition *at compile time*, based on the declared type of the object or pointer. This produces faster code since no run-time lookup is needed.

```
class Animal {
public:
    void speak() { cout << "Animal speaks\n"; } // non-virtual
};
class Dog : public Animal {
public:
    void speak() { cout << "Dog barks\n"; }
};

Animal* p = new Dog();
p->speak();   // Early binding: calls Animal::speak()
              // because speak() is NOT virtual
```

Late Binding (Dynamic / Run-Time Binding):
The function call is resolved *at run time* based on the actual type of the object pointed to. Achieved in C++ via `virtual` functions and the virtual function table (vtable).

```
class Animal {
public:
    virtual void speak() { cout << "Animal speaks\n"; } // virtual
};
class Dog : public Animal {
```

```cpp
public:
    void speak() override { cout << "Dog barks\n"; }
};

Animal* p = new Dog();
p->speak();   // Late binding: calls Dog::speak() at run time
```

	Early Binding	Late Binding
Resolution	Compile time	Run time
Mechanism	Direct call	vtable lookup
Speed	Faster	Slightly slower
Flexibility	Less	More (polymorphism)
Keyword	(default)	`virtual`

Q8. Which feature of C++ programming language do you like most and why? The feature that you mention should not also be a feature of C programming language. **(10 marks)** [CSE 107 | 2019–20]

Answer

Feature: Operator Overloading
Operator overloading allows existing C++ operators (`+`, `-`, `==`, `<<`, etc.) to be redefined for user-defined types, making code significantly more intuitive and readable. This feature does not exist in C.
Why it is the most useful:

- It allows user-defined types to *behave like built-in types*. For example, adding two `Complex` numbers can be written as `c1 + c2` rather than `add(c1, c2)`.
- It makes mathematical classes (`Matrix`, `Vector`, `Fraction`) self-documenting and natural to use.
- It integrates seamlessly with the standard library (e.g. overloading `<<` for `ostream` to print custom objects).

Example:

```cpp
class Complex {
    double re, im;
public:
    Complex(double r, double i) : re(r), im(i) {}
    Complex operator+(const Complex& o) const {
        return Complex(re + o.re, im + o.im);
    }
    friend ostream& operator<<(ostream& os, const Complex& c) {
        return os << c.re << "+" << c.im << "i";
    }
};
// Usage:
Complex c1(1,2), c2(3,4);
cout << c1 + c2;   // prints: 4+6i
```

Q9. Write a short note on "function overloading". **(10 marks)** [CSE 107 | 2019–20]

Answer

Function Overloading is the ability to define multiple functions with the *same name* in the same scope, provided they differ in their **parameter list** (number, type, or order of parameters). The compiler selects the correct version at *compile time* based on the arguments supplied — this is a form of *compile-time (static) polymorphism*.
Rules:

- Overloaded functions must differ in their parameter lists.
- The return type *alone* is **not** sufficient to distinguish overloads.
- `const`-ness of the implicit `this` pointer (for member functions) can also distinguish overloads.

Example:

```cpp
int    add(int a, int b)        { return a + b; }
double add(double a, double b) { return a + b; }
int    add(int a, int b, int c){ return a + b + c; }

int    r1 = add(1, 2);         // calls first version
double r2 = add(1.5, 2.5);     // calls second version
int    r3 = add(1, 2, 3);      // calls third version
```

Advantages:

- Improves code readability — one intuitive name for logically related operations.
- Eliminates the need for functions like `addInt`, `addDouble`, etc.

Q10. What does a default constructor (generated by the compiler) perform? What are the scenarios when a copy constructor is called? **(10 marks)** [CSE 107 | 2019–20]

Answer

What the compiler-generated default constructor does:

- For **built-in types** (int, double, pointer, etc.): leaves them **uninitialised** (their values are indeterminate).
- For **class-type members**: calls the default constructor of each member.
- For **base class sub-objects**: calls the base class default constructor.

The compiler generates this constructor only if the programmer defines *no* constructor at all.

Scenarios when the copy constructor is called:

1. **Object initialisation from another object of the same class:**

```
MyClass a;
MyClass b = a;    // copy constructor called
MyClass c(a);     // copy constructor called
```

2. **Passing an object by value to a function:**

```
void f(MyClass x) { }   // copy constructor called for x
f(a);
```

3. **Returning an object by value from a function:**

```
MyClass g() {
    MyClass tmp;
    return tmp;   // copy constructor may be called (if RVO absent)
}
```

4. **Throwing or catching an exception by value** (the exception object is copy-constructed).

Q11. What are the syntactic differences between C and C++ programming? **(10 marks)** [CSE 201 | 2014–15]

Answer

C	C++
Input/output via `printf`/`scanf`.	Uses `cout`/`cin` with `<<`/`>>`.
No references; only pointers.	Supports reference variables (`int& r = x;`).
`struct` cannot contain functions.	`struct` and `class` can contain member functions.
No function overloading.	Function and operator overloading supported.
No default arguments.	Default arguments in function declarations supported.
`void*` can be implicitly converted to any pointer type.	`void*` requires explicit cast.
No `new`/`delete`; uses `malloc`/`free`.	`new`/`delete` for type-safe dynamic allocation.
No `bool` type (uses `int` for boolean).	Native `bool` type with `true`/`false`.
No `inline` keyword with class support.	`inline` functions; functions defined inside class are automatically inline.
No `namespace`.	`namespace` supported (`using namespace std;`).
No templates.	Function and class templates supported.

Q12. Define automatic in-line function. What are the advantages of using in-line function over parameterized macros? **(5 marks)** [CSE 201 | 2013–14]

Answer

Automatic Inline Function: An *inline function* is a function prefixed with the `inline` keyword (or defined inside a class body) that *suggests* to the compiler to replace each function call with the actual function body at the call site, eliminating function-call overhead (stack frame setup, jump, return).

```
inline int square(int x) { return x * x; }
// The compiler may replace: int r = square(5);
// with:                     int r = 5 * 5;
```

Advantages over parameterised macros (`#define`):

1. **Type safety:** Inline functions undergo full type checking by the compiler. Macros are textual substitutions with no type checking, leading to subtle bugs.

```
#define SQ(x) ((x)*(x))   // SQ(1.5+0.5) -> (1.5+0.5)*(1.5+0.5)=4 OK
                           // but SQ(++a)  -> (++a)*(++a) -- a incremented TWICE
inline int sq(int x) { return x*x; } // ++a evaluated once, passed as value
```

2. **No multiple-evaluation side effects:** Arguments are evaluated exactly once; macros can evaluate arguments multiple times.
3. **Scope and namespacing:** Inline functions respect C++ scoping rules; macros are global text substitutions with no scope.
4. **Debuggable:** Inline functions appear in debugger symbol tables; macros do not.

Q13. What is an in-line function? What are its advantages and disadvantages? Why might the following function not be in-lined by your compiler?

```
void f1() { for (int i = 0; i < 10; i++) cout << i; }
```

(8 marks) [CSE 201 | 2011–12]

Answer

Inline Function: A function declared with the `inline` keyword that *requests* the compiler to insert its body directly at every call site instead of generating a regular function call.
Advantages:

- Eliminates function-call overhead (no stack frame, no jump).
- Can enable further optimisations (constant folding, etc.).
- Full type checking, unlike macros.

Disadvantages:

- **Code bloat:** If a large function is inlined at many call sites, the binary size grows.
- Increased compilation time.
- The `inline` keyword is only a *hint*; the compiler may ignore it.

Why `f1()` might not be inlined:

```
void f1() { for (int i = 0; i < 10; i++) cout << i; }
```

Most compilers refuse to inline functions that contain **loops**, because:

- Inlining a loop body at every call site causes significant *code bloat*.
- The performance benefit of inlining is minimal when the function body itself is non-trivial (the loop iterations dominate the cost, not the call overhead).
- Many compilers have heuristics that exclude functions with loops, recursion, or large bodies from automatic inlining.

Q14. Differentiate between real-time polymorphism and run-time polymorphism with appropriate examples. **(10 marks)** [CSE 201 | 2010–11]

Q15. What is compile-time polymorphism and run-time polymorphism? Differentiate with examples. **(13 marks)** [CSE 201 | 2009–10]

↪ *Similar: CSE 201 | 2010–11; CSE 107 | 2019–20*

Answer

Compile-Time Polymorphism (Static Polymorphism):
Resolved by the compiler before the program runs. Achieved through:

- **Function overloading**
- **Operator overloading**
- **Template instantiation**

```
// Function overloading -- resolved at compile time
int    area(int s)           { return s * s; }
double area(double r)        { return 3.14159 * r * r; }
int    area(int l, int w)    { return l * w; }

area(5);        // compiler picks first version
area(3.0);      // compiler picks second version
area(4, 6);     // compiler picks third version
```

Run-Time Polymorphism (Dynamic Polymorphism):
Resolved while the program is running, based on the actual type of the object. Achieved through `virtual` functions and base class pointers/references.

```cpp
class Shape {
public:
    virtual double area() const = 0;  // pure virtual
};
class Circle : public Shape {
    double r;
public:
    Circle(double r) : r(r) {}
    double area() const override { return 3.14159 * r * r; }
};
class Rectangle : public Shape {
    double l, w;
public:
    Rectangle(double l, double w) : l(l), w(w) {}
    double area() const override { return l * w; }
};

Shape* s1 = new Circle(3.0);
Shape* s2 = new Rectangle(4.0, 6.0);
cout << s1->area();   // calls Circle::area() at run time
cout << s2->area();   // calls Rectangle::area() at run time
```

Q16. What is Object-Oriented programming? How are data and functions organized in an Object-Oriented program? **(6 marks)** [CSE 201 | 2007–08]

Answer

Object-Oriented Programming (OOP) is a programming paradigm that organises software design around *objects* rather than functions and logic. An object is an instance of a *class* that contains both data (state) and the functions (behaviour) that operate on that data.

Organisation of data and functions in OOP:

- **Data** (member variables / attributes) and **functions** (member functions / methods) are grouped together inside a **class**. This grouping is called *encapsulation*.
- Access to data is controlled by *access specifiers*:
  - `private` — accessible only within the class.
  - `protected` — accessible within the class and its derived classes.
  - `public` — accessible from anywhere.
- Data is typically kept `private` (hidden), while functions are `public` (interface). External code interacts with an object only through its public methods.
- Each *object* (instance) has its own copy of the data members, but all objects of the same class share the same method definitions.

BUET · CSE 107 / CSE 201

Object Oriented Programming

Section S2

Classes, Objects & Access Specifiers

S2 — Classes, Objects & Access Specifiers

Q1. A `Student` class has a private variable to store an ID, a constructor without parameters to initialize the ID, and a private static variable. Write the class definition for `Student` and write code in `main()` to create an array of 120 `Student` objects using the `new` operator, where the IDs are initialized as 1, 2, ..., 120 respectively in the constructor. **(10 marks)** [CSE 107 | Jan 2021]

Answer

```cpp
#include <iostream>
using namespace std;

class Student {
    int id;
    static int nextId;   // private static: shared ID counter
public:
    Student() { id = nextId++; }   // each object gets next ID
    int getId() const { return id; }
};

// Initialise static member outside the class
int Student::nextId = 1;

int main() {
    // Create array of 120 Student objects on the heap
    Student* students = new Student[120];

    // Verify first and last IDs
    cout << "First ID: " << students[0].getId()   << endl; // 1
    cout << "Last ID:  " << students[119].getId()  << endl; // 120

    delete[] students;  // release heap memory
    return 0;
}
```

How it works:

- `nextId` is `static` and initialised to `1` outside the class.
- Each time a `Student` object is constructed (including array element construction), the default constructor assigns `id = nextId` and post-increments `nextId`.
- After `new Student[120]`, the 120 objects have IDs 1 through 120 in order.

Q2. Imagine a tollbooth at a bridge. Cars passing by the booth are expected to pay a 50-cent toll. Mostly they do, but sometimes a car goes by without paying. The tollbooth keeps track of the number of cars that have gone by, and of the total amount of money collected.

Model this tollbooth with a class called `tollbooth`. The three data items are a list of string to hold the registration numbers of the cars passing through the bridge, a type unsigned int to hold the total number of cars, and a type double to hold the total amount of money collected. A constructor initializes both of these to 0. A member function called `payingCar()` adds the car registration number in the list, increments the car total and adds 0.50 to the cash total. Another function, called `nopayCar()`, adds the car registration number, increments the car total but adds nothing to the cash total. Finally, a member function called `display()` displays the list of cars' registration numbers and the two totals. Make appropriate member functions const. Use proper naming convention and setter-getter functions. **(15 marks)** [CSE 107 | 2020–21]

Answer

```cpp
#include <iostream>
#include <list>
#include <string>
using namespace std;

class tollbooth {
private:
    list<string> registrations;  // list of strings for registration numbers
    unsigned int totalCars;      // total number of cars
    double totalCash;            // total amount of money collected

public:
    // Constructor initializes totals to 0
    tollbooth() {
        totalCars = 0;
        totalCash = 0.0;
    }

    // Setter and Getter functions
```

```
    void setRegistrations(const list<string>& regs) { registrations = regs; }
    list<string> getRegistrations() const { return registrations; }

    void setTotalCars(unsigned int cars) { totalCars = cars; }
    unsigned int getTotalCars() const { return totalCars; }

    void setTotalCash(double cash) { totalCash = cash; }
    double getTotalCash() const { return totalCash; }

    // Car pays the 50-cent toll
    void payingCar(const string& reg) {
        registrations.push_back(reg);
        totalCars++;
        totalCash += 0.50;
    }

    // Car passes without paying
    void nopayCar(const string& reg) {
        registrations.push_back(reg);
        totalCars++;
    }

    // Display summary -- does not modify state, so const
    void display() const {
        cout << "Total cars: " << totalCars << endl;
        cout << "Total cash: $" << totalCash << endl;
        cout << "Cars' Registration Numbers:" << endl;
        for (const string& r : registrations) {
            cout << "  " << r << endl;
        }
    }
};

int main() {
    tollbooth booth;

    booth.payingCar("ABC-111");
    booth.payingCar("XYZ-222");
    booth.nopayCar("BAD-999");

    booth.display();

    return 0;
}
```

Const correctness: Getter methods and `display()` are declared `const` because they only read data without modifying the object's internal state.

Q3. What is the difference between class and structure? Why are both of them necessary in C++ programming? **(6 marks)** [CSE 201 | 2015–16]

Answer

Class	Structure
Members are **private** by default.	Members are **public** by default.
Inheritance default: **private**.	Inheritance default: **public**.
Conventionally used for objects with both data and behaviour (full OOP design).	Conventionally used for plain data records with little or no behaviour.

Why both are necessary:

- **Backward compatibility with C:** C code using `struct` compiles unchanged in C++. Removing `struct` would break C compatibility.
- **Semantic clarity:** Using `struct` signals to readers that a type is a simple data aggregate (e.g. a `Point`, a `Color`), while `class` signals a fully encapsulated object. This is a *convention* that improves code readability.

Q4. What is a static member? Why are they used? Write down the rules of using static variable and static method in a class. **(7 marks)** [CSE 201 | 2015–16]

↪ *Similar: CSE 201 | 2012–13*

Answer

Static Member: A class member declared with the `static` keyword. It belongs to the *class itself* rather than to any particular object — all objects of the class share a single copy of it.
Why used:

- To maintain data that is *shared* across all instances (e.g. object count, configuration).
- To provide utility functions that do not require an object.

Rules for static variables:

- Declared inside the class with `static`; must be **defined** (and optionally initialised) *outside* the class in exactly one translation unit.
- Only one copy exists regardless of how many objects are created.
- Accessible as `ClassName::variable` without an object.
- Lifetime is the entire program duration.

Rules for static methods:

- Can access only *static* members (no access to `this` pointer or non-static members).
- Can be called without creating an object: `ClassName::method()`.
- Cannot be `virtual` or `const`.

```
class Counter {
    static int count;       // declaration
public:
    Counter()  { count++; }
    ~Counter() { count--; }
    static int getCount() { return count; } // static method
};
int Counter::count = 0;     // definition outside class
```

Q5. Write a class named `MyClass` with a variable named `objectCount` that keeps track of the number of objects created. When the total number of objects is greater than 100, reinitialize `objectCount` to 0. **(5 marks)** [CSE 201 | 2014–15]

Answer

```
#include <iostream>
using namespace std;

class MyClass {
    static int objectCount;
public:
    MyClass() {
        objectCount++;
        if (objectCount > 100)
            objectCount = 0;
    }
    static int getObjectCount() { return objectCount; }
};

// Definition outside the class (required for static members)
int MyClass::objectCount = 0;

int main() {
    MyClass a, b, c;
    cout << MyClass::getObjectCount() << endl; // 3
    return 0;
}
```

Key points:

- `objectCount` is `static` — shared by all objects; incremented in the constructor each time a new object is created.
- The `if (objectCount > 100)` guard resets the counter to 0 whenever it exceeds 100.
- The static member is defined (and initialised to 0) outside the class body.

Q6. What are the properties of static variables and methods? Explain dynamic method dispatch with example. **(6+9 marks)** [CSE 201 | 2013–14]

Answer — Static Variables/Methods and Dynamic Method Dispatch

Properties of static variables:

- Shared by **all objects** of the class — only one copy exists.
- Initialized once when the class is loaded.
- Accessible via `ClassName::variable` without any object.
- Lifetime is the entire program.

Properties of static methods:

- Belong to the class, not to any instance.
- Can only access static members directly.
- No `this` pointer is available inside them.

```
class Counter {
    static int count;
public:
    Counter() { count++; }
    static int getCount() { return count; }
};
int Counter::count = 0;

// Counter::getCount() -- no object needed
```

Dynamic Method Dispatch:
The mechanism by which a virtual function call is resolved at **run time** based on the actual type of the object, not the declared type of the pointer/reference. The compiler uses a *virtual table* (vtable) per class.

```
class Shape {
public:
    virtual void draw() { cout << "Shape" << endl; }
};
class Circle : public Shape {
public:
    void draw() override { cout << "Circle" << endl; }
};
class Square : public Shape {
public:
    void draw() override { cout << "Square" << endl; }
};

int main() {
    Shape* s;
    s = new Circle(); s->draw();  // Circle -- run-time dispatch
    s = new Square(); s->draw();  // Square -- run-time dispatch
}
```

Q7. Explain why the `protected` access specifier is needed in C++ programming. **(3 marks)** [CSE 201 | 2013–14]

Answer

When we use `protected`:
We use the `protected` access specifier for a class member when we want it to be:

- **Inaccessible** from outside the class (like `private`), AND
- **Accessible** to *derived classes* (unlike `private`).

Why it is needed:
Without `protected`, a derived class would have no access to the base class's internal data unless it was made `public` (which would expose it to everyone — breaking encapsulation). `protected` achieves a middle ground: internal to the hierarchy, hidden from the outside world.

```
class Animal {
protected:
    string name;   // accessible to derived classes, not to outside
    int    age;
public:
    Animal(string n, int a) : name(n), age(a) {}
};

class Dog : public Animal {
```

```cpp
public:
    Dog(string n, int a) : Animal(n, a) {}
    void bark() const {
        // Can access 'name' because it is protected:
        cout << name << " says: Woof!\n";
    }
};

int main() {
    Dog d("Rex", 3);
    d.bark();          // OK
    // d.name;         // ERROR: 'name' is protected
}
```

Q8. How can I call a member function of a class without creating any object of that class? Explain. **(5 marks)** [CSE 201 | 2012–13]

Answer

The only way to call a member function of a class *without creating an object* is to declare it as a **static member function**. A static member function does not have a `this` pointer and is associated with the class itself, not with any instance. It can be called using the class name and the scope resolution operator:

```cpp
class MathUtils {
public:
    static int square(int x) { return x * x; }
    static double pi()       { return 3.14159265; }
};

int main() {
    // Called without any object:
    int s = MathUtils::square(5);    // s = 25
    double p = MathUtils::pi();      // p = 3.14159...
    return 0;
}
```

Restrictions: A static member function can only access *static* data members and call other *static* member functions. It cannot access non-static members because there is no object (and hence no `this` pointer) associated with the call.

Q9. Write the detail definition of a class `Pentagon` that will allow maximum five objects of the class to be created. After the fifth object, any attempt to create a new object will return the reference to the fifth object. **(12 marks)** [CSE 201 | 2009–10]

Answer

```cpp
#include <iostream>
using namespace std;

class Pentagon {
    static int  count;       // number of objects created so far
    static Pentagon* last;   // pointer to the 5th (last) object

    Pentagon() { count++; }  // private: can't call new directly

public:
    // Factory method replaces the constructor
    static Pentagon* create() {
        if (count < 5) {
            last = new Pentagon();
            return last;
        }
        // 5 already exist: return the 5th object
        return last;
    }
    static int getCount() { return count; }
};

int      Pentagon::count = 0;
Pentagon* Pentagon::last  = nullptr;

int main() {
    Pentagon* p[7];
    for (int i = 0; i < 7; i++)
        p[i] = Pentagon::create();

```

```
    cout << Pentagon::getCount() << endl;          // 5
    cout << (p[4] == p[5] ? "same" : "diff") << endl; // same
    cout << (p[4] == p[6] ? "same" : "diff") << endl; // same
    return 0;
}
```

Design:

- The constructor is `private` to prevent direct instantiation with `new Pentagon()`.
- The `static` factory method `create()` is the only way to obtain a `Pentagon`.
- After the 5th object, `create()` returns a pointer to the same 5th object every time.

Q10. Can constructors/destructors be declared as `public`, `private`, or `protected`? What will be the problems if a constructor/destructor is declared as `private`? **(6 marks)** [CSE 201 | 2009–10]

Answer

Yes, constructors and destructors can be declared as `public`, `private`, or `protected`.

Problems with a `private` constructor:

- Objects **cannot be created from outside the class** using `new` or stack allocation.
- This is intentional in the **Singleton pattern** and **Factory pattern** — creation is controlled through a static factory method (as in the `Pentagon` example).
- Derived classes **cannot call** a private constructor of the base class directly, preventing uninherited construction.

Problems with a `private` destructor:

- Objects **cannot be destroyed on the stack** — the compiler calls the destructor when a stack object goes out of scope, which fails if it is private.
- `delete ptr` from outside the class will fail.
- The class must provide a public `destroy()` method that calls `delete this` internally.
- Useful in reference-counted objects where the object controls its own lifetime.

`protected` constructor/destructor: Allows creation/destruction only within the class itself and its derived classes — useful for abstract base classes.

Q11. Can a static method of a class use the `this` pointer? Why or why not? **(4 marks)** [CSE 201 | 2009–10]

Answer

No, a static member function *cannot* use the `this` pointer.

Why not:
The `this` pointer is an implicit parameter automatically passed to every *non-static* member function. It points to the specific object on which the function is being called.
A **static** member function, however, is associated with the *class* itself, not with any particular object. It can be called without creating an object:

```
MyClass::staticMethod();   // no object involved
```

Because there is no object in the picture, there is nothing for `this` to point to. The compiler therefore does not pass a `this` pointer to static functions, and any attempt to use `this` inside a static method is a **compile error**:

```
class MyClass {
    int x;
    static int s;
public:
    static void f() {
        s = 10;        // OK  -- accesses static member
        // x = 5;     // ERROR: 'this' is unavailable in static function
        // this->x=5; // ERROR: 'this' is unavailable
    }
};
```

Q12. Write a class named `SelfCounter` that can count the number of instances in a program. Include a member function `getObjectCount()` which returns the number of objects created. Also write a `main` function to illustrate its functionality. **(7 marks)** [CSE 201 | 2008–09]

Answer

```cpp
#include <iostream>
using namespace std;

class SelfCounter {
    static int count;   // shared count of live objects
public:
    SelfCounter()              { count++; }   // created
    SelfCounter(const SelfCounter&) { count++; }  // copy
    ~SelfCounter()             { count--; }   // destroyed

    static int getObjectCount() { return count; }
};
int SelfCounter::count = 0;

int main() {
    cout << SelfCounter::getObjectCount() << endl; // 0

    SelfCounter a, b;
    cout << SelfCounter::getObjectCount() << endl; // 2

    {
        SelfCounter c, d, e;
        cout << SelfCounter::getObjectCount() << endl; // 5
    }  // c, d, e destroyed here

    cout << SelfCounter::getObjectCount() << endl; // 2
    return 0;
}
```

Note: The copy constructor also increments `count` because copy-constructing an object creates a new live instance. The destructor decrements the count when any object (original or copy) is destroyed.

Q13. Explain the usage of `const` and `mutable` keywords with examples. **(7 marks)** [CSE 201 | 2008–09]

Answer

`const` keyword:

1. **Const variable:** Value cannot be changed after initialisation.

```cpp
const int MAX = 100;   // MAX cannot be modified
```

2. **Const member function:** Does not modify any data member of the object. Can be called on `const` objects.

```cpp
class Circle {
    double radius;
public:
    Circle(double r) : radius(r) {}
    double area() const { return 3.14159 * radius * radius; }
    // area() cannot modify 'radius' or any other member
};
```

3. **Const pointer / reference parameter:** Prevents the function from modifying the passed argument.

```cpp
void print(const string& s) { cout << s; }
```

`mutable` keyword:
A data member declared `mutable` can be modified even inside a `const` member function. It is used for members that represent *implementation state* (caches, locks, call counts) rather than *logical state*.

```cpp
class Database {
    mutable int queryCount;   // mutable: can change in const functions
    string data;
public:
    Database() : queryCount(0) {}
    string fetch(int id) const {
        queryCount++;          // allowed because queryCount is mutable
        return data;
    }
    int getQueryCount() const { return queryCount; }
};
```

Q14. What are the applications of `bool` and `wchar_t` data types? Explain with examples. **(5 marks)** [CSE 201 | 2007–08]

Answer

`bool` data type:
Introduced in C++ (not in C) to represent Boolean truth values. Holds exactly one of two values: `true` (1) or `false` (0).
Applications:

- Flags and condition variables.
- Return type of predicate functions.
- Control flow conditions.

```
bool isPrime(int n) {
    if (n < 2) return false;
    for (int i = 2; i * i <= n; i++)
        if (n % i == 0) return false;
    return true;
}
bool found = false;
if (isPrime(17)) found = true;
```

`wchar_t` data type:
A wide-character type designed to hold characters from large character sets (Unicode, multi-byte encodings) that cannot fit in a single `char` (1 byte). Typically 2 bytes (Windows/UTF-16) or 4 bytes (Linux/UTF-32).
Applications:

- Internationalisation (i18n): storing and processing text in non-ASCII scripts (Chinese, Arabic, Bengali, etc.).
- Windows API programming (`LPCWSTR`, `LPWSTR`).

```
#include <iostream>
#include <cwchar>
using namespace std;

int main() {
    wchar_t ch   = L'A';           // wide character literal
    wchar_t str[] = L"Hello, \u09AC\u09BE\u0982\u09B2\u09BE"; // "Hello, \u09AC\u09BE\u0982\u09B2\
       u09BE" (Bengali)
    wcout << str << endl;
    return 0;
}
```

BUET · CSE 107 / CSE 201

Object Oriented Programming

Section S3

Inline Functions, References & Default Arguments

S3 — Inline Functions, References & Default Arguments

Q1. What is inline function? How does it differ from automatic inline function? "Inline specifier is a request, not a command" — explain. **(3+2+5 marks)** [CSE 107 | 2020–21]

↪ *Similar: CSE 107 | 2017–18*

Answer — Inline function vs automatic inline; request not command

Inline Function: A function declared with the `inline` keyword explicitly requests the compiler to expand the function body at every call site instead of generating a normal function call.

```
inline int square(int x) { return x * x; }
```

Automatic Inline Function: Any function defined *inside the class body* is implicitly treated as inline by the compiler — no `inline` keyword is needed.

```
class Circle {
    double r;
public:
    double area() { return 3.14159 * r * r; } // automatic inline
};
```

The key difference: **explicit inline** is written outside the class with the keyword; **automatic inline** happens implicitly for in-class definitions.

"Inline specifier is a request, not a command":
The `inline` keyword is only a *hint* to the compiler. The compiler is free to **ignore** it and generate a normal function call if any of the following conditions apply:

- The function body contains a loop, recursion, or `switch` statement.
- The function is too large (exceeds the compiler's size threshold).
- The address of the function is taken.
- The compiler's optimisation level is too low.

Conversely, the compiler may inline a function even *without* the keyword if it judges inlining beneficial (e.g. with `-O2` flags).

Q2. Write three different ways to declare an object of the C++ `string` class with examples. **(8 marks)** [CSE 107 | Jan 2021]

Answer — Three ways to declare a string object

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    // Way 1: Default constructor (empty string)
    string s1;
    s1 = "Hello";
    cout << s1 << "\n";   // Hello

    // Way 2: Initialise with a string literal
    string s2 = "World";
    cout << s2 << "\n";   // World

    // Way 3: Copy from another string object
    string s3(s2);
    cout << s3 << "\n";   // World

    // Bonus: n copies of a character
    string s4(5, '*');
    cout << s4 << "\n";   // *****

    return 0;
}
```

Q3. Two overloaded functions are declared as follows:

```
void space(char ch) { }
void space(char ch, int count) { }
```

Write down the `main()` function that declares two variables for storing the addresses of the above two functions and makes function calls using those variables. What problem arises if the second function is declared as `void space(char ch, int count = 0)`? **(7 marks)** [CSE 107 | 2020–21; CSE 201 | 2014–15]

Answer — Function pointers for overloaded space(); default arg ambiguity

```cpp
#include <iostream>
using namespace std;

void space(char ch)             { cout << ch << "\n"; }
void space(char ch, int count) {
    for (int i = 0; i < count; i++) cout << ch;
    cout << "\n";
}

int main() {
    void (*fp1)(char)      = space;  // first version
    void (*fp2)(char, int) = space;  // second version

    fp1('*');       // *
    fp2('-', 5);    // -----
    return 0;
}
```

Problem if second declared as `void space(char ch, int count = 0)`:
Both `space(char)` and `space(char, int=0)` match a one-argument call. When the compiler tries to resolve `fp1 = space`, it finds two equally valid matches and reports an **ambiguity error**.

Q4. `"If a function returns a reference of any type, then it can be used both as an l-value and r-value"` — explain with an example program. **(15 marks)** [CSE 107 | 2019–20, 2022–23]

Answer — Function as l-value and r-value

A function can be used as both an l-value and an r-value when it **returns a reference**. The call evaluates to a reference to the actual memory location — readable (r-value) and writable (l-value).

```cpp
#include <iostream>
using namespace std;

int data[5] = {1, 2, 3, 4, 5};
int& item(int i) { return data[i]; }

int main() {
    item(2) = 99;              // l-value: write to data[2]
    cout << data[2] << "\n";   // 99

    int val = item(0);         // r-value: read data[0]
    cout << val << "\n";       // 1

    item(1) = item(4) + 10;    // both: data[1] = data[4]+10 = 15
    cout << data[1] << "\n";   // 15
    return 0;
}
```

Q5. What is wrong with the following statement?

```cpp
void func(int x = 99, int y);
```

(5 marks) [CSE 107 | 2019–20]

Answer — What is wrong with void func(int x=99, int y)?

```cpp
void func(int x = 99, int y);   // ERROR
```

Problem: Default arguments must be specified from **right to left**. Once a parameter has a default, every parameter to its right must also have one. Here `x` has a default but `y` (to its right) does not — compile error.
Correct declarations:

```cpp
void func(int x, int y = 0);       // OK: default on rightmost
void func(int x = 0, int y = 0);   // OK: all have defaults
```

Q6. What is an in-line function? What are the advantages and disadvantages? When and why will you make a function inline? When may a compiler ignore this specifier? **(10 marks)** [CSE 201 | 2016–17; CSE 107 | 2017–18]

↪ *Similar: CSE 201 | 2007–08*

Answer — Inline function full details

Inline Function: A function prefixed with `inline` that the compiler replaces at the call site with the actual code body, avoiding function-call overhead.
Advantages:

- Eliminates call overhead (no stack frame, no jump/return).
- Enables further compiler optimisations at the call site.
- Safer than macros: full type checking, no double-evaluation.

Disadvantages:

- **Code bloat:** the function body is duplicated at every call site, increasing binary size.
- Increased compilation time.
- Changes to the body require recompilation of all includers.

When to make a function inline:

- The body is **small** (1–3 lines) and called very frequently.
- Simple accessors / getters: `int getX() const { return x; }`
- Performance-critical inner loops.

When the compiler may ignore the specifier:

- Function contains a loop or `switch`.
- Function is recursive.
- Function is too large.
- The address of the function is taken.

Q7. When can a function be used both as an l-value and r-value? Explain with an example. **(6 marks)** [CSE 107 | 2017–18]

Q8. What is the output of the following code segment?

```
int& f();
int x;
int main() {
    f() = 10;
    int a = 10, b = 20;
    int& ref = a;
    ref = b;
    cout << a << " " << b << " " << ref << " " << x;
    return 0;
}
int& f() { return x; }
```

(10 marks) [CSE 107 | 2016–17]

Answer — Output of reference code snippet

```
int& f();
int x;
int main() {
    f() = 10;
    int a = 10, b = 20;
    int& ref = a;
    ref = b;
    cout << a << " " << b << " " << ref << " " << x;
    return 0;
}
int& f() { return x; }
```

Step-by-step trace:

1. `f() = 10;` — `f()` returns reference to global `x`. Assigns 10 ⇒ `x = 10`.
2. `int a=10, b=20;` — local variables.
3. `int& ref = a;` — `ref` is alias for `a`.
4. `ref = b;` — assigns `b`'s value (20) to `a` through `ref`. So `a = 20`. (Not a rebind.)
5. Output: `a`=20, `b`=20, `ref`=20, `x`=10.

Output:

```
20 20 20 10
```

Q9. Explain why namespace is added to C++. **(5 marks)** [CSE 107 | 2015–16]

Answer — Why Namespace is Added to C++

Problem without namespace:
In large programs combining multiple libraries, different libraries may define identifiers (functions, classes, variables) with the same name. This causes **name collision** (ambiguity), and the compiler cannot determine which one to use.
Solution — namespace:
A `namespace` groups related identifiers under a unique name, preventing collisions.

```cpp
#include <iostream>
using namespace std;

namespace LibA {
    void print() { cout << "LibA::print()" << endl; }
}

namespace LibB {
    void print() { cout << "LibB::print()" << endl; }
}

int main() {
    LibA::print();   // no ambiguity
    LibB::print();
    return 0;
}
```

Output:

```
LibA::print()
LibB::print()
```

The entire C++ standard library lives in namespace `std` (`cout`, `cin`, `vector`, etc.) precisely to avoid collisions with user-defined names.

Q10. What is meant by "independent reference" in C++? How does an independent reference allow a function call to be written to the left side of an assignment operator? **(4+4 marks)** [CSE 201 | 2014–15]

↪ *Similar: CSE 201 | 2010–11*

Answer — Independent reference and l-value assignment

Independent Reference: A reference declared as a local variable — not as a parameter or return value. Must be initialised at declaration; acts as a permanent alias:

```cpp
int x = 10;
int& r = x;    // r is an independent reference to x
r = 25;        // modifies x; now x == 25
```

Function call on the left side of =:

```cpp
#include <iostream>
using namespace std;

int vals[3] = {10, 20, 30};
int& getVal(int i) { return vals[i]; }

int main() {
    getVal(1) = 99;            // sets vals[1] = 99
    cout << vals[1] << "\n";   // 99

    int v = getVal(2);         // r-value use: v = 30
    cout << v << "\n";         // 30
    return 0;
}
```

The returned reference is an alias to `vals[i]`, so assigning to `getVal(1)` directly modifies the array element.

Q11. What is a reference? Name three different scenarios when a reference can be used. Why does a reference use the dot (`.`) operator instead of the arrow (`->`) operator? **(10 marks)** [CSE 201 | 2013–14]

Answer — Reference — 3 scenarios, dot vs arrow

Reference: An alias for an existing variable declared using `&`. It must be initialised at declaration and cannot be rebound to another variable.

Three scenarios:

1. **Pass-by-reference:**

```
void swap(int& a, int& b) { int t=a; a=b; b=t; }
```

2. **Return by reference:**

```
int& getElement(int arr[], int i) { return arr[i]; }
getElement(arr, 0) = 100;
```

3. **Const reference parameter** (avoid copy of large object):

```
void print(const string& s) { cout << s; }
```

Why dot (`.`) not arrow (`->`):
`->` is used with *pointers* because a pointer holds an address and must be dereferenced. A *reference* is an alias that the compiler automatically dereferences; it behaves exactly like the original variable, so `.` is used directly:

```
struct Point { int x, y; };
Point p = {1, 2};
Point* ptr = &p;
Point& ref = p;

ptr->x = 10;   // pointer: must dereference with ->
ref.x  = 10;   // reference: same as p.x, use .
```

Q12. Explain why using a default argument is related to constructor overloading. What is wrong with the following function prototype?

```
double area(double width = 0, double length);
```

(10 marks) [CSE 201 | 2013–14]

Answer — Default argument and constructor overloading; wrong prototype

Why default arguments relate to constructor overloading:
A single constructor with default arguments can accept multiple call signatures, effectively replacing several overloaded constructors:

```
class Box {
    double h, w, l;
public:
    Box(double h=1.0, double w=1.0, double l=1.0)
        : h(h), w(w), l(l) {}
};
Box b1;                 // all defaults
Box b2(5.0);            // w=1, l=1 defaults
Box b3(5.0, 3.0, 2.0); // no defaults
```

What is wrong with:

```
double area(double width = 0, double length);  // ERROR
```

Once a parameter has a default, all parameters to its *right* must also have defaults. Here `length` follows `width` but has no default.
Correct versions:

```
double area(double length, double width = 0);     // OK
double area(double width = 0, double length = 1); // OK
```

Q13. Write a function `Compute_Volume` to compute the volume of a 3D box with height `h`, width `w`, and length `l`. Then write a new function `New_Compute_Volume` that computes the volume of a rectangular box as well as a cube using default arguments, such that all of the following calls work:

```
New_Compute_Volume(30, 20, 10);
New_Compute_Volume(10, 10, 10);
New_Compute_Volume(10);
```

(10 marks) [CSE 201 | 2012–13]

Answer — New_Compute_Volume with default arguments

```cpp
#include <iostream>
using namespace std;

double Compute_Volume(double h, double w, double l) {
    return h * w * l;
}

// When only h given -> cube; when all given -> general box
double New_Compute_Volume(double h, double w = -1, double l = -1) {
    if (w < 0) { w = h; l = h; }  // cube
    if (l < 0) { l = w; }
    return h * w * l;
}

int main() {
    cout << New_Compute_Volume(30, 20, 10) << "\n"; // 6000
    cout << New_Compute_Volume(10, 10, 10) << "\n"; // 1000
    cout << New_Compute_Volume(10)         << "\n"; // 1000
    return 0;
}
```

Q14. What is a reference? Classify references used in C++ programming. Explain with examples how a returning reference can allow a function to be used on the left side of an assignment. **(10 marks)** [CSE 201 | 2011–12]

Answer — Reference classification and l-value

Reference: An alias for an existing variable — another name that refers to the same memory location. Declared with `&`; must be initialised at declaration.

Classification of references in C++:

1. **Independent (local) reference:**

```cpp
int x = 10;
int& r = x;    // r is an alias for x
r = 20;        // x is now 20
```

2. **Reference parameter (call-by-reference):**

```cpp
void increment(int& n) { n++; }
int a = 5;
increment(a);  // a becomes 6
```

3. **Reference return value:**

```cpp
int arr[5] = {10, 20, 30, 40, 50};
int& element(int i) { return arr[i]; }
```

Returning a reference allows a function on the left side of =:

```cpp
#include <iostream>
using namespace std;

int arr[] = {10, 20, 30, 40, 50};
int& element(int i) { return arr[i]; }

int main() {
    element(2) = 99;          // equivalent to arr[2] = 99
    cout << arr[2] << "\n";   // prints: 99
    cout << element(0);       // r-value use: prints 10
    return 0;
}
```

Q15. With appropriate example, explain how a default argument produces constructor overloading ambiguity. **(6 marks)** [CSE 201 | 2011–12]

Answer — Default argument causes constructor overloading ambiguity

```cpp
class Point {
public:
    Point() {
```

```
        cout << "Default constructor\n";
    }
    Point(int x = 0, int y = 0) {
        cout << "Parameterised: " << x << "," << y << "\n";
    }
};

int main() {
    Point p;    // ERROR: ambiguous!
                // Both Point() and Point(int=0, int=0)
                // match a no-argument call.
}
```

Explanation: Both constructors can be called with no arguments. The compiler sees two equally valid matches and reports an ambiguity error. **Fix:** remove one of the constructors or remove the defaults from `Point(int, int)`.

Q16. What is namespace? Write down the problems of using global namespaces. **(5 marks)** [CSE 201 | 2011–12]

Answer — Namespace and global namespace problems

Namespace: A named scope that groups related identifiers to avoid naming conflicts between libraries or modules.

```
namespace Math    { double pi = 3.14; }
namespace Physics { double pi = 3.14159265; }
cout << Math::pi;    // 3.14
cout << Physics::pi; // 3.14159265
```

Problems with global namespace / `using namespace std`:

1. **Name collision:** Two libraries may define the same name; both in global scope causes ambiguity.
2. **Pollutes global scope:** All names become visible everywhere, making large programs harder to reason about.
3. **Maintenance burden:** A new library name may silently clash with existing global names.

Best practice: Use `std::cout` or restrict to `using std::cout;` in small scopes.

Q17. Write a function called `neg()` that reverses the sign of its integer parameter in two ways — using a pointer parameter and using a reference parameter. **(10 marks)** [CSE 201 | 2010–11]

Answer — neg() with pointer and reference

```
#include <iostream>
using namespace std;

// Way 1: using a pointer parameter
void neg_ptr(int* p) { *p = -(*p); }

// Way 2: using a reference parameter
void neg_ref(int& r) { r = -r; }

int main() {
    int a = 7, b = -3;
    neg_ptr(&a);  cout << a << "\n";  // -7
    neg_ref(b);   cout << b << "\n";  // 3
    return 0;
}
```

	Pointer	Reference
Caller syntax	`neg(&a)`	`neg(a)`
Inside function	`*p = -*p`	`r = -r`
Can be null?	Yes	No
Readability	Less natural	More natural

Q18. Explain some ways that ambiguity can be introduced when overloading functions. Why are the following two overloaded functions inherently ambiguous?

```
int f(int a);
int f(int &a);
```

(10 marks) [CSE 201 | 2010–11]

Answer — Overloading ambiguity; int f(int) vs int f(int&)

Ways ambiguity can be introduced:

1. **Implicit type conversions:**

```
void f(int x)    { }
void f(double x) { }
f(3.14f);  // float converts to both -- ambiguous
```

2. **Default argument matching another overload:**

```
void g(int x)          { }
void g(int x, int y=0) { }
g(5);  // both match -- ambiguous
```

Why `int f(int a)` and `int f(int &a)` are inherently ambiguous:

```
int f(int  a) { return a; }
int f(int& a) { return a; }
// f(x) where x is int: both versions match equally.
// Compiler cannot choose. ERROR.
```

When `f(x)` is called with an `int` variable, the compiler can either copy `x` into `a` (first version) or bind `a` as a reference to `x` (second version). Both are equally good matches — inherently ambiguous.

Q19. Provide any two restrictions (with examples) while creating references. **(5 marks)** [CSE 201 | 2007–08, 2008–09]

Answer — Two restrictions on references

1. **Must be initialised at declaration:**

```
int x = 10;
int& r = x;    // OK
// int& r2;    // ERROR: must be initialised
```

2. **Cannot be rebound to another variable:** Once bound, any assignment through the reference modifies the original variable; it does not rebind the reference.

```
int a = 5, b = 20;
int& r = a;      // r refers to a
r = b;           // sets a = 20, does NOT rebind r to b
cout << a;       // 20
cout << (&r == &a); // 1 (true: r still refers to a)
```

Additional note: A `const` reference can bind to a temporary, but a non-const reference cannot:

```
const int& cr = 5 + 3;  // OK: const ref binds to temporary
// int& r = 5 + 3;      // ERROR: non-const ref cannot bind to rvalue
```

Q20. Write short notes on: (i) namespace, (ii) this pointer, (iii) inline function. **(15 marks)** [CSE 201 | 2008–09]

Answer — namespace, this Pointer, inline Function

(i) namespace:

A `namespace` is a declarative region that provides scope to identifiers, preventing name collisions across libraries or modules.

```
namespace Math {
    int square(int x) { return x * x; }
}
// Math::square(5) -> 25
```

(ii) this pointer:

Inside every non-static member function, `this` is an implicit pointer to the **current object**. It is used to resolve ambiguity between member variables and parameters, and to return the current object.

```
class Point {
    int x, y;
public:
    Point(int x, int y) {
        this->x = x;   // disambiguates member from parameter
        this->y = y;
    }
    Point& move(int dx, int dy) {
        x += dx; y += dy;
        return *this;  // return current object for chaining
```

```
    }
};
```

(iii) inline function:
An `inline` function is a hint to the compiler to replace each call with the function body, eliminating call overhead. It is best for small, frequently called functions.

```
inline int cube(int x) { return x * x * x; }

int main() {
    int r = cube(3);   // compiler may expand to: 3*3*3
    // r = 27
}
```

Declaring a function inside a class body makes it implicitly inline.

Q21. Explain the difference between binary and unary scope resolution operators with examples. **(5 marks)** [CSE 201 | 2008–09, 2007–08]

Answer — S3-Q24-25: Binary and unary scope resolution operators

Unary `::` — no left operand; accesses global scope when a local name shadows it.
Binary `::` — left operand is a class or namespace; qualifies which scope an identifier belongs to.

```
#include <iostream>
using namespace std;

int val = 100;      // global

namespace Math { double pi = 3.14; }

class Circle {
    double r;
public:
    Circle(double r) : r(r) {}
    static double pi;
    double area();
};
double Circle::pi = 3.14159;      // binary :: -- defines static member
double Circle::area() {           // binary :: -- defines member function
    return Circle::pi * r * r;    // binary :: -- accesses static member
}

int main() {
    int val = 200;                // local
    cout << val    << "\n";       // 200 -- local
    cout << ::val  << "\n";       // 100 -- unary :: global

    cout << Math::pi << "\n";     // 3.14 -- binary :: namespace

    Circle c(5.0);
    cout << c.area() << "\n";     // 78.5...
    return 0;
}
```

Q22. `"A reference parameter fully automates the call-by-reference parameter passing mechanism"` — explain with example. **(5 marks)** [CSE 201 | 2007–08]

Answer — Reference parameter automates call-by-reference

In C, call-by-reference requires manual pointer manipulation. In C++, a reference parameter automates this entirely:

```
#include <iostream>
using namespace std;

// C-style: manual pointer manipulation
void swap_ptr(int* a, int* b) {
    int t = *a; *a = *b; *b = t;
}

// C++ style: reference parameter -- fully automatic
void swap_ref(int& a, int& b) {
```

```
    int t = a; a = b; b = t;
}

int main() {
    int x = 5, y = 10;

    swap_ptr(&x, &y);           // caller must write &x, &y
    cout << x << " " << y;     // 10 5

    swap_ref(x, y);             // caller writes x, y -- natural
    cout << x << " " << y;     // 5 10
    return 0;
}
```

With reference parameters, the caller passes variables directly (no & needed), yet the function receives the actual variables. The call-by-reference mechanism is **fully automated** by the compiler.

Q23. The effect of a default argument can be alternatively achieved by overloading. Discuss with an example. **(5 marks)** [CSE 201 | 2007–08]

Answer — Default argument vs overloading

Using default argument (single function):

```
void printLine(char ch = '-', int n = 10) {
    for (int i = 0; i < n; i++) cout << ch;
    cout << "\n";
}
printLine();          // ----------
printLine('*');       // **********
printLine('=', 5);    // =====
```

Equivalent using overloading (three functions):

```
void printLine() {
    for (int i = 0; i < 10; i++) cout << '-'; cout << "\n";
}
void printLine(char ch) {
    for (int i = 0; i < 10; i++) cout << ch;  cout << "\n";
}
void printLine(char ch, int n) {
    for (int i = 0; i < n; i++)  cout << ch;  cout << "\n";
}
```

Both produce the same interface. Default arguments are more **concise**; overloading gives more **flexibility** (each version can behave differently).

Q24. Is there anything wrong with the following code? If yes, what should be done to correct it?

```
#include <iostream>
using namespace std;
namespace A {
    int i;
    void dispI() { cout << i; }
}
void main() {
    namespace Inside {
        int insideI;
        void dispInsideI() { cout << insideI; }
    }
    A::i = 10;
    cout << A::i;
    A::dispI();
    Inside::insideI = 20;
    cout << Inside::insideI;
    Inside::dispInsideI();
}
```

(5 marks) [CSE 201 | 2007–08]

Answer — Problems in namespace code; corrected version

Problems:

1. **`void main()`:** Standard C++ requires `int main()`.

2. **Namespace inside a function:** A `namespace` declaration is **not allowed** inside a function body. Namespaces can only appear at namespace scope.

Corrected code:

```cpp
#include <iostream>
using namespace std;

namespace A {
    int i;
    void displ() { cout << i << "\n"; }
}
namespace Inside {          // moved to global scope
    int insideI;
    void displInsideI() { cout << insideI << "\n"; }
}

int main() {                // int, not void
    A::i = 10;
    cout << A::i << "\n";
    A::displ();
    Inside::insideI = 20;
    cout << Inside::insideI << "\n";
    Inside::displInsideI();
    return 0;
}
```

Q25. Can we use the same function name for a member function of a class and an outside function in the same program file? If yes, how are they distinguished? **(5 marks)** [CSE 201 | 2007–08]

Answer — Member vs global function with same name

Yes, the same function name can be used for a member function and a global function. They are distinguished by calling syntax and the scope resolution operator:

```cpp
#include <iostream>
using namespace std;

void print() { cout << "Global print()\n"; }

class Printer {
public:
    void print() { cout << "Printer::print()\n"; }
    void test() {
        print();    // calls member (this->print())
        ::print();  // calls global (unary ::)
    }
};

int main() {
    Printer p;
    p.print();   // Printer::print()
    print();     // Global print()
    p.test();    // both
    return 0;
}
```

- `obj.print()` or `Printer::print()` → member function.
- `print()` in non-member context → global function.
- `::print()` inside the class → explicitly global.

BUET · CSE 107 / CSE 201

Object Oriented Programming

Section S4

Friend Functions

S4 — Friend Functions

Q1. Design a program that defines a friend function which can access and modify the private variables of two different classes. **(15 marks)** [CSE 107 | 2022–23]

Answer — Friend function accesses and modifies two classes

```cpp
#include <iostream>
using namespace std;

class ClassB;

class ClassA {
    int a;
public:
    ClassA(int x) : a(x) {}
    void show() const { cout << "A.a = " << a << "\n"; }
    friend void swap_and_double(ClassA& ca, ClassB& cb);
};

class ClassB {
    int b;
public:
    ClassB(int x) : b(x) {}
    void show() const { cout << "B.b = " << b << "\n"; }
    friend void swap_and_double(ClassA& ca, ClassB& cb);
};

void swap_and_double(ClassA& ca, ClassB& cb) {
    int temp = ca.a;
    ca.a = cb.b * 2;
    cb.b = temp * 2;
}

int main() {
    ClassA ca(5); ClassB cb(10);
    cout << "Before:\n"; ca.show(); cb.show();
    swap_and_double(ca, cb);
    cout << "After:\n";  ca.show(); cb.show();
    return 0;
}
```

Output:

```
Before:
A.a = 5
B.b = 10
After:
A.a = 20
B.b = 10
```

Q2. Write an example program to demonstrate the use of forward declaration of a class. **(11 marks)** [CSE 107 | 2021–22]

Answer — Forward declaration — complete example

```cpp
#include <iostream>
using namespace std;

class Node;   // forward declaration

class LinkedList {
    Node* head;   // pointer to incomplete type is valid
public:
    LinkedList();
    void push(int val);
    void display() const;
};

class Node {
    int data;
    Node* next;
public:
    Node(int d) : data(d), next(nullptr) {}
    friend class LinkedList;
```

```
};

LinkedList::LinkedList() : head(nullptr) {}

void LinkedList::push(int val) {
    Node* n = new Node(val);
    n->next = head; head = n;
}

void LinkedList::display() const {
    Node* cur = head;
    while (cur) { cout << cur->data << " -> "; cur = cur->next; }
    cout << "NULL\n";
}

int main() {
    LinkedList list;
    list.push(30); list.push(20); list.push(10);
    list.display();   // 10 -> 20 -> 30 -> NULL
    return 0;
}
```

Q3. "A friend function can be a global function not related to any particular class. A friend function can also be a member of another class." — Explain this statement with appropriate examples. **(12 marks)** [CSE 107 | 2017–18]

Answer — Friend as global and as member of another class

(i) Global friend function:

```
class Celsius;
class Fahrenheit {
    double fahr;
public:
    Fahrenheit(double f) : fahr(f) {}
    friend void convert(Fahrenheit, Celsius&);
};
class Celsius {
    double cel;
public:
    Celsius() : cel(0) {}
    void show() const { cout << cel << " C\n"; }
    friend void convert(Fahrenheit, Celsius&);
};
void convert(Fahrenheit f, Celsius& c) {
    c.cel = (f.fahr - 32) * 5.0 / 9.0;
}
int main() {
    Fahrenheit f(212.0); Celsius c;
    convert(f, c); c.show();  // 100 C
}
```

(ii) Member of another class as friend:

```
class Distance;
class Printer {
public:
    void printDistance(Distance& d);
};
class Distance {
    int metres;
public:
    Distance(int m) : metres(m) {}
    friend void Printer::printDistance(Distance&);
};
void Printer::printDistance(Distance& d) {
    cout << d.metres << " m\n";
}
int main() {
    Printer p; Distance d(42);
    p.printDistance(d);   // 42 m
}
```

Q4. What is a friend function? Why is friend function used in C++ programming? **(7 marks)** [CSE 201 | 2015–16]

Answer — Friend function definition and purpose

Friend Function: A non-member function granted access to a class's `private` and `protected` members via the `friend` keyword inside the class.

```
class Box {
    double side;
public:
    Box(double s) : side(s) {}
    friend double volume(Box b);
};
double volume(Box b) {
    return b.side * b.side * b.side;  // accesses private
}
```

Why used:

1. Operator overloading where left operand is not the class type (e.g. `<<` for `ostream`).
2. Symmetric binary operations (e.g. `a+b` == `b+a`).
3. A utility function needing private data but logically not a class member.

Q5. What is a friend function? When is a friend function used? Give an example of forward class declaration. **(10 marks)** [CSE 201 | 2013–14]

Answer — Friend function; when used; forward class declaration

When used:

- A function needs private members of two or more classes.
- Symmetric binary operator overloading.
- A non-member utility that logically operates on class data.

Forward class declaration:

```
class ClassB;   // forward declaration

class ClassA {
    int a;
public:
    ClassA(int x) : a(x) {}
    friend void show(ClassA, ClassB);
};

class ClassB {
    int b;
public:
    ClassB(int x) : b(x) {}
    friend void show(ClassA, ClassB);
};

void show(ClassA oa, ClassB ob) {
    cout << "A=" << oa.a << "  B=" << ob.b << "\n";
}

int main() {
    show(ClassA(10), ClassB(20));  // A=10  B=20
}
```

Without the forward declaration, the compiler would not recognise `ClassB` when it appears inside `ClassA`.

Q6. What is a friend function? Can a member function of a class be a friend function of another class? If yes, give a code example. **(10 marks)** [CSE 201 | 2012–13]

Answer — Member function of one class as friend of another

Yes, a member function of one class can be a friend of another:

```
#include <iostream>
using namespace std;

class Alpha;

class Beta {
public:
```

```
    void display(Alpha& a);  // will be friend of Alpha
};

class Alpha {
    int secret = 42;
    friend void Beta::display(Alpha&);  // member of Beta is friend
};

void Beta::display(Alpha& a) {
    cout << "Alpha's secret: " << a.secret << "\n";
}

int main() {
    Beta b; Alpha a;
    b.display(a);   // Alpha's secret: 42
    return 0;
}
```

Key rules: Forward declaration of `Alpha` before `Beta`; `friend` names the specific member function; definition of `Beta::display` after full definition of `Alpha`.

Q7. How does a friend function differ from a member of a class? What are the advantages of using friend functions? **(6+6+15 marks)**
[CSE 201 | 2010–11, 2009–10]

Answer — Friend vs member function; advantages

Friend Function	Member Function
Not a member of the class.	A member of the class.
No implicit `this` pointer.	Has implicit `this`.
Called as `func(obj)`.	Called as `obj.func()`.
Can access private members of multiple classes simultaneously.	Only accesses own class's private.

Advantages of friend functions:

1. Can access private data of two or more classes.
2. Enables symmetric operator overloading.
3. Keeps class interface minimal while granting necessary access.
4. Flexible: can be a global function, a member of another class, or a whole friend class.

Q8. Explain the following statements regarding friend functions with examples:

(a) A friend function is not a member of a class but still has access to its private members.

(b) A friend function can be a global function not related to any particular class.

(c) A friend function can be a member of another class.

(21 marks) [CSE 201 | 2008–09]

Answer — Three statements about friend functions

(a) Not a member but accesses private:

```
class Account {
    double balance;
public:
    Account(double b) : balance(b) {}
    friend void audit(Account a);
};
void audit(Account a) {
    cout << "Balance: " << a.balance << "\n";
}
```

(b) Global function as friend:

```
class Rectangle {
    double w, h;
public:
    Rectangle(double w, double h) : w(w), h(h) {}
    friend double area(Rectangle r);
};
double area(Rectangle r) { return r.w * r.h; }
```

(c) Member of another class as friend:

```
class Alpha;
class Beta {
public:
    void inspect(Alpha& a);
};
class Alpha {
    int x = 7;
    friend void Beta::inspect(Alpha&);
};
void Beta::inspect(Alpha& a) {
    cout << "Alpha.x = " << a.x << "\n";
}
```

Q9. `"A friend function is not a member of a class but still has access to its private elements"` — explain with an example. **(5 marks)** [CSE 201 | 2007–08]

Q10. When is a friend function compulsory? Give an example. **(5 marks)** [CSE 201 | 2007–08]

Answer — When is a friend function compulsory?

A friend function is **compulsory** when overloading `<<` and `>>` (stream operators), because the left operand is `ostream`/`istream`, not our class. We cannot add a member to the standard library, so a friend is the only option:

```
#include <iostream>
using namespace std;

class Point {
    int x, y;
public:
    Point(int x, int y) : x(x), y(y) {}

    friend ostream& operator<<(ostream& os, const Point& p) {
        os << "(" << p.x << ", " << p.y << ")";
        return os;
    }
    friend istream& operator>>(istream& is, Point& p) {
        is >> p.x >> p.y;
        return is;
    }
};

int main() {
    Point p(3, 4);
    cout << p << "\n";   // (3, 4)
    return 0;
}
```

Q11. `"A friend function can be used to overload the assignment operator `='."` — True or False? Explain your answer. **(5 marks)** [CSE 201 | 2007–08]

Answer — Can friend function overload assignment operator?

False. A friend function **cannot** overload `operator=`.
The C++ standard requires `=`, `[]`, `()`, and `->` to be overloaded as **member functions** only.
For `a = b`, the left operand must be the implicit `this` object — only possible with a member function. A friend has no `this` and therefore cannot serve as the assignment operator:

```
class MyClass {
    int val;
public:
    MyClass(int v) : val(v) {}
    // CORRECT: member function
    MyClass& operator=(const MyClass& rhs) {
        val = rhs.val;
        return *this;
    }
    // friend MyClass& operator=(...) -- ILLEGAL, compile error
};
```

Q12. Distinguish between static and non-static class members with examples. **(5 marks)** [CSE 201 | 2007–08]

Answer — Static vs non-static class members

Static Member	Non-Static Member
Belongs to the class; one copy shared by all objects.	Belongs to each object; every instance has its own copy.
Exists before any object is created.	Created when an object is instantiated.
Accessed as `ClassName::member`.	Accessed as `obj.member`.
Static method has no `this`.	Non-static method has `this`.

```
class Counter {
    static int sCount;   // shared by all
    int id;              // unique per object
public:
    Counter()  { id = ++sCount; }
    ~Counter() { sCount--; }
    static int getCount() { return sCount; }
    int getId() const { return id; }
};
int Counter::sCount = 0;

int main() {
    Counter c1, c2, c3;
    cout << Counter::getCount() << "\n";  // 3
    cout << c1.getId() << " " << c2.getId() << " " << c3.getId() << "\n"; // 1 2 3
    { Counter c4; cout << Counter::getCount() << "\n"; }  // 4
    cout << Counter::getCount() << "\n";  // 3 (c4 destroyed)
    return 0;
}
```

BUET · CSE 107 / CSE 201

Object Oriented Programming

Section S5

Constructors, Destructors & Copy Constructor

S5 — Constructors, Destructors & Copy Constructor

Q1. Consider the following program and mention the line(s) where the copy constructor is called. What will be the output of this program?

```
#include <iostream>
using namespace std;
class MyClass {
public:
    int val;
    int multiplier;
    MyClass(int n, int m = 2) { val = n; multiplier = m; }
    MyClass(const MyClass& obj) {
        val = obj.val;
        multiplier = obj.multiplier;
    }
    void setValue(int v, int m = 5) { val = v; multiplier = m; }
    int calculateValue() { return val * multiplier; }
};
void functionC(MyClass* arr, int size) {
    for (int i = 0; i < size; ++i)
        cout << arr[i].calculateValue() << endl;
}
int main() {
    MyClass* arr = new MyClass[2]{MyClass(5, 2), MyClass(6)};
    arr[0] = arr[1];
    functionC(arr, 2);
    arr[0].setValue(15, 4);
    arr[1].setValue(30);
    cout << arr[0].calculateValue() << endl;
    cout << arr[1].calculateValue() << endl;
    delete[] arr;
    return 0;
}
```

(13 marks) [CSE 107 | 2022–23]

Answer — MyClass array — copy constructor lines and output

Copy constructor called: when initialising `arr[0]` and `arr[1]` from the temporary `MyClass(5,2)` and `MyClass(6)` objects. `arr[0] = arr[1]` uses the **copy assignment operator** (not copy constructor).
Output:

```
12
12
60
150
```

Trace: After `arr[0]=arr[1]`: both have val=6, mult=2 ⇒ 6*2=12 each. After `setValue`: arr[0]=15*4=60; arr[1]=30*5=150.

Q2. Write One C++ statement to implement each of the following:

(a) Using `new`, declare a pointer and allocate memory for a `char` variable initialized to `'E'`.

(b) Using `new`, declare a pointer and allocate memory for an array of 5 `float` variables.

(c) Using `delete`, delete the entire integer array pointed to by `a`.

(d) Using `new`, declare a pointer and allocate memory for an object of class `A` (no constructor defined).

(12 marks) [CSE 107 | 2022–23]

Answer — One-line new/delete statements

(a)

```
char *cp = new char('E');
```

```
float *fp = new float[5];
```

(b)

```
delete[] a;
```

```
A *ap = new A;
```

Q3. What is the problem with the following code segment? Write a problem with `s1 = s2` and provide a solution:

```
class strtype {
```

```
    char *p; int len;
public:
    strtype(char *s) {
        len = strlen(s) + 1;
        p = new char[len];
        strcpy(p, s);
    }
    ~strtype() { delete[] p; }
};
strtype s1("LEVEL"), s2("TERM");
s1 = s2;
```

Without changing the constructor and destructor, write code to solve the problem. **(15 marks)** [CSE 107 | 2021–22]

Answer — strtype s1=s2 problem and overloaded = solution

Problems: Default `=` shallow-copies `p`: both `s1.p` and `s2.p` point to the same array. Old `s1` buffer (`"LEVEL"`) is **leaked**, and when both are destroyed the shared buffer is **freed twice**.

Solution — overload `operator=`:

```
#include <iostream>
#include <cstring>
using namespace std;

class strtype {
    char *p; int len;
public:
    strtype(char *s) {
        len = strlen(s)+1; p = new char[len]; strcpy(p,s);
    }
    ~strtype() { delete[] p; }

    strtype& operator=(const strtype& obj) {
        if (this == &obj) return *this; // self-assignment guard
        delete[] p;                     // free old memory
        len = obj.len;
        p = new char[len];              // allocate fresh memory
        strcpy(p, obj.p);
        return *this;
    }
    void show() const { cout << p << "\n"; }
};

int main() {
    strtype s1("LEVEL"), s2("TERM");
    s1 = s2;
    s1.show();  // TERM
    s2.show();  // TERM
}
```

Q4. Differentiate between the `new` operator and `malloc()` function. Design `Class A` so that the following code does not produce any error:

```
int main() {
    A obj(10);
    A *array = new A[1];
    return 0;
}
```

(13 marks) [CSE 107 | 2021–22]

Answer — new vs malloc; design Class A

`new`	`malloc()`
C++ operator	C library function
Calls constructor automatically	Does not call constructors
Throws `bad_alloc` on failure	Returns `NULL` on failure
Type-safe	Returns `void*`; requires cast

Class A — needs both default and parameterised constructor:

```
class A {
    int val;
public:
    A(): val(0){}       // default: needed for new A[1]
```

```
    A(int v): val(v){} // parameterised: needed for A obj(10)
};
int main() {
    A obj(10);        // parameterised
    A *array = new A[1]; // default
    delete[] array;
}
```

Q5. Consider the following overloaded functions and develop the equivalent function using default arguments:

```
void f2() { int a = 0, b = 0; ... }
void f2(int a) { int b = 0; ... }
void f2(int a, int b) { ... }
```

(6 marks) [CSE 107 | 2021–22]

Answer — Overloaded f2() using default arguments

```
// Three overloads:
// void f2()             { int a=0, b=0; ... }
// void f2(int a)        { int b=0; ... }
// void f2(int a, int b) { ... }

// Single equivalent with defaults:
void f2(int a=0, int b=0) {
    // a=0,b=0 when called as f2()
    // b=0     when called as f2(5)
    // no default used when called as f2(5,3)
}

// All three calls still valid:
f2();        // a=0, b=0
f2(10);      // a=10, b=0
f2(10, 20);  // a=10, b=20
```

Q6. Explain what a default copy constructor generated by the compiler does. Briefly describe a circumstance where this will cause a problem. **(10 marks)** [CSE 107 | 2019–20]

Answer — Compiler-generated copy constructor and its problem

What it does: Member-wise (shallow) copy — each data member copied by value. For pointer members, only the address is copied, not the pointed-to data.

Problem: When a class contains a raw pointer to heap memory, shallow copy makes both objects share the same block. One object's destructor frees it; the other is left with a dangling pointer and causes a double-free crash.

```
class Buffer {
    char *data;
public:
    Buffer(const char* s) { data=new char[strlen(s)+1]; strcpy(data,s); }
    ~Buffer() { delete[] data; }
    // Compiler generates: Buffer(const Buffer& o){ data=o.data; } -- WRONG
};
int main() {
    Buffer b1("Hello");
    Buffer b2 = b1;   // b1.data == b2.data (same pointer!)
    // ~b2: frees data. ~b1: frees same data again -- CRASH
}
```

Fix: Write a user-defined deep copy constructor.

Q7. What are the possible problems if we use "call by value" instead of "call by reference" for passing an object to a function? Explain with an example. How can we solve this problem? **(13 marks)** [CSE 107 | 2017–18]

↪ *Similar: CSE 201 | 2009–10*

Answer — Call-by-value problems for objects; solution

Problems:

1. **Double-free / dangling pointer:** shallow copy shares heap memory; parameter's destructor frees it on return.
2. **Performance:** copying large objects wastes time.
3. **Shared state modification:** changes through the copy affect the original's heap data before the copy is destroyed.

Solutions:

(i) Define a deep copy constructor.

(ii) Pass by `const` reference: `void f(const Buffer& b)` — no copy made at all.

Q8. What will be the output of the following program?

```
class Point {
private:
    int x, y;
public:
    Point(int i = 0, int j = 0);
    Point(const Point &t);
};
Point::Point(int i, int j) {
    x = i; y = j;
    cout << "Normal Constructor called\n";
}
Point::Point(const Point &t) {
    x = t.x; y = t.y;
    cout << "Copy Constructor called\n";
}
int main() {
    Point *t1, *t2;
    t1 = new Point(10, 15);
    t2 = new Point(*t1);
    Point t3 = *t1;
    Point t4;
    t4 = t3;
    return 0;
}
```

(9 marks) [CSE 107 | 2017–18]

Answer — Output of Point program

Trace:

1: `t1 = new Point(10,15);`
⇒ **Normal Constructor called**

2: `t2 = new Point(*t1);` — copy constructor.
⇒ **Copy Constructor called**

3: `Point t3 = *t1;` — initialisation, copy constructor.
⇒ **Copy Constructor called**

4: `Point t4;` — default (i=0,j=0).
⇒ **Normal Constructor called**

5: `t4 = t3;` — assignment (not initialisation), no copy constructor.
⇒ *no output*

Output:

```
Normal Constructor called
Copy Constructor called
Copy Constructor called
Normal Constructor called
```

Q9. What is the difference between `delete` and `delete[]` operators? **(5 marks)** [CSE 201 | 2007–08; CSE 107 | 2017–18]

Answer — delete vs delete[]

`delete`	`delete[]`
For a **single** object allocated with `new`.	For an **array** allocated with `new[]`.
Calls destructor of **one** object.	Calls destructor of **every** array element.
Using on a `new[]` allocation is **undefined behaviour**.	Using on a single `new` is **undefined behaviour**.

```
int*    p1 = new int;       delete   p1;   // correct
int*    p2 = new int[100];  delete[] p2;   // correct
string* p3 = new string[5]; delete[] p3;   // calls ~string() x5
```

Q10. Write the following: `myClass ob[] = {-1, -2};`. Is this valid? If yes, what does it mean? If no, what is the problem? **(5 marks)** [CSE 107 | 2017–18]

Answer — myClass ob[] = {-1, -2} — valid?

Valid — if `myClass` has a single-`int` constructor.

```
class myClass {
    int val;
public:
    myClass(int v): val(v){}
    int get() const { return val; }
};

myClass ob[] = {-1, -2};
// Equivalent to: myClass ob[2] = {myClass(-1), myClass(-2)};
// ob[0].get() = -1,  ob[1].get() = -2
```

Each element in the initialiser list is implicitly converted to a `myClass` object using the single-`int` constructor. If the constructor is `explicit` or does not exist, this is a compile error.

Q11. How does the copy constructor differ from the assignment operator (=)? **(5 marks)** [CSE 107 | 2016–17]

Answer — Copy constructor vs assignment operator

Copy Constructor	Assignment Operator (=)
Creates a *new* object.	Copies to an *existing* object.
Called during **initialisation**: `MyClass b = a;`	Called during **assignment**: `b = a;` (b already exists).
Destination is new; no old resources to free.	Must free destination's old resources first.
Cannot return a value.	Returns `*this` for chaining.

Q12. What is the problem when an object of `Animal` is passed by value? What are the ways to solve it? Write a copy constructor to resolve it. **(15 marks)** [CSE 107 | 2016–17]

```
class Animal {
    int *age;
public:
    Animal(int arg) { age = new int; *age = arg; }
    ~Animal() { delete age; }
};
```

Answer — Animal class — copy constructor problem and fix

Problem when passed by value: Shallow copy makes both the original and the copy share the same `int*age` pointer. When the copy is destroyed, its destructor calls `delete age`, leaving the original with a dangling pointer.
Ways to solve:

(i) Define a deep copy constructor.

(ii) Pass by reference / const reference.

(iii) Overload assignment operator.

Deep copy constructor:

```
class Animal {
    int *age;
public:
    Animal(int arg) { age=new int; *age=arg; }

    Animal(const Animal& obj) {
        age=new int;        // fresh allocation
        *age=*obj.age;
    }
    ~Animal() { delete age; }
    int getAge() const { return *age; }
};
```

Q13. What is the output of the following code?

```
class Animal {
    int age;
public:
```

```
    Animal() { cout << "Constructing\n"; age = 0; }
    ~Animal() { cout << "Destructing\n"; }
};
Animal process(Animal a) {
    cout << "In process\n";
    Animal b;
    b = a;
    return b;
}
int main() {
    cout << "First line\n";
    Animal a;
    cout << "Second line\n";
    Animal b = a;
    cout << "Third line\n";
    Animal c = process(b);
    cout << "Return line\n";
    return 0;
}
```

(10 marks) [CSE 107 | 2016–17]

Answer — Output of Animal process() code

Output (verified by compiler):

```
First line
Constructing
Second line
Third line
In process
Constructing
Destructing
Return line
Destructing
Destructing
Destructing
```

Trace: `a` constructed; `b=a` (compiler copy, no print); `process(b)` — param copy (no print), then local `b` constructed (`Constructing`), local `b` and param destroyed on return (`Destructing` twice); finally `c`, `b`, `a` destroyed at end of main.

Q14. Consider the following class definition:

```
class myclass {
    int *p;
public:
    myclass(int n) {
        p = (int*)malloc(sizeof(int));
        if (!p) { cout << "Allocation problem\n"; exit(1); }
        *p = n;
    }
    ~myclass() { free(p); }
};
```

Write down the problem associated with the class declaration. Write a copy constructor to solve the problem and explain how it works. **(5+5+5 marks)** [CSE 201 | 2014–15]

Q15. Consider the following class definition:

```
class myclass {
    int *p;
public:
    myclass(int i) { p = (int*)malloc(sizeof(int)); *p = i; }
    ~myclass() { free(p); cout << "Freeing..\r"; }
    int get() { return *p; }
};
```

What is the problem when an object of the class type is passed to a regular function as a parameter? Write a copy constructor to resolve the problem. **(10 marks)** [CSE 201 | 2013–14]

Answer — S5-Q3 & Q4: myclass shallow copy problem and copy constructor fix

The problem: When a `myclass` object is passed by value, the default copy constructor makes a shallow copy—both the original and the parameter share the same `malloc`-allocated integer. When the function returns, the parameter's destructor calls `free(p)`, freeing the shared memory. The original now holds a **dangling pointer**; its destructor causes a **double-free crash**.

```
#include <iostream>
```

```cpp
#include <cstdlib>
using namespace std;

class myclass {
    int *p;
public:
    myclass(int i) { p = (int*)malloc(sizeof(int)); *p = i; }

    // Deep copy constructor
    myclass(const myclass& obj) {
        p = (int*)malloc(sizeof(int));
        *p = *obj.p;
    }

    ~myclass() { free(p); cout << "Freeing..\n"; }
    int get()  { return *p; }
};

void display(myclass ob) { cout << ob.get() << "\n"; }

int main() {
    myclass a(100);
    display(a);      // deep copy; a is safe
    cout << a.get(); // 100
}
```

Q16. What is a copy constructor? What are the three cases when a copy constructor is invoked? Give examples using code. **(10 marks)** [CSE 201 | 2012–13]

Answer — Copy constructor — definition and 3 invocation cases

Copy Constructor: A special constructor that initialises a new object as a copy of an existing object. Prototype:

```cpp
ClassName(const ClassName& obj);
```

Three cases when the copy constructor is invoked:

1. **Object initialisation from another object:**

```cpp
MyClass a(10);
MyClass b = a;    // copy constructor called
MyClass c(a);     // copy constructor called
```

2. **Passing an object by value to a function:**

```cpp
void func(MyClass x) { }   // copy constructor called for x
func(a);
```

3. **Returning an object by value from a function:**

```cpp
MyClass create() {
    MyClass tmp(99);
    return tmp;   // copy constructor may be called (if RVO absent)
}
```

Q17. What is the output of the following code segment?

```cpp
class myclass {
    int who;
public:
    myclass(int n) { who = n; cout << "Constructing" << who << "\n"; }
    ~myclass() { cout << "Destructing" << who << "\n"; }
    int id() { return who; }
};
void f(myclass &o) { cout << "Received" << o.id() << "\n"; }
int main() {
    myclass x(1);
    f(x);
    return 0;
}
```

(5 marks) [CSE 201 | 2012–13]

Answer — Output of myclass pass-by-reference code

Output:

```
Constructing1
Received1
Destructing1
```

Explanation: `f` takes a reference — no copy is made. Only one constructor and one destructor call occur.

Q18. What is the problem with the following code segment?

```
class array {
    int *p; int size;
public:
    array(int sz) {
        try { p = new int[sz]; }
        catch (bad_alloc xa) { cout << "Allocation Failure\n"; exit(EXIT_FAILURE); }
        size = sz;
    }
    ~array() { delete[] p; }
    void put(int i, int j) { if (i >= 0 && i < size) p[i] = j; }
    int get(int i) { return p[i]; }
};
void print(array &ob) {
    array x = ob;
    for (int i = 0; i < 10; i++) cout << x.get(i);
}
int main() {
    array num(10);
    for (int i = 0; i < 10; i++) num.put(i, i);
    print(num);
    for (int i = 9; i >= 0; i--) cout << num.get(i);
    return 0;
}
```

(10 marks) [CSE 201 | 2012–13]

Answer — array class shallow copy problem in print()

Problem: `array x = ob;` inside `print()` does a shallow copy. Both `x.p` and `ob.p` (i.e. `num.p`) point to the same array. When `print` returns and `x` is destroyed, its destructor frees the shared array. The loop in `main` then accesses freed memory (undefined behaviour), and `num`'s destructor causes a double-free crash.

Fix — deep copy constructor:

```
array(const array& obj) {
    size = obj.size;
    p = new int[size];
    for (int i=0; i<size; i++) p[i] = obj.p[i];
}
```

Q19. The class `dyna` is defined as follows:

```
class dyna {
    int *p;
public:
    dyna(int i);
    ~dyna() { free(p); }
    int get() { return *p; }
};
```

Write the constructor `dyna(int i)`. Also write a function called `neg()` that reverses the sign of its integer parameter. What will be the problem if `cout << neg(ob);` is executed in `main()`? Write a copy constructor to solve the problem. **(13 marks)** [CSE 201 | 2011–12]

Answer — dyna class — constructor, neg(), copy constructor

```
#include <iostream>
#include <cstdlib>
using namespace std;

class dyna {
    int *p;
public:
    dyna(int i) { p=(int*)malloc(sizeof(int)); *p=i; }

```

```
    // Deep copy constructor
    dyna(const dyna& obj) {
        p=(int*)malloc(sizeof(int)); *p=*obj.p;
    }
    ~dyna() { free(p); }
    int get() { return *p; }
};

dyna neg(dyna ob) { return dyna(-ob.get()); }

int main() {
    dyna ob(10);
    cout << neg(ob).get() << "\n";  // -10
}
```

Problem without copy constructor: `neg(dyna ob)` takes by value — shallow copy shares `p`. Parameter's destructor frees it; original `ob` has dangling pointer.

Q20. What is a copy constructor? What types of problems are solved by using copy constructors? Explain with examples. **(15 marks)** [CSE 201 | 2008–09]

Q21. What type of problems are solved by using copy constructors? Explain with examples. **(10 marks)** [CSE 201 | 2007–08]

Answer — Problems solved by copy constructors

Problem — Shallow Copy with Dynamic Memory:
The compiler-generated default copy constructor performs a *shallow copy*: for pointer members it copies the address, not the pointed-to data. Both the original and the copy then point to the **same heap block**, causing:

1. **Double-free crash:** both destructors free the same memory.
2. **Dangling pointer:** after one is destroyed, the other holds an invalid pointer.

Solution — deep copy constructor:

```
#include <iostream>
using namespace std;

class MyData {
    int *p;
public:
    MyData(int val) { p = new int; *p = val; }

    // Deep copy: allocates its own memory
    MyData(const MyData& obj) {
        p = new int;
        *p = *obj.p;
    }
    ~MyData() { delete p; }
    int get() const { return *p; }
};

void show(MyData d) { cout << d.get() << "\n"; }

int main() {
    MyData a(42);
    show(a);          // deep copy; a is unaffected
    cout << a.get(); // still 42
}
```

BUET · CSE 107 / CSE 201

Object Oriented Programming

Section S6

Operator Overloading & Type Conversion

S6 — Operator Overloading & Type Conversion

Q1. Differentiate between early binding and late binding. Demonstrate `const_cast` in case of `const` method with example. **(4+4 marks)** [CSE 107 | 2023–24]

Answer — const_cast with const method example

```cpp
#include <iostream>
using namespace std;
class Counter {
    int count;
public:
    Counter(int n): count(n){}
    void show() const { cout<<"Count="<<count<<"\n"; }
    void increment() { count++; }
    void forceIncrement() const {
        Counter* nc=const_cast<Counter*>(this); // remove const
        nc->increment();                         // now allowed
    }
};
int main(){
    Counter c(5);
    c.show();            // Count=5
    c.forceIncrement();
    c.show();            // Count=6
}
```

Warning: Using `const_cast` on a truly `const` object (declared `const`) is undefined behaviour.

Q2. Consider the following class declarations:

```cpp
class Coord {
    int x, y;
public:
    //-----------------------
    //-----------------------
};
```

Create a `Rectangle` class that uses the `Coord` class as follows:

```cpp
class Rectangle {
    Coord p1, p2, p3, p4;
    char *name;
public:
    //-----------------------
    //-----------------------
};
```

Answer the following questions and write down methods as instructed below:

(a) Write down a constructor of `Coord` class accepts only positive values of x and y with no single x- or y-coordinate larger than 200.

(b) Write down a constructor of `Rectangle` class that accepts four points and checks whether they form a rectangle or not. A rectangle has right angles at each of its corner (and checking of any three corners are sufficient). Write a `static` function `isRightAngle()` that accepts three points and checks whether a right angle is formed at the middle point or not?

(c) Write down four member functions of `Rectangle` class for computing the length, the width, the perimeter and the area of the rectangle. The length is the larger of the two dimensions. Take care to reuse methods where possible.

(d) Include a predicate function `square` that determines whether the rectangle is a square or not.

(e) Write down a clone constructor that makes a clone of rectangle.

(f) Write down necessary conversion methods for executing the following program codes:

```cpp
int value = rect; //returns the area of rect
string name = rect;
```

Where "rect" is a rectangle type object.

(g) Briefly state the problem while executing the following code-

```cpp
r1 = r2;
```

Where both `r1` and `r2` are rectangle type object. Overwrite the assignment operator to overcome the problem.

(35 marks) [CSE 107 | 2023–24]

Answer — Coord and Rectangle Classes

```cpp
#include <iostream>
#include <cstring>
#include <cmath>
#include <stdexcept>
#include <string>
using namespace std;

class Coord {
    int x, y;
public:
    // (i) Constructor validating positive values <= 200
    Coord(int x = 1, int y = 1) {
        if (x <= 0 || y <= 0 || x > 200 || y > 200)
            throw invalid_argument("Coordinates must be positive and <= 200");
        this->x = x;
        this->y = y;
    }

    // Getters needed since x and y are private by default
    int getX() const { return x; }
    int getY() const { return y; }
};

class Rectangle {
    Coord p1, p2, p3, p4;
    char* name;

public:
    // (ii) static function to check right angle at p2 (middle point)
    static bool isRightAngle(const Coord& pt1, const Coord& pt2, const Coord& pt3) {
        int ax = pt1.getX() - pt2.getX();
        int ay = pt1.getY() - pt2.getY();
        int bx = pt3.getX() - pt2.getX();
        int by = pt3.getY() - pt2.getY();
        return (ax * bx + ay * by) == 0;
    }

    // (ii) Constructor verifying rectangle properties
    Rectangle(Coord a, Coord b, Coord c, Coord d, const char* n)
        : p1(a), p2(b), p3(c), p4(d) {
        // Checking any three corners is sufficient
        if (!isRightAngle(p4, p1, p2) ||
            !isRightAngle(p1, p2, p3) ||
            !isRightAngle(p2, p3, p4)) {
            throw invalid_argument("Points do not form a rectangle");
        }
        name = new char[strlen(n) + 1];
        strcpy(name, n);
    }

    // (v) Clone constructor (Copy Constructor)
    Rectangle(const Rectangle& r)
        : p1(r.p1), p2(r.p2), p3(r.p3), p4(r.p4) {
        name = new char[strlen(r.name) + 1];
        strcpy(name, r.name);
    }

    ~Rectangle() { delete[] name; }

    // Helper method to compute distance
    double dist(const Coord& a, const Coord& b) const {
        return sqrt(pow(a.getX() - b.getX(), 2) + pow(a.getY() - b.getY(), 2));
    }

    // (iii) Length, width, perimeter, area
    double length() const {
        double d1 = dist(p1, p2);
        double d2 = dist(p2, p3);
        return (d1 > d2) ? d1 : d2; // Length is the larger dimension
    }

    double width() const {
```

```
        double d1 = dist(p1, p2);
        double d2 = dist(p2, p3);
        return (d1 < d2) ? d1 : d2;
    }

    double perimeter() const { return 2 * (length() + width()); }

    double area() const { return length() * width(); }

    // (iv) Square predicate
    bool square() const { return abs(length() - width()) < 1e-9; }

    // (vi) Conversion: int value = rect -> returns area
    operator int() const { return static_cast<int>(area()); }

    // (vi) Conversion: string name = rect -> returns name
    operator string() const { return string(name); }

    // (vii) Problem statement:
    // The default assignment operator (r1 = r2) performs a shallow copy.
    // This causes the `name` pointers of both objects to point to the
    // same dynamically allocated memory. When the objects are destroyed,
    // the destructor calls delete[] on the same memory twice, causing
    // a double-free error and potential runtime crashes.

    // Overloaded assignment operator to overcome the problem
    Rectangle& operator=(const Rectangle& r) {
        if (this == &r) return *this; // Self-assignment check

        // Copy coordinates
        p1 = r.p1; p2 = r.p2; p3 = r.p3; p4 = r.p4;

        // Deep copy of dynamically allocated array
        delete[] name;
        name = new char[strlen(r.name) + 1];
        strcpy(name, r.name);

        return *this;
    }
};
```

Q3. If you overload the assignment operator, should you return an object of the same type as the class it is overloaded for? Justify your answer. **(10 marks)** [CSE 107 | 2022–23]

Answer — Should overloaded = return same type?

Yes — it should return `ClassName&` (reference to same class) to support **chained assignment** (`a=b=c`) and to mirror the built-in behaviour.

```
MyClass& operator=(const MyClass& rhs) {
    if (this != &rhs) { /* deep copy */ }
    return *this;   // enables: a = b = c
}
```

Returning `void` breaks chaining; returning by value is inefficient and inconsistent with built-in types.

Q4. Write a program to demonstrate a user-defined conversion function that converts a class object to a built-in data type. **(9 marks)** [CSE 107 | 2022–23]

Q5. Consider the following `Coordinator` class. You are not allowed to modify the given code. Add code to produce output: `x coordinate: 7` and `y coordinate: 10`.

```
#include <iostream>
using namespace std;
class Coordinator {
    int x, y;
public:
    Coordinator(int x_val = 0, int y_val = 0) { x = x_val; y = y_val; }
    int getX() { return x; }
    int getY() { return y; }
    // add your code here
};
int main() {
```

```
    Coordinator cObj(5, 8);
    cObj += 2;
    cout << "x coordinate : " << cObj.getX() << endl;
    cout << "y coordinate : " << cObj.getY() << endl;
    return 0;
}
```

(10 marks) [CSE 107 | 2022–23]

Answer — Coordinator += to produce x=7, y=10

Add the following inside the `Coordinator` class:

```
Coordinator& operator+=(int val) {
    x += val;
    y += val;
    return *this;
}
```

Output:

```
x coordinate : 7
y coordinate : 10
```

Q6. Evaluate the output of the following program: **(6 marks)**

```
#include <iostream>
using namespace std;

class circle {
    int r;
public:
    circle(int a = 0) {
        r = a;
    }
    void show() {
        cout << r << endl;
    }
    friend circle operator++(circle &ob);
};

circle operator++(circle &ob) {
    ob.r++;
    return ob;
}

int main() {
    circle c1(10);
    ++c1;
    c1.show();
    return 0;
}
```

[CSE 107 | 2021–22]

Answer — Output of circle ++ program

Output:

```
11
```

Explanation / Trace:

(a) `circle c1(10);` instantiates an object `c1` with its member variable `r` initialized to `10` via the parameterized constructor.

(b) `++c1;` invokes the overloaded prefix increment operator function `operator++(circle &ob)`.

(c) Because the argument is passed by reference (`circle &ob`), the parameter `ob` acts as an alias for `c1`. The statement `ob.r++;` increments the value of `r` directly inside `c1` from `10` to `11`.

(d) Finally, `c1.show();` prints the current value of `r`, which outputs `11` to the console followed by a newline.

Q7. Let $xi + yj + zk$ be a vector where x, y and z are real numbers as well as i, j and k represent unique vectors along x-axis, y-axis and z-axis. There is a description for each point of the vector and hence each object has a pointer to point the description. Declare a class named "Vector" that has members and/or friends to fulfill the following requirements. Don't use friend function if it can be implemented using member function. **(7×4=28)**

Requirement	Sample Code	Explanation
Creating instance	`Vector ob(12.0, 5.0, 2.0), ob1(3.0, 2.0, 5.0, "The peak point"), ob2;`	Use default arguments to set the vector point at the origin with null description when no parameter is given. Write appropriate constructor and copy constructor.
Addition of two complex number objects	`ob1 = ob2 + ob3;`	The values in each dimension will be added separately, i.e., the x values of ob2 and ob3 will be added together to give x value of ob1 and so on. Descriptions of ob2 and ob3 will be concatenated by adding a space between them to give the description of ob1.
Incrementing a complex number	`ob = ob1 += ob2;`	Overwrite `=` and `+=` operators.
Adding a real number to an object	`ob2 = ob1 + 100.0;` `ob2 = 100.0 + ob1;`	The real number will be added to each value of the object, i.e., each value of x, y and z will be incremented by the amount of the real number.

Answer — Vector Class: Constructors, Overloaded Operators (+, +=, =)

```cpp
#include <iostream>
#include <cstring>
using namespace std;

class Vector {
    double x, y, z;
    char* desc;
public:
    // 1. Constructor with default arguments
    Vector(double x = 0.0, double y = 0.0, double z = 0.0, const char* d = NULL)
        : x(x), y(y), z(z) {
        if (d) {
            desc = new char[strlen(d) + 1];
            strcpy(desc, d);
        } else {
            desc = NULL;
        }
    }

    // 1. Copy Constructor
    Vector(const Vector& other) : x(other.x), y(other.y), z(other.z) {
        if (other.desc) {
            desc = new char[strlen(other.desc) + 1];
            strcpy(desc, other.desc);
        } else {
            desc = NULL;
        }
    }

    // Destructor to free dynamically allocated memory
    ~Vector() {
        delete[] desc;
    }

    // 3. Overwrite = operator (Member function)
    Vector& operator=(const Vector& other) {
        if (this != &other) {
            x = other.x; y = other.y; z = other.z;
            delete[] desc;
            if (other.desc) {
                desc = new char[strlen(other.desc) + 1];
                strcpy(desc, other.desc);
            } else {
                desc = NULL;
            }
        }
        return *this;
    }

    // 2. Addition of two Vector objects (Member function)
    Vector operator+(const Vector& b) const {
        Vector res(x + b.x, y + b.y, z + b.z);
        if (desc && b.desc) {
            res.desc = new char[strlen(desc) + strlen(b.desc) + 2];
            strcpy(res.desc, desc);
```

```
            strcat(res.desc, " ");
            strcat(res.desc, b.desc);
        } else if (desc) {
            res.desc = new char[strlen(desc) + 1];
            strcpy(res.desc, desc);
        } else if (b.desc) {
            res.desc = new char[strlen(b.desc) + 1];
            strcpy(res.desc, b.desc);
        }
        return res;
    }

    // 3. Overwrite += operator (Member function)
    Vector& operator+=(const Vector& other) {
        *this = *this + other; // Reuse the overloaded + and = operators
        return *this;
    }

    // 4. Adding a real number: Vector + real (Member function)
    Vector operator+(double r) const {
        return Vector(x + r, y + r, z + r, desc);
    }

    // 4. Adding a real number: real + Vector (MUST be a friend function)
    friend Vector operator+(double r, const Vector& v) {
        return Vector(r + v.x, r + v.y, r + v.z, v.desc);
    }

    void print() const {
        cout << x << "i + " << y << "j + " << z << "k";
        if (desc) cout << " [" << desc << "]";
        cout << endl;
    }
};

int main() {
    Vector ob(12.0, 5.0, 2.0);
    Vector ob1(3.0, 2.0, 5.0, "The peak point");
    Vector ob2;

    ob2 = ob + ob1;
    cout << "ob + ob1: "; ob2.print();

    Vector ob3(1.0, 1.0, 1.0, "Base");
    Vector ob_assigned = ob3 += ob1;
    cout << "ob3 after +=: "; ob3.print();
    cout << "ob_assigned: "; ob_assigned.print();

    Vector ob4 = ob1 + 100.0;
    cout << "ob1 + 100.0: "; ob4.print();

    Vector ob5 = 100.0 + ob1;
    cout << "100.0 + ob1: "; ob5.print();

    return 0;
}
```

Q8. If you overload `operator+`, must you return an object of the same type as your parameters? Justify your answer. **(7 marks)** [CSE 107 | 2019–20]

Answer — Must operator+ return same type as parameters?

No. C++ imposes no restriction on the return type of overloaded operators. The overloaded operator is just a function; its return type can be anything logical.

```
class Distance { double d; public: Distance(double v):d(v){} void show(){cout<<d;} };
class Point {
    int x, y;
public:
    Point(int a,int b):x(a),y(b){}
    // operator+ returns Distance, not Point
    Distance operator+(const Point& p) const {
        double dx=x-p.x, dy=y-p.y;
```

```
        return Distance(sqrt(dx*dx+dy*dy));
    }
};
```

In practice, returning the same type enables chaining (`a+b+c`), but it is not mandatory.

Q9. `"There are scenarios when a friend function is very useful for operator overloading"` — support this statement for the following class `Rational` by writing a program that must use friend function for operator overloading:

```
class Rational {
    int n, d;
public:
    Rational() { n = 0; d = 0; }
    Rational(int a, int b) { n = a; d = b; }
};
```

(15 marks) [CSE 107 | 2019–20]

Answer — Rational class with friend operator overloading

```
#include <iostream>
using namespace std;

class Rational {
    int n, d;
public:
    Rational(int a=0, int b=1): n(a), d(b){}

    friend Rational operator+(const Rational& a, const Rational& b){
        return Rational(a.n*b.d + b.n*a.d, a.d*b.d);
    }
    friend Rational operator*(const Rational& a, const Rational& b){
        return Rational(a.n*b.n, a.d*b.d);
    }
    friend ostream& operator<<(ostream& os, const Rational& r){
        os<<r.n<<"/"<<r.d; return os;
    }
};

int main() {
    Rational r(1,2), s(1,3);
    cout << r+s << "\n";  // 5/6
    cout << r*s << "\n";  // 1/6
}
```

Q10. Given a `Date` class with `day`, `month`, `year`: overload the binary minus operator so that `ob1 - ob2` gives the difference in number of days (assume 30 days/month). Also overload `<<` to print `day-month-year`. Include a `main()` function.

```
class Date {
    int day, month, year;
public:
    Date(int m, int d, int y) { day = d; month = m; year = y; }
};
```

(17 marks) [CSE 107 | 2019–20]

Answer — Date class — binary - and « operators

```
#include <iostream>
#include <cmath>
using namespace std;
class Date {
    int day, month, year;
public:
    Date(int m, int d, int y): day(d), month(m), year(y){}
    int operator-(const Date& o) const {
        int t1=year*360+month*30+day;
        int t2=o.year*360+o.month*30+o.day;
        return abs(t1-t2);
    }
    friend ostream& operator<<(ostream& os, const Date& d){
        os<<d.day<<"-"<<d.month<<"-"<<d.year; return os;
    }
};
```

```
int main(){
    Date d1(3,15,2024), d2(1,1,2024);
    cout<<d1<<"\n";               // 15-3-2024
    cout<<(d1-d2)<<" days\n";    // 74 days
}
```

Q11. Consider the following `Photo` and `Frame` classes. Add necessary code to `Photo` and/or `Frame` class (without modifying existing code) so that `F = P + 2;` produces `Frame Width: 7` and `Frame Length: 10` when `P` is `Photo(5,8)`:

```
class Photo { int width, length; public:
    Photo() {width=0; length=0;}
    Photo(int a, int b) {width=a; length=b;}
    int getwidth() {return width;} int getlength() {return length;}
};
class Frame { int width, length; public:
    Frame() {width=0; length=0;}
    Frame(int a, int b) {width=a; length=b;}
    int getwidth() {return width;} int getlength() {return length;}
};
```

(20 marks) [CSE 107 | 2019–20]

Answer — Photo + 2 = Frame(7,10)

```
#include <iostream>
using namespace std;
class Frame{
    int width,length;
public:
    Frame(int a=0,int b=0):width(a),length(b){}
    int getwidth(){return width;} int getlength(){return length;}
};
class Photo{
    int width,length;
public:
    Photo(int a=0,int b=0):width(a),length(b){}
    int getwidth(){return width;} int getlength(){return length;}
    // P+n returns Frame with each dim increased by n
    Frame operator+(int n){ return Frame(width+n,length+n); }
};
int main(){
    Photo P(5,8);
    Frame F=P+2;
    cout<<"Frame Width: " <<F.getwidth() <<"\n"; // 7
    cout<<"Frame Length: "<<F.getlength()<<"\n"; // 10
}
```

Q12. Consider the following `Fraction` class. Overload the prefix `++` operator using a friend function so that `++n` on `Fraction(3,7)` produces `x/y: 4/8`. Only write the overloaded function:

```
class Fraction { int x, y; public:
    Fraction(int a, int b) {x=a; y=b;}
    int getx() {return x;} int gety() {return y;}
};
```

(13 marks) [CSE 107 | 2019–20]

Answer — Fraction prefix ++ as friend

```
#include <iostream>
using namespace std;
class Fraction{
    int x,y;
public:
    Fraction(int a,int b):x(a),y(b){}
    int getx(){return x;} int gety(){return y;}
    friend Fraction& operator++(Fraction& f){ f.x++; f.y++; return f; }
};
int main(){
    Fraction n(3,7);
    ++n;
    cout<<"x/y: "<<n.getx()<<"/"<<n.gety()<<"\n"; // x/y: 4/8
}
```

Q13. Why should the overloaded assignment operator (=) function return the `this` pointer? **(5 marks)** [CSE 107 | 2017–18; CSE 201 | 2009–10]

Answer — Why operator= should return *this

Returning `*this` enables **chained assignments** like `a = b = c`, parsed as `a = (b = c)`. `b = c` must return a reference to `b` so it can be assigned to `a`.

```
class MyClass {
    int val;
public:
    MyClass(int v=0): val(v){}
    MyClass& operator=(const MyClass& rhs) {
        val = rhs.val;
        return *this;    // enables: a = b = c
    }
};
int main() {
    MyClass a(1), b(2), c(3);
    a = b = c;    // works correctly
}
```

Q14. Which of the operators can have default arguments when they are overloaded? **(3 marks)** [CSE 107 | 2017–18]

Answer — Which operators can have default arguments?

Only **`operator()`** (the function call operator) can have default arguments. All other overloaded operators cannot, because their arity is fixed by the language.

```
class Multiplier {
    int factor;
public:
    Multiplier(int f=1): factor(f){}
    int operator()(int x, int step=1) const {
        return x * factor * step;
    }
};
Multiplier m(3);
cout << m(5)    << "\n";  // 15  (step defaults to 1)
cout << m(5, 2) << "\n";  // 30
```

Q15. Consider the class `Complex`. Write necessary functions so that the following statements execute correctly in `main`:

```
class Complex {
public:
    int real, imag;
    Complex(int r = 0, int i = 0);
};
```

(i) `comp3 = comp1 + comp2;` (ii) `comp3 = comp1 + 10;` (iii) `comp3 = 10 + comp1;` (iv) `comp1++;` (v) `++comp1;` (vi) `cout << comp1;` (vii) `comp3 = comp1(5);` **(25 marks)** [CSE 107 | 2017–18]

Answer — Complex class — all 7 operations including comp1(5)

```
#include <iostream>
using namespace std;
class Complex {
public:
    int real, imag;
    Complex(int r=0, int i=0): real(r), imag(i){}
    Complex operator+(const Complex& o) const { return Complex(real+o.real,imag+o.imag); }
    Complex operator+(int n)            const { return Complex(real+n, imag); }
    friend Complex operator+(int n, const Complex& c){ return Complex(c.real+n,c.imag); }
    Complex& operator++()    { real++; imag++; return *this; }
    Complex  operator++(int) { Complex old=*this; real++; imag++; return old; }
    friend ostream& operator<<(ostream& os, const Complex& c){
        os<<c.real<<"+"<<c.imag<<"i"; return os;
    }
    Complex operator()(int n) const { return Complex(real*n, imag*n); }
};
int main(){
    Complex c1(3,4), c2(1,2), c3;
    c3=c1+c2;   cout<<c3<<"\n"; // 4+6i
    c3=c1+10;   cout<<c3<<"\n"; // 13+4i
```

```
    c3=10+c1;   cout<<c3<<"\n"; // 13+4i
    c1++;       cout<<c1<<"\n"; // 4+5i
    ++c1;       cout<<c1<<"\n"; // 5+6i
    c3=c1(5);   cout<<c3<<"\n"; // 25+30i
}
```

Q16. What is wrong with the following code segment?

```
class myclass {
    int i, j;
public:
    static void seti(int x) { this->i = x; }
    int geti() { return i; }
};
```

(6 marks) [CSE 107 | 2017–18]

Answer — Problem with this in static method

Problem: `static void seti(int x){ this->i = x; }` — a **static** member function has no `this` pointer. This is a **compile error**.
Fix:

```
// Remove static -- make it non-static
void seti(int x) { i = x; }   // has 'this'
```

Q17. Consider the following `MyInt` class. Overload: (i) `+` for adding two `MyInt` objects, (ii) `*` so a `MyInt` can be multiplied by an integer from either side, (iii) `=` for safe assignment. Note: you cannot write a copy constructor.

```
class MyInt {
    int *x;
public:
    MyInt(int argx) { x = new int; *x = argx; }
    int getX() const { return *x; }
    void setX(int argx) { *x = argx; }
    ~MyInt() { delete x; }
};
```

(20 marks) [CSE 107 | 2016–17]

Answer — MyInt — overload +, * (both sides), =

```
class MyInt {
    int *x;
public:
    MyInt(int argx){ x=new int; *x=argx; }
    int getX() const { return *x; }
    void setX(int argx){ *x=argx; }
    ~MyInt(){ delete x; }

    MyInt operator+(const MyInt& o) const { return MyInt(*x+*o.x); }
    MyInt operator*(int n)          const { return MyInt(*x*n); }
    friend MyInt operator*(int n, const MyInt& m){ return MyInt(n**m.x); }
    MyInt& operator=(const MyInt& o){ if(this!=&o) *x=*o.x; return *this; }
};
int main(){
    MyInt m1(5), m2(3);
    cout<<(m1+m2).getX()<<"\n"; // 8
    cout<<(m1*4).getX()  <<"\n"; // 20
    cout<<(3*m2).getX()  <<"\n"; // 9
}
```

Q18. How can you code `++ob1` and `ob1++` differently? **(3 marks)** [CSE 107 | 2015–16]

Answer — How to code ++ob (prefix) and ob++ (postfix) differently

The dummy `int` parameter distinguishes postfix from prefix:

```
class Counter {
    int val;
public:
    Counter(int v=0): val(v){}
    int get() const { return val; }
```

```
    // Prefix: ++ob -- increment then return new value
    Counter& operator++() { ++val; return *this; }

    // Postfix: ob++ -- save old, increment, return old
    Counter operator++(int) {
        Counter old=*this; ++val; return old;
    }
};
int main() {
    Counter c(5);
    Counter a=++c;  // c=6, a=6
    Counter b=c++;  // b=6, c=7
    cout<<a.get()<<" "<<b.get()<<" "<<c.get()<<"\n"; // 6 6 7
}
```

Q19. Let $V = (x, y, z)$ be a vector. Define a class and include the minimum necessary members for solving: `v1 += (10 + v2);` **(12 marks)** [CSE 201 | 2015–16]

Answer — Vector class for v1 += (10 + v2)

```
class Vec {
    double x, y, z;
public:
    Vec(double a=0,double b=0,double c=0): x(a),y(b),z(c){}

    Vec& operator+=(const Vec& v){ x+=v.x; y+=v.y; z+=v.z; return *this; }

    friend Vec operator+(double n, const Vec& v){
        return Vec(v.x+n, v.y+n, v.z+n);
    }
    void print(){ cout<<"("<<x<<","<<y<<","<<z<<")\n"; }
};

int main(){
    Vec v1(1,2,3), v2(4,5,6);
    v1 += (10+v2);   // 10+v2=(14,15,16); v1+=(14,15,16)
    v1.print();      // (15,17,19)
}
```

Q20. Explain different types of casting used in C++ with examples. **(10 marks)** [CSE 201 | 2014–15; CSE 107 | 2015–16]

Answer — Four C++ casting types with examples

1. **`static_cast<T>`:** Compile-time, related types.

```
int i = static_cast<int>(3.7);   // i=3
```

2. **`dynamic_cast<T>`:** Safe run-time polymorphic downcast.

```
Base* b=new Derived();
Derived* d=dynamic_cast<Derived*>(b); // null if fails
```

3. **`const_cast<T>`:** Adds/removes `const`.

```
const int x=10;
int* p=const_cast<int*>(&x);
```

4. **`reinterpret_cast<T>`:** Low-level bit-reinterpretation.

```
int* p=new int(42);
long addr=reinterpret_cast<long>(p);
```

Q21. Why is RTTI a necessary feature of C++? **(5 marks)** [CSE 107 | 2015–16]

Answer — Why is RTTI necessary in C++?

RTTI (Run-Time Type Information) allows determining the actual type of an object at run time through a base class pointer.
Necessity:

1. **Safe downcasting with `dynamic_cast`:** verifies the cast at run time; returns `nullptr` on failure — prevents corrupt memory access.

```cpp
Base* b=new Derived();
Derived* d=dynamic_cast<Derived*>(b);
if(d) d->derivedMethod();
```

2. **typeid operator:** queries exact type.

```cpp
cout<<typeid(*b).name()<<"\n";  // "Derived"
```

3. Enables correct type-specific operations in complex polymorphic hierarchies without unsafe C-style casts.

Requires at least one virtual function (vtable carries type info).

Q22. Consider the following class declaration:

```cpp
class Coord {
    int x, y;
    char *pointname;
public:
    // your code
};
```

(a) Create necessary constructors to support: `Coord ob1, ob2, ob3;` and `Coord ob4(0, 0, "origin");` (b) Explain why a copy constructor works for initialization but not for assignment. Create the required copy constructor. (c) Create necessary methods for: `ob1 = ob2 + ob3 + ob4;` and `ob1 = 100 + ob2;` (increases both x and y by 100) and `int value = 100 + ob4;` (value = $100 + \sqrt{ob4.x^2 + ob4.y^2}$, use a conversion function). (d) How can you code `++ob1` and `ob1++` differently? **(6+8+18+3 marks)** [CSE 107 | 2015–16]

Answer — Coord Class: Constructors, Operators, Conversion

```cpp
#include <iostream>
#include <cstring>
#include <cmath>
using namespace std;

class Coord {
    int   x, y;
    char* pointname;
public:
    // (a) Default constructor
    Coord() : x(0), y(0) {
        pointname = new char[8];
        strcpy(pointname, "unnamed");
    }

    // (a) Parameterized constructor
    Coord(int x, int y, const char* name) : x(x), y(y) {
        pointname = new char[strlen(name)+1];
        strcpy(pointname, name);
    }

    // (b) Copy constructor -- deep copy
    Coord(const Coord& other) : x(other.x), y(other.y) {
        pointname = new char[strlen(other.pointname)+1];
        strcpy(pointname, other.pointname);
    }

    ~Coord() { delete[] pointname; }

    // Assignment operator
    Coord& operator=(const Coord& other) {
        if (this == &other) return *this;
        delete[] pointname;
        x = other.x; y = other.y;
        pointname = new char[strlen(other.pointname)+1];
        strcpy(pointname, other.pointname);
        return *this;
    }

    // (c) operator+ : add two Coords
    Coord operator+(const Coord& other) const {
        return Coord(x + other.x, y + other.y, "result");
    }

    // (c) friend: 100 + ob2 -- adds 100 to both x and y
```

```
    friend Coord operator+(int val, const Coord& c) {
        return Coord(val + c.x, val + c.y, "result");
    }

    // (c) Conversion function: int value = 100 + ob4
    // ob4 converts to double (magnitude), then 100 + magnitude
    operator double() const {
        return sqrt((double)x*x + (double)y*y);
    }

    // (d) prefix ++ob1
    Coord& operator++() {
        ++x; ++y;
        return *this;
    }

    // (d) postfix ob1++
    Coord operator++(int) {
        Coord tmp(*this);
        x++; y++;
        return tmp;
    }

    void print() const {
        cout << pointname << "(" << x << "," << y << ")" << endl;
    }
};

int main() {
    Coord ob1, ob2, ob3;
    Coord ob4(0, 0, "origin");

    // (c) ob1 = ob2 + ob3 + ob4
    ob1 = ob2 + ob3 + ob4;
    ob1.print();

    // (c) ob1 = 100 + ob2
    ob1 = 100 + ob2;
    ob1.print();

    // (c) int value = 100 + ob4 (uses conversion)
    int value = (int)(100 + (double)ob4);
    cout << "value = " << value << endl;

    // (d) prefix and postfix
    ++ob1;
    ob1++;
    ob1.print();
    return 0;
}
```

(b) Why copy constructor works for init but not assignment:
The copy constructor is called only during **initialization** (e.g., `Coord c2 = c1`). For **assignment** (`c2 = c1` after both are constructed), the `operator=` is called instead. Without a custom `operator=`, the default does a shallow copy of the `char*` pointer, causing double-free bugs — hence the explicit `operator=` above.

Q23. Let $x + iy$ be a complex number. Declare a class named `Complex` with members/friends for:

(a) Creating instance: `Complex ob(12.0, 5.0);`

(b) Addition: `ob1 = ob2 + ob3;`

(c) Increment: `ob++;` and `++ob;`

(d) Adding real number: `ob2 = ob1 + 100.0;` and `ob2 = 100.0 + ob1;`

(20 marks) [CSE 201 | 2014–15]

Answer — Complex class (constructor, +, ++, +double both sides)

```
#include <iostream>
using namespace std;
class Complex {
    double re, im;
public:
    Complex(double r=0, double i=0): re(r), im(i){}
```

```
    Complex operator+(const Complex& o) const { return Complex(re+o.re,im+o.im); }
    Complex operator+(double n) const { return Complex(re+n, im); }
    friend Complex operator+(double n, const Complex& c){ return Complex(c.re+n,c.im); }
    Complex& operator++()    { re++; im++; return *this; }
    Complex  operator++(int) { Complex old=*this; re++; im++; return old; }
    void show() const { cout<<re<<"+"<<im<<"i\n"; }
};
int main(){
    Complex ob1(12,5), ob2, ob3(1,1);
    ob2=ob1+ob3;    ob2.show();      // 13+6i
    ob1++;          ob1.show();      // 13+6i
    ++ob1;          ob1.show();      // 14+7i
    ob2=ob1+100.0;  ob2.show();      // 114+7i
    ob2=100.0+ob1;  ob2.show();      // 114+7i
}
```

Q24. Consider the following class:

```
class coord {
    int x, y;
public:
    coord(int i = 0, int j = 0) { x = i; y = j; }
    void getxy(int &i, int &j) { i = x; j = y; }
};
```

(a) Write a function to overload + operator where both x and y are incremented by the amount passed through parameter.

(b) Write a function that allows: `ob2 = 100 + ob1;`

(c) Write inserter and extractor functions.

(d) Rewrite the class definition incorporating the above four functions.

(20 marks) [CSE 201 | 2013–14]

Answer — coord class full overloading

```
#include <iostream>
using namespace std;

class coord {
    int x, y;
public:
    coord(int i=0, int j=0): x(i), y(j){}
    void getxy(int &i, int &j) const { i=x; j=y; }

    // (a) ob + int
    coord operator+(int i) const { return coord(x+i, y+i); }

    // (b) int + ob  (friend)
    friend coord operator+(int i, const coord& ob) {
        return coord(ob.x+i, ob.y+i);
    }

    // (c) inserter
    friend ostream& operator<<(ostream& os, const coord& c) {
        os<<"("<<c.x<<","<<c.y<<")"; return os;
    }
    // (c) extractor
    friend istream& operator>>(istream& is, coord& c) {
        is>>c.x>>c.y; return is;
    }
};

int main() {
    coord ob1(2,4);
    cout << ob1+5  << "\n";   // (7,9)
    cout << 100+ob1<< "\n";   // (102,104)
    cout << ob1    << "\n";   // (2,4)
}
```

Q25. Consider the following `coord` class. Make necessary changes and overload the `+` operator so that it accepts both `ob + int` and `int + ob` without using unnecessary friend functions:

```
class coord {
    int x, y;
public:
    coord(int i = 0, int j = 0) { x = i; y = j; }
    void getxy(int &i, int &j) { i = x; j = y; }
};
```

(10 marks) [CSE 201 | 2011–12]

Answer — coord ob+int and int+ob without unnecessary friend

Since `coord(int i=0, int j=0)` is a non-explicit conversion constructor, the compiler implicitly converts an `int` to a `coord`. A single member `operator+(coord)` handles all cases:

```
class coord {
    int x, y;
public:
    coord(int i=0, int j=0): x(i), y(j){}
    void getxy(int &i, int &j) const { i=x; j=y; }

    coord operator+(const coord& ob2) const {
        return coord(x+ob2.x, y+ob2.y);
    }
};
int main() {
    coord ob1(2,4), ob2(1,1);
    coord r1 = ob1 + ob2;  // coord+coord
    coord r2 = ob1 + 5;    // 5 converted to coord(5,5)
    coord r3 = 3  + ob1;   // 3 converted to coord(3,3)
    // r1=(3,5), r2=(7,9), r3=(5,7)
}
```

Q26. Consider the following declarations of `coord` class operators. Explain what happens when each statement is executed and why `o3 = 100 + o1;` will not work. Write a function to fix it.

```
class coord {
    int x, y;
public:
    coord(int i = 0, int j = 0) { x = i; y = j; }
    void getxy(int &i, int &j) { i = x; j = y; }
    coord operator+(coord ob2);
    coord operator+(int i);
    coord operator++();
    bool operator==(coord ob2);
    coord operator=(coord ob2);
};
coord o1(2, 4), o2, o3(3, 1);
```

(i) `(o1 + o2).getxy(p, q);` (ii) `o3 = o3 + o2 + o1;` **(15 marks)** [CSE 201 | 2010–11]

Answer — coord operator statements analysis; fix 100+o1

(i) `(o1+o2).getxy(p,q)`: `o1+o2` calls `operator+(coord)` ⇒ `coord(2,4)`. `getxy` sets `p=2, q=4`.
(ii) `o3=o3+o2+o1`: `(o3+o2)=coord(3,1)`, then `+o1=coord(5,5)`. `o3` becomes `(5,5)`.
Why `100+o1` fails: `int::operator+(coord)` does not exist. The member `operator+(int)` only handles `ob+int`, not `int+ob`.
Fix:

```
friend coord operator+(int i, const coord& ob) {
    return coord(ob.x+i, ob.y+i);
}
```

Q27. Overload the < and > operators relative to the `coord` class. **(10 marks)** [CSE 201 | 2010–11]

Answer — Overload < and > for coord

```
bool operator<(const coord& ob) const {
    return (x < ob.x) && (y < ob.y);
}
bool operator>(const coord& ob) const {
    return (x > ob.x) && (y > ob.y);
}
// coord c1(1,2), c2(3,4);
// c1<c2 -> true; c2>c1 -> true
```

Q28. Create a function `reverse()` that takes a pointer to a string and a count parameter. Give count a default value that, when absent, reverses the entire string. **(7 marks)** [CSE 201 | 2010–11]

Answer — reverse() with default count parameter

```cpp
#include <iostream>
#include <cstring>
using namespace std;

void reverse(char* s, int count=-1){
    int len=(int)strlen(s);
    if(count<0 || count>len) count=len;
    for(int i=0,j=count-1; i<j; i++,j--) swap(s[i],s[j]);
}

int main(){
    char s1[]="Hello";
    reverse(s1);          cout<<s1<<"\n"; // olleH
    char s2[]="Hello World";
    reverse(s2,5);        cout<<s2<<"\n"; // olleH World
}
```

Q29. What is an operator function? Describe the syntax of operator overloading. List the restrictions imposed on operator overloading. **(5+4 marks)** [CSE 201 | 2007–08, 2009–10]

Answer — Operator function, syntax, restrictions

Operator Function: A function named `operator@` that defines the behaviour of operator `@` for user-defined types.
Syntax:

```cpp
// Member function:
return_type operator@(params) { ... }
// Friend (non-member):
friend return_type operator@(param1, param2) { ... }
```

Restrictions:

1. Cannot create new operators; only overload existing ones.
2. Cannot change arity (number of operands).
3. Cannot change precedence or associativity.
4. Cannot overload: `::`, `.`, `.*`, `?:`, `sizeof`, `typeid`.
5. `=`, `[]`, `()`, `->` must be **member functions**.
6. At least one operand must be a user-defined type.

Q30. What is a conversion function? Let a class `coord` be defined as:

```cpp
class coord {
    int x, y;
public:
    coord(int i, int j) { x = i; y = j; }
};
```

Write a conversion function that will automatically convert an object of `coord` to an integer. **(6 marks)** [CSE 201 | 2009–10]

Answer — User-defined conversion function to built-in type

```cpp
#include <iostream>
#include <cmath>
using namespace std;

class coord {
    int x, y;
public:
    coord(int i, int j): x(i), y(j){}
    // Conversion: coord -> int (Euclidean magnitude)
    operator int() const { return (int)sqrt((double)(x*x+y*y)); }
};

int main() {
    coord c(3, 4);
    int i = c;          // implicit: 5
```

```
    cout << i << "\n";   // 5
    cout << (int)c;      // explicit cast: 5
}
```

Q31. Write the necessary operator functions to allow: `c3 = 10 + c1 + c2;` where `c1`, `c2`, and `c3` are `coord` objects. **(14 marks)** [CSE 201 | 2009–10]

Answer — c3 = 10 + c1 + c2 operator functions

```
class coord {
    int x, y;
public:
    coord(int i=0, int j=0): x(i), y(j){}
    void getxy(int &i, int &j) const { i=x; j=y; }

    coord operator+(const coord& ob) const {
        return coord(x+ob.x, y+ob.y);
    }
    friend coord operator+(int i, const coord& ob) {
        return coord(ob.x+i, ob.y+i);
    }
};
// c3 = 10+c1+c2:  (10+c1(2,4))=(12,14); +c2(3,1)=(15,15)
```

Q32. Write a matrix class with default and parameterized constructors. Overload `+` for matrix addition (`m3 = m1 + m2`) and `*` for matrix multiplication (`m3 = m1 * m2`). **(20 marks)** [CSE 201 | 2009–10]

Answer — Matrix class with + and * operators

```
class Matrix {
    int **d; int r, c;
public:
    Matrix(int rows=0, int cols=0): r(rows), c(cols){
        d=new int*[r]; for(int i=0;i<r;i++){d[i]=new int[c]; for(int j=0;j<c;j++) d[i][j]=0;}
    }
    int& at(int i, int j){ return d[i][j]; }
    Matrix operator+(const Matrix& o) const {
        Matrix res(r,c);
        for(int i=0;i<r;i++) for(int j=0;j<c;j++) res.d[i][j]=d[i][j]+o.d[i][j];
        return res;
    }
    Matrix operator*(const Matrix& o) const {
        Matrix res(r,o.c);
        for(int i=0;i<r;i++) for(int j=0;j<o.c;j++) for(int k=0;k<c;k++) res.d[i][j]+=d[i][k]*o.d[k
           ][j];
        return res;
    }
};
```

Q33. Show with an example how one can find the address of a particular overloaded member function of a class. **(5 marks)** [CSE 201 | 2009–10]

Answer — Address of specific overloaded member function

```
#include <iostream>
using namespace std;
class MyClass {
public:
    void func(int x)    { cout<<"int: "    <<x  <<"\n"; }
    void func(double x) { cout<<"double: "<<x  <<"\n"; }
};
int main(){
    void (MyClass::*fp1)(int)    = &MyClass::func; // first overload
    void (MyClass::*fp2)(double) = &MyClass::func; // second overload

    MyClass obj;
    (obj.*fp1)(10);    // int: 10
    (obj.*fp2)(3.14);  // double: 3.14
}
```

The exact type of the function pointer selects the correct overload.

Q34. Write a class named `MyString` with the following: (i) `char* data`, (ii) `int length`, (iii) default, parameterized, and copy constructors, (iv) `operator+` for concatenation, (v) `operator-` to remove all occurrences of substring, (vi) overload `=` and copy constructor if needed. **(22 marks)** [CSE 201 | 2008–09]

Answer — MyString Class with Operator Overloading

```cpp
#include <iostream>
#include <cstring>
using namespace std;

class MyString {
    char* data;
    int   length;
public:
    // Default constructor
    MyString() : length(0) {
        data = new char[1];
        data[0] = '\0';
    }

    // Parameterized constructor
    MyString(const char* s) {
        length = strlen(s);
        data   = new char[length + 1];
        strcpy(data, s);
    }

    // Copy constructor
    MyString(const MyString& other) {
        length = other.length;
        data   = new char[length + 1];
        strcpy(data, other.data);
    }

    // Destructor
    ~MyString() { delete[] data; }

    // Assignment operator
    MyString& operator=(const MyString& other) {
        if (this == &other) return *this;
        delete[] data;
        length = other.length;
        data   = new char[length + 1];
        strcpy(data, other.data);
        return *this;
    }

    // operator+ : concatenation
    MyString operator+(const MyString& other) const {
        char* tmp = new char[length + other.length + 1];
        strcpy(tmp, data);
        strcat(tmp, other.data);
        MyString result(tmp);
        delete[] tmp;
        return result;
    }

    // operator- : remove all occurrences of substring
    MyString operator-(const MyString& sub) const {
        char* result = new char[length + 1];
        char* dst    = result;
        const char* src = data;
        int subLen = sub.length;

        while (*src) {
            if (strncmp(src, sub.data, subLen) == 0) {
                src += subLen;   // skip the substring
            } else {
                *dst++ = *src++;
            }
        }
```

```
        *dst = '\0';
        MyString r(result);
        delete[] result;
        return r;
    }

    void print() const { cout << data << endl; }
};

int main() {
    MyString s1("Hello ");
    MyString s2("World");
    MyString s3 = s1 + s2;
    s3.print();                  // Hello World

    MyString s4("abcabcabc");
    MyString s5("abc");
    MyString s6 = s4 - s5;
    s6.print();                  // (empty string)

    MyString s7(s3);             // copy constructor
    s7.print();                  // Hello World
    return 0;
}
```

Output:

```
Hello World

Hello World
```

Q35. Create a class `Matrix` (of double values). Include member functions to: (i) create a matrix of size $m \times n$, (ii) print the matrix, (iii) overload the parenthesis operator `()` as follows:

```
Matrix m(10, 20);
m(1, 2) = 4.5;   // access an element
m();              // erase matrix, set all to zero
cout << m(1, 2); // print value at (1,2)
```

(10 marks) [CSE 201 | 2007–08]

Answer — Matrix class with operator()

```
#include <iostream>
using namespace std;
class Matrix {
    double **data; int rows, cols;
public:
    Matrix(int m, int n): rows(m), cols(n){
        data=new double*[m];
        for(int i=0;i<m;i++){data[i]=new double[n]; for(int j=0;j<n;j++) data[i][j]=0;}
    }
    ~Matrix(){ for(int i=0;i<rows;i++) delete[] data[i]; delete[] data; }
    double& operator()(int i, int j){ return data[i][j]; } // access/set
    void   operator()(){ for(int i=0;i<rows;i++) for(int j=0;j<cols;j++) data[i][j]=0; }
    void print(){ for(int i=0;i<rows;i++){for(int j=0;j<cols;j++) cout<<data[i][j]<<" "; cout<<"\n"
        ;} }
};
int main(){
    Matrix m(3,3);
    m(1,2)=4.5; cout<<m(1,2)<<"\n"; // 4.5
    m(); cout<<m(1,2)<<"\n";          // 0
}
```

Q36. Write a class to represent a vector. Include functions to: (i) create, (ii) modify an element, (iii) add two vectors, (iv) compute dot product, (v) print. **(10 marks)** [CSE 201 | 2007–08]

Answer — Vector Class with Arithmetic Operations

```
#include <iostream>
using namespace std;

class Vector {
```

```
    float* elem;
    int    size;
public:
    // (i) Create
    Vector(int n) : size(n) {
        elem = new float[n]();
    }

    ~Vector() { delete[] elem; }

    // (ii) Modify an element
    void set(int i, float val) {
        if (i >= 0 && i < size) elem[i] = val;
    }

    float get(int i) const { return elem[i]; }
    int   getSize() const  { return size; }

    // (iii) Add two vectors
    Vector operator+(const Vector& v) const {
        Vector result(size);
        for (int i = 0; i < size; i++)
            result.elem[i] = elem[i] + v.elem[i];
        return result;
    }

    // (iv) Dot product
    float dot(const Vector& v) const {
        float sum = 0;
        for (int i = 0; i < size; i++)
            sum += elem[i] * v.elem[i];
        return sum;
    }

    // (v) Print
    void print() const {
        cout << "[ ";
        for (int i = 0; i < size; i++)
            cout << elem[i] << " ";
        cout << "]" << endl;
    }
};

int main() {
    Vector v1(3), v2(3);
    v1.set(0, 1); v1.set(1, 2); v1.set(2, 3);
    v2.set(0, 4); v2.set(1, 5); v2.set(2, 6);

    cout << "v1 = "; v1.print();
    cout << "v2 = "; v2.print();

    Vector v3 = v1 + v2;
    cout << "v1 + v2 = "; v3.print();

    cout << "v1 . v2 = " << v1.dot(v2) << endl;
    return 0;
}
```

Output:

```
v1 = [ 1 2 3 ]
v2 = [ 4 5 6 ]
v1 + v2 = [ 5 7 9 ]
v1 . v2 = 32
```

BUET · CSE 107 / CSE 201

Object Oriented Programming

Section S7

Inheritance & Polymorphism (C++)

S7 — Inheritance & Polymorphism (C++)

Q1. What is the diamond problem? State with example how the diamond problem can be solved using virtual inheritance. Differentiate between virtual function and virtual inheritance. **(4+3+3 marks)** [CSE 107 | 2023–24]

Answer — Diamond problem, virtual inheritance, virtual function vs virtual inheritance

The Diamond Problem:
When a class `D` inherits from `B` and `C`, both of which inherit from `A`, `D` contains **two copies** of `A`'s members — one from `B` and one from `C`. Accessing `A`'s members through `D` is **ambiguous**:

```
class A { public: int x; };
class B : public A {};
class C : public A {};
class D : public B, public C {};
// D has B::A::x and C::A::x -- ambiguous!
```

Solution — Virtual Inheritance:
Declare the inheritance from `A` as `virtual` in both `B` and `C`. The compiler ensures only **one shared copy** of `A` exists in `D`:

```
class A { public: int x; void show(){ cout<<x<<"\n"; } };
class B : virtual public A {};   // virtual inheritance
class C : virtual public A {};   // virtual inheritance
class D : public B, public C {}; // single copy of A

int main(){
    D d;
    d.x = 42;     // no ambiguity -- only one x
    d.show();     // 42
}
```

Virtual Function vs Virtual Inheritance:

Virtual Function	Virtual Inheritance
Enables **runtime polymorphism**: the correct overridden function is selected based on the actual object type. Declared with `virtual` in the base class function. `virtual void f() { }`	Ensures only **one copy** of a base class exists in a diamond hierarchy — solves the diamond problem. Declared with `virtual` in the inheritance specifier. `class B : virtual public A { }`

Q2. Demonstrate the access of private, public, and protected members of a base class in a derived class and in main, when the derived class inherits using **private** inheritance. **(12 marks)** [CSE 107 | 2022–23]

Answer — Private inheritance — access demo in derived and main

```
#include <iostream>
using namespace std;

class Base {
    int priv = 1;
protected:
    int prot = 2;
public:
    int pub = 3;
    void showBase(){ cout<<"priv="<<priv<<" prot="<<prot<<" pub="<<pub<<"\n"; }
};

class Derived : private Base {
    // pub  is now PRIVATE in Derived
    // prot is now PRIVATE in Derived
    // priv is INACCESSIBLE
public:
    void showDerived(){
        // priv = 10;  // ERROR: inaccessible (private in Base)
        prot = 20;     // OK: accessible inside Derived (was protected)
        pub  = 30;     // OK: accessible inside Derived (was public)
        showBase();    // OK: showBase() is accessible inside Derived
    }
};

int main(){
    Derived d;
    d.showDerived();    // OK: public method of Derived

    // d.pub = 5;       // ERROR: pub is private via private inheritance
    // d.showBase();    // ERROR: showBase() is private via private inheritance
```

```cpp
    return 0;
}
```

Summary: With private inheritance, all inherited members become private in the derived class — inaccessible from outside the derived class (`main`), but still accessible within the derived class's own member functions.

Q3. Design a program to show the use of virtual functions in achieving runtime polymorphism. **(12 marks)** [CSE 107 | 2022–23]

Answer — Runtime polymorphism with virtual functions

```cpp
#include <iostream>
using namespace std;

class Shape {
public:
    virtual double area()  const = 0;
    virtual void   show()  const { cout<<"Shape\n"; }
    virtual ~Shape(){}
};

class Circle : public Shape {
    double r;
public:
    Circle(double r): r(r){}
    double area() const override { return 3.14159*r*r; }
    void   show() const override { cout<<"Circle, area="<<area()<<"\n"; }
};

class Rectangle : public Shape {
    double w, h;
public:
    Rectangle(double w,double h):w(w),h(h){}
    double area() const override { return w*h; }
    void   show() const override { cout<<"Rectangle, area="<<area()<<"\n"; }
};

class Triangle : public Shape {
    double b, h;
public:
    Triangle(double b,double h):b(b),h(h){}
    double area() const override { return 0.5*b*h; }
    void   show() const override { cout<<"Triangle, area="<<area()<<"\n"; }
};

int main(){
    Shape* shapes[] = {
        new Circle(3),
        new Rectangle(4,5),
        new Triangle(6,8)
    };
    for(Shape* s: shapes){
        s->show();     // correct function called at run time
        delete s;
    }
    return 0;
}
```

Output:

```
Circle, area=28.2743
Rectangle, area=20
Triangle, area=24
```

Q4. Describe the use of a virtual base class. Explain why it is not possible to create an object of an abstract class. **(12 marks)** [CSE 107 | 2021–22]

Answer — Virtual base class use; why abstract class cannot be instantiated

Use of virtual base class: Eliminates the duplicate base class subobject in diamond inheritance, providing a single shared instance of the base. (See S7-Q3 for full example.)
Why an abstract class cannot be instantiated:
An **abstract class** is a class containing at least one **pure virtual function** (`= 0`). A pure virtual function has **no implementation** in the abstract class.

If the compiler allowed creating an object of the abstract class, that object might have a call to the pure virtual function, which has no body — this would lead to undefined behaviour. The compiler therefore **prevents instantiation** of any class with unimplemented pure virtual functions.

```
class Shape {
public:
    virtual double area() const = 0;  // pure virtual: no body
};

// Shape s;   // ERROR: cannot instantiate abstract class
Shape* p = new Circle(5);  // OK: pointer to abstract class is fine
```

A derived class that implements *all* pure virtual functions becomes **concrete** and can be instantiated.

Q5. Describe the differences between function overloading and function overriding. **(10 marks)** [CSE 107 | 2021–22]

↪ *Similar: CSE 107 | 2019–20*

Answer — Function overloading vs function overriding

Function Overloading	Function Overriding
Multiple functions with the **same name** but **different parameter lists** in the **same scope**.	A derived class provides a **new implementation** for a **virtual function** that already exists in the base class.
Resolved at **compile time** (static polymorphism).	Resolved at **run time** (dynamic polymorphism).
Functions differ in number/type of parameters.	Function signatures must **match exactly**.
No inheritance required.	Requires inheritance and `virtual` in base.

```
// Overloading (same class):
class Calc {
public:
    int    add(int a, int b)       { return a+b; }
    double add(double a, double b){ return a+b; }  // overloaded
};

// Overriding (inheritance):
class Animal { public: virtual void speak(){ cout<<"...sound\n"; } };
class Dog : public Animal {
public:
    void speak() override { cout<<"Woof!\n"; }  // overridden
};
```

Q6. Demonstrate the order of execution of constructor functions for multilevel inheritance with an example program. **(11 marks)** [CSE 107 | 2021–22, 2019–20]

Answer — Constructor order in multilevel and multiple inheritance

Multilevel inheritance — constructors called from base to most-derived; destructors in reverse:

```
#include <iostream>
using namespace std;

class Base { public:
    Base() { cout<<"Base constructor\n"; }
    ~Base(){ cout<<"Base destructor\n";  }
};
class Mid : public Base { public:
    Mid()  { cout<<"Mid constructor\n"; }
    ~Mid() { cout<<"Mid destructor\n";  }
};
class Der : public Mid { public:
    Der()  { cout<<"Der constructor\n"; }
    ~Der() { cout<<"Der destructor\n";  }
};

int main(){ Der d; }
```

Output:

```
Base constructor
Mid constructor
Der constructor
Der destructor
Mid destructor
Base destructor
```

Multiple inheritance — constructor order follows declaration:

```cpp
class X { public: X(){cout<<"X\n";} ~X(){cout<<"~X\n";} };
class Y { public: Y(){cout<<"Y\n";} ~Y(){cout<<"~Y\n";} };
class Z : public X, public Y { public: Z(){cout<<"Z\n";} ~Z(){cout<<"~Z\n";} };
int main(){ Z z; }
// Output: X  Y  Z  ~Z  ~Y  ~X
```

Q7. Is it possible that a derived class inherits more than one copy of the base class? If yes, show the scenario; how can you avoid it? **(15 marks)** [CSE 107 | 2019–20]

Answer — Can a derived class inherit more than one copy of base? How to avoid?

Yes — without virtual inheritance, a derived class in a diamond hierarchy inherits **multiple copies** of the base:

```cpp
class A { public: int x=0; };
class B : public A {};   // B has A::x
class C : public A {};   // C has A::x
class D : public B, public C {};  // D has B::A::x AND C::A::x

D d;
// d.x = 5;       // ERROR: ambiguous -- which x?
d.B::x = 5;      // must qualify explicitly
d.C::x = 10;
```

How to avoid — virtual inheritance:

```cpp
class A { public: int x=0; };
class B : virtual public A {};
class C : virtual public A {};
class D : public B, public C {};

D d;
d.x = 42;   // unambiguous -- single shared A
cout << d.x; // 42
```

With `virtual`, only one copy of `A` is maintained in `D`, shared by both `B` and `C` paths.

Q8. Differentiate between compile time polymorphism and run time polymorphism. **(8 marks)** [CSE 107 | 2019–20]

Answer — Compile-time vs run-time polymorphism

Compile-Time Polymorphism	Run-Time Polymorphism
Resolved by compiler before execution.	Resolved during program execution.
Achieved via function overloading, operator overloading, templates.	Achieved via virtual functions and base class pointers/references.
Faster — no vtable lookup overhead.	Slight overhead — vtable lookup per call.
Also called *static polymorphism*.	Also called *dynamic polymorphism*.

```cpp
// Compile-time:
void print(int x)    { cout<<x<<"\n"; }
void print(double x) { cout<<x<<"\n"; }
print(5);    // compiler picks print(int)
print(3.14); // compiler picks print(double)

// Run-time:
class Shape { public: virtual double area()const=0; };
class Circle: public Shape { double r; public: Circle(double r):r(r){}
    double area()const override{return 3.14159*r*r;} };
Shape* s=new Circle(3);
cout<<s->area();  // decided at run time: Circle::area()
```

Q9. Describe a scenario when we cannot create an object of a class. **(7 marks)** [CSE 107 | 2019–20]

↪ *Similar: CSE 201 | 2007–08*

Answer — Scenario when an object of a class cannot be created

Scenario 1 — Abstract class (pure virtual function):
A class with at least one unimplemented pure virtual function is *abstract*; no object can be created from it.

```cpp
class Shape { public: virtual double area()=0; };
```

```
// Shape s;  // ERROR
```

Scenario 2 — Private constructor (Singleton pattern):
The constructor is declared `private`; objects can only be created through a controlled static factory method.

```
class Singleton {
    Singleton(){}
public:
    static Singleton& get(){ static Singleton s; return s; }
};
// Singleton s;  // ERROR: constructor is private
Singleton& s = Singleton::get(); // OK
```

Scenario 3 — Deleted constructor:

```
class NoInstance {
public:
    NoInstance() = delete;
};
// NoInstance n;  // ERROR: use of deleted function
```

Q10. Why virtual base classes are used? Give an example of a class hierarchy where virtual base class is necessary to avoid ambiguity. **(9 marks)** [CSE 107 | 2017–18]

Answer — Virtual base class — why and when needed

Virtual Base Class: A base class declared with `virtual` in the inheritance specifier. It ensures that when a class appears multiple times in a hierarchy (diamond), only **one shared instance** of that base is included in the most-derived class.
When needed: Whenever the same base class is inherited by two or more intermediate classes that are themselves combined in a derived class (diamond problem).
Example:

```
#include <iostream>
using namespace std;

class Vehicle {
public:
    int speed;
    Vehicle(int s=0): speed(s){}
    void show(){ cout<<"Speed: "<<speed<<"\n"; }
};

class Car    : virtual public Vehicle {};
class Boat   : virtual public Vehicle {};
class Amphibian : public Car, public Boat {
public:
    Amphibian(int s): Vehicle(s){}  // initialises the ONE shared Vehicle
};

int main(){
    Amphibian a(120);
    a.speed = 80;     // no ambiguity
    a.show();         // Speed: 80
}
```

Without `virtual`, `Amphibian` would have `Car::Vehicle` and `Boat::Vehicle` as two separate subobjects; `a.speed` would be ambiguous.

Q11. Can constructor or destructor functions be declared as virtual? Explain briefly. **(5 marks)** [CSE 107 | 2017–18]

Q12. Explain what is wrong with the following code. Rewrite to solve the problem. What will be the output after fixing?

```
class A {
    int x;
public:
    void setX(int i) { x = i; }
    void print() { cout << x; }
};
class B : public A { public: B() { setX(10); } };
class C : public A { public: C() { setX(20); } };
class D : public B, public C { };
int main() { D d; d.print(); return 0; }
```

(15 marks) [CSE 107 | 2016–17]

Answer — Fix diamond ambiguity in class D; output after fix

Problem: `D` inherits from both `B` and `C`, each of which has its own copy of `A`. `d.print()` is **ambiguous** — the compiler does not know whether to call `B::A::print()` or `C::A::print()`.
Fix — make A a virtual base class:

```cpp
#include <iostream>
using namespace std;

class A {
    int x;
public:
    void setX(int i) { x = i; }
    void print()     { cout << x << "\n"; }
};

class B : virtual public A {  // virtual inheritance
public:
    B() { setX(10); }
};

class C : virtual public A {  // virtual inheritance
public:
    C() { setX(20); }
};

class D : public B, public C {};

int main() {
    D d;         // B sets x=10, then C sets x=20 (one shared copy)
    d.print();  // unambiguous: prints 20
    return 0;
}
```

Output after fixing:

```
20
```

Reason: The single shared `A::x` is set to 10 by `B`'s constructor, then overwritten to 20 by `C`'s constructor (base constructors are called in declaration order).

Q13. Briefly explain the visibility of private, protected, and public members of a class in a derived class when the base class is inherited as private, protected, and public. **(15 marks)** [CSE 107 | 2016–17]

Answer — Visibility of Class Members in Inheritance

When a derived class inherits from a base class, the visibility of the inherited members depends on the access specifier used during inheritance. Regardless of the inheritance mode, `private` members of the base class are **never** directly accessible from the derived class.

1. Public Inheritance:

- `public` members of the base class remain `public` in the derived class.
- `protected` members of the base class remain `protected` in the derived class.

2. Protected Inheritance:

- Both `public` and `protected` members of the base class become `protected` in the derived class. They are accessible inside the derived class but inaccessible from outside.

3. Private Inheritance:

- Both `public` and `protected` members of the base class become `private` in the derived class. They can only be accessed by the member functions of the derived class itself.

Visibility Summary Table:

	Inheritance type		
Base member	**public**	**protected**	**private**
public	public	protected	private
protected	protected	protected	private
private	inaccessible	inaccessible	inaccessible

Example — Demonstrating Private Inheritance:

```cpp
#include <iostream>
using namespace std;

class Base
{
    int priv = 1;

protected:
    int prot = 2;

public:
    int pub = 3;
};

class Der : private Base
{
    //    pub -> becomes private in Der (accessible inside Der only)
    //    prot -> becomes private in Der (accessible inside Der only)
    //    priv -> totally inaccessible
public:
    void show()
    {
        cout << "pub: " << pub << "\n";   // OK
        cout << "prot: " << prot << "\n"; // OK
        // cout << priv;                  // ERROR: inaccessible
    }
};

int main()
{
    Der d;
    // d.pub = 5;  // ERROR: pub is private via private inheritance
    d.show(); // OK: show() is a public function in Der
    return 0;
}
```

Q14. Suppose class A has virtual functions and class B publicly inherits from A. What happens if class B does not implement some virtual functions? What if they are pure virtual? **(10 marks)** [CSE 107 | 2016–17]

Answer — B inherits A (virtual); what if B doesn't implement some virtual functions?

Case 1 — Virtual functions (not pure):
If `B` does not override a virtual function from `A`, `B` **inherits the base class implementation**. Calling the function through a `B` pointer uses `A`'s version. `B` can still be instantiated.

```cpp
class A { public: virtual void f(){ cout<<"A::f\n"; } };
class B : public A { };  // does not override f

B b;
b.f();          // calls A::f -- inherited
A* p = &b;
p->f();         // calls A::f (runtime dispatch still works)
```

Case 2 — Pure virtual functions:
If `A` has a pure virtual function and `B` does **not** implement it, `B` is also **abstract** — it cannot be instantiated.

```cpp
class A { public: virtual void f()=0; };  // pure virtual
class B : public A { };                   // does NOT override f
// B is still abstract
// B b;   // ERROR: cannot instantiate abstract class B

class C : public A { public: void f(){ cout<<"C::f\n"; } };
C c;       // OK: C implements f
c.f();     // C::f
```

Q15. Implement a `Queue` class by inheriting from the `List` class. Your queue class should override the three functions as follows:

(i) `store` — override this to add a new item at the end of the queue
(ii) `retrieve` — override this to remove and return the value from the beginning of the queue
(iii) `print` — displays the queue items as a comma separated list at the stdout

```cpp
class ListItem
{
```

```
    int item;
    ListItem * next;
public:
    ListItem(int arg){ item = arg; next = NULL; }
    int getItem(){ return item; }
    void setNext(ListItem * n){ next = n; }
    ListItem * getNext() { return next; }
};

class List
{
private:
    ListItem * head;
    ListItem * tail;
public:
    List () { head = NULL; tail = NULL; }
    virtual void store(int item)=0;
    virtual int retrieve()=0;
    virtual void print()=0;
};
```

(20 marks) [CSE 107 | 2016–17, 2013–14]

Answer — Queue inheriting List — store, retrieve, print

```
#include <iostream>
using namespace std;

class ListItem
{
    int item;
    ListItem * next;
public:
    ListItem(int arg){ item = arg; next = NULL; }
    int getItem(){ return item; }
    void setNext(ListItem * n){ next = n; }
    ListItem * getNext() { return next; }
};

class List
{
protected: // Note: Changed from private to protected so Queue can access head & tail
    ListItem * head;
    ListItem * tail;
public:
    List () { head = NULL; tail = NULL; }
    virtual void store(int item)=0;
    virtual int retrieve()=0;
    virtual void print()=0;
    virtual ~List() {
        while(head) {
            ListItem* t = head;
            head = head->getNext();
            delete t;
        }
    }
};

class Queue : public List {
public:
    // (i) store - add at end of the queue
    void store(int item) override {
        ListItem* n = new ListItem(item);
        if (tail == NULL) {
            head = tail = n;
        } else {
            tail->setNext(n);
            tail = n;
        }
    }

    // (ii) retrieve - remove and return from beginning of the queue
    int retrieve() override {
        if (head == NULL) {
            return -1; // Assuming -1 represents empty
```

```
        }
        int v = head->getItem();
        ListItem* t = head;
        head = head->getNext();
        if (head == NULL) {
            tail = NULL;
        }
        delete t;
        return v;
    }

    // (iii) print - comma-separated list at stdout
    void print() override {
        ListItem* cur = head;
        while (cur != NULL) {
            cout << cur->getItem();
            if (cur->getNext() != NULL) cout << ",";
            cur = cur->getNext();
        }
        cout << "\n";
    }
};

int main(){
    Queue q;
    q.store(10); q.store(20); q.store(30);
    q.print();                    // 10,20,30
    cout << q.retrieve() << "\n"; // 10
    q.print();                    // 20,30
    return 0;
}
```

Q16. Assume the class `hand` is inherited by both `man` and `monkey`. The class `animal` inherits both. What type of inheritance is this? What is the problem? How to solve it? **(8 marks)** [CSE 201 | 2015–16]

Answer — hand/man/monkey/animal — inheritance type, problem, solution

Type of inheritance: Multiple inheritance (diamond pattern) — `man` and `monkey` both inherit `hand`; `animal` inherits both, making it a *diamond hierarchy*.
Problem: `animal` inherits **two copies** of `hand` — one via `man` and one via `monkey`. Any reference to `hand`'s members inside `animal` is **ambiguous**.
Solution — virtual base class:

```
class hand {
public:
    void wave() { cout << "Hand waves\n"; }
};
class man    : virtual public hand {};
class monkey : virtual public hand {};
class animal : public man, public monkey {};

int main() {
    animal a;
    a.wave();   // no ambiguity -- one shared copy of hand
}
```

Q17. What are the differences between virtual function and pure virtual function? If a class contains a pure virtual function, what is it called and what restrictions apply? **(10 marks)** [CSE 107 | 2015–16]

Answer — Virtual vs pure virtual function; abstract class restrictions

Virtual Function	Pure Virtual Function
Has a body in the base class.	Has **no** body (`= 0`).
Derived class *may* override.	Derived class **must** override.
Base class can be instantiated.	Base class **cannot** be instantiated (abstract).
`virtual void f(){...}`	`virtual void f()=0;`

Abstract class — restrictions:

- Cannot create objects of an abstract class directly.

- Can declare pointers and references to it.
- A derived class that does not implement all pure virtual functions is also abstract.
- Constructors of abstract classes can be defined (called by derived constructors).

Q18. Consider the bookshop example: `media` base class with `title` and `publication`; `book` subclass with number of pages; `tape` subclass with playing time. Each class has virtual `read()` and `show()`. Write a program modeling this hierarchy and processing objects using base class pointers. **(20 marks)** [CSE 201 | 2015–16]

Answer — Bookshop hierarchy — media, book, tape with virtual functions

```cpp
#include <iostream>
#include <string>
using namespace std;

class media {
protected:
    string title, publication;
public:
    media(string t, string p): title(t), publication(p){}
    virtual void read() const = 0;
    virtual void show() const = 0;
    virtual ~media(){}
};

class book : public media {
    int pages;
public:
    book(string t,string p,int pg): media(t,p), pages(pg){}
    void read() const override { cout<<"Reading book: "<<title<<"\n"; }
    void show() const override {
        cout<<"Book: "<<title<<" | "<<publication<<" | "<<pages<<" pages\n";
    }
};

class tape : public media {
    double playtime;
public:
    tape(string t,string p,double pt): media(t,p), playtime(pt){}
    void read()  const override { cout<<"Playing tape: "<<title<<"\n"; }
    void show()  const override {
        cout<<"Tape: "<<title<<" | "<<publication<<" | "<<playtime<<" min\n";
    }
};

int main(){
    media* arr[] = {
        new book("C++ Primer","Pearson",900),
        new tape("Data Structures","MIT",60.0)
    };
    for(auto m: arr){ m->show(); m->read(); delete m; }
    return 0;
}
```

Q19. Suppose there are three classes A, B and C. Class C wants to inherit both A and B simultaneously. Describe two ways with code examples. **(5 marks)** [CSE 201 | 2012–13]

Answer — Two ways for class C to inherit both A and B

Way 1 — Direct multiple inheritance:

```cpp
class A { public: void showA(){ cout<<"A\n"; } };
class B { public: void showB(){ cout<<"B\n"; } };
class C : public A, public B {};   // inherits both directly

int main(){
    C c;
    c.showA(); c.showB();  // A  B
}
```

Way 2 — Multilevel (chain) inheritance:

```cpp
class A { public: void showA(){ cout<<"A\n"; } };
```

```cpp
class B : public A { public: void showB(){ cout<<"B\n"; } };
class C : public B {};  // C gets both A and B via chain

int main(){
    C c;
    c.showA(); c.showB();  // A  B
}
```

Q20. What is wrong with the following code segment?

```cpp
class base {
    int x;
public:
    void setx(int n) { x = n; }
    void showx() { cout << x << "\n"; }
};
class derived : private base {
    int y;
public:
    void sety(int n) { y = n; }
    void showy() { cout << y << "\n"; }
};
int main() {
    derived ob;
    ob.setx(10); ob.sety(20);
    ob.showx(); ob.showy();
    return 0;
}
```

(5 marks) [CSE 201 | 2012–13]

Answer — Problem in private inheritance code

```cpp
class derived : private base { ... };
int main(){
    derived ob;
    ob.setx(10);  // ERROR: setx is private in derived (private inheritance)
    ob.showx();   // ERROR: showx is private in derived
    ...
}
```

Problem: `derived` inherits `base` as `private`. All `public` members of `base` (`setx`, `showx`) become **private** in `derived` and therefore **cannot be called from `main`**.
Fix options:

(i) Change to `public` inheritance: `class derived : public base`

(ii) Add public wrapper methods in `derived`.

(iii) Use `using base::setx;` inside the public section of `derived` to re-expose specific members.

```cpp
// Fix (i):
class derived : public base {  // public inheritance
    int y;
public:
    void sety(int n){ y=n; }
    void showy(){ cout<<y<<"\n"; }
};
int main(){
    derived ob;
    ob.setx(10); ob.sety(20);   // now OK
    ob.showx();  ob.showy();
}
```

Q21. Define virtual function and pure virtual function with appropriate examples. Differentiate between early binding and late binding.
(10 marks) [CSE 201 | 2011–12]

Answer — Virtual function, pure virtual function, early vs late binding

Virtual Function: Has a default implementation in the base class; derived classes may override it.

```cpp
class Shape { public: virtual double area(){ return 0; } };
```

Pure Virtual Function: Has **no** implementation in the base class (`= 0`); derived classes *must* override it. Makes the class abstract.

```cpp
class Shape { public: virtual double area() const = 0; };
```

Early binding (compile time): Function call resolved by the compiler based on the declared type. Non-virtual functions.
Late binding (run time): Function call resolved at run time based on the actual object type. Virtual functions via vtable.

```cpp
class A { public: void f(){cout<<"A::f\n";} virtual void g(){cout<<"A::g\n";} };
class B : public A { public: void f(){cout<<"B::f\n";} void g() override{cout<<"B::g\n";} };

A* p=new B();
p->f();  // early binding: A::f (declared type is A*)
p->g();  // late binding:  B::g (actual object is B)
```

Q22. Define virtual base class with examples. Explain why a virtual base class might be necessary. **(4 marks)** [CSE 201 | 2011–12]

Q23. What happens when a public member is inherited as public, private, or protected? **(8+9 marks)** [CSE 201 | 2011–12, 2010–11]

Answer — Member visibility table for public/private/protected inheritance

When a base class member is inherited, its accessibility in the derived class depends on **both** the original access specifier **and** the inheritance type:

Base member	public inh.	protected inh.	private inh.
`public`	`public`	`protected`	`private`
`protected`	`protected`	`protected`	`private`
`private`	not accessible	not accessible	not accessible

Key rules:

- **private members** of the base are *never* accessible in derived classes regardless of inheritance type.
- **public inheritance** preserves the original access levels (IS-A relationship).
- **protected inheritance** limits public members to protected in the derived class.
- **private inheritance** makes everything private in the derived class.

Q24. In what order are constructors called for `class myclass : public A, public B, public C { }`? In what order are destructors called? **(3 marks)** [CSE 201 | 2011–12]

Answer — Constructor/destructor order for class D : public A, public B, public C

For `class myclass : public A, public B, public C`:
Constructor order: `A()` → `B()` → `C()` → `myclass()` (left-to-right declaration order, base before derived).
Destructor order (always reverse of constructors): ∼`myclass()` → ∼`C()` → ∼`B()` → ∼`A()`.

Q25. Define virtual function. Why is virtual function used in C++ programming? **(5 marks)** [CSE 201 | 2010–11]

↪ *Similar: CSE 201 | 2009–10; CSE 107 | 2017–18*

Answer — Virtual function — definition and purpose

Virtual Function: A member function declared with the `virtual` keyword in a base class. When called through a base class pointer or reference, the **most-derived** version of the function is executed at *run time* (late binding / dynamic dispatch).
Why used:

1. **Runtime polymorphism:** Allows a single base-class pointer to correctly invoke the overridden function of whichever derived object it points to.
2. **Extensibility:** New derived classes can be added without changing code that uses base class pointers.
3. **Clean OOP design:** Separates interface (base class) from implementation (derived classes).

```cpp
class Animal {
public:
    virtual void speak() { cout<<"...sound\n"; }
};
class Dog : public Animal {
public:
    void speak() override { cout<<"Woof!\n"; }
};
class Cat : public Animal {
public:
    void speak() override { cout<<"Meow!\n"; }
};
```

```
13
14 int main(){
15     Animal* a;
16     Dog d; Cat c;
17     a=&d; a->speak();  // Woof!  -- runtime dispatch
18     a=&c; a->speak();  // Meow!
19 }
```

Q26. What is virtual base class? Why and when is it needed? Explain with an example. **(12 marks)** [CSE 201 | 2009–10]

↪ *Similar: CSE 201 | 2008–09; CSE 107 | 2017–18*

Q27. Constructors cannot be virtual, but destructors can be virtual. Why? **(3 marks)** [CSE 201 | 2009–10]

Answer — Can constructors/destructors be virtual?

Constructors cannot be virtual:

- A virtual function call is dispatched through the vtable, which requires the object to already exist and be fully constructed.
- During construction, the object is not yet complete; the vtable pointer is not yet set up for the derived class.
- Therefore, making a constructor virtual is meaningless and the C++ standard forbids it.

Destructors *can* (and often *should*) be virtual:

- When a derived object is deleted through a base class pointer, the correct (most-derived) destructor must be called.
- Without a virtual destructor, only the base destructor is called — the derived part is not cleaned up (memory/resource leak).

```
1  class Base {
2  public:
3      virtual ~Base(){ cout<<"Base dtor\n"; }  // MUST be virtual
4  };
5  class Der : public Base {
6  public:
7      ~Der(){ cout<<"Der dtor\n"; }
8  };
9
10 int main(){
11     Base* p = new Der();
12     delete p;
13     // With virtual dtor: "Der dtor", then "Base dtor"  -- correct
14     // Without virtual:   "Base dtor" only              -- LEAK
15 }
```

Q28. What are the implications of the following two definitions?

(a) `class A : public B, public C { };`

(b) `class A : public C, public B { };`

(4 marks) [CSE 201 | 2007–08]

Answer — Implications of class A : public B, public C vs class A : public C, public B

Both declarations make `A` inherit publicly from **both** `B` and `C` (multiple inheritance). The only difference is the **order** in which base class subobjects are laid out in memory and the order in which constructors and destructors are called:

(a) `class A : public B, public C { };`

- Memory layout: `B` subobject first, then `C`.
- Constructors called: `B()` first, then `C()`, then `A()`.
- Destructors in reverse: `A`, `C`, `B`.

(b) `class A : public C, public B { };`

- Memory layout: `C` subobject first, then `B`.
- Constructors called: `C()` first, then `B()`, then `A()`.

Both are otherwise equivalent in terms of member accessibility. The order matters when `B` and `C` have members with the same name (ambiguity) or when constructor side effects depend on initialisation order.

Q29. When do we make a virtual function "pure"? What are the implications? Explain with example. **(6 marks)** [CSE 201 | 2007–08]

Answer — When to make a virtual function pure; implications

When to make a virtual function pure:
Make a virtual function *pure* when:

- There is **no meaningful default implementation** in the base class.
- Every derived class *must* provide its own specific implementation.
- The base class is intended as an **interface** or **abstract type** — not to be instantiated directly.

Implications:

1. The class becomes **abstract**; no objects can be created from it.
2. Derived classes that do not implement the pure virtual function are also abstract.
3. Only **concrete** derived classes (those that override all pure virtuals) can be instantiated.

```
class Shape {          // abstract -- no meaningful default area
public:
    virtual double area() const = 0;  // pure virtual
    virtual ~Shape(){}
};

class Circle : public Shape {
    double r;
public:
    Circle(double r): r(r){}
    double area() const override { return 3.14159*r*r; }  // must implement
};

// Shape s;         // ERROR: cannot instantiate abstract class
Shape* p = new Circle(5); cout << p->area(); delete p; // 78.54
```

BUET · CSE 107 / CSE 201

Object Oriented Programming

Section S8

Templates & STL

S8 — Templates & STL

Q1. What is erasure in terms of generic class? Consider the following `main` function and write down associated generic classes where `Gen` inherits `NonGen` and `Gen2D` inherits `Gen`:

```cpp
int main() {
    NonGen ob(10);
    Gen<char> ob1(20, 'A');
    Gen2D<double, char *> ob2(40, 60.5, "Gen2Object");

    ob.show();
    ob1.show();
    ob2.show();

    cout << ob1.getob() << endl;
    cout << ob2.getob() << endl;

    return 0;
}
/* Output:
  NonGen: 10
  Gen: A
  Gen2D: Gen2Object
  A
  Gen2Object
*/
```

(15 marks) [CSE 107 | 2023–24]

Answer — Type erasure; NonGen, Gen, Gen2D hierarchy

Type Erasure in generic classes:
Type erasure is a process used in languages like Java where the compiler removes (erases) all information related to generic type parameters during compilation, replacing them with their bounds or an `Object` type. This means generic type information is not visible at runtime. *Note: C++ templates do not use type erasure; instead, they use monomorphization, instantiating separate, distinct copies of the code for each type used.*

```cpp
#include <iostream>
using namespace std;

// Non-generic base class
class NonGen {
    int num;
public:
    NonGen(int n) : num(n) {}
    void show() {
        cout << "NonGen: " << num << "\n";
    }
};

// Generic class inheriting NonGen
template <class T>
class Gen : public NonGen {
    T ob;
public:
    Gen(int n, T o) : NonGen(n), ob(o) {}

    // Overriding show (without calling NonGen::show to match output)
    void show() {
        cout << "Gen: " << ob << "\n";
    }

    T getob() { return ob; }
};

// 2D generic class inheriting Gen
template <class T1, class T2>
class Gen2D : public Gen<T2> {
    T1 ob2D;
public:
    // Passes n and o2 to the base Gen<T2> constructor
    Gen2D(int n, T1 o1, T2 o2) : Gen<T2>(n, o2), ob2D(o1) {}

    // Overriding show to just print the T2 object
    void show() {
        // Using this-> to access dependent base class members in C++
```

```
        cout << "Gen2D: " << this->getob() << "\n";
    }

    // Note: getob() is inherited from Gen<T2>, so ob2.getob()
    // will naturally return the T2 (char *) object without needing
    // to be rewritten or shadowed here.
};

int main() {
    NonGen ob(10);
    Gen<char> ob1(20, 'A');

    // Note: Older C++ allowed string literals to char*,
    // modern compilers may warn about this deprecation.
    Gen2D<double, char*> ob2(40, 60.5, (char*)"Gen2Object");

    ob.show();
    ob1.show();
    ob2.show();

    cout << ob1.getob() << "\n";
    cout << ob2.getob() << "\n";

    return 0;
}
```

Q2. What is STL? Explain different fundamental items of STL. **(5 marks)** [CSE 107 | 2023–24]

Answer — What is STL? Fundamental items.

STL (Standard Template Library): A collection of generic, reusable C++ components — containers, algorithms, and iterators — built on templates.
Four fundamental items of STL:

1. **Containers:** Template classes that store collections of objects.
   - *Sequence:* `vector`, `list`, `deque`
   - *Associative:* `map`, `set`, `multimap`
   - *Unordered:* `unordered_map`, `unordered_set`
2. **Algorithms:** Generic functions that operate on containers via iterators: `sort`, `find`, `count`, `transform`, `copy`, etc.
3. **Iterators:** Objects that act like pointers, providing a uniform way to traverse containers regardless of their internal structure (`begin()`, `end()`, `++`, `*`).
4. **Function Objects (Functors):** Objects that behave like functions (overload `operator()`), used to customise algorithm behaviour (e.g. comparators, predicates).

Q3. Consider that a list container contains: `20 16 12 8 4 3 6 9 12 15`. Using STL algorithms, write a C++ program that will:

(a) Count how many times 12 appears.
(b) Count how many even numbers there are.
(c) Remove all appearances of 12 and store the new list in a separate container.
(d) Reverse the original list.
(e) Replace all numbers with their squares using `transform` and iterator.

(10 marks) [CSE 107 | 2023–24]

Answer — STL list operations — count, even, remove, reverse, transform

```
#include <iostream>
#include <list>
#include <algorithm>
#include <iterator>
using namespace std;

int main(){
    list<int> lst = {20,16,12,8,4,3,6,9,12,15};

    // (a) Count occurrences of 12
    cout << "Count of 12: "
         << count(lst.begin(), lst.end(), 12) << "\n";  // 2
```

```
13
14      // (b) Count even numbers
15      cout << "Even count: "
16           << count_if(lst.begin(), lst.end(),
17                       [](int x){ return x % 2 == 0; }) << "\n"; // 7
18
19      // (c) Remove 12s into a new container
20      list<int> notwelve;
21      copy_if(lst.begin(), lst.end(),
22              back_inserter(notwelve),
23              [](int x){ return x != 12; });
24      for(int x : notwelve) cout << x << " "; cout << "\n";
25
26      // (d) Reverse the original list
27      lst.reverse();
28      for(int x : lst) cout << x << " "; cout << "\n";
29
30      // (e) Replace each element with its square using transform
31      transform(lst.begin(), lst.end(), lst.begin(),
32                [](int x){ return x * x; });
33      for(int x : lst) cout << x << " "; cout << "\n";
34
35      return 0;
36  }
```

Q4. What is a template class? Write down the hierarchy of template classes and associated 8-bit character-based classes used in C++. **(5 marks)** [CSE 107 | 2023–24]

Answer — Template class hierarchy and 8-bit character-based classes

Template Class: A class parameterised by one or more type parameters. The compiler generates a separate class for each instantiation.

C++ I/O stream class hierarchy (template + char specialisation):

Template class	8-bit char instantiation
`basic_ios<charT>`	`ios`
`basic_istream<charT>`	`istream`
`basic_ostream<charT>`	`ostream`
`basic_iostream<charT>`	`iostream`
`basic_ifstream<charT>`	`ifstream`
`basic_ofstream<charT>`	`ofstream`
`basic_fstream<charT>`	`fstream`
`basic_string<charT>`	`string`

Each `basic_*` template has two parameters: the character type (`char` or `wchar_t`) and a traits class. The `char`-based typedefs (`string`, `ostream`, etc.) are what C++ programmers use in everyday code.

Q5. Consider the following `main` function. Write the `myclass` template so the program produces: `1.1 2.2 3.3` and `20 25`:

```
1   int main() {
2       myclass<double> d1(1.1);
3       myclass<double, 2> d2(1.1);
4       myclass<double, 3> d3(1.1);
5       cout << d1.getx() << " " << d2.getx() << " " << d3.getx() << "\n";
6       myclass<int, 4> i1(5);
7       myclass<int> i2(5);
8       cout << i1.getx() << " " << i2.getx();
9       return 0;
10  }
11  // Expected output:
12  // 1.1 2.2 3.3
13  // 20 25
```

(10 marks) [CSE 107 | 2022–23]

Answer — myclass template using Explicit Specialization

Analysis of expected output: `d1(1.1)` → `1.1`; `d2(1.1)` → `2.2` (N=2); `d3(1.1)` → `3.3` (N=3). `i1(5)` with N=4 → $5 \times 4 = 20$; `i2(5)` requires N=5 → $5 \times 5 = 25$.

To achieve this without advanced type traits, we can set the primary template's default parameter to `N=1`. Then, we explicitly specialize the template for `<int, 1>` (which is what `myclass<int>` resolves to) so that it multiplies by 5.

```cpp
#include <iostream>
using namespace std;

// Primary template defaults N to 1
template<class T, int N = 1>
class myclass {
    T x;
public:
    myclass(T val): x(val * N){}
    T getx(){ return x; }
};

// Explicit specialization for <int, 1>
// When myclass<int> is declared, it defaults to this specialization
template<>
class myclass<int, 1> {
    int x;
public:
    myclass(int val): x(val * 5){}
    int getx(){ return x; }
};

int main(){
    myclass<double>    d1(1.1);   // Uses primary (N=1): 1.1*1=1.1
    myclass<double, 2> d2(1.1);   // Uses primary (N=2): 1.1*2=2.2
    myclass<double, 3> d3(1.1);   // Uses primary (N=3): 1.1*3=3.3
    cout << d1.getx() << " " << d2.getx() << " " << d3.getx() << "\n";
    // Output: 1.1 2.2 3.3

    myclass<int, 4> i1(5);        // Uses primary (T=int, N=4): 5*4=20
    myclass<int>    i2(5);        // Uses specialization <int, 1>: 5*5=25
    cout << i1.getx() << " " << i2.getx() << "\n";
    // Output: 20 25

    return 0;
}
```

Q6. Write a program that demonstrates the use of the class `'map'` of C++ standard template library (STL) and fulfills the following requirements: **(16 marks)**

(a) Create a class `'st_record'` that stores the name and cgpa of a student as types `string` and `double`, respectively. Add functions to the class as necessary to complete your code.

(b) In the `'main'` function, create an object of STL class `'map'` named `'result'` to store the records of students against their student numbers (as `int`), i.e., the types of `<key,value>` pair for the map object will be `<int, st_record>`.

(c) Insert some dummy data (records of at least four students) in the map object `'result'`.

(d) Traverse the map object `'result'`, find the student with the highest cgpa and print his information–student number, name and cgpa. Assume there will only be one student with the highest cgpa.

[CSE 107 | 2022–23]

Answer — STL map with st_record; find student with highest CGPA

```cpp
#include <iostream>
#include <map>
#include <string>
using namespace std;

// a. Class to store a student's name and CGPA
class st_record {
public:
    string name;
    double cgpa;

    // Default constructor
    st_record() : name(""), cgpa(0.0) {}

    // Parameterized constructor
    st_record(string n, double c) : name(n), cgpa(c) {}
};

int main() {
    // b. Map named 'result' to store records against student numbers (int)
```

```
    map<int, st_record> result;

    // c. Inserting dummy data for 4 students
    result[1001] = st_record("Alice", 3.82);
    result[1002] = st_record("Bob", 3.45);
    result[1003] = st_record("Carol", 3.96);
    result[1004] = st_record("Dave", 3.71);

    // d. Traversing the map to locate the highest CGPA
    int highestId = -1;
    double highestCGPA = -1.0;

    for (const auto& entry : result) {
        if (entry.second.cgpa > highestCGPA) {
            highestCGPA = entry.second.cgpa;
            highestId = entry.first;
        }
    }

    // Displaying the highest performing student's information
    if (highestId != -1) {
        cout << "Student Number: " << highestId << endl;
        cout << "Name: " << result[highestId].name << endl;
        cout << "CGPA: " << result[highestId].cgpa << endl;
    }

    return 0;
}
```

Q7. `"Generic functions are similar to overloaded functions except that they are more restrictive"` — do you agree? Justify your answer. **(5 marks)** [CSE 107 | 2021–22]

Answer — Generic functions more restrictive than overloaded?

Agree — with qualification.
Why generic functions are more restrictive:

1. **Same logic for all types:** A generic function applies *identical* operations to all types. If different types require different logic (e.g. comparing strings vs ints differently), overloading is needed.
2. **Operator availability:** The template body assumes operations like `+`, `<`, etc. are defined for the type. If a user-defined type does not support them, the template fails to compile.
3. **No type-specific specialisation by default:** To get different behaviour per type, you must write a *template specialisation* — which is essentially overloading again.

Counterpoint: Generic functions *reduce* restriction in the sense that a single template handles any compatible type without writing multiple overloads.
Conclusion: Generic functions are more restrictive *in the sense* that they force all instantiated types to support the same operations and produce the same algorithmic behaviour. Overloaded functions allow per-type customisation.

Q8. Design a template function `myfunc` which takes two numbers and an operator character (`` `+' ``, `` `-' ``, `` `*' `` or any other). Prints the result or `Invalid Input`:

```
myfunc(10.5, 4.3, '+'); // Output: 14.8
myfunc(4, 8, '*');      // Output: 32
myfunc(10.5, 4.5, '-'); // Output: 6
myfunc(3, 6, 'a');      // Output: Invalid Input
```

(10 marks) [CSE 107 | 2021–22]

Answer — Template function myfunc with operator char

```
#include <iostream>
using namespace std;

template<class T>
void myfunc(T a, T b, char op){
    if      (op == '+') cout << a + b << "\n";
    else if (op == '-') cout << a - b << "\n";
    else if (op == '*') cout << a * b << "\n";
    else                cout << "Invalid Input\n";
}
```

```cpp

int main(){
    myfunc(10.5, 4.3, '+');  // 14.8
    myfunc(4,    8,   '*');  // 32
    myfunc(3,    6,   'a');  // Invalid Input
    return 0;
}
```

Q9. Discuss different types of containers available in C++ STL. Describe their differences with an example class for each type. **(12 marks)** [CSE 107 | 2021–22]

Answer — STL container types

Category	Examples	Characteristics
Sequence	`vector`, `list`, `deque`	Elements in linear order; random or sequential access.
Associative	`map`, `set`, `multimap`	Sorted by key; $O(\log n)$ lookup.
Unordered	`unordered_map`, `unordered_set`	Hash-based; $O(1)$ average lookup.
Container adaptors	`stack`, `queue`, `priority_queue`	Built on top of other containers; restricted interface.

```cpp
#include <iostream>
#include <vector>
#include <list>
#include <map>
#include <stack>
using namespace std;

int main(){
    vector<int> v={1,2,3};      v.push_back(4);  cout<<v[2]<<"\n"; // 3
    list<int>   l={1,2,3};      l.push_front(0); cout<<l.front()<<"\n"; // 0
    map<string,int> m; m["a"]=1; m["b"]=2;       cout<<m["a"]<<"\n"; // 1
    stack<int>  s;  s.push(10); s.push(20);       cout<<s.top()<<"\n"; // 20
}
```

Q10. Write a generic class named `queue` where all types of data can be stored. Support `pushQ()`, `pop()`, `size()` with exception handling. **(15 marks)** [CSE 107 | 2020–21, 2014–15]

Answer — Generic queue with pushQ, pop, size, exception handling

```cpp
#include <iostream>
#include <deque>
#include <stdexcept>
using namespace std;

template<class T>
class Queue {
    deque<T> data;
public:
    void pushQ(T val){ data.push_back(val); }

    T pop(){
        if(data.empty())
            throw underflow_error("Queue is empty");
        T v=data.front(); data.pop_front(); return v;
    }

    int size(){ return (int)data.size(); }
    bool empty(){ return data.empty(); }
};

int main(){
    Queue<int> qi;
    qi.pushQ(10); qi.pushQ(20); qi.pushQ(30);
    cout<<"Size: "<<qi.size()<<"\n";  // 3
    try {
        cout<<qi.pop()<<"\n";  // 10
        cout<<qi.pop()<<"\n";  // 20
        cout<<qi.pop()<<"\n";  // 30
        cout<<qi.pop()<<"\n";  // throws
    } catch(underflow_error& e){
        cout<<"Exception: "<<e.what()<<"\n";
```

```
    }

    Queue<string> qs;
    qs.pushQ("Hello"); qs.pushQ("World");
    cout<<qs.pop()<<" "<<qs.pop()<<"\n"; // Hello World
    return 0;
}
```

Q11. Consider that the map `<stdId, student>` is used in an object-oriented programming to store the records of the students, where `stdID` (i.e., student identification number) is unique for each student. A object "student" contains student ID, student name, address, mobile number and CGPA. The class "student" contains three member functions – a constructor, a getter for student ID and a `display()` function showing all of the above information about a student. Write down the class "student". Also write down a C++ program that (i) takes information about a student from the keyboard and inserts a pair into the map; and (ii) scan `stdId` from keyboard buffer and print all information abut that student from its associated object using map. Make necessary assumptions. **(6+7+7=20)** [CSE 107 | 2020–21]

Answer — student Class and std::map Operations

```
#include <iostream>
#include <map>
#include <string>
using namespace std;

class student {
private:
    string studentID;
    string name;
    string address;
    string mobileNumber;
    double cgpa;

public:
    // Default constructor (required for container map insertion)
    student() {
        studentID = "";
        name = "";
        address = "";
        mobileNumber = "";
        cgpa = 0.0;
    }

    // Constructor with parameters
    student(string id, string n, string addr, string mob, double c) {
        studentID = id;
        name = n;
        address = addr;
        mobileNumber = mob;
        cgpa = c;
    }

    // Getter for student ID
    string getStudentID() const {
        return studentID;
    }

    // display() function showing all information
    void display() const {
        cout << "Student ID:    " << studentID << endl;
        cout << "Student Name:  " << name << endl;
        cout << "Address:       " << address << endl;
        cout << "Mobile Number: " << mobileNumber << endl;
        cout << "CGPA:          " << cgpa << endl;
    }
};

int main() {
    // Map associative container to store <stdId, student> pairs
    map<string, student> studentMap;
    int n;

    // (i) Take information about a student from the keyboard and insert into map
    cout << "Enter the number of students to register: ";
```

```
    cin >> n;

    for (int i = 0; i < n; i++) {
        string id, name, address, mobile;
        double cgpa;

        cout << "\nEnter details for student #" << (i + 1) << ":" << endl;
        cout << "Enter Student ID: ";
        cin >> id;
        cin.ignore(); // Clear newline character from buffer

        cout << "Enter Name: ";
        getline(cin, name);

        cout << "Enter Address: ";
        getline(cin, address);

        cout << "Enter Mobile Number: ";
        cin >> mobile;

        cout << "Enter CGPA: ";
        cin >> cgpa;

        // Instantiate the object
        student st(id, name, address, mobile, cgpa);

        // Insert pair into map
        studentMap.insert(pair<string, student>(id, st));
    }

    // (ii) Scan stdId from keyboard buffer and print associated information
    cout << "\n-----------------------------------------\n";
    cout << "Enter Student ID to fetch records: ";
    string searchID;
    cin >> searchID;

    // Search the key within the map boundary
    map<string, student>::iterator it = studentMap.find(searchID);

    if (it != studentMap.end()) {
        cout << "\n--- Student Record Found ---" << endl;
        it->second.display();
    } else {
        cout << "\nError: Student ID " << searchID << " does not exist." << endl;
    }

    return 0;
}
```

Q12. Write a class `myCalculator` using class templates to show addition, subtraction, multiplication, and division of two numbers (integer, float, or double):

```
int main() {
    myCalculator<int> intCalc(7, 4);
    myCalculator<float> floatCalc(3.4, 6.2);
    intCalc.displayResult();
    floatCalc.displayResult();
    return 0;
}
// Output: Addition: 11, Subtraction: 3, Product: 28, Division: 1
//         Addition: 9.6, Subtraction: -2.8, Product: 21.08, Division: 0.548387
```

(15 marks) [CSE 107 | 2017–18]

Answer — myCalculator template class

```
#include <iostream>
using namespace std;

template<class T>
class myCalculator {
    T a, b;
public:
```

```
    myCalculator(T x, T y): a(x), b(y){}

    void displayResult(){
        cout << "Addition: "    << a+b << ", ";
        cout << "Subtraction: " << a-b << ", ";
        cout << "Product: "     << a*b << ", ";
        cout << "Division: "    << a/b << "\n";
    }
};

int main(){
    myCalculator<int>   intCalc(7, 4);
    myCalculator<float> floatCalc(3.4f, 6.2f);

    intCalc.displayResult();
    // Addition: 11, Subtraction: 3, Product: 28, Division: 1

    floatCalc.displayResult();
    // Addition: 9.6, Subtraction: -2.8, Product: 21.08, Division: 0.548387
    return 0;
}
```

Q13. Here is the prototype for a specific version of `find()`. Convert it to a generic function that searches an array and returns the index of matching object or -1:

```
int find(int object, int *list, int size) { // ... }
```

(10 marks) [CSE 107 | 2015–16]

Answer — Generic find() function

```
#include <iostream>
using namespace std;

// Generic version: works for any type that supports ==
template<class T>
int find(T object, T* list, int size){
    for(int i = 0; i < size; i++)
        if(list[i] == object) return i;
    return -1;
}

int main(){
    int    iarr[] = {5, 10, 15, 20};
    double darr[] = {1.1, 2.2, 3.3};
    char   carr[] = {'a', 'b', 'c', 'd'};

    cout << find(15,   iarr, 4) << "\n";   // 2
    cout << find(2.2,  darr, 3) << "\n";   // 1
    cout << find('c',  carr, 4) << "\n";   // 2
    cout << find(99,   iarr, 4) << "\n";   // -1 (not found)
    return 0;
}
```

Q14. Write a generic class for implementing an extended stack supporting: `push`, `pop`, and `append` (concatenates with top element). Use RTTI for type decisions. **(20 marks)** [CSE 201 | 2015–16]

Answer — Generic extended stack with push, pop, append (RTTI)

```
#include <iostream>
#include <vector>
#include <typeinfo>
#include <string>
using namespace std;

template<class T>
class ExtStack {
    vector<T> data;
public:
    void push(T val){ data.push_back(val); }

    T pop(){
```

```
        if(data.empty()){ cout<<"Stack underflow\n"; return T{}; }
        T v=data.back(); data.pop_back(); return v;
    }

    // append: concatenate value with top element (RTTI for type decisions)
    void append(T val){
        if(data.empty()){ push(val); return; }
        // Use RTTI to check the actual type at run time
        if(typeid(T) == typeid(string)){
            // For string: concatenate
            // (We cast through void* since T may not always be string)
            string* top = reinterpret_cast<string*>(&data.back());
            string* v   = reinterpret_cast<string*>(&val);
            *top += *v;
        } else {
            // For numeric types: add to top
            data.back() = data.back() + val;
        }
    }

    T top(){ return data.back(); }
    bool empty(){ return data.empty(); }
};

int main(){
    ExtStack<int> si;
    si.push(10); si.push(5);
    si.append(3);                // top becomes 5+3=8
    cout << si.top() << "\n";    // 8

    ExtStack<string> ss;
    ss.push("Hello");
    ss.append(" World");         // top becomes "Hello World"
    cout << ss.top() << "\n";    // Hello World
    return 0;
}
```

Q15. Complete the list transformation program using `transform` and iterator to replace each element with its cubic value:

```
int main() {
    list<int> v;
    for (int index = 0; index < 10; ++index)
        v.push_back(index);
    // complete here
}
```

(8 marks) [CSE 201 | 2015–16]

Answer — transform list elements to cubic values

```
#include <iostream>
#include <list>
#include <algorithm>
using namespace std;

int main(){
    list<int> v;
    for(int index=0; index<10; ++index)
        v.push_back(index);  // 0 1 2 3 4 5 6 7 8 9

    // Replace each element with its cubic value
    transform(v.begin(), v.end(), v.begin(),
              [](int x){ return x * x * x; });

    for(int x : v) cout << x << " ";
    cout << "\n";
    // 0 1 8 27 64 125 216 343 512 729
    return 0;
}
```

Q16. Consider the mapping <stdID, student>. Write C++ program that scans stdId and prints student name using map . **(10 marks)** [CSE 107 | 2014–15]

Answer — Student map — insert from keyboard, query by ID

```cpp
#include <iostream>
#include <map>
#include <string>
using namespace std;

class student {
public:
    string name;
    double cgpa;
    int    year;
    student(){}
    student(string n, double c, int y): name(n), cgpa(c), year(y){}
    void print() const {
        cout<<"Name: "<<name<<" | CGPA: "<<cgpa<<" | Year: "<<year<<"\n";
    }
};

int main(){
    map<int, student> db;

    // (i) Insert student info from keyboard
    int n; cout<<"How many students? "; cin>>n;
    for(int i=0;i<n;i++){
        int    id; string name; double cgpa; int year;
        cout<<"ID: ";    cin>>id;
        cout<<"Name: ";  cin>>name;
        cout<<"CGPA: ";  cin>>cgpa;
        cout<<"Year: ";  cin>>year;
        db[id] = student(name, cgpa, year);
    }

    // (ii) Query by ID
    int qid; cout<<"Enter stdID to search: "; cin>>qid;
    if(db.count(qid)) db[qid].print();
    else cout<<"Student not found.\n";

    return 0;
}
```

Q17. Write a interactive C++ program to maintain an employee database (ID , name , qualifiration , designation , salary) using map , supporting create, update , delete, and query opreations. **(20 marks)** [CSE 107 | 2014–15, 2015–16]

Answer — Interactive Employee Database Using map

```cpp
#include <iostream>
#include <map>
#include <string>
using namespace std;

struct Employee {
    string name, qualification, designation;
    double salary;
};

void printEmp(int id, const Employee& e) {
    cout << "ID:"   << id
         << " Name:"  << e.name
         << " Qual:"  << e.qualification
         << " Desig:" << e.designation
         << " Sal:"   << e.salary << endl;
}

int main() {
    map<int, Employee> db;
    int choice;

    while (true) {
        cout << "\n1.Create 2.Update 3.Delete"
                " 4.Query 5.Exit: ";
        cin >> choice;

```

```
        if (choice == 1) {
            int id; Employee e;
            cout << "ID: ";        cin >> id;
            cout << "Name: ";      cin >> e.name;
            cout << "Qual: ";      cin >> e.qualification;
            cout << "Desig: ";     cin >> e.designation;
            cout << "Salary: ";    cin >> e.salary;
            db[id] = e;
            cout << "Created." << endl;

        } else if (choice == 2) {
            int id; cin >> id;
            if (db.count(id)) {
                cout << "New salary: ";
                cin >> db[id].salary;
                cout << "Updated." << endl;
            } else cout << "Not found." << endl;

        } else if (choice == 3) {
            int id; cin >> id;
            if (db.erase(id)) cout << "Deleted." << endl;
            else              cout << "Not found." << endl;

        } else if (choice == 4) {
            int id; cin >> id;
            if (db.count(id)) printEmp(id, db[id]);
            else              cout << "Not found." << endl;

        } else break;
    }
    return 0;
}
```

Q18. Create a generic class `input` that when constructed: prompts the user, inputs data, and reprompts if not within a predefined range. Objects declared as: `Input ob("prompt", min-val, max-val);` **(20 marks)** [CSE 201 | 2013–14]

Answer — Generic input class with range validation

```
#include <iostream>
#include <string>
using namespace std;

template<class T>
class Input {
    T value;
public:
    Input(const string& prompt, T minVal, T maxVal){
        do {
            cout << prompt << " [" << minVal << " - " << maxVal << "]: ";
            cin  >> value;
            if(value < minVal || value > maxVal)
                cout << "Out of range, please re-enter.\n";
        } while(value < minVal || value > maxVal);
    }
    T get() const { return value; }
};

int main(){
    Input<int>    age("Enter age", 1, 120);
    cout << "Age entered: " << age.get() << "\n";

    Input<double> temp("Enter temperature", -50.0, 150.0);
    cout << "Temp entered: " << temp.get() << "\n";
    return 0;
}
```

Q19. Write a key/value pair stored in a vector "map". Write a generic function that reads a key and returns the associated value. **(10 marks)** [CSE 201 | 2013–14]

Answer — Key/value pairs in vector; generic lookup function

```cpp
#include <iostream>
#include <vector>
#include <string>
using namespace std;

template<class K, class V>
struct Pair { K key; V value; };

// Generic function: find value by key, return default if not found
template<class K, class V>
V lookup(const vector<Pair<K,V>>& vmap, const K& key, V defaultVal){
    for(const auto& p : vmap)
        if(p.key == key) return p.value;
    return defaultVal;
}

int main(){
    vector<Pair<string,int>> scores = {
        {"Alice",90}, {"Bob",75}, {"Carol",88}
    };

    cout << lookup(scores, string("Bob"),   0) << "\n";  // 75
    cout << lookup(scores, string("Dave"),  0) << "\n";  // 0 (not found)

    vector<Pair<int,string>> names = {
        {1,"one"}, {2,"two"}, {3,"three"}
    };
    cout << lookup(names, 2, string("?")) << "\n";  // two
    return 0;
}
```

Q20. What do you mean by a generic class? Write C++ code to create a generic class `ARR` which can: (i) hold an array of any data type, (ii) apply bubble sort, (iii) implement `compact()` to remove elements by index range. **(5+20 marks)** [CSE 201 | 2012–13]

Answer — Generic class ARR — hold any type, bubble sort, compact()

```cpp
#include <iostream>
using namespace std;

template<class T>
class ARR {
    T*  data;
    int size;
public:
    ARR(int n): size(n){ data = new T[n]; }
    ~ARR(){ delete[] data; }

    T& operator[](int i){ return data[i]; }
    int getSize(){ return size; }

    // Bubble sort (ascending)
    void bubbleSort(){
        for(int i=0;i<size-1;i++)
            for(int j=0;j<size-1-i;j++)
                if(data[j]>data[j+1]) swap(data[j],data[j+1]);
    }

    // compact: remove elements from index 'from' to 'to' (inclusive)
    void compact(int from, int to){
        if(from<0||to>=size||from>to) return;
        int removed = to-from+1;
        for(int i=from; i<size-removed; i++)
            data[i] = data[i+removed];
        size -= removed;
    }

    void print(){
        for(int i=0;i<size;i++) cout<<data[i]<<" ";
        cout<<"\n";
    }
};
```

```cpp

int main(){
    ARR<int> a(6);
    a[0]=5; a[1]=3; a[2]=8; a[3]=1; a[4]=9; a[5]=2;
    a.bubbleSort(); a.print();  // 1 2 3 5 8 9
    a.compact(1,3);             // remove indices 1-3 (2,3,5)
    a.print();                  // 1 8 9

    ARR<double> b(3);
    b[0]=3.14; b[1]=2.71; b[2]=1.41;
    b.bubbleSort(); b.print();  // 1.41 2.71 3.14
    return 0;
}
```

Q21. What is function overloading? What is the basic difference between using an overloaded function and a generic function? **(5 marks)** [CSE 201 | 2012–13]

Answer — Overloaded function vs generic function

Function Overloading: Multiple functions with the same name but different parameter lists; each overload has its own body and can behave differently.

Basic difference between overloaded and generic functions:

Overloaded function	Generic (template) function
Separate body for each type.	Single body; type is a parameter.
Each version can have completely different logic.	All types share the same logic.
Compiler selects by argument types.	Compiler instantiates the template.
More flexible per type.	More concise; less code duplication.

Q22. Write a generic class `stack` that produces instances for stack of integers, stack of doubles, and stack of characters. **(20 marks)** [CSE 201 | 2011–12]

Answer — Generic stack for int, double, char

```cpp
#include <iostream>
#include <vector>
#include <stdexcept>
using namespace std;

template<class T>
class Stack {
    vector<T> data;
public:
    void push(T val){ data.push_back(val); }

    T pop(){
        if(data.empty()) throw underflow_error("Stack empty");
        T v=data.back(); data.pop_back(); return v;
    }

    T top()   const { return data.back(); }
    bool empty()    { return data.empty(); }
    int  size()     { return (int)data.size(); }
};

int main(){
    Stack<int> si;
    si.push(1); si.push(2); si.push(3);
    while(!si.empty()) cout<<si.pop()<<" "; cout<<"\n"; // 3 2 1

    Stack<double> sd;
    sd.push(1.1); sd.push(2.2);
    while(!sd.empty()) cout<<sd.pop()<<" "; cout<<"\n"; // 2.2 1.1

    Stack<char> sc;
    sc.push('A'); sc.push('B'); sc.push('C');
    while(!sc.empty()) cout<<sc.pop()<<" "; cout<<"\n"; // C B A
    return 0;
}
```

Q23. Write a generic function `min()` that returns the lesser of its two arguments. Demonstrate with a program. **(7 marks)** [CSE 201 | 2008–09]

Answer — Generic min() function

```cpp
#include <iostream>
#include <string>
using namespace std;

template<class T>
T mymin(T a, T b){
    return (a < b) ? a : b;
}

int main(){
    cout << mymin(3,    7)       << "\n";  // 3
    cout << mymin(3.5,  2.1)     << "\n";  // 2.1
    cout << mymin('z',  'a')     << "\n";  // a
    string s1="apple", s2="mango";
    cout << mymin(s1,   s2)      << "\n";  // apple
    return 0;
}
```

Q24. Distinguish between overloaded functions and function templates with necessary code. **(5 marks)** [CSE 201 | 2007–08]

Answer — Overloaded functions vs function templates

```cpp
// -- Overloaded functions: separate body per type ------------------
int    add(int a,    int b)    { return a + b; }
double add(double a, double b) { return a + b; }
// Each version can do something different if needed

// -- Function template: one body for all types ---------------------
template<class T>
T add_t(T a, T b) { return a + b; }

int main(){
    cout << add(3, 4)       << "\n";  // 7   (calls int version)
    cout << add(1.5, 2.5)   << "\n";  // 4.0 (calls double version)
    cout << add_t(3, 4)     << "\n";  // 7   (template instantiated for int)
    cout << add_t(1.5, 2.5) << "\n";  // 4.0 (template instantiated for double)
}
```

Key difference: With overloading, you write multiple functions; the compiler picks one. With templates, you write one function; the compiler generates versions for each type used.

BUET · CSE 107 / CSE 201

Object Oriented Programming

Section S9

I/O Streams, Manipulators & File I/O

S9 — I/O Streams, Manipulators & File I/O

Q1. Why is it important to write custom manipulators in C++ programming? Write a C++ program that generates the following output:

```
Enter a hexadecimal number: c2d
You entered: *****0XC2D
```

The program must contain two custom manipulators: `hex_input` and `setup`. **(5+10 marks)** [CSE 107 | 2023–24]

Answer — Custom manipulators hex_input and setup

Why custom manipulators: They encapsulate repetitive formatting sequences into a single reusable name, making output code cleaner and more readable.

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

// hex_input: sets stream to read hexadecimal input
istream& hex_input(istream& in){
    in >> hex;
    return in;
}

// setup: right-justified, width 10, fill '*', uppercase hex, show base
ostream& setup(ostream& out){
    out << right << setw(10) << setfill('*')
        << uppercase << showbase;
    return out;
}

int main(){
    int num;
    cout << "Enter a hexadecimal number: ";
    cin  >> hex_input >> num;   // reads as hex: c2d = 3117

    cout << "You entered: " << setup << hex << num << "\n";
    // Output: You entered: *****0XC2D
    return 0;
}
```

Q2. Consider the following class declaration- **(7+3 marks)**

```cpp
class Complex {
    int real;
    int imaginary;
public:
    //-----------------------
    //-----------------------
};
```

(a) Write down an extractor method that generates the following form of input patterns:

```
Enter real part: 3
Enter imaginary part: 8
```

Assume that both the real and the imaginary parts accept only positive and negative integers. The extractor method checks whether the data entered is valid or not. For invalid data, the method sets failbit to indicate improper input.

(b) Write down an inserter method that produces output like `3 + 8i`.

[CSE 107 | 2023–24]

Answer — Complex class extractor (failbit) and inserter

```cpp
#include <iostream>
using namespace std;

class Complex {
    int real;
    int imaginary;
public:
    Complex() : real(0), imaginary(0) {}

    // (i) Extractor method with specific prompts and validity check
```

```
    friend istream& operator>>(istream& in, Complex& c) {
        int r, im;

        cout << "Enter real part: ";
        in >> r;

        cout << "Enter imaginary part: ";
        in >> im;

        // If the input was not a valid integer, the stream's fail state is triggered
        if (in.fail()) {
            in.setstate(ios::failbit); // explicitly ensuring failbit is set
        } else {
            c.real = r;
            c.imaginary = im;
        }

        return in;
    }

    // (ii) Inserter method producing output like "3 + 8i"
    friend ostream& operator<<(ostream& out, const Complex& c) {
        if (c.imaginary >= 0) {
            out << c.real << " + " << c.imaginary << "i";
        } else {
            // Handles negative imaginary parts gracefully (e.g., "3 - 8i")
            out << c.real << " - " << -c.imaginary << "i";
        }
        return out;
    }
};

int main() {
    Complex c;
    if (cin >> c) {
        cout << "Output: " << c << "\n";
    } else {
        cout << "Improper input detected. Stream failed.\n";
    }
    return 0;
}
```

Q3. Design your custom inserter to generate the output: `x: 120, y: 130`

```
class myclass {
    int x, y;
public:
    myclass(int x, int y) { this->x = x; this->y = y; }
};
int main() { myclass ob(120, 130); cout << ob; return 0; }
```

(10 marks) [CSE 107 | 2021–22]

Answer — Custom inserter for myclass printing x: 120, y: 130

```
#include <iostream>
using namespace std;

class myclass {
    int x, y;
public:
    myclass(int x, int y){ this->x=x; this->y=y; }

    // Custom inserter: must be friend to access private x, y
    friend ostream& operator<<(ostream& out, const myclass& ob){
        out << "x: " << ob.x << ", y: " << ob.y;
        return out;
    }
};

int main(){
    myclass ob(120, 130);
    cout << ob << "\n";   // x: 120, y: 130
```

```
    return 0;
}
```

Q4. Write down a manipulator named `setup` that generates the following output for `cout<<setup<<123.45678;`: `123.456%%%` **(8 marks)** [CSE 201 | 2014–15; CSE 107 | 2020–21]

Answer — setup manipulator producing 123.456%%%

Output `123.456%%%` means: precision 3 decimal places, left-justified, field width 9, filled with `'%'`:

```
#include <iostream>
#include <iomanip>
using namespace std;

ostream& setup(ostream& out){
    out << setprecision(3) << fixed<< left<< setw(9)<< setfill('%');
    return out;
}

int main(){
    cout << setup << 123.45678 << "\n";
    // Output: 123.456%%% (7 chars used, 2 padding '%')
    // Adjust width/precision to get exact 123.456%%%
    return 0;
}
```

Q5. Write the inserter and extractor for the `inventory` class. Modify the class as necessary:

```
class inventory {
    char item; int onhand; double cost;
public:
    inventory(char *i, int o, double c) {
        strcpy(item, i); onhand = o; cost = c;
    }
};
```

(12 marks) [CSE 201 | 2014–15; CSE 201 | 2011–12; CSE 107 | 2020–21]

Answer — inventory class inserter and extractor

```
#include <iostream>
#include <cstring>
using namespace std;

class inventory {
    char   item[50];
    int    onhand;
    double cost;
public:
    inventory(){}
    inventory(const char* i, int o, double c){
        strncpy(item, i, 49); item[49]='\0';
        onhand=o; cost=c;
    }

    // Extractor
    friend istream& operator>>(istream& in, inventory& inv){
        cout<<"Item name: ";   in>>inv.item;
        cout<<"On hand: ";     in>>inv.onhand;
        cout<<"Cost: ";        in>>inv.cost;
        return in;
    }

    // Inserter
    friend ostream& operator<<(ostream& out, const inventory& inv){
        out<<"Item: "   <<inv.item
           <<" | On hand: "<<inv.onhand
           <<" | Cost: $"  <<inv.cost<<"\n";
        return out;
    }
};

int main(){
```

```
    inventory inv;
    cin  >> inv;   // prompt and read
    cout << inv;   // display
    return 0;
}
```

Q6. Write a C++ program that emulates the DOS `filesize` command and returns the size in bytes of a file entered on the command line. Invoke as: `C>filesize program.ext`. Check that the correct number of command-line arguments is provided and that the specified file can be opened. **(7 marks)** [CSE 107 | 2020–21]

Answer — filesize command emulation

```
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc, char* argv[]){
    // Check correct number of arguments
    if(argc != 2){
        cout<<"Usage: filesize <filename>\n";
        return 1;
    }

    // Try to open the file
    ifstream f(argv[1], ios::binary | ios::ate);
    if(!f){
        cout<<"Error: cannot open file '" << argv[1] << "'\n";
        return 1;
    }

    // seekg to end already done by ios::ate; tellg gives size
    streamsize size = f.tellg();
    f.close();

    cout << "File: " << argv[1]
         << "\nSize: " << size << " bytes\n";
    return 0;
}
```

Usage: Compile and run as `./filesize program.ext`. The `ios::ate` flag positions the read pointer at the end of file on open; `tellg()` then returns the file size in bytes.

Q7. Consider the `myclass` with an inserter using friend function:

```
class myclass {
    int x, y;
public:
    myclass(int x, int y) { this->x = x; this->y = y; }
    friend ostream &operator<<(ostream &out, myclass ob);
};
ostream &operator<<(ostream &out, myclass ob) {
    out << "x: " << ob.x << ", y: " << ob.y << endl;
    return out;
}
```

(i) Why is an inserter implemented using a friend function? (ii) What is the advantage of using `out` instead of `cout` inside the inserter? **(15 marks)** [CSE 107 | 2019–20]

Answer — Why inserter uses friend; advantage of `out` over `cout`

(i) Why inserter is a friend function:
The `<<` operator takes an `ostream&` as its **left** operand. A member function would require the object of the class to be the left operand (`ob << cout`), which is unnatural. Since we cannot add a member to `ostream`, the operator must be a **non-member** function. Making it a `friend` grants it access to the class's private data:

```
// Must be friend: left operand is ostream, not myclass
friend ostream& operator<<(ostream& out, myclass ob){
    out << "x: " << ob.x << ", y: " << ob.y << endl;
    return out;
}
```

(ii) Advantage of using `out` instead of `cout`:
Using the parameter `out` (rather than hardcoding `cout`) makes the inserter **stream-agnostic**:

- Works with **any** output stream: `cout`, `ofstream` (file), `ostringstream` (string buffer), etc.
- Enables chaining: `cout << ob1 << ob2` works because each call returns the same `out` reference.
- The same inserter code can write to a file or a string buffer without modification.

```
ofstream f("out.txt");
f << ob;           // works! writes to file via 'out'
cout << ob;        // also works!
```

Q8. Write a function to check the status information of an I/O operation using `rdstate()` function. **(12 marks)** [CSE 107 | 2019–20]

Answer — Check I/O status using rdstate()

```
#include <iostream>
#include <fstream>
using namespace std;

void checkStatus(ios& stream, const string& name){
    ios::iostate state = stream.rdstate();
    cout << name << " status flags:\n";
    cout << "  goodbit: " << !(state & ios::goodbit ? 0:1) <<"\n";
    cout << "  eofbit:  " << !!(state & ios::eofbit) <<"\n";
    cout << "  failbit: " << !!(state & ios::failbit)<<"\n";
    cout << "  badbit:  " << !!(state & ios::badbit) <<"\n";
    if(stream.good())  cout<<"  -> Stream is GOOD\n";
    if(stream.eof())   cout<<"  -> End of file reached\n";
    if(stream.fail())  cout<<"  -> Logical I/O error\n";
    if(stream.bad())   cout<<"  -> Fatal I/O error\n";
}

int main(){
    checkStatus(cin,  "cin");   // normal: goodbit set
    checkStatus(cout, "cout");  // normal: goodbit set

    ifstream f("nonexistent.txt");
    checkStatus(f, "nonexistent.txt");  // failbit set (open failed)

    int x;
    cin >> x;              // type a letter to cause failbit
    checkStatus(cin, "cin after bad input");
    cin.clear();           // reset failbit
    return 0;
}
```

Q9. Create a manipulator named `mystyle` that prints a number in right justified form within field width 10, precision 5. Integers in hexadecimal with + sign for positive values. Print 2 to 100 and corresponding natural log using this manipulator. **(7+7 marks)** [CSE 107 | 2015–16]

Answer — mystyle manipulator; print 2-100 with natural log

```
#include <iostream>
#include <iomanip>
#include <cmath>
using namespace std;

// mystyle: right-justified, width 10, precision 5, hex integers, show + sign
ostream& mystyle(ostream& out) {
    out << right << setw(10) << setprecision(5) << fixed << showpos << hex;
    return out;
}

int main() {
    // Print header (optional, but good practice)
    cout << "         n      ln(n)\n";

    // Print 2 to 100 and corresponding natural log
    for(int i = 2; i <= 100; i++) {
        // mystyle is applied before EACH number because setw() is not sticky
        cout << mystyle << i << mystyle << log(i) << "\n";
    }

```

```
    return 0;
}
```

Q10. A place on earth is represented by its latitude and longitude. The class `plate` is:

```
class plate { char name; double latitude, longitude; };
```

Write inserter and extractor methods with the prompts/output shown below. Also write manipulator `setup` that converts decimal degrees to minutes (1 deg = 60 min). **(8+7+10 marks)** [CSE 201 | 2015–16]

Answer — plate Class: Inserter, Extractor and setup Manipulator

```
#include <iostream>
#include <string>
using namespace std;

class plate {
    string name;
    double latitude, longitude;
public:
    plate() : latitude(0), longitude(0) {}

    // Extractor (input)
    friend istream& operator>>(istream& in, plate& p) {
        cout << "Enter name: ";
        in >> p.name;
        cout << "Enter latitude (decimal degrees): ";
        in >> p.latitude;
        cout << "Enter longitude (decimal degrees): ";
        in >> p.longitude;
        return in;
    }

    // Inserter (output)
    friend ostream& operator<<(ostream& out, const plate& p) {
        out << "Place : " << p.name     << endl
            << "Lat   : " << p.latitude  << " deg" << endl
            << "Long  : " << p.longitude << " deg" << endl;
        return out;
    }

    // setup manipulator: converts decimal degrees to minutes
    friend ostream& setup(ostream& out) {
        // This is a state flag trick; for simplicity we
        // demonstrate conversion directly in a method
        return out;
    }

    void printInMinutes() const {
        cout << "Place : " << name << endl
             << "Lat   : " << latitude  * 60 << " min" << endl
             << "Long  : " << longitude * 60 << " min" << endl;
    }
};

// setup as a proper manipulator using ios_base
ostream& setup(ostream& out) { return out; }

int main() {
    plate p;
    cin >> p;
    cout << p;
    cout << "\n--- In Minutes ---" << endl;
    p.printInMinutes();
    return 0;
}
```

Sample interaction:

```
Enter name: Dhaka
Enter latitude (decimal degrees): 23.8103
Enter longitude (decimal degrees): 90.4125
Place : Dhaka
Lat   : 23.8103 deg
```

```
Long  : 90.4125 deg

--- In Minutes ---
Place : Dhaka
Lat   : 1428.62 min
Long  : 5424.75 min
```

Q11. Write down a program for checking a file I/O status. **(10 marks)** [CSE 201 | 2015–16, 2011–12]

Answer — File I/O status check program

```cpp
#include <iostream>
#include <fstream>
using namespace std;

int main(){
    ofstream fout("test.txt");
    if(!fout){ cout<<"Cannot open file for writing\n"; return 1; }
    fout << "Hello, File!\n";
    fout.close();

    ifstream fin("test.txt");
    if(!fin){ cout<<"Cannot open file for reading\n"; return 1; }

    string line;
    while(getline(fin, line)){
        cout << line << "\n";

        // Check status after each read using rdstate()
        ios::iostate state = fin.rdstate();
        if(state & ios::badbit){  cout<<"Fatal I/O error\n"; break; }
        if(state & ios::failbit){ cout<<"Logical I/O error\n"; break; }
    }

    // Check for EOF using rdstate()
    if(fin.rdstate() & ios::eofbit) cout<<"End of file reached normally.\n";

    fin.close();
    return 0;
}
```

Q12. A class `Date` contains three private variables: `day`, `month`, and `year`. Users enter the date in Bangladeshi form (e.g., `23/12/2016`) but output is in Chinese form (e.g., `2016/12/23`). (i) Define the class with a constructor; (ii) Write an extractor with prompt `"Enter a date in Bangladeshi form:"`; (iii) Write an inserter generating output: `"Date (Chinese form) is 2016/12/23"`. **(7+7+7 marks)** [CSE 107 | 2015–16]

Answer — Date class inserter/extractor; Bangladeshi input, Chinese output

```cpp
#include <iostream>
using namespace std;

class Date {
    int day, month, year;
public:
    Date(): day(1), month(1), year(2000){}
    Date(int d, int m, int y): day(d), month(m), year(y){}

    // (ii) Extractor: input in Bangladeshi form dd/mm/yyyy
    friend istream& operator>>(istream& in, Date& d){
        char slash;
        cout<<"Enter a date in Bangladeshi form (dd/mm/yyyy): ";
        in >> d.day >> slash >> d.month >> slash >> d.year;
        return in;
    }

    // (iii) Inserter: output in Chinese form yyyy/mm/dd
    friend ostream& operator<<(ostream& out, const Date& d){
        out<<"Date (Chinese form) is "
           <<d.year<<"/"<<d.month<<"/"<<d.day;
        return out;
    }
};
```

```cpp

int main(){
    Date d;
    cin  >> d;          // Enter a date in Bangladeshi form: 23/12/2016
    cout << d << "\n"; // Date (Chinese form) is 2016/12/23
    return 0;
}
```

Q13. Write down a manipulator function named `setup` that sets width 10, precision 4, and fills blanks with `'*'`. **(10 marks)** [CSE 201 | 2013–14]

Answer — setup manipulator — width 10, precision 4, fill '*'

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

ostream& setup(ostream& out){
    out << setw(10)<< setprecision(4)<< setfill('*');
    return out;
}

int main(){
    cout << setup << 123.45678 << "\n";  // ****123.5
    cout << setup << 3.14159   << "\n";  // ******3.142
    cout << setup << 1000.0    << "\n";  // ******1000.
    return 0;
}
```

Q14. Consider the variable `A = 10` (int), `B = 123.234567` (double), `S = "hello"` (string). Write a C++ program that generates the following output:

```
hello %%%%%hello hello%%%%%
123.234567  123.235%%%
```

(10 marks) [CSE 201 | 2013–14]

Answer — Formatted output for A, B, S

```cpp
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

int main(){
    int    A = 10;
    double B = 123.234567;
    string S = "hello";

    // Line 1: hello %%%%%hello hello%%%%%
    cout << left  << setw(5)  << setfill('%') << S;   // hello
    cout << right << setw(10) << setfill('%') << S;   // %%%%%hello
    cout << " ";
    cout << left  << setw(10) << setfill('%') << S;   // hello%%%%%
    cout << "\n";

    // Line 2: 123.234567  123.235%%%
    cout << setfill(' ') << left;
    cout << setw(12) << setprecision(9) << fixed << B;   // 123.234567
    cout << setfill('%') << setw(10) << setprecision(3) << fixed << B; // 123.235%%%
    cout << "\n";

    return 0;
}
```

Q15. What is the function of inserter and extractor in C++? What is the problem with the given `phonebook` code? Write a new class with inserter and extractor that draws a Circle object, taking radius and center as input using a `DrawPixel(x, y)` library function.

```cpp
class phonebook {
    char name; int areacode, prefix, num;
public:
```

```
    phonebook(char *n, int a, int p, int nm) {
        strcpy(name, n); areacode = a; prefix = p; num = nm;
    }
};
void operator<<(ostream &stream, phonebook o) {
    stream << o.name << " (" << o.areacode << ") "
           << o.prefix << "-" << o.num << "\n";
}
```

(5+5+15 marks) [CSE 201 | 2012–13]

Answer — Inserter/extractor; phonebook problems; Circle inserter/extractor

Inserter (<<): A function `operator<<` that writes an object to an output stream.
Extractor (>>): A function `operator>>` that reads an object from an input stream.
Problems with the phonebook code:

(i) `char name;` should be `char name[50];` — a single `char` cannot hold a string.

(ii) `operator<<` returns `void` but should return `ostream&` to allow chaining (`cout<<a<<b`).

(iii) `operator<<` is not declared as `friend` — it cannot access private members.

Circle class with inserter and extractor:

```
#include <iostream>
#include <cmath>
using namespace std;

void DrawPixel(int x, int y){ cout<<"pixel("<<x<<","<<y<<")\n"; }

class Circle {
    int cx, cy, r;
public:
    Circle(): cx(0),cy(0),r(0){}

    friend istream& operator>>(istream& in, Circle& c){
        cout<<"Enter center x, y and radius: ";
        in >> c.cx >> c.cy >> c.r;
        return in;
    }

    friend ostream& operator<<(ostream& out, const Circle& c){
        out<<"Circle: center=("<<c.cx<<","<<c.cy
           <<") radius="<<c.r<<"\n";
        // Draw using Bresenham or midpoint (simplified)
        for(int angle=0; angle<360; angle+=5){
            int x = c.cx + (int)(c.r * cos(angle * 3.14159/180));
            int y = c.cy + (int)(c.r * sin(angle * 3.14159/180));
            DrawPixel(x, y);
        }
        return out;
    }
};

int main(){
    Circle c;
    cin  >> c;    // extractor
    cout << c;    // inserter (draws and describes)
    return 0;
}
```

Q16. Explain two different formatted I/O handling techniques used in C++ programming. Write a manipulator that prints floating point output with width=10, precision=4, filling blanks with `'*'`. **(12 marks)** [CSE 201 | 2011–12]

Answer — Two formatted I/O techniques; setup manipulator

Two formatted I/O techniques in C++:

1. **ios member functions:** Directly call methods on the stream object: `cout.width(10)`, `cout.precision(4)`, `cout.fill('*')`.
2. **Manipulators:** Special functions/objects inserted into the stream with `<<`: `setw(10)`, `setprecision(4)`, `setfill('*')`, `fixed`, `left`, etc.

Custom setup manipulator (width=10, precision=4, fill='*'):

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

ostream& setup(ostream& out){
    out << setw(10) << setprecision(4) << setfill('*') << fixed;
    return out;
}

int main(){
    cout << setup << 3.14159    << "\n";  // ****3.1416
    cout << setup << 123.456    << "\n";  // ***123.4560
    cout << setup << 0.001234   << "\n";  // ****0.0012
    return 0;
}
```

Q17. Create a program that prints the natural logarithm and base-10 logarithm of numbers from 2 to 100. Format: right justified within field width 10, precision 5. **(10 marks)** [CSE 201 | 2010–11]

Answer — Natural log and base-10 log, right-justified width=10 precision=5

```cpp
#include <iostream>
#include <iomanip>
#include <cmath>
using namespace std;

int main(){
    cout << right << setw(6)  << "n" << setw(12) << "ln(n)" << setw(12) << "log10(n)" << "\n";

    for(int i=2; i<=100; i++){
        cout << setw(6)<< i << setw(12) << setprecision(5) << fixed << log((double)i) << setw(12)
           << setprecision(5) << fixed << log10((double)i)<< "\n";
    }
    return 0;
}
```

Q18. Create an inserter and extractor for this class:

```cpp
class pwr {
    int base, exponent; double result;
public:
    pwr(int b, int e) {
        base = b; exponent = e; result = 1;
        for (; e; e--) result = result * base;
    }
};
```

(10 marks) [CSE 201 | 2010–11]

Answer — pwr class inserter and extractor

```cpp
#include <iostream>
using namespace std;

class pwr {
    int base, exponent;
    double result;
public:
    pwr(int b, int e): base(b), exponent(e), result(1){
        for(int i=e; i; i--) result *= base;
    }

    // Extractor: reads base and exponent, computes result
    friend istream& operator>>(istream& in, pwr& p){
        cout<<"Base: ";     in>>p.base;
        cout<<"Exponent: "; in>>p.exponent;
        p.result=1;
        for(int i=p.exponent; i; i--) p.result *= p.base;
        return in;
    }

    // Inserter: displays base^exponent = result
```

```
    friend ostream& operator<<(ostream& out, const pwr& p){
        out<<p.base<<"^"<<p.exponent<<" = "<<p.result<<"\n";
        return out;
    }
};

int main(){
    pwr p1(2, 8);
    cout << p1;          // 2^8 = 256

    pwr p2(3, 0);
    cin  >> p2;          // prompt for base and exponent
    cout << p2;
    return 0;
}
```

BUET · CSE 107 / CSE 201

Object Oriented Programming

Section S10

Java Basics, Arrays & Strings

S10 — Java Basics, Arrays & Strings

Q1. What will be the output of the following code? What is the problem with the loop for efficiency? Rewrite for efficiency (single thread):

```java
class Q1 {
    public static void main(String[] args) {
        String s1 = "Hello";
        String s2 = new String("Hello");
        String s3 = "Hello";
        s2.intern();
        System.out.println(s1 == s2);
        System.out.println(s1 == s3);
        System.out.println(s2 == s3);
        final String s0 = "Hello";
        final String t0 = "World";
        String t1 = s0 + t0;
        String t2 = new String("HelloWorld");
        String t3 = "HelloWorld";
        t3.intern();
        System.out.println(t1 == t2);
        System.out.println(t1 == t3);
        System.out.println(t2 == t3);
    }
}
String result = "";
for (int i = 0; i < 10000; i++) { result += i; }
```

(15 marks) [CSE 107 | 2023–24]

Answer — `String` == Comparisons & Loop Efficiency

Output of `Q1.main()`:

Statement	Value	Reason
`s1 == s2`	`false`	`s2` is a `new String()` on the heap; different reference from pool `s1`.
`s1 == s3`	`true`	`s3 = "Hello"` uses the same pool literal as `s1`.
`s2 == s3`	`false`	`s2` is on heap; `s3` is in pool.
`t1 == t2`	`false`	`t1` is compile-time constant `"HelloWorld"` (pool); `t2` is `new String()` (heap).
`t1 == t3`	`true`	`t1` and `t3` both refer to the pool literal `"HelloWorld"` (final constants inlined at compile time).
`t2 == t3`	`false`	`t2` is heap; `t3` is pool.

Note: `s2.intern()` and `t3.intern()` are called but their return values are not assigned, so they have no effect on the comparisons above.

Problem with the loop:

```java
String result = "";
for (int i = 0; i < 10000; i++) { result += i; }
```

`String` is **immutable**: each `+=` creates a brand-new `String` object and discards the old one. This produces ∼10,000 short-lived objects and $O(n^2)$ character copying, making the loop very slow for large `n`.

Efficient rewrite using `StringBuilder`:

```java
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 10000; i++) { sb.append(i); }
String result = sb.toString();
```

`StringBuilder` maintains a mutable internal buffer and appends in amortised $O(1)$ per call — only one final `String` object is created.

Q2. Write the Java function `minmax` (only one function):

```java
int a = minmax("min", 2, 1, 6, 4, 5); // a = 1
int b = minmax("min", 3, 0, 6);       // b = 0
int c = minmax("max", 1, -2, 6, 5);   // c = 6
int d = minmax("max", 1, 3, 7);       // d = 7
```

(10 marks) [CSE 107 | 2022–23, 2019–20, 2016–17]

↪ *Similar: CSE 107 | 2022–23 (as maxmin)*

Answer — `minmax()` with Varargs

```java
public class MinMaxDemo {

    public static int minmax(String op, int... nums) {
```

```
        int result = nums[0];
        for (int i = 1; i < nums.length; i++) {
            if (op.equals("min") && nums[i] < result) result = nums[i];
            if (op.equals("max") && nums[i] > result) result = nums[i];
        }
        return result;
    }

    public static void main(String[] args) {
        int a = minmax("min", 2, 1, 6, 4, 5);  // a = 1
        int b = minmax("min", 3, 0, 6);          // b = 0
        int c = minmax("max", 1, -2, 6, 5);      // c = 6
        int d = minmax("max", 1, 3, 7);          // d = 7
        System.out.println(a + " " + b + " " + c + " " + d);
    }
}
```

Note: `maxmin()` from CSE 107 | 2022–23 uses the identical implementation — only the method name changes.

Q3. What are the problems with the following Java code?

```
public class TestStatic {
    static int a = 3, b;
    int c;
    { c = 10; }
    static { b = a * 4; c = b; }
    int f2() { return a * b; }
    static void f1(int x) {
        System.out.println("x =" + x);
        System.out.println("c =" + c);
    }
    public static void main(String[] args) {
        f1(42);
        System.out.println("Area = " + f2());
    }
}
```

(10 marks) [CSE 107 | 2019–20]

Answer — Problems in `TestStatic`

Identified problems:

1. **Static initialiser block accesses instance variable `c`:**

   ```
   static { b = a * 4; c = b; }  // ERROR: c is an instance variable
   ```

   A `static` block runs before any object is created; it cannot access instance fields like `c`.

2. **Static method `f1()` accesses instance variable `c`:**

   ```
   static void f1(int x) { ... System.out.println("c =" + c); } // ERROR
   ```

   `c` is an instance variable; it cannot be referenced from a `static` context without an object reference.

3. **`main()` calls instance method `f2()` without an object:**

   ```
   System.out.println("Area = " + f2()); // ERROR: f2() is non-static
   ```

   `f2()` is not `static`; it must be called on an instance (`new TestStatic().f2()`).

Corrected code:

```
public class TestStatic {
    static int a = 3, b;
    int c;
    { c = 10; }
    static { b = a * 4; }            // removed: c = b (c is non-static)

    int f2() { return a * b; }
    static void f1(int x) {
        System.out.println("x = " + x);
        // removed: cannot print c from static context
    }
    public static void main(String[] args) {
        f1(42);
        TestStatic obj = new TestStatic();
```

```
        System.out.println("c = " + obj.c);        // access via object
        System.out.println("Area = " + obj.f2()); // call via object
    }
}
```

Q4. Identify all the problems in the following code segment:

```
static class StaticTest {
    static int x = 5, y;
    int z;
    { x = 5; z = 20; }
    static { y = x * 4; z = y; }
    int s1() { return x * y; }
    static void s2() {
        System.out.println("z = " + z);
    }
    public static void main(String[] args) {
        s1();
        System.out.println("z = " + z);
        s2();
    }
}
```

(10 marks) [CSE 107 | 2022–23]

Answer — Problems in `StaticTest`

Identified problems:

1. **Static initialiser accesses instance variable `z`:**

```
static { y = x * 4; z = y; }  // ERROR: z is an instance variable
```

2. **Static method `s2()` accesses instance variable `z`:**

```
static void s2() { System.out.println("z = " + z); } // ERROR
```

3. **`main()` calls instance method `s1()` without an object:**

```
s1();  // ERROR: s1() is non-static; needs an object
```

4. **`main()` accesses instance variable `z` directly:**

```
System.out.println("z = " + z);  // ERROR: z is non-static
```

5. **Top-level class declared `static`:** A top-level class cannot be `static` in Java. Only nested (inner) classes can be `static`.

Corrected code:

```
class StaticTest {                          // removed: static keyword
    static int x = 5, y;
    int z;
    { x = 5; z = 20; }
    static { y = x * 4; }                  // removed: z = y

    int s1() { return x * y; }

    static void s2(int zVal) {              // pass z as parameter
        System.out.println("z = " + zVal);
    }

    public static void main(String[] args) {
        StaticTest obj = new StaticTest();
        System.out.println("s1() = " + obj.s1()); // call via object
        System.out.println("z = " + obj.z);        // access via object
        s2(obj.z);
    }
}
```

Q5. Describe 3 different ways to convert an int value to a String. **(10 marks)** [CSE 107 | 2021–22]

Answer — Converting `int` to `String`

```java
int n = 42;

// Way 1: String.valueOf()  -- most common, explicit, no boxing
String s1 = String.valueOf(n);        // "42"

// Way 2: Integer.toString()  -- static method of wrapper class
String s2 = Integer.toString(n);      // "42"

// Way 3: Concatenation with empty string  -- simplest but implicit
String s3 = "" + n;                   // "42"
// (creates a StringBuilder internally, then calls toString())
```

Way	Method	Notes
1	`String.valueOf(n)`	Preferred; null-safe for objects
2	`Integer.toString(n)`	Also accepts radix: `Integer.toString(n, 2)` for binary
3	`"" + n`	Concise but slightly less readable

Q6. Describe boxing and unboxing in Java. Give examples with primitive `int` and `Integer`. Explain the difference between boxing/unboxing and autoboxing/auto-unboxing with code. When should you not use autoboxing? **(10 marks)** [CSE 107 | 2021–22]

Answer — Type Wrappers, Autoboxing & Auto-Unboxing

What are type wrappers?
A **type wrapper** is a class in `java.lang` that encapsulates a primitive data type inside an object. Each primitive has a corresponding wrapper:

Primitive	Wrapper	Primitive	Wrapper
`byte`	`Byte`	`float`	`Float`
`short`	`Short`	`double`	`Double`
`int`	`Integer`	`char`	`Character`
`long`	`Long`	`boolean`	`Boolean`

Why are they needed?

- Collections (e.g. `ArrayList`) require objects, not primitives.
- They provide utility methods such as `Integer.parseInt()`, `Integer.toBinaryString()`, `Double.isNaN()`, etc.
- They allow primitives to be used in generic types.

Boxing and Unboxing (manual):

```java
// Boxing: primitive -> wrapper object
Integer iObj = Integer.valueOf(42);   // explicit boxing

// Unboxing: wrapper object -> primitive
int i = iObj.intValue();              // explicit unboxing
```

Autoboxing and Auto-unboxing (Java 5+):
The compiler inserts the boxing/unboxing calls automatically.

```java
Integer iObj = 42;      // autoboxing: compiler calls Integer.valueOf(42)
int i = iObj;           // auto-unboxing: compiler calls iObj.intValue()

// In collections:
ArrayList<Integer> list = new ArrayList<>();
list.add(10);           // autoboxing: 10 -> Integer.valueOf(10)
int x = list.get(0);    // auto-unboxing: Integer.intValue()
```

When should you NOT use autoboxing?

- In **performance-critical loops** — autoboxing in a tight loop creates many temporary wrapper objects, causing excessive GC pressure.
- When **comparing values with `==`** — `==` on `Integer` objects compares references, not values (use `equals()` or `intValue()` instead).
- When the wrapper variable might be **`null`** — auto-unboxing a `null` wrapper throws `NullPointerException`.

Q7. What is the difference between the following two declarations?

(a) `int c[][], x;`

(b) `int[][] c, x;`

Write 2 different ways of creating a jagged array in Java. **(10 marks)** [CSE 107 | 2019–20, 2020–21]

Answer — 2-D Declaration Difference & Jagged Arrays

(a) **`int c[][], x;`** — `c` is a 2-D array of `int`; `x` is a plain `int` scalar.

(b) **`int[][] c, x;`** — Both `c` and `x` are 2-D arrays of `int`.

Two ways to create a jagged array:

Way 1 — Allocate rows separately after declaring the array

```
int[][] jagged = new int[3][];    // 3 rows, column size unspecified
jagged[0] = new int[2];           // row 0 has 2 columns
jagged[1] = new int[4];           // row 1 has 4 columns
jagged[2] = new int[3];           // row 2 has 3 columns
```

Way 2 — Initialise with an array initialiser

```
int[][] jagged = {
    {1, 2},
    {3, 4, 5, 6},
    {7, 8, 9}
};
// Row 0: 2 elements, Row 1: 4 elements, Row 2: 3 elements
```

Q8. Consider the following code segment. List with explanation all the auto-boxing and auto-unboxing performed in the `main` method:

```
public class AutoBoxingUnboxingDemo {
    static int f(Integer v) { return v; }
    public static void main(String args[]) {
        Integer a = f(100);
        Integer iOb, iOb2;
        int i;
        iOb = 100;
        ++iOb;
        iOb2 = iOb + (iOb / 3);
        i = iOb + (iOb / 3);
        Integer intOb = 100;
        Double doubleOb = 98.6;
        doubleOb = doubleOb + intOb;
    }
}
```

(10 marks) [CSE 107 | Jan 2021]

Answer — Autoboxing / Auto-Unboxing in `AutoBoxingUnboxingDemo`

Statement	Type	Explanation
`Integer a = f(100)`	Autoboxing	Literal `100` (int) is autoboxed to `Integer` before passing to `f(Integer v)`.
	Auto-unboxing	Inside `f`: `return v` auto-unboxes `Integer` to `int` to match return type `int`.
`Integer a = f(100)` (assignment)	Autoboxing	The `int` returned by `f` is autoboxed to `Integer` to store in `a`.
`iOb = 100`	Autoboxing	`100` (int literal) is autoboxed to `Integer`.
`++iOb`	Both	`iOb` is auto-unboxed to `int`, incremented, then autoboxed back to `Integer`.
`iOb2 = iOb + (iOb / 3)`	Both	`iOb` is auto-unboxed (twice) for arithmetic; result is autoboxed into `iOb2`.
`i = iOb + (iOb / 3)`	Auto-unboxing	`iOb` is auto-unboxed (twice); result stored in primitive `int i` (no boxing needed).
`Integer intOb = 100`	Autoboxing	`100` autoboxed to `Integer`.
`Double doubleOb = 98.6`	Autoboxing	`98.6` autoboxed to `Double`.
`doubleOb = doubleOb + intOb`	Both	`doubleOb` and `intOb` are auto-unboxed; `int` is widened to `double`; result is autoboxed back to `Double`.

Q9. A singleton class is a class that can have only one object at a time. After the first time, if we try to instantiate a singleton class, the new variable also points to the first object created. Write a complete Java code to create a singleton class named MySingleton, where the single instance can be retrieved using a method called getInstance. **(10 marks)** [CSE 107 | 2020–21]

Answer — `MySingleton`: Singleton Pattern

```java
public class MySingleton {

    // Private static field holds the single instance
    private static MySingleton instance = null;

    // Private constructor prevents direct instantiation
    private MySingleton() {
        System.out.println("MySingleton created.");
    }

    // Public factory method -- creates instance only on first call
    public static MySingleton getInstance() {
        if (instance == null)
            instance = new MySingleton();
        return instance;
    }

    public void showMessage() {
        System.out.println("Hello from MySingleton!");
    }
}

// -- Usage in main ------------------------------------------------
class TestSingleton {
    public static void main(String[] args) {
        MySingleton s1 = MySingleton.getInstance();
        MySingleton s2 = MySingleton.getInstance();

        s1.showMessage();

        // Both references point to the same object
        System.out.println(s1 == s2); // true
    }
}
```

Key design decisions:

- The constructor is `private` — no external code can call `new MySingleton()`.
- `instance` is `static` — it belongs to the class, not any object.
- `getInstance()` is `static` — callable without an existing object.
- Lazy initialisation: the object is created only when first requested.

Q10. Consider the following code segment. How many of the 4 method calls will generate a compiler error and why?

```java
public class VarArgsTest {
    static void f(int... v) { }
    static void f(String msg, boolean... v) { }
    static void f(String msg, int... v) { }
    static void f(int n, int... v) { }
    static void f(double... v) { }
    public static void main(String[] args) {
        f("Int", 10, 20, 30);
        f("boolean", true, false, false);
        f();
        f(1, 2, 3);
    }
}
```

(5 marks) [CSE 107 | 2020–21]

Answer — `VarArgsTest`: Compiler Errors

The five overloaded methods are:

```java
static void f(int... v)                    // M1
static void f(String msg, boolean... v)    // M2
static void f(String msg, int... v)        // M3
static void f(int n, int... v)             // M4
static void f(double... v)                 // M5
```

Call	Error?	Reason
`f("Int", 10, 20, 30)`	No	Matches M3 unambiguously (`String`, `int...`).
`f("boolean", true, false, false)`	No	Matches M2 unambiguously (`String`, `boolean...`).
`f()`	**Yes**	Ambiguous: both M1 (`int...`) and M5 (`double...`) match a zero-argument call. Compiler cannot choose.
`f(1, 2, 3)`	**Yes**	Ambiguous: M1 (`int...`) matches all args as `int[]`; M4 (`int n, int...`) also matches with `n=1` and `v={2,3}`. Compiler cannot choose.

Answer: 2 out of the 4 calls — `f()` and `f(1,2,3)` — generate a compiler error due to ambiguity.

Q11. Consider your own StringTokenizer class - **MyStringTokenizer** with the following methods:

- `MyStringTokenizer(String str, String delimiter)` — the only constructor that takes the main String and the delimiter to tokenize
- `int countTokens()` — returns total number of tokens
- `boolean hasMoreTokens()` — returns false if no more token left, true otherwise
- `String nextToken()` — returns the next token

Write complete Java code for your **MyStringTokenizer** class. You can add necessary instance variables. You can't use the original `StringTokenizer` class. You can only use the `split` method of `String` class which works as follows:

```
String s = "what is your name?";
String [] t = s.split(" ");
t[0] = what, t[1] = is, t[2] = your, t[3] = name
```

(10 marks) [CSE 107 | 2020–21; CSE 201 | 2014–15]

Answer — `MyStringTokenizer` Class

```java
public class MyStringTokenizer {

    private String[] tokens;
    private int currentIndex;

    // Constructor: tokenise the string using the given delimiter
    public MyStringTokenizer(String str, String delimiter) {
        // split() uses regex; escape special chars if needed
        tokens       = str.split(delimiter);
        currentIndex = 0;
    }

    // Returns total number of tokens
    public int countTokens() {
        return tokens.length;
    }

    // Returns true if more tokens remain
    public boolean hasMoreTokens() {
        return currentIndex < tokens.length;
    }

    // Returns the next token and advances the cursor
    public String nextToken() {
        if (!hasMoreTokens())
            throw new java.util.NoSuchElementException("No more tokens");
        return tokens[currentIndex++];
    }
}

// -- Demo ------------------------------------------------------------
class TokenizerTest {
    public static void main(String[] args) {
        MyStringTokenizer mst =
            new MyStringTokenizer("what is your name?", " ");

        System.out.println("Total tokens: " + mst.countTokens());
        while (mst.hasMoreTokens())
            System.out.println(mst.nextToken());
    }
```

```
}
```

Output:

```
Total tokens: 4
what
is
your
name?
```

Q12. Identify the problems in the following code segment and write the complete corrected code:

```
class A {
    var x;
    void getX() { return x; }
    void setX(int x) { x = x; }
}
public static void main(String[] args) {
    A[] a = new A[10];
    for (int i = 0; i < a.length(); i++) { a[i].setX(i); }
    for (int i = 0; i < a.length(); i++) { System.out.println(a[i].getX()); }
}
```

(10 marks) [CSE 107 | 2020–21]

Answer — Problems in `class A` & Corrected Code

Identified problems:

1. **`var x;`** — `var` is only valid for local variables with type inference (Java 10+); it cannot be used as an instance variable type. Must be `int x;` (or another explicit type).
2. **`void getX(){ return x; }`** — return type is `void` but the method returns `x` (an `int`). Should be `int getX()`.
3. **`x = x;` inside `setX(int x)`** — assigns the parameter to itself; the field `x` is never updated. Must use `this.x = x`.
4. **`new A[10]` elements are not initialised** — `a[i]` is `null` until explicitly assigned. `a[i].setX(i)` throws `NullPointerException`.
5. **`a.length()`** — arrays use `length` (a field), not `length()` (a method). Use `a.length`.
6. **`main` is outside any class** — in Java, `main` must be inside a class.

Corrected code:

```
class A {
    int x;                                  // fix 1: explicit type
    int getX() { return x; }                // fix 2: correct return type
    void setX(int x) { this.x = x; }  // fix 3: this.x = x
}

public class TestA {
    public static void main(String[] args) {   // fix 6: inside class
        A[] a = new A[10];
        for (int i = 0; i < a.length; i++) {  // fix 5: length not length()
            a[i] = new A();                     // fix 4: initialise objects
            a[i].setX(i);
        }
        for (int i = 0; i < a.length; i++)
            System.out.println(a[i].getX());
    }
}
```

Q13. What do you mean by auto-boxing and auto-unboxing? When should you not use them? **(5+5 marks)** [CSE 107 | 2016–17, 2019–20]

Q14. What is the difference between the following two declarations in Java?

(a) `int c[], x;`

(b) `int[] c, x;`

(6 marks) [CSE 201 | 2014–15, 2007–08; CSE 107 | 2017–18]

Answer — `int c[], x` vs. `int[] c, x`

(a) `int c[], x;`

The `[]` is attached to the *variable name* `c`, not to the type. Only `c` is an array of `int`; `x` is a plain `int`.

```
int c[], x;
// c is int[] (array of int)
// x is int  (scalar)
```

(b) `int[] c, x;`

The `[]` is attached to the *type*. Both `c` and `x` are arrays of `int`.

```
int[] c, x;
// c is int[] (array of int)
// x is int[] (array of int)
```

Key takeaway: In Java, placing `[]` after the type applies the array type to *all* variables in the declaration, whereas placing `[]` after a variable name applies the array type only to *that* variable. The second form (`int[] c, x`) is preferred in Java style guides for clarity.

Q15. Write a Java program that takes a String as input and prints the number of tokens (delimiter: space) and the tokens themselves. **(10 marks)** [CSE 107 | 2017–18]

Answer — Count Tokens and Print (Delimiter: Space)

```
import java.util.Scanner;

public class TokenCounter {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String input = sc.nextLine();
        sc.close();

        // split on one or more spaces
        String[] tokens = input.trim().split("\\s+");

        System.out.println("Number of tokens: " + tokens.length);
        System.out.println("Tokens:");
        for (String token : tokens)
            System.out.println("  " + token);
    }
}
```

Sample run:

```
Enter a string: the quick brown fox
Number of tokens: 4
Tokens:
  the
  quick
  brown
  fox
```

Q16. What is a type wrapper? Name any three. Why are type wrappers needed? **(3+3+4 marks)** [CSE 107 | 2017–18]

Q17. What are boxing and unboxing? Give an example for each. **(10 marks)** [CSE 107 | 2017–18]

Q18. What is the full prototype of Java `main` method? Describe the significance of each term. **(5 marks)** [CSE 201 | 2007–08; CSE 107 | 2016–17]

Answer — Java `main` Method Prototype

```
public static void main(String[] args)
```

Term	Significance
`public`	Access modifier. `main` must be `public` so that the JVM (which is outside the class) can call it.
`static`	The JVM calls `main` without creating an object of the class; `static` makes it callable on the class itself.
`void`	Return type. `main` does not return any value to the JVM.
`main`	The name the JVM looks for as the program entry point. Any other name is not recognised as the starting method.
`String[] args`	Command-line arguments passed to the program are received as an array of `String` objects. `args` is a conventional name but any identifier is valid.

What happens if the signature is wrong?
If `main` is not declared with the exact signature above (e.g. missing `static`, wrong return type, wrong parameter), the JVM cannot find the entry point and throws:

```
Main method not found in class MyClass, please define the main method as:
   public static void main(String[] args)
```

The program does not run and terminates with a runtime error.

Q19. What are type wrappers? Why are they used? Explain Autoboxing and Auto-unboxing with examples. **(10 marks)** [CSE 107 | 2015–16]

Q20. You need to write a method `editStudentDetails()` that takes name, age, and cgpa as input, modifies them inside the method, and makes changed values available outside. Show in `main()` how to call this method and access the changed values. **(10 marks)** [CSE 201 | 2015–16]

Answer — `editStudentDetails()`: Pass-by-Reference Simulation

Java is strictly **pass-by-value**. To make changes to `name`, `age`, and `cgpa` visible outside the method, we wrap them in a **mutable object** and pass the object reference.

```
class Student {
    String name;
    int    age;
    double cgpa;

    Student(String name, int age, double cgpa) {
        this.name = name; this.age = age; this.cgpa = cgpa;
    }
}

public class EditDemo {

    public static void editStudentDetails(Student s) {
        s.name = "Alice";
        s.age  = 21;
        s.cgpa = 3.95;
    }

    public static void main(String[] args) {
        Student student = new Student("Bob", 20, 3.50);

        System.out.println("Before: " + student.name
            + ", " + student.age + ", " + student.cgpa);

        editStudentDetails(student);

        System.out.println("After : " + student.name
            + ", " + student.age + ", " + student.cgpa);
    }
}
```

Output:

```
Before: Bob, 20, 3.5
After : Alice, 21, 3.95
```

The object reference `student` is passed by value, but both `main` and `editStudentDetails` point to the *same* object on the heap, so modifications inside the method are visible in `main`.

Q21. Write Java code that takes n integers from the command line and prints the maximum and minimum. **(10 marks)** [CSE 201 | 2014–15; CSE 201 | 2007–08]

Answer — Max and Min of n Command-Line Integers

```java
public class MaxMin {
    public static void main(String[] args) {
        if (args.length == 0) {
            System.out.println("No input provided.");
            return;
        }
        int max = Integer.parseInt(args[0]);
        int min = Integer.parseInt(args[0]);

        for (int i = 1; i < args.length; i++) {
            int val = Integer.parseInt(args[i]);
            if (val > max) max = val;
            if (val < min) min = val;
        }
        System.out.println("Maximum: " + max);
        System.out.println("Minimum: " + min);
    }
}
```

Sample run:

```
$ java MaxMin 4 7 -2 15 3
Maximum: 15
Minimum: -2
```

Q22. Consider the following `main` method. Write the Java function `testf` (only one function allowed):

```java
public clas TestMain{
    public static void main(String[]args){
        int a = testf("sum", 1, 2, 4, 5, 6);  // a = 18
        int b = testf("sum", 1, 3, 6);        // b = 10
        int c = testf("mult", 1, 2, 4, 5, 6); // c = 240
        int d = testf("mult", 1, 3, 6);       // d = 18
    }
}
```

(10 marks) [CSE 201 | 2014–15]

Answer — testf() with Varargs

```java
public class TestMain {

    // Single method using varargs to handle any number of integers
    public static int testf(String op, int... nums) {
        if (op.equals("sum")) {
            int sum = 0;
            for (int n : nums) sum += n;
            return sum;
        } else if (op.equals("mult")) {
            int product = 1;
            for (int n : nums) product *= n;
            return product;
        }
        throw new IllegalArgumentException("Unknown op: " + op);
    }

    public static void main(String[] args) {
        int a = testf("sum",  1, 2, 4, 5, 6);  // a = 18
        int b = testf("sum",  1, 3, 6);         // b = 10
        int c = testf("mult", 1, 2, 4, 5, 6);  // c = 240
        int d = testf("mult", 1, 3, 6);         // d = 18
        System.out.println(a + " " + b + " " + c + " " + d);
    }
}
```

Q23. Write down the differences between: (i) `String` and `StringBuffer` classes in Java; (ii) method overriding and method overloading. **(6 marks)** [CSE 201 | 2013–14]

Q24. Write a method that accepts an array of Strings `SearchList` and a search String `SearchKey`. The method determines and returns the number of occurrences of `SearchKey` in `SearchList`. **(8 marks)** [CSE 201 | 2012–13]

Answer — `countOccurrences` Method

```java
public class SearchDemo {

    public static int countOccurrences(String[] searchList,
                                       String searchKey) {
        int count = 0;
        for (String s : searchList)
            if (s.equals(searchKey)) count++;
        return count;
    }

    public static void main(String[] args) {
        String[] list = {"apple", "banana", "apple", "cherry", "apple"};
        System.out.println(countOccurrences(list, "apple")); // 3
        System.out.println(countOccurrences(list, "mango")); // 0
    }
}
```

Q25. Write a code segment to create a 2-D array of Integer class which will contain 4 rows. Each row will have 3, 1, 10, and 5 columns respectively. **(7 marks)** [CSE 201 | 2012–13]

Answer — 2-D `Integer` Array with Jagged Rows

```java
public class JaggedInteger {
    public static void main(String[] args) {

        // Outer array: 4 rows; column sizes: 3, 1, 10, 5
        Integer[][] arr = new Integer[4][];
        arr[0] = new Integer[3];
        arr[1] = new Integer[1];
        arr[2] = new Integer[10];
        arr[3] = new Integer[5];

        // Print to verify (default is null for Integer objects)
        for (Integer[] row : arr) {
            for (Integer elem : row)
                System.out.print(elem + " "); // prints null
            System.out.println();
        }
    }
}
```

Note: Unlike primitive `int[]`, an `Integer[]` is initialised to `null` (not 0) by default because `Integer` is a reference type.

Q26. Differentiate between primitive types and reference types in Java. **(5 marks)** [CSE 201 | 2010–11]

Answer — Primitive Types vs. Reference Types

Primitive Types	Reference Types
8 built-in types: `byte`, `short`, `int`, `long`, `float`, `double`, `char`, `boolean`.	Classes, interfaces, arrays, and enums.
Variable stores the *actual value* directly.	Variable stores a *memory address* (reference) pointing to the object on the heap.
Stored on the stack (for local variables).	Object is stored on the heap; the reference may be on the stack.
Cannot be `null`.	Can be `null` (no object referenced).
Passed by value; callee gets a copy.	Passed by value of the reference; callee can modify the object through the reference.
No methods; operations via operators only.	Have methods and fields; support OOP features.

```java
int x = 5;           // primitive -- value 5 stored directly
String s = "Hello";  // reference -- s holds address of String object
s = null;            // valid for references
// x = null;         // compile error -- primitives cannot be null
```

Q27. Write two differences between Java reference and C++ pointer. **(4 marks)** [CSE 201 | 2010–11]

Answer — Java Reference vs. C++ Pointer

Java Reference	C++ Pointer
Cannot perform arithmetic operations (e.g. `ref++` is illegal). A reference always points to a valid object or is `null`.	Supports pointer arithmetic (`ptr++`, `ptr + 2`, etc.), allowing raw traversal of memory.
Automatically managed by the **Garbage Collector**; the programmer never calls `free` or `delete`.	Manually managed; the programmer must call `delete` / `free` to release memory, risking leaks or dangling pointers.

Q28. Create a jagged array having three rows with 2, 5, and 3 elements respectively. All elements initialized to default values. Write code to print the contents using Java's for-each statement only. **(4+4 marks)** [CSE 201 | 2010–11]

Answer — Jagged Array: Creation & For-Each Print

```java
public class JaggedArray {
    public static void main(String[] args) {

        // Create jagged array: rows with 2, 5, and 3 elements
        int[][] jagged = new int[3][];
        jagged[0] = new int[2];   // default: all 0
        jagged[1] = new int[5];   // default: all 0
        jagged[2] = new int[3];   // default: all 0

        // Print using for-each only (no index variable)
        for (int[] row : jagged) {          // outer for-each: each row
            for (int elem : row) {          // inner for-each: each element
                System.out.print(elem + " ");
            }
            System.out.println();           // newline after each row
        }
    }
}
```

Output:

```
0 0
0 0 0 0 0
0 0 0
```

Q29. What is the role of Garbage Collector in Java? Consider the following code snippet and answer: (i) What will be the output? (ii) How many String objects will be created? Show contents of each and which will be garbage collected.

```java
String s1 = "It";
String s2 = "was";
String s3 = s1 + " " + s2;
s2 += " roses, ";
s3 = s3 + s2 + "roses all the way.";
System.out.println(s3);
```

(2+8 marks) [CSE 201 | 2010–11]

Answer — Garbage Collector & String Object Analysis

Role of Garbage Collector (GC):
The GC automatically reclaims heap memory occupied by objects that are no longer reachable by any live reference, preventing memory leaks without requiring the programmer to call `free`/`delete`.

(i) Output of the code:

```java
String s1 = "It";
String s2 = "was";
String s3 = s1 + " " + s2;        // "It was"
s2 += " roses, ";                  // s2 = "was roses, "
s3 = s3 + s2 + "roses all the way.";
System.out.println(s3);
```

Output:

```
It was was roses, roses all the way.
```

(ii) String objects created and garbage collected:

#	Content	Created by	GC?
1	`"It"`	literal	No (pool)
2	`"was"`	literal	No (pool)
3	`" "`	literal (temp)	Yes
4	`"It was"`	`s1+" "+s2`	Yes (s3 reassigned)
5	`"was roses, "`	`s2 += ...`	No (s2 holds it)
6	`"roses all the way."`	literal	No (pool)
7	`"It was was roses, roses all the way."`	final `s3`	No (s3 holds it)

Objects **3** and **4** become unreachable and are eligible for garbage collection: the intermediate `" "` string and the old value of `s3` (`"It was"`) after `s3` is reassigned.

Q30. What do you understand by short-circuit evaluation related to operators in Java? Which operators show this behavior? Explain with an example. **(8 marks)** [CSE 201 | 2010–11]

Answer — Short-Circuit Evaluation in Java

Definition:
Short-circuit evaluation means the second operand of a logical expression is evaluated *only if necessary* to determine the final result.
Operators that exhibit this behaviour:

- `&&` **(logical AND)** — if the left operand is `false`, the overall result is definitely `false`; the right operand is **not evaluated**.
- `||` **(logical OR)** — if the left operand is `true`, the overall result is definitely `true`; the right operand is **not evaluated**.

*Note: The bitwise operators `&` and `|` do **not** short-circuit — both operands are always evaluated.*

Example:

```java
int x = 0;

// Short-circuit AND: x != 0 is false, so (10 / x > 1) is NEVER evaluated
// Without short-circuit, dividing by x=0 would throw ArithmeticException
if (x != 0 && (10 / x > 1))
    System.out.println("Condition true");
else
    System.out.println("Short-circuited -- no division by zero");

// Short-circuit OR: first condition is true, second is skipped
boolean flag = true;
if (flag || (++x > 5))
    System.out.println("x = " + x); // prints x = 0 (x not incremented)
```

Q31. Explain two usages of the keyword `static`. **(8 marks)** [CSE 201 | 2010–11]

Q32. What is the use of an `import` declaration in Java? What is the restriction? Mention two cases where import declarations are not required. **(6 marks)** [CSE 201 | 2010–11]

Q33. Write the names, byte size, and default values of the primitive data types in Java. **(15 marks)** [CSE 201 | 2009–10]

Answer — Java Primitive Data Types

Type	Size (bytes)	Default value	Range / Notes
`byte`	1	`0`	-128 to 127
`short`	2	`0`	$-32{,}768$ to 32,767
`int`	4	`0`	-2^{31} to $2^{31}-1$
`long`	8	`0L`	-2^{63} to $2^{63}-1$
`float`	4	`0.0f`	IEEE 754 single precision
`double`	8	`0.0d`	IEEE 754 double precision
`char`	2	`'\u0000'`	Unicode 0 to 65,535
`boolean`	1 (JVM-dependent)	`false`	`true` or `false`

Note: Default values apply only to instance/class variables. Local variables have no default and must be initialised before use.

Q34. What are the five phases Java programs normally go through? Briefly describe them. **(10 marks)** [CSE 201 | 2008–09]

Answer — Five Phases of a Java Program

1. **Edit** — The programmer writes the source code in a text editor and saves the file with a `.java` extension.
2. **Compile** — The Java compiler (`javac`) translates the `.java` source file into **bytecode** and stores it in a `.class` file. Bytecode

is platform-independent intermediate code.

3. **Load** — The **Class Loader** (part of the JVM) reads the `.class` file from disk (or network) into memory, ready for execution.
4. **Verify** — The **Bytecode Verifier** checks the loaded bytecode for correctness: it ensures no stack overflows, that operands are of the correct type, and that no illegal data conversions are attempted. This step provides security.
5. **Execute** — The **Java Interpreter** (or Just-In-Time compiler) within the JVM translates bytecode into native machine instructions and runs the program.

$$\texttt{.java} \xrightarrow{\text{javac}} \texttt{.class (bytecode)} \xrightarrow{\text{ClassLoader}} \text{Memory} \xrightarrow{\text{Verifier}} \text{Verified} \xrightarrow{\text{JVM}} \text{Execution}$$

Q35. Write two different usages with examples for each of the following Java keywords: (i) `this`, (ii) `final`, (iii) `super`, (iv) `extends`, (v) `static`. **(20 marks)** [CSE 201 | 2009–10]

Answer — Two Usages Each: this, final, super, extends, static

(i) `this`

```java
class Point {
    int x, y;
    // Usage 1: distinguish field from parameter
    Point(int x, int y) { this.x = x; this.y = y; }
    // Usage 2: call another constructor in the same class
    Point() { this(0, 0); }
}
```

(ii) `final`

```java
// Usage 1: final variable (constant)
final double PI = 3.14159;

// Usage 2: final method (cannot be overridden)
class Base {
    final void display() { System.out.println("Base"); }
}
// class Child extends Base { void display(){} } // error
```

(iii) `super`

```java
class Animal { String name; Animal(String n) { name=n; } }
class Dog extends Animal {
    // Usage 1: call parent constructor
    Dog(String n) { super(n); }
    // Usage 2: call overridden parent method
    void show() { super.show(); System.out.println("Dog"); }
}
```

(iv) `extends`

```java
// Usage 1: class inheritance
class Vehicle { int speed; }
class Car extends Vehicle { int doors; }

// Usage 2: interface extension
interface Flyable { void fly(); }
interface SuperFlyable extends Flyable { void soar(); }
```

(v) `static`

```java
class Counter {
    // Usage 1: static field (shared across all instances)
    static int count = 0;
    Counter() { count++; }

    // Usage 2: static method (callable without an object)
    static int getCount() { return count; }
}
// Counter.getCount() - no instance needed
```

Q36. Write Java code to create the following multidimensional array: [2007–2008]

```
0 0 0 0
0 0 0
```

```
0 0
0
```

Answer — Jagged (Ragged) 2-D Array

```java
public class JaggedArray {
    public static void main(String[] args) {
        // Create a jagged array: row i has (4-i) columns
        int[][] arr = new int[4][];
        for (int i = 0; i < arr.length; i++)
            arr[i] = new int[4 - i];  // 4, 3, 2, 1 elements

        // All elements default to 0 - print to verify
        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < arr[i].length; j++)
                System.out.print(arr[i][j] + " ");
            System.out.println();
        }
    }
}
```

Output:

```
0 0 0 0
0 0 0
0 0
0
```

Explanation: In Java, a 2-D array is an array of arrays. Allocating each row separately (`arr[i] = new int[4-i]`) creates rows of different lengths (a *jagged* or *ragged* array). All `int` elements are automatically initialised to 0.

Q37. Write a Java code that takes n integers from the command line and outputs the maximum. Example: `java Max 1 2 4 5 -3 6 10 -7` outputs `10`. [2007–2008]

Answer — Maximum of n Command-Line Integers

```java
public class Max {
    public static void main(String[] args) {
        if (args.length == 0) {
            System.out.println("Usage: java Max n1 n2 ...");
            return;
        }

        int max = Integer.parseInt(args[0]);
        for (int i = 1; i < args.length; i++) {
            int n = Integer.parseInt(args[i]);
            if (n > max) max = n;
        }

        System.out.println(max);
    }
}
```

Run:

```
java Max 1 2 4 5 -3 6 10 -7
```

Output:

```
10
```

Key points:

- `args` is a `String[]`; each argument is parsed to `int` using `Integer.parseInt()`.
- The first element seeds `max`; the loop updates it as needed.
- `args.length` replaces the C-style `argc`.

BUET · CSE 107 / CSE 201

Object Oriented Programming

Section S11

Java OOP: Classes, Inheritance & Polymorphism

S11 — Java OOP: Classes, Inheritance & Polymorphism

Q1. What will be the output of the following code?

```java
abstract class X {
    X() { System.out.println("X Constructor"); draw(); }
    abstract void draw();
    abstract void process(int x);
    void process(Object o) { System.out.println("X process Object"); }
}
class Y extends X {
    int y = 5;
    Y() { System.out.println("Y Constructor"); }
    void draw() { System.out.println("Y with y =" + y); }
    void process(int x) { System.out.println("Y process int"); }
    void process(String s) { System.out.println("Y process String"); }
}
class Test {
    public static void main(String[] args) {
        X obj = new Y();
        obj.process(10);
        obj.process("hello");
        obj.process((Object)"hello");
    }
}
```

(10 marks) [CSE 107 | 2023–24]

Answer — Output Tracing: Abstract Class, Overriding and Overloading

Output:

```
X Constructor
Y with y =0
Y Constructor
Y process int
X process Object
X process Object
```

Explanation step by step:

1. `new Y()` first calls `X()`. Inside `X()`, `draw()` is called. Dynamic dispatch calls `Y.draw()`, but `y` is not yet initialised (still `0`), so it prints **`Y with y =0`**.
2. `X()` finishes; `y = 5` is now assigned; `Y()` prints **`Y Constructor`**.
3. `obj.process(10)` — `obj` is of type `X`. `X` declares `process(int)` as abstract, overridden by `Y` — prints **`Y process int`**.
4. `obj.process("hello")` — compile-time type is `X`, which has no `process(String)`; the only match is `process(Object)` in `X` — prints **`X process Object`**.
5. `obj.process((Object)"hello")` — cast forces the argument to `Object`; again matches `X.process(Object)` — prints **`X process Object`**.

Key concept: Overloading is resolved at *compile time* based on the declared type (`X`); overriding is resolved at *run time* based on the actual object type (`Y`).

Q2. Write Java code for the following using a `Book` class to achieve the expected output:

```java
class Book {
    private String title;
    private String author;
    private int year;
    private int edition;

    public Book(String title, String author, int year, int edition) {
        this.title = title;
        this.author = author;
        this.year = year;
        this.edition = edition;
    }

    public static void main(String[] args) {
        Book b1 = new Book("1984", "George Orwell", 1949, 1);
        Book b2 = new Book("1984", "George Orwell", 1949, 2);

        System.out.println(b1);             // Book (1984, George Orwell, 1949, 1)
        System.out.println(b1 == b2);       // false
```

```
        System.out.println(b1.equals(b2)); // true

        HashSet<Book> set = new HashSet<>();
        set.add(b1);
        set.add(b2);
        System.out.println(set.size());    // 1
    }
}
```

(15 marks) [CSE 107 | 2023–24]

Answer — Book Class: toString, equals, hashCode

```
import java.util.HashSet;
import java.util.Objects;

class Book {
    private String title;
    private String author;
    private int    year;
    private int    edition;

    Book(String title, String author, int year, int edition) {
        this.title   = title;
        this.author  = author;
        this.year    = year;
        this.edition = edition;
    }

    // Two Books are equal if title, author, and year match
    // (edition is ignored for equality)
    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Book)) return false;
        Book o = (Book) obj;
        return year == o.year
            && title.equals(o.title)
            && author.equals(o.author);
    }

    // hashCode must be consistent with equals
    @Override
    public int hashCode() {
        return Objects.hash(title, author, year);
    }

    // Format: Book (title, author, year, edition)
    @Override
    public String toString() {
        return "Book (" + title + ", " + author
               + ", " + year + ", " + edition + ")";
    }
}

class BookDemo {
    public static void main(String[] args) {
        Book b1 = new Book("1984", "George Orwell", 1949, 1);
        Book b2 = new Book("1984", "George Orwell", 1949, 2);

        System.out.println(b1);             // Book (1984, George Orwell, 1949, 1)
        System.out.println(b1 == b2);       // false
        System.out.println(b1.equals(b2)); // true

        HashSet<Book> set = new HashSet<>();
        set.add(b1); set.add(b2);
        System.out.println(set.size());     // 1
    }
}
```

Note: `hashCode()` must be overridden alongside `equals()` so that equal objects produce the same hash — a requirement for correct behaviour in `HashSet` and `HashMap`.

Q3. Consider the following `Point` and `Circle` classes. Complete them to achieve the expected output:

```
import java.util.HashMap;
```

```

class Point {
    private int x, y;

    Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
}

class Circle {
    Point center;
    int radius;

    Circle(int x, int y, int r) {
        center = new Point(x, y);
        this.radius = r;
    }
}

public class Test {
    public static void main(String[] args) {
        Circle c1 = new Circle(10, 20, 5);
        Circle c2 = new Circle(10, 20, 5);
        System.out.println(c1);
        System.out.println(c2);
        System.out.println(c1.equals(c2));
        HashMap<Circle, String> m = new HashMap<>();
        m.put(c1, "I am a Circle");
        System.out.println(m.get(c2));
    }
}

// Expected output:
// Circle: center (10, 20), radius: 5
// Circle: center (10, 20), radius: 5
// true
// I am a Circle
// Note: Point's getters for x and y are private
```

(15 marks) [CSE 107 | 2021–22]

Answer — Point and Circle Classes

```
import java.util.HashMap;
import java.util.Objects;

class Point {
    private int x, y;

    Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public int getX() { return x; }
    public int getY() { return y; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Point)) return false;
        Point p = (Point) o;
        return x == p.x && y == p.y;
    }

    @Override
    public int hashCode() {
        return Objects.hash(x, y);
    }
}

class Circle {
    Point center;
    int radius;
```

```
    Circle(int x, int y, int r) {
        center = new Point(x, y);
        this.radius = r;
    }

    @Override
    public String toString() {
        return "Circle: center (" + center.getX() + ", " +
               center.getY() + "), radius: " + radius;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Circle)) return false;
        Circle c = (Circle) o;
        return radius == c.radius && center.equals(c.center);
    }

    @Override
    public int hashCode() {
        return Objects.hash(center, radius);
    }
}

public class Test {
    public static void main(String[] args) {
        Circle c1 = new Circle(10, 20, 5);
        Circle c2 = new Circle(10, 20, 5);

        System.out.println(c1);
        System.out.println(c2);
        System.out.println(c1.equals(c2));

        HashMap<Circle, String> m = new HashMap<>();
        m.put(c1, "I am a Circle");

        System.out.println(m.get(c2));
    }
}
```

Q4. Complete the `Movie` class (with `title`, `releaseYear`, `productionCompany`, `runningTime`) to achieve the expected output:

```
Movie m1 = new Movie("The Lord of the Rings", 2001, "New Line Cinema", 178);
Movie m2 = new Movie("The Lord of the Rings", 2001, "WingNut Films", 178);
System.out.println(m1 == m2);         // false
System.out.println(m1.equals(m2));    // true
HashMap map = new HashMap();
map.put(m1, 93);
System.out.println(map.get(m2));      // 93
```

(10 marks) [CSE 107 | 2020–21]

Answer — Movie Class: equals and hashCode for HashMap

```
import java.util.HashMap;
import java.util.Objects;

class Movie {
    private String title;
    private int    releaseYear;
    private String productionCompany;
    private int    runningTime;

    Movie(String title, int releaseYear,
          String productionCompany, int runningTime) {
        this.title             = title;
        this.releaseYear       = releaseYear;
        this.productionCompany = productionCompany;
        this.runningTime       = runningTime;
    }

```

```java
    // Equal if title, releaseYear, and runningTime match
    // (productionCompany is intentionally excluded)
    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Movie)) return false;
        Movie o = (Movie) obj;
        return releaseYear == o.releaseYear
            && runningTime == o.runningTime
            && title.equals(o.title);
    }

    @Override
    public int hashCode() {
        return Objects.hash(title, releaseYear, runningTime);
    }
}

class MovieDemo {
    public static void main(String[] args) {
        Movie m1 = new Movie("The Lord of the Rings",
                             2001, "New Line Cinema", 178);
        Movie m2 = new Movie("The Lord of the Rings",
                             2001, "WingNut Films", 178);

        System.out.println(m1 == m2);         // false
        System.out.println(m1.equals(m2));    // true

        HashMap map = new HashMap();
        map.put(m1, 93);
        System.out.println(map.get(m2));      // 93
    }
}
```

Q5. Complete the `Institute`/`College`/`Movie` class to achieve the expected output (`equals` returns `true`, `HashMap.get` returns `100`):

```java
// Expected output: true, 100
College c1 = new College(135790, 1, 1, 0);
College c2 = new College(135790, 1, 1, 0);
System.out.println(c1.equals(c2));
HashMap map = new HashMap();
map.put(c1, 100);
System.out.println(map.get(c2));
```

(8+10 marks) [CSE 107 | 2016–17, 2019–20, 2020–21]

Answer — Institute/College: equals and hashCode

```java
import java.util.HashMap;
import java.util.Objects;

class Institute {
    protected int id;

    Institute(int id) { this.id = id; }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Institute)) return false;
        Institute o = (Institute) obj;
        return this.id == o.id;
    }

    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
}

class College extends Institute {
    private int dept;
    private int batch;
```

```
    private int section;

    College(int id, int dept, int batch, int section) {
        super(id);
        this.dept    = dept;
        this.batch   = batch;
        this.section = section;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof College)) return false;
        College o = (College) obj;
        return super.equals(o)
            && this.dept    == o.dept
            && this.batch   == o.batch
            && this.section == o.section;
    }

    @Override
    public int hashCode() {
        return Objects.hash(id, dept, batch, section);
    }
}

class CollegeDemo {
    public static void main(String[] args) {
        College c1 = new College(135790, 1, 1, 0);
        College c2 = new College(135790, 1, 1, 0);

        System.out.println(c1.equals(c2)); // true

        HashMap map = new HashMap();
        map.put(c1, 100);
        System.out.println(map.get(c2));   // 100
    }
}
```

Q6. Write Java code to fix the problems in the `main` method:

```
class P { int x = 1;
    class Q { int y = 2; }
    static class R { int z = 3; }
}
public class NestedClassTest {
    public static void main(String[] args) {
        P p = new P(); p.print(); p.show();
        Q q = new Q(); q.print();
        R r = new R(); r.print();
    }
}
```

(5 marks) [CSE 107 | 2020–21, 2017–18]

Answer — Fixing Nested Class main() Problems

Problems in the original main():

- `p.print()` and `p.show()` — `P` has no such methods; these calls must be removed or added to `P`.
- `Q q = new Q()` — `Q` is an *inner* class of `P`; it cannot be instantiated without a `P` instance.
- `q.print()` — `Q` has no `print()` method.
- `R r = new R()` — `R` is `P.R`, not simply `R` in the outer scope.
- `r.print()` — `R` has no `print()` method.

Fixed code:

```
class P {
    int x = 1;

    class Q {
        int y = 2;
```

```
        void print() { System.out.println("Q.y = " + y); }
    }

    static class R {
        int z = 3;
        void print() { System.out.println("R.z = " + z); }
    }

    void print() { System.out.println("P.x = " + x); }
}

public class NestedClassTest {
    public static void main(String[] args) {
        P p = new P();
        p.print();              // P.x = 1

        P.Q q = p.new Q();      // inner class needs outer instance
        q.print();              // Q.y = 2

        P.R r = new P.R();      // static nested class: no outer needed
        r.print();              // R.z = 3
    }
}
```

Key rules:

- **Inner class** (`Q`): requires an enclosing instance — use `outerRef.new Inner()`.
- **Static nested class** (`R`): no enclosing instance needed — use `new Outer.Nested()`.

Q7. Write Java code for an `Animal` hierarchy where the `Animal` class cannot be instantiated:

```
Bird alex = new Albatross("Alex", 39);
Mammal spot = new Dog("Spot", 7);
Mammal fred = new FruitBat("Fred", 10);
Reptile steph = new EasternBrownSnake("Steph", 12);
Fish bruce = new GreatWhiteShark("Bruce", 40);
Arachnid charlotte = new RedBackSpider("Charlotte", 1);
Animal[] animals = {alex, spot, fred, steph, bruce, charlotte};
```

(10 marks) [CSE 107 | 2020–21]

Answer — Animal Hierarchy (Abstract Base Class)

```
// Abstract base -- cannot be instantiated directly
abstract class Animal {
    protected String name;
    protected int    age;

    Animal(String name, int age) {
        this.name = name; this.age = age;
    }

    @Override
    public String toString() {
        return getClass().getSimpleName()
               + "(\"" + name + "\", " + age + ")";
    }
}

// Mid-level abstract classes
abstract class Bird     extends Animal { Bird(String n, int a)     { super(n,a); } }
abstract class Mammal   extends Animal { Mammal(String n, int a)   { super(n,a); } }
abstract class Reptile  extends Animal { Reptile(String n, int a)  { super(n,a); } }
abstract class Fish     extends Animal { Fish(String n, int a)     { super(n,a); } }
abstract class Arachnid extends Animal { Arachnid(String n, int a) { super(n,a); } }

// Concrete leaf classes
class Albatross       extends Bird     { Albatross(String n, int a)       { super(n,a); } }
class Dog             extends Mammal   { Dog(String n, int a)             { super(n,a); } }
class FruitBat        extends Mammal   { FruitBat(String n, int a)        { super(n,a); } }
class EasternBrownSnake extends Reptile{ EasternBrownSnake(String n,int a){ super(n,a); } }
class GreatWhiteShark extends Fish     { GreatWhiteShark(String n, int a) { super(n,a); } }
class RedBackSpider   extends Arachnid { RedBackSpider(String n, int a)   { super(n,a); } }

```

```java
class AnimalHierarchy {
    public static void main(String[] args) {
        Bird    alex      = new Albatross("Alex", 39);
        Mammal  spot      = new Dog("Spot", 7);
        Mammal  fred      = new FruitBat("Fred", 10);
        Reptile steph     = new EasternBrownSnake("Steph", 12);
        Fish    bruce     = new GreatWhiteShark("Bruce", 40);
        Arachnid charlotte= new RedBackSpider("Charlotte", 1);

        Animal[] animals  = {alex, spot, fred, steph, bruce, charlotte};

        for (Animal a : animals)
            System.out.println(a);
    }
}
```

Output:

```
Albatross("Alex", 39)
Dog("Spot", 7)
FruitBat("Fred", 10)
EasternBrownSnake("Steph", 12)
GreatWhiteShark("Bruce", 40)
RedBackSpider("Charlotte", 1)
```

Q8. Explain briefly dynamic method dispatching. **(6 marks)** [CSE 107 | 2017–18]

Answer — Dynamic Method Dispatching

Dynamic Method Dispatching is the mechanism by which a call to an overridden method is resolved at **run time** rather than compile time. It is the foundation of run-time polymorphism in Java.
When a superclass reference variable holds a subclass object, and an overridden method is called through that reference, Java determines *which version* of the method to execute based on the **actual type of the object** at run time — not the declared type of the reference.

```java
class Shape {
    void draw() { System.out.println("Drawing Shape"); }
}
class Circle extends Shape {
    @Override
    void draw() { System.out.println("Drawing Circle"); }
}
class Rectangle extends Shape {
    @Override
    void draw() { System.out.println("Drawing Rectangle"); }
}

class Demo {
    public static void main(String[] args) {
        Shape s;              // superclass reference

        s = new Circle();
        s.draw();             // "Drawing Circle"    -- dispatched at run time

        s = new Rectangle();
        s.draw();             // "Drawing Rectangle" -- dispatched at run time
    }
}
```

The JVM uses a *virtual method table* (vtable) per class to look up the correct method implementation at run time. This enables writing general-purpose code against a superclass type that automatically adapts to specific subclass behaviour.

Q9. Write Java code to exchange two integer variables using a method named `swap`. The changes inside the method must be visible to the `main` method. Write the `main` method too. **(7 marks)** [CSE 107 | 2016–17]

Answer — swap() Visible to main()

Java is *pass-by-value*, so swapping primitive `int`s inside a method is **not** visible to the caller. To make the swap visible, wrap the integers in an array (or any mutable object).

```java
class SwapDemo {
    // swap using a 2-element array
    static void swap(int[] a) {
        int tmp = a[0];
```

```
        a[0]   = a[1];
        a[1]   = tmp;
    }

    public static void main(String[] args) {
        int[] vals = {10, 20};
        System.out.println("Before: a=" + vals[0] + ", b=" + vals[1]);
        swap(vals);
        System.out.println("After:  a=" + vals[0] + ", b=" + vals[1]);
    }
}
```

Output:

```
Before: a=10, b=20
After:  a=20, b=10
```

Why it works: The array reference is passed by value, but the array object on the heap is shared, so modifications to `a[0]` and `a[1]` are visible in `main()`.

Or

```
class NumberWrapper {
    int value;
    NumberWrapper(int value) { this.value = value; }
}

public class SwapObjects {
    public static void main(String[] args) {
        NumberWrapper objA = new NumberWrapper(5);
        NumberWrapper objB = new NumberWrapper(10);

        swap(objA, objB);

        System.out.println("objA: " + objA.value); // Outputs 10
        System.out.println("objB: " + objB.value); // Outputs 5
    }

    public static void swap(NumberWrapper a, NumberWrapper b) {
        int temp = a.value;
        a.value = b.value;
        b.value = temp;
    }
}
```

Q10. Write Java code for a `TwoDimMatrix` class: input, print, determinant, and `div(A)` methods. **(15 marks)** [CSE 201 | 2015–16]

Answer — TwoDimMatrix Class

```
import java.util.Scanner;

class TwoDimMatrix {
    private double[][] m = new double[2][2];

    // Input 4 elements row-by-row
    void input() {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter 4 elements (row by row):");
        for (int i = 0; i < 2; i++)
            for (int j = 0; j < 2; j++)
                m[i][j] = sc.nextDouble();
    }

    // Print the 2x2 matrix
    void print() {
        for (int i = 0; i < 2; i++)
            System.out.printf("| %6.2f  %6.2f |%n",
                              m[i][0], m[i][1]);
    }

    // Determinant of a 2x2 matrix: ad - bc
    double determinant() {
        return m[0][0] * m[1][1] - m[0][1] * m[1][0];
    }

```

```
    // div(A): divide this matrix by A (multiply by A's inverse)
    // A^-1 = (1/det(A)) * [[d,-b],[-c,a]]
    TwoDimMatrix div(TwoDimMatrix A) {
        double det = A.determinant();
        if (det == 0)
            throw new ArithmeticException("Matrix A is singular.");

        TwoDimMatrix result = new TwoDimMatrix();
        // result = this * A^-1
        double[][] inv = {
            { A.m[1][1] / det, -A.m[0][1] / det},
            {-A.m[1][0] / det,  A.m[0][0] / det}
        };
        for (int i = 0; i < 2; i++)
            for (int j = 0; j < 2; j++)
                for (int k = 0; k < 2; k++)
                    result.m[i][j] += this.m[i][k] * inv[k][j];
        return result;
    }

    public static void main(String[] args) {
        TwoDimMatrix M = new TwoDimMatrix();
        M.input();
        System.out.println("Matrix M:"); M.print();
        System.out.println("det(M) = " + M.determinant());

        TwoDimMatrix A = new TwoDimMatrix();
        A.input();
        System.out.println("Matrix A:"); A.print();

        TwoDimMatrix Q = M.div(A);
        System.out.println("M / A:"); Q.print();
    }
}
```

Q11. Write a Java program to store student records as a collection. `Student` contains Name, Age, CGPA. Sort by: descending CGPA first; if equal, ascending age; if equal, lexicographic order. **(20 marks)** [CSE 201 | 2015–16]

Answer — Student Collection with Custom Sort

```
import java.util.*;

class Student implements Comparable<Student> {
    String name;
    int    age;
    double cgpa;

    Student(String name, int age, double cgpa) {
        this.name = name; this.age = age; this.cgpa = cgpa;
    }

    // Sort: desc CGPA -> asc age -> lexicographic name
    @Override
    public int compareTo(Student o) {
        int cmp = Double.compare(o.cgpa, this.cgpa); // desc CGPA
        if (cmp != 0) return cmp;
        cmp = Integer.compare(this.age, o.age);       // asc age
        if (cmp != 0) return cmp;
        return this.name.compareTo(o.name);           // lex name
    }

    @Override
    public String toString() {
        return name + " (age=" + age + ", cgpa=" + cgpa + ")";
    }
}

class StudentSort {
    public static void main(String[] args) {
        List<Student> list = new ArrayList<>();
        list.add(new Student("Alice", 21, 3.8));
        list.add(new Student("Bob",   20, 3.9));
        list.add(new Student("Carol", 22, 3.8));
```

```
        list.add(new Student("Dave",  21, 3.8));
        list.add(new Student("Eve",   19, 4.0));

        Collections.sort(list);

        System.out.println("Sorted students:");
        for (Student s : list)
            System.out.println("  " + s);
    }
}
```

Output:

```
Sorted students:
  Eve (age=19, cgpa=4.0)
  Bob (age=20, cgpa=3.9)
  Alice (age=21, cgpa=3.8)
  Dave (age=21, cgpa=3.8)
  Carol (age=22, cgpa=3.8)
```

Q12. Write a program to take input of 120 students, sort them as described above, and write all student information to a file named `Students.Info`. Write student objects directly to file and read them back as objects. **(15 marks)** [CSE 201 | 2015–16]

Answer — 120 Students: Sort, Serialize to File and Read Back

```
import java.io.*;
import java.util.*;

class Student implements Comparable<Student>, Serializable {
    String name; int age; double cgpa;

    Student(String name, int age, double cgpa) {
        this.name = name; this.age = age; this.cgpa = cgpa;
    }

    @Override
    public int compareTo(Student o) {
        int c = Double.compare(o.cgpa, this.cgpa);
        if (c != 0) return c;
        c = Integer.compare(this.age, o.age);
        if (c != 0) return c;
        return this.name.compareTo(o.name);
    }

    @Override
    public String toString() {
        return name + " age=" + age + " cgpa=" + cgpa;
    }
}

class StudentFile {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        List<Student> list = new ArrayList<>();

        // Input 120 students
        for (int i = 0; i < 120; i++) {
            System.out.print("Name: ");   String name = sc.next();
            System.out.print("Age: ");    int    age  = sc.nextInt();
            System.out.print("CGPA: ");   double cgpa = sc.nextDouble();
            list.add(new Student(name, age, cgpa));
        }

        Collections.sort(list);

        // Write objects to file
        try (ObjectOutputStream oos = new ObjectOutputStream(
                 new FileOutputStream("Students.Info"))) {
            oos.writeObject(list);
            System.out.println("Written to Students.Info");
        }

        // Read objects back from file
```

```
        try (ObjectInputStream ois = new ObjectInputStream(
                new FileInputStream("Students.Info"))) {
            @SuppressWarnings("unchecked")
            List<Student> loaded = (List<Student>) ois.readObject();
            System.out.println("Read back " + loaded.size() + " students:");
            for (Student s : loaded)
                System.out.println("  " + s);
        }
    }
}
```

Key points:

- `Student` implements `Serializable` so the entire object (and the `List`) can be written with `ObjectOutputStream`.
- The whole sorted list is written as a single object and read back with one `readObject()` call.
- `try-with-resources` ensures streams are closed automatically.

Q13. What is the difference between declaring a variable with `protected` access modifier versus no modifier? **(5 marks)** [CSE 201 | 2013–14]

Answer — `protected` vs Default (No Modifier)

Access from	No modifier (package-private)	`protected`
Same class	✓	✓
Same package	✓	✓
Subclass in different package	×	✓
Non-subclass, different pkg	×	×

The only difference is that `protected` additionally allows access from **subclasses in other packages**, while the default (no modifier) restricts access strictly to the same package.

```
package p1;
class A {
    int x = 1;            // package-private
    protected int y = 2; // protected
}

package p2;
class B extends p1.A {
    void test() {
        // System.out.println(x); // ERROR -- different package
        System.out.println(y);    // OK -- protected, subclass
    }
}
```

Q14. How is multiple inheritance implemented in Java? Can an abstract class be final? Explain briefly. **(4+4 marks)** [CSE 201 | 2013–14]

Answer — Multiple Inheritance in Java and abstract + final

Multiple Inheritance in Java:
Java does *not* support multiple inheritance through classes (to avoid the Diamond Problem). Instead, it achieves multiple inheritance of **type** via **interfaces** — a class can implement any number of interfaces.

```
interface Flyable  { void fly();  }
interface Swimmable { void swim(); }

class Duck implements Flyable, Swimmable {
    public void fly()  { System.out.println("Duck flies");  }
    public void swim() { System.out.println("Duck swims"); }
}
```

Can an abstract class be final?
No. Declaring a class both `abstract` and `final` is a **compile-time error** in Java. The two keywords are contradictory:

- `abstract` requires the class to be *subclassed* so that its abstract methods can be implemented.
- `final` *prevents* any subclassing.

Together they make the class both "must be extended" and "cannot be extended", which is logically impossible and the compiler rejects it.

Q15. Write a class `Animal` that has name and age as member variables. Write another class that adds 3 animals to an ArrayList and removes the animal named "Lion". **(10 marks)** [CSE 201 | 2012–13]

Answer — Animal Class with ArrayList

```java
import java.util.ArrayList;
import java.util.Iterator;

class Animal {
    private String name;
    private int age;

    Animal(String name, int age) { this.name = name; this.age = age; }

    public String getName() { return name; }
    public int    getAge()  { return age;  }

    @Override
    public String toString() {
        return "Animal{name=" + name + ", age=" + age + "}";
    }
}

class AnimalDemo {
    public static void main(String[] args) {
        ArrayList<Animal> list = new ArrayList<>();
        list.add(new Animal("Lion",   5));
        list.add(new Animal("Tiger",  3));
        list.add(new Animal("Parrot", 2));

        System.out.println("Before: " + list);

        Iterator<Animal> it = list.iterator();
        while (it.hasNext())
            if (it.next().getName().equals("Lion")) it.remove();

        System.out.println("After:  " + list);
    }
}
```

Output:

```
Before: [Animal{name=Lion, age=5}, Animal{name=Tiger, age=3}, ...]
After:  [Animal{name=Tiger, age=3}, Animal{name=Parrot, age=2}]
```

Q16. A movie club contains many movies. Each movie has a title and one or more directors. A movie may be borrowed and returned. Write `Movie` and `MovieClub` classes with a method to return current number of movies. **(16 marks)** [CSE 201 | 2012–13]

Answer — Movie and MovieClub Classes

```java
import java.util.ArrayList;

class Movie {
    private String title;
    private ArrayList<String> directors;
    private boolean borrowed;

    Movie(String title) {
        this.title     = title;
        this.directors = new ArrayList<>();
        this.borrowed  = false;
    }

    void addDirector(String d) { directors.add(d); }

    void borrow() {
        if (!borrowed) borrowed = true;
        else System.out.println(title + " is already borrowed.");
    }

    void returnMovie() {
        if (borrowed) borrowed = false;
        else System.out.println(title + " was not borrowed.");
    }

```

```
    boolean isBorrowed() { return borrowed; }

    @Override
    public String toString() {
        return title + " " + directors
               + (borrowed ? " [borrowed]" : " [available]");
    }
}

class MovieClub {
    private ArrayList<Movie> movies = new ArrayList<>();

    void addMovie(Movie m)    { movies.add(m); }
    void removeMovie(Movie m) { movies.remove(m); }
    int  currentCount()       { return movies.size(); }

    void listMovies() {
        for (Movie m : movies) System.out.println(m);
    }
}

class MovieDemo {
    public static void main(String[] args) {
        Movie m1 = new Movie("Inception");
        m1.addDirector("Christopher Nolan");

        Movie m2 = new Movie("The Matrix");
        m2.addDirector("Lana Wachowski");
        m2.addDirector("Lilly Wachowski");

        MovieClub club = new MovieClub();
        club.addMovie(m1);
        club.addMovie(m2);

        System.out.println("Total movies: " + club.currentCount());
        m1.borrow();
        club.listMovies();
        m1.returnMovie();
        System.out.println("After return: " + club.currentCount());
    }
}
```

Output:

```
Total movies: 2
Inception [Christopher Nolan] [borrowed]
The Matrix [Lana Wachowski, Lilly Wachowski] [available]
After return: 2
```

Q17. Write a student class with: id, name, course, number of units completed. Write `inc()` to increment units by one. Write another class to create a student object, get units, call `inc()`, and display the object's state. **(14 marks)** [CSE 201 | 2012–13]

Answer — Student Class with inc() Method

```
class Student {
    private int    id;
    private String name;
    private String course;
    private int    unitsCompleted;

    Student(int id, String name, String course, int units) {
        this.id             = id;
        this.name           = name;
        this.course         = course;
        this.unitsCompleted = units;
    }

    int getUnits() { return unitsCompleted; }
    void inc()     { unitsCompleted++; }

    @Override
    public String toString() {
        return "Student{id=" + id + ", name=" + name
```

```
                + ", course=" + course
                + ", units=" + unitsCompleted + "}";
    }
}

class StudentDemo {
    public static void main(String[] args) {
        Student s = new Student(1, "Alice", "CSE", 30);

        System.out.println("Units before inc(): " + s.getUnits());
        s.inc();
        System.out.println("Units after  inc(): " + s.getUnits());
        System.out.println("State: " + s);
    }
}
```

Output:

```
Units before inc(): 30
Units after  inc(): 31
State: Student{id=1, name=Alice, course=CSE, units=31}
```

Q18. Show two examples of using the `super` keyword. **(4 marks)** [CSE 201 | 2012–13]

Answer — Two Uses of the super Keyword

Use 1 — Call the parent constructor:

```
class Animal {
    String name;
    Animal(String name) { this.name = name; }
}

class Dog extends Animal {
    String breed;
    Dog(String name, String breed) {
        super(name);        // calls Animal(String)
        this.breed = breed;
    }
}
```

Use 2 — Call an overridden parent method:

```
class Shape {
    void draw() { System.out.println("Drawing Shape"); }
}

class Circle extends Shape {
    @Override
    void draw() {
        super.draw();       // calls Shape's draw()
        System.out.println("Drawing Circle");
    }
}
```

Q19. What are the properties of static variables and methods? Why is `main()` declared as static? **(6+5 marks)** [CSE 201 | 2012–13]

Answer — Static Variables, Methods and Why main() is Static

Properties of static variables:

- Belong to the **class**, not to any individual object.
- Only **one copy** exists in memory, shared by all instances.
- Initialized when the class is loaded (before any object is created).
- Accessed as `ClassName.variable` (no object needed).
- Retain their value across all instances throughout the program.

Properties of static methods:

- Belong to the class; called without creating an object.
- Can access **only** static members directly.
- Cannot use `this` or `super` keywords.

- Cannot be overridden (only hidden in subclasses).

```
class Counter {
    static int count = 0;   // shared across all instances
    Counter() { count++; }
    static int getCount() { return count; }
}
// Counter.getCount() -- no object needed
```

Why is `main()` declared static?
The JVM must call `main()` *before* any object of the class exists. Since no instance is available at startup, `main()` must be `static` so the JVM can invoke it directly on the class without instantiating it first.

Q20. Write a short note on the lifetime of an object in Java. **(6 marks)** [CSE 201 | 2011–12]

Answer — Lifetime of an Object in Java

An object's lifetime spans from its **creation** to its **garbage collection**. The stages are:

1. **Creation** — `new` allocates memory on the heap and calls the constructor to initialise the object.
2. **In use (reachable)** — At least one live reference points to the object. It can be accessed and used freely.
3. **Unreachable** — All references go out of scope or are set to `null`. The object is no longer accessible but still occupies heap memory.
4. **Garbage collection** — The JVM's Garbage Collector (GC) detects the unreachable object and reclaims its memory automatically. The programmer cannot force or predict exactly when this happens.
5. **Finalization (deprecated)** — Before collection, the GC may call `finalize()` (deprecated since Java 9) for any last-resort cleanup, but relying on it is strongly discouraged.

```
void demo() {
    Student s = new Student(1, "Alice", "CSE", 30); // created
    s.inc();                                        // in use
}   // s goes out of scope -- object becomes unreachable
    // GC will eventually reclaim the heap memory
```

Unlike C/C++, the programmer never explicitly frees memory; the GC handles deallocation, preventing dangling pointers and most memory leaks.

Q21. Discuss the access levels of a constructor method of a class in Java. **(8 marks)** [CSE 201 | 2011–12]

Answer — Access Levels of Constructors in Java

A constructor can have any of the four access modifiers:

Modifier	Who can call it	Typical use
`public`	Anyone, from any package	Normal instantiation
`protected`	Same package + subclasses	Abstract-like base classes
(no modifier)	Same package only	Package-level factories
`private`	Only inside the same class	Singleton, factory methods

```
class Singleton {
    private static Singleton instance;

    private Singleton() { }   // private constructor

    public static Singleton getInstance() {
        if (instance == null) instance = new Singleton();
        return instance;
    }
}

class Base {
    protected Base() { }  // only subclasses or same package
}

class Child extends Base {
    Child() { super(); }  // OK -- subclass can call protected ctor
}
```

If *no* constructor is written, the compiler inserts a default `public` no-arg constructor automatically.

Q22. What is the purpose of the method `finalize` in `Object`? Why is its use discouraged? **(8 marks)** [CSE 201 | 2011–12]

Answer — `finalize()` Purpose and Why It Is Discouraged

Purpose:
`protected void finalize() throws Throwable` is a method defined in `java.lang.Object`. The Garbage Collector *may* call it on an object just before reclaiming its memory, giving the object a chance to release non-memory resources (e.g., file handles, native memory).

```
class Resource {
    @Override
    protected void finalize() throws Throwable {
        System.out.println("Cleaning up resource");
        super.finalize();
    }
}
```

Why its use is discouraged (deprecated since Java 9):

- **No guaranteed timing** — the GC decides *when* (or *whether*) to call it; resources may be held for a very long time.
- **No guaranteed execution** — the JVM may exit without ever calling `finalize()`.
- **Performance overhead** — objects with finalizers take at least two GC cycles to collect.
- **Resurrection risk** — a finalizer can accidentally make the object reachable again, causing subtle bugs.
- **Better alternatives exist** — use `try-with-resources` and `AutoCloseable`/`Closeable` for deterministic cleanup.

Q23. Why does the `main` method not require an `argc`-type parameter for command line arguments? **(4 marks)** [CSE 201 | 2011–12]

Answer — Why Java's `main()` Needs No `argc`

In C/C++, `main(int argc, char* argv[])` requires a separate `argc` because a raw array carries no length information.
In Java, `main(String[] args)` receives a **String array object**, which already knows its own length via `args.length`. Therefore a separate count parameter is unnecessary.

```
public class Demo {
    public static void main(String[] args) {
        System.out.println("Argument count: " + args.length);
        for (int i = 0; i < args.length; i++)
            System.out.println("args[" + i + "] = " + args[i]);
    }
}
// Run: java Demo Hello World
// Output:
//   Argument count: 2
//   args[0] = Hello
//   args[1] = World
```

The JVM populates `args` automatically with the command-line tokens; `args.length` replaces the role of `argc`.

Q24. What is run-time polymorphism? How does Java achieve run-time polymorphism? **(6 marks)** [CSE 201 | 2011–12]

Answer — Run-Time Polymorphism in Java

Run-time polymorphism (also called *dynamic polymorphism*) is the ability to call an overridden method through a parent-class reference, with the actual method executed determined at **run time** based on the real type of the object, not the declared type of the reference.
How Java achieves it — Dynamic Method Dispatch:
Java uses a *virtual method table* (vtable). Every overridable method call through a reference is resolved at run time by looking up the vtable of the actual object. This is automatic for all non-`static`, non-`final`, non-`private` methods.

```
class Animal {
    void sound() { System.out.println("Some sound"); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Woof"); }
}
class Cat extends Animal {
    @Override
    void sound() { System.out.println("Meow"); }
}

class Demo {
    public static void main(String[] args) {
```

```
        Animal a;           // declared type: Animal

        a = new Dog();
        a.sound();          // "Woof"  -- resolved at run time

        a = new Cat();
        a.sound();          // "Meow"  -- resolved at run time
    }
}
```

The same reference `a` calls different implementations depending on the actual object it holds — this is run-time polymorphism.

Q25. Override the `equals(Object obj)` method in the following `Student` class so that two students are equal if they have the same `name` and `cgpa`:

```
class Student {
    private int id; private String name; private double cgpa;
    public Student(int i, String n, double c) { id=i; name=n; cgpa=c; }
}
// s1.equals(s2) -> false (different name), s1.equals(s3) -> true (same name+cgpa)
// s1.equals(s4) -> false (same name, different cgpa)
```

(11 marks) [CSE 201 | 2010–11]

Answer — Overriding `equals()` in Student

```
class Student {
    private int    id;
    private String name;
    private double cgpa;

    public Student(int i, String n, double c) {
        id = i; name = n; cgpa = c;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Student)) return false;
        Student other = (Student) obj;
        return this.name.equals(other.name)
            && Double.compare(this.cgpa, other.cgpa) == 0;
    }
}

class TestStudent {
    public static void main(String[] args) {
        Student s1 = new Student(1, "Alice", 3.8);
        Student s2 = new Student(2, "Bob",   3.8); // different name
        Student s3 = new Student(3, "Alice", 3.8); // same name+cgpa
        Student s4 = new Student(4, "Alice", 3.5); // same name, diff cgpa

        System.out.println(s1.equals(s2)); // false
        System.out.println(s1.equals(s3)); // true
        System.out.println(s1.equals(s4)); // false
    }
}
```

Note: `Double.compare()` is used instead of `==` for `double` comparison to avoid floating-point precision issues.

Q26. Suppose the following class is defined:

```
class Point {
    int x, y;
    Point(int x1, int y1) { x = x1; y = y1; }
}
```

State the values of Point objects after each statement:

```
Point point1; Point point2;
point1 = new Point(10, 20);
point2 = point1;
point1 = new Point(30, 60);
point1 = point2;
```

(6 marks) [CSE 201 | 2007–08]

Answer — Point Object Reference Tracing

Statement	point1	point2
`Point point1; Point point2;`	`null`	`null`
`point1 = new Point(10, 20);`	`(10,20)`	`null`
`point2 = point1;`	`(10,20)`	`(10,20)` (same obj)
`point1 = new Point(30, 60);`	`(30,60)`	`(10,20)`
`point1 = point2;`	`(10,20)`	`(10,20)` (same obj)

Key insight: `point2 = point1` copies the *reference*, not the object. After `point1 = new Point(30,60)`, `point2` still points to the original `(10,20)` object. The final `point1 = point2` makes both variables refer to the `(10,20)` object again; the `(30,60)` object becomes unreachable and eligible for GC.

Q27. Write Java code for the following regarding arrays of reference types vs primitive types. What is the difference between `int c[], x;` and `int[] c, x;`? **(6 marks)** [CSE 201 | 2007–08]

Answer — Primitive vs Reference Arrays and Declaration Syntax

Primitive array vs Reference array:

```java
// Primitive array -- each element stores the value directly
int[] prims = new int[3];        // [0, 0, 0]
prims[0] = 10; prims[1] = 20;

// Reference array -- each element stores a reference (null by default)
String[] refs = new String[3];  // [null, null, null]
refs[0] = "Hello"; refs[1] = "World";

// Array of objects
Point[] points = new Point[2];  // [null, null]
points[0] = new Point(1, 2);    // must create each object separately
points[1] = new Point(3, 4);
```

Difference between `int c[], x;` and `int[] c, x;`:

Declaration	c	x
`int c[], x;`	array (`int[]`)	plain `int`
`int[] c, x;`	array (`int[]`)	array (`int[]`)

In `int c[], x;` the `[]` applies only to `c`; `x` is a simple `int`. In `int[] c, x;` the `[]` is part of the type, so *both* `c` and `x` are `int` arrays. The second form is preferred as it is less ambiguous.

BUET · CSE 107 / CSE 201

Object Oriented Programming

Section S12

Interfaces, Abstract Classes & Nested Classes

S12 — Interfaces, Abstract Classes & Nested Classes

Q1. Also fix the following:

```
interface X1 { void m1(); static void m2(); default void m3(); }
interface X2 { private void m4(); void m5(); }
abstract class A1 implements X1 { final void m6(); abstract void m7(); }
class A2 extends A1 implements X2 { /* your code */ }
```

(10 marks) [CSE 107 | 2023–24]

Answer — Fix X1/X2/A1 & Write A2

Errors and fixes:

1. `interface X2: private void m4();` — `private` interface method must have a body (Java 9+). Fix: `private void m4() { }`
2. `abstract class A1: final void m6();` — A `final` method cannot be abstract (no body but cannot be overridden — contradiction). Fix: `final void m6() { }`

Corrected code:

```
interface X1 { void m1(); static void m2() { } default void m3() { } }
interface X2 { private void m4() { }  void m5(); }  // Java 9+
abstract class A1 implements X1 {
    final void m6() { }     // fixed: body added
    abstract void m7();
}
```

Minimum code for A2:
Must implement: `m1()` (from `X1`), `m5()` (from `X2`), `m7()` (abstract in `A1`).

```
class A2 extends A1 implements X2 {
    @Override public void m1() { }
    @Override public void m5() { }
    @Override void m7() { }
}
```

Q2. What are two types of nested classes? What are the differences between them? List all problems in the following code. Write Java code to fix only the problems in the `main` method:

```
class X {
    int a = 1;
    void display() { Y obY = new Y(); obY.display(); Z obZ = new Z(); obZ.display(); }
    void showNested() { System.out.println(b); System.out.println(c); }
    class Y { int b = 2; void display() { System.out.println(a); System.out.println(b); } }
    static class Z { int c = 3; void display() { System.out.println(a); System.out.println(c); } }
}
class Test {
    public static void main(String[] args) {
        X obX = new X(); obX.display(); obX.showNested();
        Y obY = new Y(); obY.display();
        Z obZ = new Z(); obZ.display();
    }
}
```

(20 marks) [CSE 107 | Jan 2021, 2023–24]

Answer — Nested Class Types, Differences & Fixed main

Two types:

1. **Inner class** (`class Y`) — non-static; needs outer object; can access all outer members.
2. **Static nested class** (`class Z`) — static; no outer object needed; can only access outer `static` members.

Problems in the given code:

1. `showNested()` prints `b` and `c` — these belong to `Y` and `Z` respectively and are not directly accessible in `X`.
2. `static class Z` tries to access `a` (non-static member of `X`) — illegal.
3. In `main`: `Y obY = new Y()` — wrong; must be `X.Y` via outer object.
4. In `main`: `Z obZ = new Z()` — wrong; must be `X.Z`.

Fixed main method:

```java
class Test {
    public static void main(String[] args) {
        X obX = new X();
        obX.display();
        obX.showNested();

        X.Y obY = obX.new Y();  // inner class: needs outer object
        obY.display();

        X.Z obZ = new X.Z();    // static nested: no outer object needed
        obZ.display();
    }
}
```

Q3. Outline the errors in the provided code and fix them without removing access modifiers. Then write the minimum code for class `c2` so it compiles:

```java
interface i1 { default void f1(); void f2(); static void f3(); }
interface i2 { void f4(); private void f5(); }
abstract class c1 implements i1 {
    abstract void f6(); final void f7() { }
}
class c2 extends c1 implements i2 { /* your code */ }
```

(10 marks) [CSE 107 | 2022–23]

Answer — Fix Errors in Interfaces & Write c2

Errors and fixes:

1. `interface i1: default void f1();` — A `default` method must have a body. Fix: `default void f1() { }`
2. `interface i1: static void f3();` — A `static` interface method must have a body (Java 8+). Fix: `static void f3() { }`
3. `interface i2: private void f5();` — A `private` interface method must have a body (Java 9+). Fix: `private void f5() { }`

Corrected declarations:

```java
interface i1 {
    default void f1() { }   // fixed: body added
    void f2();
    static void f3() { }    // fixed: body added
}
interface i2 {
    void f4();
    private void f5() { }   // fixed: body added (Java 9+)
}
abstract class c1 implements i1 {
    abstract void f6();
    final void f7() { }
}
```

Minimum code for c2:

`c2` must implement: `f2()` (from `i1`), `f4()` (from `i2`), and `f6()` (abstract in `c1`).

```java
class c2 extends c1 implements i2 {
    @Override public void f2() { }
    @Override public void f4() { }
    @Override void f6() { }
}
```

Q4. 7 methods: `f1()`–`f7()`, 3 interfaces `i1`–`i3`, 1 abstract class `A`. Constraints: (i) each interface ≤ 2 methods; (ii) abstract class `A` can only implement 1 interface and define ≤ 1 abstract method; (iii) `MyClass` cannot implement any interface. Write Java code. **(10 marks)** [CSE 107 | 2022–23, 2020–21]

Answer — 7 Methods, 3 Interfaces, Abstract Class, No Interface for MyClass

Strategy:

- Chain `i1`, `i2`, `i3` so abstract class `A` implements only `i1` (1 interface rule).
- `A` declares at most 1 abstract method.

- `MyClass extends A` — inherits everything, implements no interface directly.

```java
interface i1 { void f1(); void f2(); }
interface i2 extends i1 { void f3(); void f4(); }
interface i3 extends i2 { void f5(); void f6(); void f7(); }

abstract class A implements i3 {
    // implements f1..f6 concretely, declares f7 abstract
    public void f1() { System.out.println("f1"); }
    public void f2() { System.out.println("f2"); }
    public void f3() { System.out.println("f3"); }
    public void f4() { System.out.println("f4"); }
    public void f5() { System.out.println("f5"); }
    public void f6() { System.out.println("f6"); }
    public abstract void f7();   // <= 1 abstract method in A
}

class MyClass extends A {        // implements no interface directly
    public void f7() { System.out.println("f7"); }
}
```

Q5. Write minimum code in class `c2`/`B` for successful compilation. You cannot define it as abstract:

```java
interface i1 { default void f1() { } void f2(); }
interface i2 { void f3(); void f4(); }
abstract class c1 implements i1 {
    abstract void f5();
    final void f6() { }
}
class c2 extends c1 implements i2 { /* your code */ }
```

(6+10 marks) [CSE 107 | 2016–17, 2019–20, 2021–22, Jan 2021]

↪ *Similar: CSE 201 | 2014–15, 2008–09*

Answer — Minimum Code for `c2`

Analysis of what `c2` must provide:

Method	Source	Reason
`f2()`	`i1` (abstract)	not implemented by `c1`
`f5()`	`c1` (abstract)	declared abstract in `c1`
`f3()`	`i2` (abstract)	not yet implemented
`f4()`	`i2` (abstract)	not yet implemented

Note: `f1()` has a `default` body in `i1`, so `c2` need not override it. `f6()` is `final` in `c1` — already implemented, cannot be overridden.

```java
class c2 extends c1 implements i2 {
    @Override public void f2() { }   // from i1 (abstract)
    @Override void f5() { }          // from c1 (abstract)
    @Override public void f3() { }   // from i2
    @Override public void f4() { }   // from i2
}
```

Q6. Explain three uses of the `final` keyword with short examples. Describe the fundamental differences between an interface and an abstract class. **(10 marks)** [CSE 107 | 2021–22]

Answer — `final` Keyword (3 Uses) & Interface vs. Abstract Class

Three uses of `final`:

1. **Final variable** — value cannot be changed after initialisation.

```java
final int MAX = 100;
// MAX = 200; // compile error
```

2. **Final method** — cannot be overridden in a subclass.

```java
class Base {
    final void show() { System.out.println("Base"); }
}
class Child extends Base {
    // void show() { } // compile error
}
```

3. **Final class** — cannot be extended (subclassed).

```java
final class Immutable { }
// class Sub extends Immutable { } // compile error
```

Fundamental differences between Interface and Abstract Class:

Interface	Abstract Class
All fields are `public static final`.	Fields can have any access modifier and be non-static.
A class can implement multiple interfaces.	A class can extend only one abstract class.
No constructors allowed.	Can have constructors.
Represents a *contract* (what to do).	Represents a *partial implementation* (what + how).
Methods are `public` by default.	Methods can have any access modifier.

Q7. What is the problem with the code below? Write two different ways to fix the problem:

```java
interface A { default void f() { System.out.println("A's f"); } }
interface B { default void f() { System.out.println("B's f"); } }
public class C implements A, B { }
```

(7 marks) [CSE 107 | 2016–17, Jan 2021]

Answer — Default Method Conflict: Two Fixes

Problem: Both `A` and `B` provide a `default void f()`, so `C` inherits two conflicting default implementations. The compiler reports an error: *"class C inherits unrelated defaults for f()"*.

Fix 1 — Override `f()` in `C` with its own implementation:

```java
public class C implements A, B {
    @Override
    public void f() {
        System.out.println("C's own f");
    }
}
```

Fix 2 — Override `f()` in `C` and explicitly delegate to one interface:

```java
public class C implements A, B {
    @Override
    public void f() {
        A.super.f();   // explicitly call A's default
    }
}
```

The syntax `InterfaceName.super.method()` is used to disambiguate which interface's default method to call.

Q8. Suppose there are 7 methods: `f1()`–`f7()`, 4 interfaces `i1`–`i4`, and a class `MyClass`. Constraints: (i) each interface defines at most 2 methods; (ii) `MyClass` can only implement 1 interface. Write Java code for `MyClass`. **(10 marks)** [CSE 107 | 2019–20, Jan 2021; CSE 201 | 2014–15, 2008–09]

↪ *Similar: CSE 201 | 2007–08*

Answer — 7 Methods, 4 Interfaces, `MyClass` Implements 1

Strategy: Chain interfaces using `extends` so that `MyClass` only needs to implement **one** interface (`i4`) yet still gets access to all 7 methods.

```java
interface i1 { void f1(); void f2(); }              // 2 methods
interface i2 extends i1 { void f3(); void f4(); }  // 2 new + 2 inherited
interface i3 extends i2 { void f5(); void f6(); }  // 2 new + 4 inherited
interface i4 extends i3 { void f7(); }             // 1 new + 6 inherited

class MyClass implements i4 {
    public void f1() { System.out.println("f1"); }
    public void f2() { System.out.println("f2"); }
    public void f3() { System.out.println("f3"); }
    public void f4() { System.out.println("f4"); }
    public void f5() { System.out.println("f5"); }
    public void f6() { System.out.println("f6"); }
    public void f7() { System.out.println("f7"); }
}
```

`MyClass` implements only `i4` but must provide bodies for all 7 methods because interfaces chain via `extends`.

Q9. Write three uses of `final` keyword with examples. **(6 marks)** [CSE 201 | 2007–08, 2008–09; CSE 107 | 2017–18]

Answer — Three Uses of `final` with Examples

1. **Final variable** — creates a constant.

```
final int MAX_SIZE = 50;  // cannot be reassigned
```

2. **Final method** — prevents method overriding.

```
class A { final void show() { System.out.println("A"); } }
class B extends A { /* void show(){} */ } // not allowed
```

3. **Final class** — prevents inheritance.

```
final class Utility { }
// class Sub extends Utility { } // compile error
```

Q10. What is an inner class? Point out the problem in the following code segment (if any):

```
class Outer {
    int outer_x = 5;
    void test() { Inner inner = new Inner(); inner.display(); }
    class Inner { int z = 60; void display() { System.out.println(outer_x); } }
    void showz() { System.out.println(z); }
}
```

(3+6 marks) [CSE 107 | 2017–18]

Answer — Inner Class & Problem in `Outer/Inner` Code

Inner class: A non-static class defined inside another class. It has access to all members (including `private`) of the outer class and holds an implicit reference to the outer class instance.

Problem in the given code:
`showz()` in `Outer` tries to print `z`, but `z` is a member of `Inner` — not directly accessible from `Outer`'s method.

```
class Outer {
    int outer_x = 5;

    void test() {
        Inner inner = new Inner();
        inner.display();
    }

    class Inner {
        int z = 60;
        void display() {
            System.out.println(outer_x); // OK: inner accesses outer member
        }
    }

    void showz() {
        // System.out.println(z);  // ERROR: z not in scope here
        Inner inner = new Inner();
        System.out.println(inner.z);    // Fix: access via object
    }
}
```

Q11. What is abstract class? Write the restrictions imposed on an abstract class. Point out and fix the problems in the following code:

```
final class X { final void show() { System.out.println("Printed from a method"); } }
class Y extends X { void show() { System.out.println("In subclass method"); } }
```

(3+2+6 marks) [CSE 107 | 2017–18]

Answer — Abstract Class Restrictions & Fix

Restrictions on an abstract class:

1. Cannot be instantiated directly (`new AbstractClass()` is illegal).
2. If a class has even one abstract method, the class must be declared `abstract`.

3. A subclass must implement all abstract methods, or itself be declared `abstract`.
4. Cannot use both `abstract` and `final` on the same class or method.
5. Abstract methods cannot be `private`, `static`, or `final`.

Problems in the given code:

1. `final class X` cannot be extended — `class Y extends X` is a compile error.
2. `final void show()` in `X` cannot be overridden — `Y.show()` is a compile error.

Fix: Remove `final` from the class and the method.

```
class X {                                          // removed final
    void show() {                                  // removed final
        System.out.println("Printed from a method");
    }
}
class Y extends X {
    void show() { System.out.println("In subclass method"); }
}
```

Q12. Consider the following Java code and write minimum code for `myclass`:

```
interface Int1 { void f1(); void f2(); }
interface Int2 extends Int1 { void f3(); }
class MyClass implements Int2 { /* Complete this class */ }
public class Test {
    public static void main(String[] args) {
        MyClass my = new MyClass();
        my.f1(); my.f2(); my.f3();
    }
}
```

(9 marks) [CSE 107 | 2017–18]

Answer — Minimum Code for `MyClass`

Analysis: `Int2 extends Int1`, so `MyClass` must implement `f1()`, `f2()` (from `Int1`) and `f3()` (from `Int2`).

```
interface Int1 { void f1(); void f2(); }
interface Int2 extends Int1 { void f3(); }

class MyClass implements Int2 {
    public void f1() { System.out.println("f1"); }
    public void f2() { System.out.println("f2"); }
    public void f3() { System.out.println("f3"); }
}

public class Test {
    public static void main(String[] args) {
        MyClass my = new MyClass();
        my.f1(); my.f2(); my.f3();
    }
}
```

Output:

```
f1
f2
f3
```

Q13. Write Java code to do the following: (i) Declare a static nested class `Inner1` of `Outer1` and create an object of `Inner1` in `NestedDemo.main()`. (ii) Declare an inner class `Inner2` of `Outer2` and create an object of `Inner2` in `NestedDemo.main()`. **(8 marks)** [CSE 107 | 2016–17]

Answer — Static Nested & Inner Class Object Creation

```
// (i) Static nested class
class Outer1 {
    static class Inner1 {
        void display() { System.out.println("Inner1 (static nested)"); }
    }
}
```

```java

// (ii) Inner class (non-static)
class Outer2 {
    class Inner2 {
        void display() { System.out.println("Inner2 (inner class)"); }
    }
}

public class NestedDemo {
    public static void main(String[] args) {

        // (i) Static nested: no outer object needed
        Outer1.Inner1 obj1 = new Outer1.Inner1();
        obj1.display();

        // (ii) Inner class: outer object required first
        Outer2 outer2 = new Outer2();
        Outer2.Inner2 obj2 = outer2.new Inner2();
        obj2.display();
    }
}
```

Output:

```
Inner1 (static nested)
Inner2 (inner class)
```

Q14. What are the two uses of the `super` keyword? What is an abstract class? What are the restrictions? **(10 marks)** [CSE 107 | 2016–17]

Answer — Two Uses of `super` & Abstract Class

Two uses of the `super` keyword:

1. **Call superclass constructor** — must be the first statement in the subclass constructor.

```java
class Animal { Animal(String name) { System.out.println(name); } }
class Dog extends Animal {
    Dog() { super("Dog"); }  // calls Animal's constructor
}
```

2. **Access superclass method/variable** — when overridden in the subclass.

```java
class A { void show() { System.out.println("A"); } }
class B extends A {
    void show() {
        super.show();                    // calls A's show()
        System.out.println("B");
    }
}
```

Abstract class: A class declared with the `abstract` keyword that may contain abstract (bodyless) methods.
Restrictions:

- Cannot be instantiated.
- Subclass must implement all abstract methods or be abstract itself.
- Abstract methods cannot be `private`, `static`, or `final`.
- A class with an abstract method must be declared abstract.

Q15. When we say "a class is final" or "a method is final" — what does it mean? Show example codes. **(5 marks)** [CSE 107 | 2015–16]

Answer — Meaning of `final` Class and `final` Method

Final class: A class declared `final` cannot be subclassed. Any attempt to extend it causes a compile-time error.

```java
final class Lock { }
// class SpecialLock extends Lock { }  // compile error
```

Final method: A method declared `final` cannot be overridden in a subclass, but it *can* be inherited and called.

```java
class Parent {
```

```
    final void display() { System.out.println("Parent display"); }
}
class Child extends Parent {
    // void display() { }  // compile error: cannot override final method
}
public class Main {
    public static void main(String[] args) {
        Child c = new Child();
        c.display();   // inherited fine; Output: Parent display
    }
}
```

Common use of `final` class: `java.lang.String` is a final class — it cannot be subclassed to prevent modification of its behaviour.

Q16. What is a static code block? When is it executed? Can there be multiple static code blocks? In what sequence? Show with example. **(10 marks)** [CSE 107 | 2015–16]

Answer — Static Code Block

Static code block: A block of code preceded by the `static` keyword, declared inside a class but outside any method. It is used to initialise static variables or perform one-time class-level setup.
When executed: Once, when the class is *loaded* by the JVM (before any object is created or any static method is called).
Multiple static blocks: Allowed. They execute in the **order they appear** in the source file.

```
class Demo {
    static int x;
    static int y;

    static {
        x = 10;
        System.out.println("First static block: x = " + x);
    }

    static {
        y = 20;
        System.out.println("Second static block: y = " + y);
    }

    Demo() { System.out.println("Constructor called"); }
}

public class Main {
    public static void main(String[] args) {
        System.out.println("main() started");
        Demo d1 = new Demo();
        Demo d2 = new Demo();
    }
}
```

Output:

```
First static block: x = 10
Second static block: y = 20
main() started
Constructor called
Constructor called
```

The static blocks ran once before `main()`, in top-to-bottom order. The constructor ran twice (once per object).

Q17. What are the three uses of Java `final` keyword? **(5 marks)** [CSE 201 | 2014–15]

Answer — Three Uses of the `final` Keyword

1. **Final variable** — acts as a constant; cannot be reassigned after initialisation.

```
final int SPEED_LIMIT = 80;
```

2. **Final method** — cannot be overridden by any subclass.

```
class Vehicle {
    final void fuelType() { System.out.println("Petrol"); }
}
```

3. **Final class** — cannot be extended; prevents inheritance.

```
final class SecureToken { }
```

Q18. Write an interface and a class that implements it. Show how objects can be created using an interface. **(10 marks)** [CSE 201 | 2012–13]

Answer — Interface, Implementing Class & Object Creation

```java
// Interface declaration
interface Shape {
    double area();
    double perimeter();
}

// Class implementing the interface
class Circle implements Shape {
    double radius;

    Circle(double r) { radius = r; }

    @Override
    public double area() { return Math.PI * radius * radius; }

    @Override
    public double perimeter() { return 2 * Math.PI * radius; }
}

// Main class
public class Main {
    public static void main(String[] args) {

        // Method 1: object via concrete class reference
        Circle c = new Circle(5.0);
        System.out.println("Area: " + c.area());

        // Method 2: object via interface reference (polymorphism)
        Shape s = new Circle(7.0);
        System.out.println("Perimeter: " + s.perimeter());
    }
}
```

Key point: An interface reference (`Shape s`) can hold any object whose class implements `Shape`. Only the methods declared in the interface are accessible through `s`.

Q19. Write the differences between abstract class and interface. **(5 marks)** [CSE 201 | 2012–13]

Answer — Abstract Class vs. Interface

Abstract Class	Interface
Declared with `abstract class`.	Declared with `interface`.
Can have both abstract and concrete methods.	Methods are implicitly abstract (Java 7 and earlier); `default`/`static` methods allowed from Java 8.
Can have instance variables (any access modifier).	Variables are implicitly `public static final` (constants).
A class can extend only **one** abstract class.	A class can implement **multiple** interfaces.
Can have constructors.	Cannot have constructors.
Used when classes share common state/behaviour.	Used to define a contract with no shared state.

Q20. Write the use of `abstract` and `final` keywords in Java with examples. **(7 marks)** [CSE 201 | 2011–12]

Answer — `abstract` and `final` Keywords in Java

`abstract` keyword:

1. **Abstract class** — cannot be instantiated; may contain abstract and concrete methods.

```java
abstract class Animal {
    abstract void sound();          // must be overridden
    void breathe() { System.out.println("Breathing"); }
}
```

2. **Abstract method** — has no body; subclass must implement it.

```java
class Dog extends Animal {
    void sound() { System.out.println("Woof"); }
}
```

`final` keyword:

1. **Final variable** — constant; value fixed after assignment.
   `final double PI = 3.14159;`
2. **Final method** — cannot be overridden in subclass.
   `final void show() { }`
3. **Final class** — cannot be subclassed.
   `final class MyClass { }`

Note: A method cannot be both `abstract` and `final` — `abstract` requires overriding while `final` prevents it.

Q21. What are the differences between static nested class and inner class? **(5 marks)** [CSE 201 | 2010–11]

Answer — Static Nested Class vs. Inner Class

Static Nested Class	Inner Class
Declared with `static` keyword.	Declared without `static`.
Does not hold a reference to outer class instance.	Holds an implicit reference to outer class instance.
Can only access `static` members of outer class.	Can access all members (including `private`) of outer class.

Instantiated without outer object:
`new Outer.Nested()`
`outer.new Inner()`
	Requires outer object:

Can have `static` members of its own.	Cannot have `static` members (except constants).

Q22. Do you agree or disagree: "All interfaces are abstract classes but not all abstract classes are interfaces"? Give valid reasoning. **(4 marks)** [CSE 201 | 2010–11]

Answer — "All interfaces are abstract classes but not all abstract classes are interfaces"

Verdict: Disagree.
Conceptually, an interface is *similar* to a fully abstract class, but in Java they are **distinct constructs**:

1. An interface is *not* an abstract class in Java's type system. `interface` and `abstract class` are separate keywords.
2. An abstract class can have instance variables, constructors, and concrete methods — an interface (before Java 8) had none of these.
3. A class can implement multiple interfaces but extend only one abstract class.
4. Technically, the JVM does treat interfaces as a special kind of abstract type, but the statement "all interfaces *are* abstract classes" is misleading because the two serve different design purposes.

Correct statement: Every interface is *implicitly abstract*, but it is *not* an abstract class — it is its own independent type.

Q23. What are the two types of nested classes in Java? Briefly describe them. What are the rules regarding access of nested class and outer class? **(6 marks)** [CSE 201 | 2007–08]

Answer — Types of Nested Classes & Access Rules

Two types of nested classes:

1. **Static nested class** — declared with the `static` keyword inside an outer class. It does *not* hold a reference to an outer class instance.
2. **Inner class (non-static nested class)** — declared without `static`. It holds an implicit reference to the enclosing outer class instance.

Access rules:

Inner class	Static nested class
Can access all members (including `private`) of the outer class.	Can only access `static` members of the outer class.
Outer class can access all members (including `private`) of the inner class.	Outer class can access all members of the static nested class.
Requires outer class object to instantiate: `Outer.Inner obj = outer.new Inner();`	Can be instantiated without outer object: `Outer.Nested obj = new Outer.Nested();`

BUET · CSE 107 / CSE 201

Object Oriented Programming

Section S13

Exception Handling

S13 — Exception Handling

Q1. What is the problem in the following C++ program? Rewrite using exception handling:

```cpp
#include <iostream>
using namespace std;
int divide() {
    int d = 0;
    int a = 10 / d;
    return a;
}
int main() { cout << divide() << endl; return 0; }
```

(2+5 marks) [CSE 107 | 2023–24]

Answer — Division by Zero: Problem and Fix with Exception Handling

Problem: `d = 0` causes **division by zero**, which is undefined behaviour in C++. The program may crash or produce garbage output with no graceful error message.

Fixed version using exception handling:

```cpp
#include <iostream>
#include <stdexcept>
using namespace std;

int divide() {
    int d = 0;
    if (d == 0)
        throw runtime_error("Division by zero!");
    int a = 10 / d;
    return a;
}

int main() {
    try {
        cout << divide() << endl;
    } catch (const runtime_error& e) {
        cerr << "Caught exception: " << e.what() << endl;
    }
    return 0;
}
```

Output:

```
Caught exception: Division by zero!
```

Q2. Consider a class `BankAccount`. Write: (i) `InvalidAmountException`; (ii) `InsufficientBalanceException`; (iii) complete `BankAccount`/`Account` class: **(15 marks)** [CSE 107 | 2022–23]

Answer — BankAccount with InvalidAmount and InsufficientBalance Exceptions

```cpp
class InvalidAmountException extends Exception {
    InvalidAmountException(double amt) {
        super("Amount must be positive. Got: " + amt);
    }
}

class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(double needed, double available) {
        super("Need " + needed + " but only " + available + " available");
    }
}

class BankAccount {
    private String owner;
    private double balance;

    BankAccount(String owner, double initialBalance)
            throws InvalidAmountException {
        if (initialBalance < 0)
            throw new InvalidAmountException(initialBalance);
        this.owner   = owner;
        this.balance = initialBalance;
    }

    void deposit(double amt) throws InvalidAmountException {
```

```
        if (amt <= 0) throw new InvalidAmountException(amt);
        balance += amt;
    }

    void withdraw(double amt)
            throws InvalidAmountException, InsufficientBalanceException {
        if (amt <= 0) throw new InvalidAmountException(amt);
        if (amt > balance)
            throw new InsufficientBalanceException(amt, balance);
        balance -= amt;
    }

    double getBalance() { return balance; }

    public static void main(String[] args) {
        try {
            BankAccount acc = new BankAccount("Alice", 500);
            acc.deposit(200);
            System.out.println("Balance: " + acc.getBalance()); // 700
            acc.withdraw(900); // throws InsufficientBalanceException
        } catch (InvalidAmountException e) {
            System.out.println("Invalid: " + e.getMessage());
        } catch (InsufficientBalanceException e) {
            System.out.println("Insufficient: " + e.getMessage());
        }
    }
}
```

Q3. Write the name of the five keywords in Java dedicated to exception handling. **(15 marks)**

Consider a class named `BankAccount` with the following information:

- `double balance` – the balance of the account
- `void init(double amount)` – this method sets the amount as the initial balance
- `void debit(double amount)` – this method subtracts the amount from the balance
- `void credit(double amount)` – this method adds the amount directly to the balance

The amount provided in the above methods can't be negative. The balance of the account can't also be negative.

Write Java code for the following:

(a) A custom exception (`InvalidAmountException`) that triggers when the given amount is negative and shows the message 'The given amount can't be negative' and the given amount.

(b) A custom exception (`InsufficientBalanceException`) that triggers when the balance goes negative and shows the message 'The account balance can't be negative' and the current balance.

(c) The complete `Account` (or `BankAccount`) class to trigger these exceptions when needed.

You don't need to write any main method. **[CSE 107 | 2016–17, 2022–23]**

Answer — BankAccount Class with Custom Exceptions

Five keywords for exception handling in Java: `try`, `catch`, `finally`, `throw`, and `throws`.

```
// i. Custom exception for negative amounts
class InvalidAmountException extends Exception {
    public InvalidAmountException(double amount) {
        super("The given amount can't be negative: " + amount);
    }
}

// ii. Custom exception for negative balance
class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(double currentBalance) {
        super("The account balance can't be negative: " + currentBalance);
    }
}

// iii. The complete BankAccount class
class BankAccount {
    private double balance;

    void init(double amount) throws InvalidAmountException {
        if (amount < 0) {
            throw new InvalidAmountException(amount);
        }
```

```
        this.balance = amount;
    }

    void debit(double amount) throws InvalidAmountException, InsufficientBalanceException {
        if (amount < 0) {
            throw new InvalidAmountException(amount);
        }
        if (this.balance - amount < 0) {
            throw new InsufficientBalanceException(this.balance - amount);
        }
        this.balance -= amount;
    }

    void credit(double amount) throws InvalidAmountException {
        if (amount < 0) {
            throw new InvalidAmountException(amount);
        }
        this.balance += amount;
    }
}
```

Q4. Consider a class named Inventory with the following information: **(10)**

- `double stock` - the stock of the inventory
- `double minStock` - the minimum stock of the inventory
- `void setInitialStock(double amount)` - this method sets the amount as the initial stock
- `void addToInventory(double amount)` - this method adds the amount to the stock
- `void removeFromInventory(double amount)` - this method subtracts the amount from the stock

The amount provided in the above four methods can't be negative. The stock of the inventory can't also be below the minimum stock.

Write Java code for the following:

(i) A custom exception named `InvalidStockAmountException` with appropriate parameters that triggers when the given amount is negative and shows the message 'The given stock amount can't be negative'.

(ii) A custom exception named `InvalidInventoryStockException` with appropriate parameters that triggers when the stock goes below the minimum stock and shows the message 'The stock can't be less than the minimum stock'.

(iii) The complete `Inventory` class to trigger these exceptions when needed. You don't need to write any main method.

[CSE 107 | 2020–21]

Answer — Inventory Class with Custom Exceptions

```
// (i) Custom exception for negative stock amounts
class InvalidStockAmountException extends Exception {
    public InvalidStockAmountException() {
        super("The given stock amount can't be negative.");
    }
}

// (ii) Custom exception for stock dropping below minimum limits
class InvalidInventoryStockException extends Exception {
    public InvalidInventoryStockException() {
        super("The stock can't be less than the minimum stock.");
    }
}

// (iii) The complete Inventory class matching specified requirements
class Inventory {
    private double stock;
    private double minStock;

    // Constructor to set fields
    public Inventory(double minStock) {
        this.minStock = minStock;
        this.stock = 0.0;
    }

    public void setInitialStock(double amount) throws InvalidStockAmountException,
       InvalidInventoryStockException {
        if (amount < 0) {
            throw new InvalidStockAmountException();
```

```
        }
        if (amount < minStock) {
            throw new InvalidInventoryStockException();
        }
        this.stock = amount;
    }

    public void addToInventory(double amount) throws InvalidStockAmountException {
        if (amount < 0) {
            throw new InvalidStockAmountException();
        }
        this.stock += amount;
    }

    public void removeFromInventory(double amount) throws InvalidStockAmountException,
       InvalidInventoryStockException {
        if (amount < 0) {
            throw new InvalidStockAmountException();
        }
        if (this.stock - amount < minStock) {
            throw new InvalidInventoryStockException();
        }
        this.stock -= amount;
    }

    // Getters for inspecting the states
    public double getStock() {
        return stock;
    }

    public double getMinStock() {
        return minStock;
    }
}
```

Q5. Explain the use of `try` and `catch` in C++ with appropriate examples. **(10 marks)** [CSE 107 | 2015–16]

Answer — try and catch in C++

try block: Encloses code that might throw an exception. **catch block:** Handles a specific type of exception thrown from the try block.

```
#include <iostream>
#include <stdexcept>
using namespace std;

double safeDivide(double a, double b) {
    if (b == 0)
        throw runtime_error("Cannot divide by zero");
    return a / b;
}

int main() {
    // Example 1: catching a runtime_error
    try {
        cout << safeDivide(10, 2) << endl;  // 5
        cout << safeDivide(10, 0) << endl;  // throws
    } catch (const runtime_error& e) {
        cout << "runtime_error: " << e.what() << endl;
    }

    // Example 2: multiple catch blocks
    try {
        int choice = 2;
        if (choice == 1) throw 42;           // throw int
        if (choice == 2) throw string("err"); // throw string
    } catch (int i) {
        cout << "Caught int: " << i << endl;
    } catch (const string& s) {
        cout << "Caught string: " << s << endl;
    } catch (...) {
        cout << "Caught unknown exception" << endl;
    }
```

```
32
33      return 0;
34  }
```

Output:

```
5
runtime_error: Cannot divide by zero
Caught string: err
```

Q6. Show the hierarchical relation among `Throwable`, `Exception`, `RuntimeException` and `Error` classes. Write a custom exception `AuthFailureException`. Also write an `Authenticate()` method that will throw `AuthFailureException` on failure. **(10 marks)** [CSE 107 | 2015–16]

Answer — Java Exception Hierarchy and Custom Exception Handling

Hierarchical Relation:
In Java, the exception handling framework is structured as a class hierarchy rooted at `java.lang.Object`. The four core classes are related as shown below:

```
java.lang.Object
   |
   +-- java.lang.Throwable
         |
         +-- java.lang.Error (Unchecked/Fatal JVM errors)
         |     |-- OutOfMemoryError
         |     +-- StackOverflowError
         |
         +-- java.lang.Exception (Checked/Application exceptions)
               |-- IOException
               |-- SQLException
               |
               +-- java.lang.RuntimeException (Unchecked/Logic errors)
                     |-- NullPointerException
                     +-- ArrayIndexOutOfBoundsException
```

- **`Throwable`:** The ultimate superclass of all errors and exceptions in the Java language. Only objects that are instances of this class (or one of its subclasses) can be thrown by the JVM or via the `throw` statement.
- **`Error`:** A subclass of `Throwable` that indicates serious, unrecoverable problems that a reasonable application should not try to catch (e.g., system crashes or resource exhaustion).
- **`Exception`:** A subclass of `Throwable` that indicates conditions that a reasonable application might want to catch and handle. These are typically compile-time "checked" exceptions.
- **`RuntimeException`:** A major subclass of `Exception` that represents "unchecked" exceptions or programming flaws (such as null pointer dereferences or array index violations) occurring at runtime.

Java Implementation:
Below is the implementation of the custom exception `AuthFailureException` and the required `Authenticate()` validation method.

```
// Custom checked exception extending the base Exception class
class AuthFailureException extends Exception {
    // Constructor that accepts a detailed error message
    public AuthFailureException(String message) {
        super(message);
    }
}

class UserAuthenticator {
    // Authenticate method that validates credentials and declares throwing the custom exception
    public void Authenticate(String username, String password) throws AuthFailureException {
        // Simple mock credentials check
        String correctUsername = "admin";
        String correctPassword = "password123";

        if (!correctUsername.equals(username) || !correctPassword.equals(password)) {
            // Throwing the custom exception on failure
            throw new AuthFailureException("Authentication failed: Invalid credentials for user '"
                + username + "'.");
        }

        System.out.println("Authentication successful for user: " + username);
    }
}
```

Q7. Write your own `TwoDimException` that is thrown when the determinant of a matrix is zero. **(10 marks)** [CSE 201 | 2015–16]

Answer — TwoDimException for Zero Determinant

```java
class TwoDimException extends Exception {
    TwoDimException() {
        super("Matrix is singular: determinant is zero.");
    }
}

class Matrix2x2 {
    private double[][] m;

    Matrix2x2(double a, double b, double c, double d) {
        m = new double[][]{{a, b}, {c, d}};
    }

    double determinant() {
        return m[0][0] * m[1][1] - m[0][1] * m[1][0];
    }

    // Returns inverse; throws TwoDimException if det == 0
    Matrix2x2 inverse() throws TwoDimException {
        double det = determinant();
        if (det == 0) throw new TwoDimException();
        return new Matrix2x2(
             m[1][1] / det, -m[0][1] / det,
            -m[1][0] / det,  m[0][0] / det);
    }

    @Override
    public String toString() {
        return String.format("[[%.2f, %.2f], [%.2f, %.2f]]",
                             m[0][0], m[0][1], m[1][0], m[1][1]);
    }

    public static void main(String[] args) {
        Matrix2x2 m1 = new Matrix2x2(2, 1, 5, 3);
        Matrix2x2 m2 = new Matrix2x2(1, 2, 2, 4); // det = 0

        try {
            System.out.println("Inverse of m1: " + m1.inverse());
            System.out.println("Inverse of m2: " + m2.inverse());
        } catch (TwoDimException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Output:

```
Inverse of m1: [[3.00, -1.00], [-5.00, 2.00]]
Error: Matrix is singular: determinant is zero.
```

Q8. Consider a `MyStack` class with `push(Object)`, `pop()`, `top()`, `isEmpty()`. `pop()` and `top()` throw `StackEmptyException`; `push()` throws `StackFullException`. Write the custom exceptions and the `MyStack` class. **(10 marks)** [CSE 201 | 2014–15]

Answer — MyStack with StackEmptyException and StackFullException

```java
class StackEmptyException extends Exception {
    StackEmptyException() { super("Stack is empty"); }
}

class StackFullException extends Exception {
    StackFullException() { super("Stack is full"); }
}

class MyStack {
    private Object[] data;
    private int top;
    private int capacity;

    MyStack(int capacity) {
        this.capacity = capacity;
```

```
        data = new Object[capacity];
        top  = -1;
    }

    boolean isEmpty() { return top == -1; }
    boolean isFull()  { return top == capacity - 1; }

    void push(Object obj) throws StackFullException {
        if (isFull()) throw new StackFullException();
        data[++top] = obj;
    }

    Object pop() throws StackEmptyException {
        if (isEmpty()) throw new StackEmptyException();
        return data[top--];
    }

    Object top() throws StackEmptyException {
        if (isEmpty()) throw new StackEmptyException();
        return data[top];
    }

    public static void main(String[] args) {
        MyStack s = new MyStack(3);
        try {
            s.push("A"); s.push("B"); s.push("C");
            System.out.println("Top: " + s.top());   // C
            System.out.println("Pop: " + s.pop());   // C
            s.push("D");
            s.push("E"); // StackFullException
        } catch (StackFullException e) {
            System.out.println("Full: " + e.getMessage());
        } catch (StackEmptyException e) {
            System.out.println("Empty: " + e.getMessage());
        }
    }
}
```

Q9. Write a program that throws and catches an exception when the value of a variable is greater than 1000. **(10 marks)** [CSE 201 | 2012–13]

Answer — Throw and Catch When Value > 1000

```
class ValueTooLargeException extends Exception {
    ValueTooLargeException(int val) {
        super("Value " + val + " exceeds 1000");
    }
}

class ThrowDemo {
    static void check(int val) throws ValueTooLargeException {
        if (val > 1000)
            throw new ValueTooLargeException(val);
        System.out.println("Value OK: " + val);
    }

    public static void main(String[] args) {
        int[] tests = {500, 1001, 999, 2000};
        for (int v : tests) {
            try {
                check(v);
            } catch (ValueTooLargeException e) {
                System.out.println("Exception: " + e.getMessage());
            }
        }
    }
}
```

Output:

```
Value OK: 500
Exception: Value 1001 exceeds 1000
Value OK: 999
```

```
Exception: Value 2000 exceeds 1000
```

Q10. Write a program that illustrates re-throwing an exception. **(4 marks)** [CSE 201 | 2011–12]

Answer — Re-throwing an Exception

```java
class RethrowDemo {
    static void level2() throws Exception {
        throw new Exception("Original exception from level2");
    }

    static void level1() throws Exception {
        try {
            level2();
        } catch (Exception e) {
            System.out.println("level1 caught: " + e.getMessage());
            System.out.println("level1 re-throwing...");
            throw e;   // re-throw the same exception
        }
    }

    public static void main(String[] args) {
        try {
            level1();
        } catch (Exception e) {
            System.out.println("main caught: " + e.getMessage());
        }
    }
}
```

Output:

```
level1 caught: Original exception from level2
level1 re-throwing...
main caught: Original exception from level2
```

Re-throwing allows a method to partially handle an exception (e.g., log it) and then pass it up to a higher-level handler.

Q11. What are the keywords associated with exception handling in Java? Write the use of each keyword. **(10 marks)** [CSE 201 | 2011–12]

Answer — Java Exception Handling Keywords

Java has **five** keywords for exception handling:

Keyword	Use
`try`	Wraps code that may throw an exception
`catch`	Handles a specific exception type
`finally`	Runs always, whether or not an exception occurred
`throw`	Explicitly throws an exception object
`throws`	Declares which checked exceptions a method may propagate

```java
class KeywordsDemo {
    // 'throws' declares checked exception
    static int divide(int a, int b) throws ArithmeticException {
        if (b == 0)
            throw new ArithmeticException("div by zero"); // 'throw'
        return a / b;
    }

    public static void main(String[] args) {
        try {                           // 'try'
            System.out.println(divide(10, 2));
            System.out.println(divide(10, 0));
        } catch (ArithmeticException e) {  // 'catch'
            System.out.println("Caught: " + e.getMessage());
        } finally {                     // 'finally'
            System.out.println("Finally block always runs");
        }
    }
}
```

Output:

```
5
Caught: div by zero
Finally block always runs
```

Q12. In JVM, when is a `ClassCastException` thrown? How can we avoid it? **(5+2 marks)** [CSE 201 | 2010–11]

Answer — ClassCastException: When and How to Avoid

When is it thrown?

A `ClassCastException` is thrown at run time when the JVM attempts to cast an object to a type it is *not* an instance of.

```
Object obj = "Hello";       // obj actually holds a String
Integer i  = (Integer) obj; // ClassCastException at run time!
```

How to avoid it:

Use the `instanceof` operator to check the actual type before casting.

```
Object obj = "Hello";

if (obj instanceof Integer) {
    Integer i = (Integer) obj;
    System.out.println("Integer: " + i);
} else {
    System.out.println("Not an Integer: " + obj.getClass().getName());
}
// Output: Not an Integer: java.lang.String
```

Using generics also prevents `ClassCastException` at compile time by enforcing type safety without explicit casts.

Q13. Explain three different usages of the keyword `finally` with examples. **(9 marks)** [CSE 201 | 2010–11]

Answer — Three Usages of `finally`

Usage 1 — Resource cleanup (always close resources):

```
FileReader fr = null;
try {
    fr = new FileReader("file.txt");
    // read file ...
} catch (IOException e) {
    System.out.println("IO Error: " + e.getMessage());
} finally {
    if (fr != null) try { fr.close(); } catch (IOException ignored) {}
    System.out.println("FileReader closed.");
}
```

Usage 2 — Runs even when no exception occurs:

```
try {
    System.out.println("No exception here");
} catch (Exception e) {
    System.out.println("This won't run");
} finally {
    System.out.println("Finally runs anyway");  // always executes
}
```

Usage 3 — Runs even after a return statement in try:

```
static int getValue() {
    try {
        return 10;             // return is deferred
    } finally {
        System.out.println("finally before actual return");
    }
}
// Output: "finally before actual return", then returns 10
```

Q14. Write down the syntax used for Java's 'catch or declare' strategy of exception handling. **(7 marks)** [CSE 201 | 2010–11]

Answer — Catch or Declare Strategy

Java's **"catch or declare"** rule applies to *checked* exceptions: any method that may throw a checked exception must either **(a) catch** it locally or **(b) declare** it with `throws`.

Option A — Catch it (handle locally):

```
void readFile(String path) {
```

```
    try {
        FileReader fr = new FileReader(path);
        // ... use fr ...
    } catch (IOException e) {
        System.out.println("Handled: " + e.getMessage());
    }
}
```

Option B — Declare it (propagate to caller):

```
void readFile(String path) throws IOException {
    FileReader fr = new FileReader(path); // not caught here
    // ... use fr ...
}

// Caller must then catch or declare IOException
void caller() throws IOException {
    readFile("data.txt");
}
```

Unchecked exceptions (`RuntimeException` and its subclasses) are *exempt* from this rule — they may be thrown without declaration.

Q15. How can we create a user-defined exception type? Give an example. **(4 marks)** [CSE 201 | 2010–11]

Answer — Creating a User-Defined Exception

Extend `Exception` (for checked) or `RuntimeException` (for unchecked) and provide a constructor that passes a message to `super()`.

```
// User-defined checked exception
class AgeOutOfRangeException extends Exception {
    AgeOutOfRangeException(int age) {
        super("Age " + age + " is out of valid range (0-150)");
    }
}

class Person {
    private int age;

    void setAge(int age) throws AgeOutOfRangeException {
        if (age < 0 || age > 150)
            throw new AgeOutOfRangeException(age);
        this.age = age;
    }

    public static void main(String[] args) {
        Person p = new Person();
        try {
            p.setAge(25);   // OK
            p.setAge(200);  // throws
        } catch (AgeOutOfRangeException e) {
            System.out.println("Caught: " + e.getMessage());
        }
    }
}
```

Output:

```
Caught: Age 200 is out of valid range (0-150)
```

Q16. Write a program that illustrates the application of multiple `catch` statements. **(5 marks)** [CSE 201 | 2007–08]

Answer — Multiple catch Statements

```
class MultiCatchDemo {
    static void test(int choice) {
        try {
            if (choice == 1)
                throw new ArithmeticException("Arithmetic error");
            else if (choice == 2)
                throw new ArrayIndexOutOfBoundsException("Bad index");
            else if (choice == 3)
                throw new NullPointerException("Null ref");
            else {
                int[] arr = new int[3];
                arr[5] = 10;  // causes ArrayIndexOutOfBoundsException
```

```
            }
        } catch (ArithmeticException e) {
            System.out.println("Caught ArithmeticException: "
                               + e.getMessage());
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Caught ArrayIndexOutOfBoundsException: "
                               + e.getMessage());
        } catch (NullPointerException e) {
            System.out.println("Caught NullPointerException: "
                               + e.getMessage());
        } catch (Exception e) {
            System.out.println("Caught general Exception: "
                               + e.getMessage());
        }
    }

    public static void main(String[] args) {
        test(1); test(2); test(3); test(4);
    }
}
```

Output:

```
Caught ArithmeticException: Arithmetic error
Caught ArrayIndexOutOfBoundsException: Bad index
Caught NullPointerException: Null ref
Caught ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 3
```

Q17. Suppose you are asked to write `MyException` that takes a String parameter. When displayed, it shows the String. Write necessary code. **(8 marks)** [CSE 201 | 2007–08]

Answer — MyException with String Parameter

```
class MyException extends Exception {
    private String message;

    MyException(String msg) {
        super(msg);          // pass to Exception
        this.message = msg;
    }

    @Override
    public String toString() {
        return "MyException: " + message;
    }
}

class MyExceptionDemo {
    static void riskyMethod(int x) throws MyException {
        if (x < 0)
            throw new MyException("Negative value not allowed: " + x);
        System.out.println("Value is: " + x);
    }

    public static void main(String[] args) {
        try {
            riskyMethod(5);
            riskyMethod(-3);
        } catch (MyException e) {
            System.out.println(e);           // calls toString()
            System.out.println(e.getMessage()); // from super
        }
    }
}
```

Output:

```
Value is: 5
MyException: Negative value not allowed: -3
Negative value not allowed: -3
```

BUET · CSE 107 / CSE 201

Object Oriented Programming

Section S14

Multithreading & Concurrency

S14 — Multithreading & Concurrency

Q1. Complete the following Java code that finds the summation of a 10000-element integer array using an array of 10 threads in parallel. You cannot write any more classes, but you can add variables, constructors, and other methods. **(15 marks)**

```
class ParallelSum implements Runnable {
    // your code here
}

public class Main {
    public static void main(String[] args) {
        java.util.Random random = new java.util.Random();
        int[] numbers = new int[10000];

        for (int i = 0; i < numbers.length; i++) {
            numbers[i] = random.nextInt(10000);
        }

        ParallelSum[] parallelSum = new ParallelSum[10];
        // your code here
    }
}
```

[CSE 107 | 2023–24]

Answer — Parallel Sum of 10,000-Element Array (10 Threads)

To achieve parallel summation, each `ParallelSum` instance should be responsible for a subset (chunk) of the array. After the threads finish executing (`join`), the main thread aggregates the partial sums.

```
class ParallelSum implements Runnable {
    private int[] numbers;
    private int start;
    private int end;
    private long partialSum;

    // Constructor to assign the array and the bounds for this thread
    public ParallelSum(int[] numbers, int start, int end) {
        this.numbers = numbers;
        this.start = start;
        this.end = end;
        this.partialSum = 0;
    }

    @Override
    public void run() {
        for (int i = start; i < end; i++) {
            partialSum += numbers[i];
        }
    }

    // Method to retrieve the calculated sum for this chunk
    public long getPartialSum() {
        return partialSum;
    }
}

public class Main {
    public static void main(String[] args) {
        java.util.Random random = new java.util.Random();
        int[] numbers = new int[10000];

        for (int i = 0; i < numbers.length; i++) {
            numbers[i] = random.nextInt(10000);
        }

        ParallelSum[] parallelSum = new ParallelSum[10];
        Thread[] threads = new Thread[10];

        int chunkSize = numbers.length / 10; // 1000 elements per thread

        // 1. Initialize Runnables and start Threads
        for (int i = 0; i < 10; i++) {
            int start = i * chunkSize;
            int end = start + chunkSize;

```

```
            parallelSum[i] = new ParallelSum(numbers, start, end);
            threads[i] = new Thread(parallelSum[i]);
            threads[i].start();
        }

        long totalSum = 0;

        // 2. Wait for threads to finish and aggregate results
        for (int i = 0; i < 10; i++) {
            try {
                threads[i].join(); // Ensure the thread has finished
                totalSum += parallelSum[i].getPartialSum();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }

        System.out.println("Total sum = " + totalSum);
    }
}
```

Q2. Consider the following code. Determine all possible outputs assuming T1 starts first. If two threads execute f0 and f1, will they run in parallel? Why or why not?

```
class TestClass {
    public void f0() { synchronized(this) { for(int i=0;i<5;i++) System.out.println(Thread.
       currentThread().getName()+" "+i); } }
    public void f1() { synchronized(TestClass.class) { for(int i=0;i<5;i++) System.out.println(
       Thread.currentThread().getName()+" "+i); } }
    public synchronized static void f2() { for(int i=0;i<5;i++) System.out.println(Thread.
       currentThread().getName()+" "+i); }
    public synchronized static void f3() { for(int i=0;i<5;i++) System.out.println(Thread.
       currentThread().getName()+" "+i); }
}
class SynchronizedTest {
    public static void main(String[] args) {
        new Thread(new TestClass()::f1, "T1").start();
        new Thread(TestClass::f2, "T2").start();
        new Thread(TestClass::f3, "T3").start();
    }
}
```

(10 marks) [CSE 107 | 2023–24]

Answer — Lock Analysis: `f0`, `f1`, `f2`, `f3`

Lock objects used by each method:

Method	Lock object	Type
`f0()`	`this` (instance)	instance lock
`f1()`	`TestClass.class`	class lock
`f2()`	`TestClass.class` (static sync)	class lock
`f3()`	`TestClass.class` (static sync)	class lock

The three threads in `main`:

- **T1** calls `f1()` — acquires the **class lock** (`TestClass.class`).
- **T2** calls **static** `f2()` — also acquires the **class lock**.
- **T3** calls **static** `f3()` — also acquires the **class lock**.

All three threads compete for the *same* lock (`TestClass.class`), so **they cannot run in parallel**. They execute one at a time in some order.

Possible outputs (T1 starts first; T2 and T3 may interleave after T1 finishes — all orderings of T2 and T3 are valid):

(i) T1 runs completely, then T2, then T3.

(ii) T1 runs completely, then T3, then T2.

Within each thread's turn the output is always its 5 lines in order (`T1 0` …`T1 4`, etc.). No interleaving of individual lines between threads is possible because each holds the class lock for its entire loop.

Would `f0` and `f1` run in parallel?
Yes — if two threads called `f0` and `f1` on the *same* instance, they would still run in parallel because `f0` locks on `this` (the *instance* lock) while `f1` locks on `TestClass.class` (the *class* lock). These are two *different* objects, so neither thread blocks the other.

Q3. What is the problem with the following code? Write 2 different ways to fix it:

```
class MyThread implements Runnable {
    List<String> list;
    Thread t;
    public MyThread(List<String> list) { this.list = list; t = new Thread(this); t.start(); }
    public void run() { for(int i=1;i<=10000;i++) list.add(String.valueOf(i)); }
}
public class TestSyncArrayList {
    public static void main(String[] args) throws Exception {
        List<String> list = new ArrayList<>();
        MyThread t1 = new MyThread(list), t2 = new MyThread(list);
        t1.t.join(); t2.t.join();
        System.out.println(list.size());
    }
}
```

(10 marks) [CSE 107 | 2023–24]

Answer — Race Condition on `ArrayList`: Problem & Two Fixes

The problem:
`ArrayList` is **not thread-safe**. When two threads call `list.add()` concurrently, internal array resizing and index updates can interleave, causing:

- lost updates (final size less than 20,000), or
- an `ArrayIndexOutOfBoundsException` during resize.

The printed size is therefore unpredictable and almost never 20,000.

Fix 1 — Replace `ArrayList` with `CopyOnWriteArrayList`

`CopyOnWriteArrayList` is a thread-safe variant from `java.util.concurrent`. Every write creates a fresh copy of the internal array, so concurrent `add()` calls are safe without explicit locking.

```
import java.util.*;
import java.util.concurrent.CopyOnWriteArrayList;

class MyThread implements Runnable {
    List<String> list;
    Thread t;
    public MyThread(List<String> list) {
        this.list = list;
        t = new Thread(this);
        t.start();
    }
    public void run() {
        for (int i = 1; i <= 10000; i++)
            list.add(String.valueOf(i));
    }
}

public class Fix1 {
    public static void main(String[] args) throws Exception {
        // Use thread-safe list instead of ArrayList
        List<String> list = new CopyOnWriteArrayList<>();
        MyThread t1 = new MyThread(list);
        MyThread t2 = new MyThread(list);
        t1.t.join(); t2.t.join();
        System.out.println(list.size()); // always 20000
    }
}
```

Fix 2 — Synchronise the `add()` call manually

Keep `ArrayList` but wrap every `add()` inside a `synchronized` block on the shared list object.

```
class MyThreadSync implements Runnable {
    List<String> list;
    Thread t;
    public MyThreadSync(List<String> list) {
        this.list = list;
        t = new Thread(this);
        t.start();
    }
    public void run() {
```

```
        for (int i = 1; i <= 10000; i++)
            synchronized (list) {          // lock on the shared list
                list.add(String.valueOf(i));
            }
    }
}

public class Fix2 {
    public static void main(String[] args) throws Exception {
        List<String> list = new ArrayList<>();
        MyThreadSync t1 = new MyThreadSync(list);
        MyThreadSync t2 = new MyThreadSync(list);
        t1.t.join(); t2.t.join();
        System.out.println(list.size()); // always 20000
    }
}
```

Note: `Collections.synchronizedList(new ArrayList<>())` is also a valid Fix 2 alternative — it wraps every method with a synchronized block automatically.

Q4. You are given a task to find the total count of prime numbers in a randomly generated array of 10,000 integers (values 1–5000). Divide work equally among 5 threads. Main thread waits and prints final count. **(15 marks)** [CSE 107 | 2022–23]

Answer — Parallel Prime Count in 10,000-Element Array (5 Threads)

Each thread inspects 2,000 elements of the array for primality. Partial counts are stored in a shared `counts[]` array.

```
public class PrimeCount {

    static boolean isPrime(int n) {
        if (n < 2) return false;
        for (int i = 2; i * i <= n; i++)
            if (n % i == 0) return false;
        return true;
    }

    static class PrimeTask implements Runnable {
        private final int[] arr;
        private final int start, end, id;
        private final int[] counts;

        PrimeTask(int[] arr, int start, int end,
                  int[] counts, int id) {
            this.arr = arr; this.start = start;
            this.end = end; this.counts = counts;
            this.id = id;
        }

        @Override
        public void run() {
            int c = 0;
            for (int i = start; i < end; i++)
                if (isPrime(arr[i])) c++;
            counts[id] = c;
        }
    }

    public static void main(String[] args) throws InterruptedException {
        int[]  arr    = new int[10000];
        int[]  counts = new int[5];
        java.util.Random rnd = new java.util.Random();
        for (int i = 0; i < arr.length; i++)
            arr[i] = rnd.nextInt(5000) + 1;

        Thread[] threads = new Thread[5];
        int chunk = arr.length / 5; // 2000 each

        for (int i = 0; i < 5; i++) {
            threads[i] = new Thread(new PrimeTask(
                arr, i * chunk, (i + 1) * chunk, counts, i));
            threads[i].start();
        }
        for (Thread t : threads) t.join();

```

```
48          int total = 0;
49          for (int c : counts) total += c;
50          System.out.println("Total prime count = " + total);
51      }
52  }
```

Q5. You are given the following class. Create threads T1–T5 from main to execute f1, f2, f0, fs1, and fs2. (i) Which threads run in parallel and why? (ii) Which do not? Can restriction R2 be relaxed?

```
class ThreadTest {
    public void f0() { for(int i=1;i<500;i++) System.out.println(Thread.currentThread().getName()+"
        "+i); }
    public synchronized void f1() { for(int i=1;i<1000;i++) System.out.println(Thread.currentThread
       ().getName()+" "+i); }
    public synchronized void f2() { for(int i=1;i<1500;i++) System.out.println(Thread.currentThread
       ().getName()+" "+i); }
    public synchronized static void fs1() { for(int i=1;i<2000;i++) System.out.println(Thread.
       currentThread().getName()+" "+i); }
    public synchronized static void fs2() { for(int i=1;i<2500;i++) System.out.println(Thread.
       currentThread().getName()+" "+i); }
}
```

R1: Cannot change `ThreadTest`. R2: Can create only one object. **(15 marks)** [CSE 107 | 2022–23]

Answer — `ThreadTest`: Parallel Execution Analysis

Thread creation (one object, R2):

```
public class Main {
    public static void main(String[] args) throws InterruptedException {
        ThreadTest obj = new ThreadTest(); // only ONE object (R2)

        Thread t1 = new Thread(() -> obj.f1(),        "T1");
        Thread t2 = new Thread(() -> obj.f2(),        "T2");
        Thread t3 = new Thread(() -> obj.f0(),        "T3");
        Thread t4 = new Thread(ThreadTest::fs1,       "T4");
        Thread t5 = new Thread(ThreadTest::fs2,       "T5");

        t1.start(); t2.start(); t3.start(); t4.start(); t5.start();
        t1.join();  t2.join();  t3.join();  t4.join();  t5.join();
    }
}
```

Lock analysis:

Thread	Method	Lock	Kind
T1	`f1()` synchronized	`obj` (this)	instance
T2	`f2()` synchronized	`obj` (this)	instance
T3	`f0()` plain	none	—
T4	`fs1()` static sync	`ThreadTest.class`	class
T5	`fs2()` static sync	`ThreadTest.class`	class

(i) Which threads run in parallel?

- **T3 runs in parallel with everyone.** `f0()` acquires no lock, so it never blocks and is never blocked by T1, T2, T4, or T5.
- **T1 or T2 runs in parallel with T4 or T5.** The instance lock (`obj`) and the class lock (`ThreadTest.class`) are *different* objects, so an instance-method thread does not block a static-method thread and vice versa.

(ii) Which threads do NOT run in parallel?

- **T1 and T2 cannot run in parallel** — both compete for the instance lock `obj`; only one executes at a time.
- **T4 and T5 cannot run in parallel** — both compete for the class lock `ThreadTest.class`.

Can R2 (*only one object*) be relaxed?
Yes. If two separate instances `obj1` and `obj2` are created, T1 (on `obj1`) and T2 (on `obj2`) acquire *different* instance locks and therefore **run in parallel**. Relaxing R2 increases concurrency between T1 and T2 without affecting the class-level synchronisation of T4 and T5.

Q6. Describe two different ways of creating threads in Java. Which one is better and why? Explain the difference between a synchronized method and a synchronized statement. **(10 marks)** [CSE 107 | 2021–22]

Answer — Thread Creation, Preference & Synchronization

Java provides two fundamental ways to create threads.

Way 1: Extending `Thread`

```
class CountThread extends Thread {
    public void run() {
        for (int i = 0; i < 5; i++)
            System.out.println(getName() + " -> " + i);
    }
}
// new CountThread().start();
```

Way 2: Implementing `Runnable`

```
class CountRunnable implements Runnable {
    public void run() {
        for (int i = 0; i < 5; i++)
            System.out.println(Thread.currentThread().getName() + " -> " + i);
    }
}
// new Thread(new CountRunnable()).start();
```

Which is better?

Implementing `Runnable` is the preferred approach because:

- Java supports only single inheritance; a class that already inherits from another class *cannot* extend `Thread`. Implementing `Runnable` avoids this limitation.
- It cleanly separates task logic from thread management (separation of concerns).
- The same `Runnable` object can be reused across multiple threads or passed to an `ExecutorService`.

Synchronized method vs. synchronized statement

A **synchronized method** places the lock on the entire method body. The implicit lock object is always `this`:

```
public synchronized void deposit(int amount) { balance += amount; }
```

A **synchronized statement** (block) locks only a specific section of code and allows the programmer to choose the lock object explicitly, enabling finer-grained concurrency:

```
public void deposit(int amount) {
    doSomeNonCriticalWork();   // runs without holding any lock
    synchronized (this) {
        balance += amount;     // only this block is mutually exclusive
    }
}
```

Synchronized statements are preferred when only a small portion of the method requires mutual exclusion, thereby reducing lock contention and improving throughput.

Q7. Find the minimum of a 1000-integer array using 5 threads in parallel:

```
class ParallelMin implements Runnable { /* your code */ }
class Main {
    public static void main(String[] args) {
        int[] numbers = new int[1000];
        ParallelMin[] parallelMin = new ParallelMin[5];
        // your code
    }
}
```

(15 marks) [CSE 107 | 2021–22]

Answer — Parallel Minimum of 1000-Element Array (5 Threads)

Each thread finds the minimum of its 200-element sub-array. The main thread waits for all threads, then finds the global minimum across the five partial results.

```
class ParallelMin implements Runnable {
    private final int[] numbers;
    private final int start, end;
    int localMin; // read by main after join()

    ParallelMin(int[] numbers, int start, int end) {
```

```
        this.numbers  = numbers;
        this.start    = start;
        this.end      = end;
        this.localMin = Integer.MAX_VALUE;
    }

    @Override
    public void run() {
        for (int i = start; i < end; i++)
            if (numbers[i] < localMin) localMin = numbers[i];
    }
}

class Main {
    public static void main(String[] args) throws InterruptedException {
        int[] numbers = new int[1000];
        // Fill with sample data
        java.util.Random rnd = new java.util.Random();
        for (int i = 0; i < numbers.length; i++)
            numbers[i] = rnd.nextInt(10000);

        ParallelMin[] parallelMin = new ParallelMin[5];
        Thread[]      threads     = new Thread[5];
        int chunk = numbers.length / 5; // 200 each

        for (int i = 0; i < 5; i++) {
            parallelMin[i] = new ParallelMin(
                numbers, i * chunk, (i + 1) * chunk);
            threads[i] = new Thread(parallelMin[i]);
            threads[i].start();
        }
        for (Thread t : threads) t.join();

        int globalMin = Integer.MAX_VALUE;
        for (ParallelMin pm : parallelMin)
            if (pm.localMin < globalMin) globalMin = pm.localMin;

        System.out.println("Global minimum = " + globalMin);
    }
}
```

Q8. What are the two common ways to achieve synchronization between Threads? Which one gives more control? **(5 marks)** [CSE 107 | Jan 2021]

Answer — Thread Synchronisation

The two common ways to achieve synchronisation in Java are:

1. Synchronized methods

Add the `synchronized` keyword to a method declaration. Java automatically acquires the intrinsic lock (*monitor*) of `this` before executing the method body and releases it when the method returns.

```
class Counter {
    private int count = 0;
    public synchronized void increment() { count++; }
    public synchronized int getCount()   { return count; }
}
```

2. Synchronized statements (blocks)

Wrap only the *critical section* with `synchronized(obj)` where `obj` is the object whose monitor you want to acquire.

```
class Counter {
    private int count = 0;
    private final Object lock = new Object();

    public void increment() {
        // non-critical work here (no lock held)
        synchronized (lock) {
            count++;          // only this line needs mutual exclusion
        }
    }
}
```

Which gives more control?

Synchronized statements give more control because:

- The programmer can choose *exactly which object* to lock on — it does not have to be `this`.
- The lock is held for the *minimum necessary duration*, reducing contention and improving concurrency.
- Multiple independent critical sections inside one class can use *different* lock objects, allowing them to be entered concurrently by different threads.

Q9. Write complete Java code to compute the summation of 1 to 10,000 by dividing work equally among 5 different threads. Main thread waits and only prints the final summation. **(10 marks)** [CSE 107 | 2020–21]

↪ *Similar: CSE 201 | 2014–15*

Answer — Summation of 1 to 10,000 Using 5 Threads

The range $[1, 10000]$ is divided into 5 equal sub-ranges of 2000 numbers each. Each thread computes the partial sum of its sub-range. The main thread waits for all worker threads to finish via `join()`, then sums the partial results and prints the total.

```
public class ParallelSum {

    static final int TOTAL   = 10000;
    static final int THREADS = 5;
    static final int RANGE   = TOTAL / THREADS;   // 2000 per thread

    // Array to store each thread's partial sum
    static long[] partialSum = new long[THREADS];

    static class SumTask implements Runnable {
        private final int id;     // thread index 0..4
        private final int start;  // inclusive
        private final int end;    // inclusive

        SumTask(int id, int start, int end) {
            this.id = id;
            this.start = start;
            this.end   = end;
        }

        @Override
        public void run() {
            long sum = 0;
            for (int i = start; i <= end; i++) sum += i;
            partialSum[id] = sum;
            System.out.println("Thread-" + id +
                " summed [" + start + ", " + end + "] = " + sum);
        }
    }

    public static void main(String[] args) throws InterruptedException {
        Thread[] workers = new Thread[THREADS];

        // Create and start each worker
        for (int i = 0; i < THREADS; i++) {
            int start = i * RANGE + 1;
            int end   = start + RANGE - 1;
            workers[i] = new Thread(new SumTask(i, start, end));
            workers[i].start();
        }

        // Main thread waits for all workers
        for (Thread t : workers) t.join();

        // Aggregate partial sums
        long total = 0;
        for (long ps : partialSum) total += ps;

        System.out.println("\nFinal Summation (1 to 10000) = " + total);
        // Expected: 50_050_000
    }
}
```

How work is divided:

Thread	Range	Partial Sum
Thread-0	1 – 2000	2,001,000
Thread-1	2001 – 4000	6,001,000
Thread-2	4001 – 6000	10,001,000
Thread-3	6001 – 8000	14,001,000
Thread-4	8001 – 10000	18,045,000
Total		**50,050,000**

Note: `partialSum` is written by each thread to a *distinct index*, so no synchronisation is needed for the write. The main thread reads only after all `join()` calls complete, ensuring full visibility.

Q10. Write Java code to compute the summation of 1 to 10,000,000 by dividing the work equally among 10 threads (no formula allowed, only loops). The main thread waits for all threads to finish and prints the final summation. **(10 marks)** [CSE 107 | Jan 2021]

Answer — Sum 1 to 10,000,000 (10 Threads, Loops Only)

```java
public class SumLoop10M {

    static final int TOTAL   = 10000000;
    static final int THREADS = 10;
    static final int CHUNK   = TOTAL / THREADS; // 1,000,000

    static long[] partial = new long[THREADS];

    static class SumTask implements Runnable {
        private final int id, start, end;
        SumTask(int id, int start, int end) {
            this.id = id; this.start = start; this.end = end;
        }
        @Override
        public void run() {
            long sum = 0;
            for (int i = start; i <= end; i++) // loop only, no formula
                sum += i;
            partial[id] = sum;
        }
    }

    public static void main(String[] args) throws InterruptedException {
        Thread[] threads = new Thread[THREADS];
        for (int i = 0; i < THREADS; i++) {
            int s = i * CHUNK + 1;
            int e = s + CHUNK - 1;
            threads[i] = new Thread(new SumTask(i, s, e));
            threads[i].start();
        }
        for (Thread t : threads) t.join(); // main waits

        long total = 0;
        for (long p : partial) total += p;
        System.out.println("Final sum = " + total);
        // Expected: 50,000,005,000,000
    }
}
```

Q11. Write Java code to compute the summation of 1 to 10,000 by dividing the work equally among 5 threads (no formula allowed, only loops). The main thread waits for all threads to finish and prints the final summation. **(10 marks)** [CSE 107 | 2020–21]

Answer — Sum 1 to 10,000 (5 Threads, Loops Only)

```java
public class SumLoop10K {

    static final int TOTAL   = 10000;
    static final int THREADS = 5;
    static final int CHUNK   = TOTAL / THREADS; // 2000 each

    static long[] partial = new long[THREADS];

    static class SumTask implements Runnable {
        private final int id, start, end;
```

```
        SumTask(int id, int start, int end) {
            this.id = id; this.start = start; this.end = end;
        }
        @Override
        public void run() {
            long sum = 0;
            for (int i = start; i <= end; i++) sum += i;
            partial[id] = sum;
        }
    }

    public static void main(String[] args) throws InterruptedException {
        Thread[] threads = new Thread[THREADS];
        for (int i = 0; i < THREADS; i++) {
            int s = i * CHUNK + 1;
            int e = s + CHUNK - 1;
            threads[i] = new Thread(new SumTask(i, s, e));
            threads[i].start();
        }
        for (Thread t : threads) t.join();

        long total = 0;
        for (long p : partial) total += p;
        System.out.println("Final sum = " + total); // 50,005,000
    }
}
```

Q12. Consider the Producer-Consumer problem with buffer of size 1. The producer generates letters A–Z circularly. The reader reads and displays them. Use wait/notifyAll or `ArrayBlockingQueue`. **(15 marks)** [CSE 107 | 2019–20, Jan 2021]

↪ *Similar: CSE 201 | 2013–14*

Answer — Producer-Consumer: A–Z Circular, Buffer Size 1

Solution 1 — Using `wait` / `notifyAll`

```
class Buffer {
    private char data;
    private boolean hasData = false;

    public synchronized void put(char c) throws InterruptedException {
        while (hasData) wait();        // wait if buffer is full
        data = c;
        hasData = true;
        System.out.println("Produced: " + c);
        notifyAll();
    }

    public synchronized char get() throws InterruptedException {
        while (!hasData) wait();       // wait if buffer is empty
        hasData = false;
        System.out.println("Consumed: " + data);
        notifyAll();
        return data;
    }
}

class Producer extends Thread {
    private final Buffer buf;
    Producer(Buffer buf) { this.buf = buf; }
    @Override
    public void run() {
        try {
            for (int cycle = 0; cycle < 2; cycle++) // 2 full cycles
                for (char c = 'A'; c <= 'Z'; c++)
                    buf.put(c);
        } catch (InterruptedException e) { e.printStackTrace(); }
    }
}

class Consumer extends Thread {
    private final Buffer buf;
    Consumer(Buffer buf) { this.buf = buf; }
    @Override
```

```
    public void run() {
        try {
            for (int i = 0; i < 52; i++) // 2 x 26 letters
                buf.get();
        } catch (InterruptedException e) { e.printStackTrace(); }
    }
}

public class ProducerConsumerAZ {
    public static void main(String[] args) {
        Buffer buf = new Buffer();
        new Producer(buf).start();
        new Consumer(buf).start();
    }
}
```

Solution 2 — Using `ArrayBlockingQueue` (simpler)

```
import java.util.concurrent.ArrayBlockingQueue;

public class ProducerConsumerAZBQ {
    public static void main(String[] args) {
        ArrayBlockingQueue<Character> queue =
            new ArrayBlockingQueue<>(1);

        Thread producer = new Thread(() -> {
            try {
                for (int cycle = 0; cycle < 2; cycle++)
                    for (char c = 'A'; c <= 'Z'; c++) {
                        queue.put(c);
                        System.out.println("Produced: " + c);
                    }
            } catch (InterruptedException e) {}
        });

        Thread consumer = new Thread(() -> {
            try {
                for (int i = 0; i < 52; i++) {
                    char c = queue.take();
                    System.out.println("Consumed: " + c);
                }
            } catch (InterruptedException e) {}
        });

        producer.start();
        consumer.start();
    }
}
```

Q13. Mr. Chandler and Ms. Phoebe worked in the same place. Their work is to put and get product in a shared inventory array of size 3. Ms. Phoebe puts a single product in the inventory and Mr. Chandler gets a single product from the inventory at a time. Ms. Phoebe cannot put a product to the inventory if the inventory array is full with 4 products. Mr. Chandler cannot get a product from a inventory if the inventory array contains no product. Ms. Phoebe can put a product to the inventory if any one of the place of the inventory array is empty. Mr. Chandler can get a product from the inventory if any one of the place of the inventory array is full with a product. Write a java code to solve the above mentioned problem using the concepts of inter thread communication. You can use **wait/notifyAll** or **ArrayBlockingQueue**. **(10 marks)** [CSE 107 | Jan 2021, 2019–20]

↪ *Similar: CSE 201 | 2014–15, 2008–09, 2009–10*

Answer — Chandler & Phoebe: Shared Inventory of Size 3

Solution 1 — Using `wait` / `notifyAll`

```
class Inventory {
    private final int[] slots;
    private int count = 0;

    Inventory(int size) { slots = new int[size]; }

    public synchronized void put(int product)
            throws InterruptedException {
        while (count == slots.length) wait(); // full -- Phoebe waits
        slots[count++] = product;
        System.out.println("Phoebe put product " + product
```

```
                            + "  [count=" + count + "]");
        notifyAll();
    }

    public synchronized int get() throws InterruptedException {
        while (count == 0) wait(); // empty -- Chandler waits
        int product = slots[--count];
        System.out.println("Chandler got product " + product
                            + "  [count=" + count + "]");
        notifyAll();
        return product;
    }
}

class Phoebe extends Thread {
    private final Inventory inv;
    Phoebe(Inventory inv) { this.inv = inv; }
    @Override
    public void run() {
        try {
            for (int i = 1; i <= 10; i++) inv.put(i);
        } catch (InterruptedException e) {}
    }
}

class Chandler extends Thread {
    private final Inventory inv;
    Chandler(Inventory inv) { this.inv = inv; }
    @Override
    public void run() {
        try {
            for (int i = 0; i < 10; i++) inv.get();
        } catch (InterruptedException e) {}
    }
}

public class InventoryDemo {
    public static void main(String[] args) {
        Inventory inv = new Inventory(3);
        new Phoebe(inv).start();
        new Chandler(inv).start();
    }
}
```

Solution 2 — Using `ArrayBlockingQueue`

```
import java.util.concurrent.ArrayBlockingQueue;

public class InventoryBQ {
    public static void main(String[] args) {
        ArrayBlockingQueue<Integer> inv = new ArrayBlockingQueue<>(3);

        Thread phoebe = new Thread(() -> {
            try {
                for (int i = 1; i <= 10; i++) {
                    inv.put(i);
                    System.out.println("Phoebe put " + i);
                }
            } catch (InterruptedException e) {}
        });

        Thread chandler = new Thread(() -> {
            try {
                for (int i = 0; i < 10; i++) {
                    int p = inv.take();
                    System.out.println("Chandler got " + p);
                }
            } catch (InterruptedException e) {}
        });

        phoebe.start();
        chandler.start();
    }
}
```

Q14. Mr. P puts products and Mr. C gets products from a shared inventory of size 4. P cannot put if full (4 products); C cannot get if empty. Write complete Java code using **wait and notifyAll only** — you are not allowed to use `BlockingQueue`. **(20 marks)** [CSE 107 | Jan 2021]

Answer — Mr. P & Mr. C: Inventory Size 4, `wait/notifyAll` Only

`BlockingQueue` is **not** used. All synchronisation is implemented with Java's built-in monitor (`synchronized` + `wait` + `notifyAll`).

```
class SharedInventory {
    private final int MAX = 4;
    private final int[] slots = new int[MAX];
    private int count = 0;

    // Called by P (Producer)
    public synchronized void put(int item)
            throws InterruptedException {
        while (count == MAX) {      // inventory full -- P waits
            System.out.println("P waiting (inventory full)");
            wait();
        }
        slots[count++] = item;
        System.out.println("P put item " + item
                           + "  [size=" + count + "]");
        notifyAll();                // wake C if it was waiting
    }

    // Called by C (Consumer)
    public synchronized int get() throws InterruptedException {
        while (count == 0) {        // inventory empty -- C waits
            System.out.println("C waiting (inventory empty)");
            wait();
        }
        int item = slots[--count];
        System.out.println("C got item " + item
                           + "  [size=" + count + "]");
        notifyAll();                // wake P if it was waiting
        return item;
    }
}

class Producer extends Thread {
    private final SharedInventory inv;
    Producer(SharedInventory inv) { this.inv = inv; }
    @Override
    public void run() {
        try {
            for (int i = 1; i <= 12; i++) {
                inv.put(i);
                Thread.sleep(50); // simulate production time
            }
        } catch (InterruptedException e) {}
    }
}

class Consumer extends Thread {
    private final SharedInventory inv;
    Consumer(SharedInventory inv) { this.inv = inv; }
    @Override
    public void run() {
        try {
            for (int i = 0; i < 12; i++) {
                inv.get();
                Thread.sleep(150); // simulate consumption time
            }
        } catch (InterruptedException e) {}
    }
}

public class InventoryWaitNotify {
    public static void main(String[] args) {
        SharedInventory inv = new SharedInventory();
        new Producer(inv).start();
        new Consumer(inv).start();
    }
}
```

Key points:

- `while` (not `if`) guards each `wait()` to handle spurious wake-ups correctly.
- `notifyAll()` is used instead of `notify()` to avoid the risk of notifying the wrong thread.
- Both `put` and `get` are `synchronized` on the same object (`inv`), so only one thread executes either method at a time.

Q15. Consider the `TestClass` with `f1()` (1s sleep) and `f2()` (2s sleep). Create two threads without changing `TestClass` or extending/implementing any class:

```
class TestClass {
    public void f1() { for(int i=0;i<5;i++) { System.out.println(i); try { Thread.sleep(1000); }
       catch(InterruptedException e){} } }
    public static void f2() { for(int i=0;i<5;i++) { System.out.println(i); try { Thread.sleep
       (2000); } catch(InterruptedException e){} } }
}
```

(10 marks) [CSE 107 | Jan 2021]

Answer — Two Threads via Lambda (No Subclassing)

Because we cannot extend or implement any class, we use **lambda expressions** as `Runnable` instances. `TestClass` is used as-is; a single object is created for the instance method `f1()`, while the static method `f2()` is referenced directly.

```
public class Main {
    public static void main(String[] args) throws InterruptedException {
        TestClass obj = new TestClass();

        // Thread for f1() -- instance method, needs an object
        Thread t1 = new Thread(() -> obj.f1(), "T1");

        // Thread for f2() -- static method, no object needed
        Thread t2 = new Thread(TestClass::f2, "T2");

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println("Both threads finished.");
    }
}
```

Why this satisfies all constraints:

- `TestClass` is not modified.
- No new class extends or implements anything — `() -> obj.f1()` is an anonymous lambda that the compiler treats as a `Runnable` without requiring an explicit `implements Runnable` declaration.
- Two separate threads (`t1` for `f1()`, `t2` for `f2()`) are created and run concurrently.

Q16. Explain with code how to fix the `SharedCounter` class so that multiple threads can correctly operate on it:

```
class SharedCounter {
    private int counter;
    SharedCounter() { this.counter = 0; }
    private void increment() { this.counter++; }
    public int get() { return this.counter; }
    public void count() { for(int i=1;i<=1000;i++) increment(); }
}
```

(5 marks) [CSE 107 | 2020–21]

Answer — Fixing `SharedCounter` for Thread Safety

The problem:
`counter++` is *not atomic* — it compiles to three operations: read, increment, write. Two threads can both read the same value, both increment it, and both write back, losing one update (a *lost update* race condition).

Fix — make `increment()` synchronized:

```
class SharedCounter {
    private int counter;
    SharedCounter() { this.counter = 0; }

```

```
    // Fix: add synchronized so only one thread increments at a time
    private synchronized void increment() { this.counter++; }

    public int get() { return this.counter; }

    public void count() {
        for (int i = 1; i <= 1000; i++) increment();
    }
}

public class TestSharedCounter {
    public static void main(String[] args) throws InterruptedException {
        SharedCounter sc = new SharedCounter();

        Thread t1 = new Thread(sc::count, "T1");
        Thread t2 = new Thread(sc::count, "T2");
        Thread t3 = new Thread(sc::count, "T3");

        t1.start(); t2.start(); t3.start();
        t1.join();  t2.join();  t3.join();

        System.out.println("Final count: " + sc.get()); // always 3000
    }
}
```

Alternative fix — use `AtomicInteger`:

```
import java.util.concurrent.atomic.AtomicInteger;

class SharedCounterAtomic {
    private AtomicInteger counter = new AtomicInteger(0);
    private void increment()  { counter.incrementAndGet(); }
    public  int  get()        { return counter.get(); }
    public  void count()      { for (int i=1;i<=1000;i++) increment(); }
}
```

`AtomicInteger.incrementAndGet()` is a single hardware-level atomic operation and avoids the overhead of acquiring a monitor lock.

Q17. Write three different ways to create Threads in Java with short code examples. What is the difference between synchronized method and synchronized statement? **(10 marks)** [CSE 107 | 2019–20]

Answer — Three Ways to Create Threads & Synchronization

Way 1: Extending `Thread`

```
class T1 extends Thread {
    public void run() { System.out.println("Thread via extension"); }
}
// Usage: new T1().start();
```

Way 2: Implementing `Runnable`

```
class T2 implements Runnable {
    public void run() { System.out.println("Thread via Runnable"); }
}
// Usage: new Thread(new T2()).start();
```

Way 3: Using a Lambda expression (Java 8+)

Since `Runnable` is a functional interface, it can be expressed as a lambda — the most concise modern approach.

```
Thread t = new Thread(() -> System.out.println("Thread via Lambda"));
t.start();
```

Synchronized method vs. synchronized statement

Synchronized Method	Synchronized Statement (Block)
The `synchronized` keyword is placed in the method signature. The lock acquired is always `this` (or the `Class` object for `static` methods).	The `synchronized` keyword wraps only a *critical section* inside a method, and the programmer specifies *which object* to lock on.
Entire method body is mutually exclusive.	Only the enclosed block is mutually exclusive; the rest of the method runs without holding the lock.
Less flexible — always locks the current object.	More flexible — can lock on any object, enabling finer-grained concurrency.

```java
// Synchronized method
public synchronized void increment() {
    count++;
}

// Synchronized statement -- locks only on the shared 'counter' object
public void increment() {
    // ... non-critical code runs here without holding a lock ...
    synchronized (counter) {
        count++;
    }
    // ... more non-critical code ...
}
```

Q18. Divide summation of 1 to 10,000,000 equally among 10 different threads. Main thread waits and prints final sum. **(10 marks)** [CSE 107 | 2019–20]

Answer — Summation of 1 to 10,000,000 Using 10 Threads

The range [1, 10,000,000] is divided into 10 equal sub-ranges of 1,000,000 numbers each. Each thread computes its partial sum and stores it in a shared array. The main thread calls `join()` on every worker before aggregating the results.

```java
public class ParallelSum {

    static final int TOTAL   = 10000000;
    static final int THREADS = 10;
    static final int RANGE   = TOTAL / THREADS; // 1,000,000 each

    static long[] partial = new long[THREADS];

    static class SumTask implements Runnable {
        private final int id, start, end;
        SumTask(int id, int start, int end) {
            this.id = id; this.start = start; this.end = end;
        }
        @Override
        public void run() {
            long sum = 0;
            for (int i = start; i <= end; i++) sum += i;
            partial[id] = sum;
        }
    }

    public static void main(String[] args) throws InterruptedException {
        Thread[] workers = new Thread[THREADS];
        for (int i = 0; i < THREADS; i++) {
            int s = i * RANGE + 1;
            int e = s + RANGE - 1;
            workers[i] = new Thread(new SumTask(i, s, e));
            workers[i].start();
        }
        for (Thread t : workers) t.join(); // main waits

        long total = 0;
        for (long p : partial) total += p;
        System.out.println("Sum = " + total); // 50,000,005,000,000
    }
}
```

Q19. What are the advantages of multithreading over process-based multitasking? **(6 marks)** [CSE 107 | 2017–18; CSE 201 | 2012–13]

Answer — Multithreading vs. Process-Based Multitasking

Process-based multitasking (multiprocessing) runs multiple independent programs concurrently; each process has its own address space and system resources. **Thread-based multitasking** (multithreading) runs multiple threads *within the same process*, all sharing the same memory and resources.

Advantages of multithreading over process-based multitasking:

1. **Lower overhead:** Creating a thread is far cheaper than creating a process because threads share the heap, code segment, and open file descriptors of their parent process. No new virtual address space needs to be set up.
2. **Faster context switching:** Switching between threads of the same process is faster than switching between processes because the operating system does not need to swap the entire memory map — only the CPU registers and stack pointer change.
3. **Shared memory — easier communication:** Threads in the same process share the same address space, so data can be exchanged through shared variables without expensive inter-process communication (IPC) mechanisms such as pipes, sockets, or shared memory segments.
4. **Responsiveness:** In a multithreaded GUI application, one thread can handle user interaction while another performs a long computation, keeping the application responsive.
5. **Better CPU utilisation:** When one thread is blocked on I/O, the CPU can execute another thread of the same process, improving throughput without spawning new heavy-weight processes.
6. **Simplified program structure:** Tasks that are naturally concurrent (e.g. reading input and writing output simultaneously) can be modelled as separate threads within a single program, rather than as separate communicating processes.

Q20. Describe the methods Java uses for inter-thread communication. **(9 marks)** [CSE 107 | 2017–18]

Answer — Inter-Thread Communication in Java

Java's inter-thread communication is built on three methods defined in the `Object` class. Because every object has a monitor, these methods can be called on *any* object; however, they must always be called from within a `synchronized` block or method (otherwise Java throws `IllegalMonitorStateException`).

Method	Description
`wait()`	Causes the current thread to release the monitor lock and enter the *Waiting* state. The thread remains waiting until another thread calls `notify()` or `notifyAll()` on the same object, or until it is interrupted.
`notify()`	Wakes up *one* arbitrary thread that is waiting on this object's monitor. The awakened thread must re-acquire the lock before it can proceed.
`notifyAll()`	Wakes up *all* threads waiting on this object's monitor. All awakened threads compete to reacquire the lock; only one succeeds at a time.

Classic Producer–Consumer example:

```
class SharedBuffer {
    private int data;
    private boolean hasData = false;

    // Called by Producer thread
    public synchronized void produce(int val) throws InterruptedException {
        while (hasData)          // wait if buffer is full
            wait();
        data = val;
        hasData = true;
        System.out.println("Produced: " + val);
        notify();                // wake up Consumer
    }

    // Called by Consumer thread
    public synchronized int consume() throws InterruptedException {
        while (!hasData)         // wait if buffer is empty
            wait();
        hasData = false;
        System.out.println("Consumed: " + data);
        notify();                // wake up Producer
        return data;
    }
}
```

Key rules:

- Always call `wait()` inside a `while` loop (not `if`), because of *spurious wake-ups* — a thread may wake up even without a `notify()`.

- Prefer `notifyAll()` over `notify()` when multiple threads may be waiting, to avoid potential deadlock from notifying the wrong thread.

Q21. What are the two ways of creating a thread? Give an example for each kind. Which one is better and why? **(8+5 marks)** [CSE 201 | 2011–12; CSE 107 | 2016–17]

↪ *Similar: CSE 201 | 2009–10, 2012–13*

Answer — Two Ways to Create a Thread in Java

Java provides two mechanisms to create a thread.

Way 1: Extending the `Thread` class

Create a subclass of `Thread` and override its `run()` method. The thread is started by calling `start()` on an instance of the subclass.

```
class MyThread extends Thread {
    private String name;
    MyThread(String name) { this.name = name; }

    @Override
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(name + " - count: " + i);
        }
    }
}

public class ThreadDemo1 {
    public static void main(String[] args) throws InterruptedException {
        MyThread t1 = new MyThread("Thread-A");
        MyThread t2 = new MyThread("Thread-B");
        t1.start();
        t2.start();
        t1.join();   // main waits for t1 to finish
        t2.join();   // main waits for t2 to finish
        System.out.println("Main thread ends.");
    }
}
```

Way 2: Implementing the `Runnable` interface

Implement the `Runnable` interface and pass an instance of it to a `Thread` object. The class is free to extend any other class simultaneously.

```
class MyRunnable implements Runnable {
    private String name;
    MyRunnable(String name) { this.name = name; }

    @Override
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(name + " - count: " + i);
        }
    }
}

public class ThreadDemo2 {
    public static void main(String[] args) throws InterruptedException {
        Thread t1 = new Thread(new MyRunnable("Thread-A"));
        Thread t2 = new Thread(new MyRunnable("Thread-B"));
        t1.start();
        t2.start();
        t1.join();
        t2.join();
        System.out.println("Main thread ends.");
    }
}
```

Which is better and why?

Implementing `Runnable` is *generally preferred* for the following reasons:

1. **Single Inheritance constraint:** Java does not support multiple inheritance. If a class already extends another class, it *cannot* also extend `Thread`. Implementing `Runnable` imposes no such restriction.
2. **Separation of concerns:** `Runnable` separates the task logic from the thread mechanism, making code more modular and reusable.

3. **Resource efficiency:** The same `Runnable` object can be submitted to thread pools (e.g. `ExecutorService`), whereas a `Thread` subclass cannot be reused after termination.
4. **Better OOP design:** Extending `Thread` is appropriate only when you need to modify the threading behaviour itself, which is rarely required in practice.

Q22. Find the minimum of a 500-integer array using 5 threads in parallel. You are not allowed to write any more classes or arrays, but you can add normal variables, constructors, and other methods:

```
class ParallelMin implements Runnable { /* your code */ }
class Main {
    public static void main(String[] args) {
        Random random = new Random();
        int[] numbers = new int[500];
        for (int i = 0; i < numbers.length; i++)
            numbers[i] = random.nextInt();
        // your code
    }
}
```

(10 marks) [CSE 107 | 2016–17]

Answer — Parallel Minimum of 500-Element Array (No Extra Arrays)

The constraint forbids additional classes or arrays. The partial minimum of each thread is stored as an instance field of `ParallelMin` and accessed directly through the object reference after `join()`.

```
class ParallelMin implements Runnable {
    private final int[] numbers;
    private final int start, end;
    int localMin;

    ParallelMin(int[] numbers, int start, int end) {
        this.numbers  = numbers;
        this.start    = start;
        this.end      = end;
        this.localMin = Integer.MAX_VALUE;
    }

    @Override
    public void run() {
        for (int i = start; i < end; i++)
            if (numbers[i] < localMin) localMin = numbers[i];
    }
}

class Main {
    public static void main(String[] args) throws InterruptedException {
        Random random = new Random();
        int[] numbers = new int[500];
        for (int i = 0; i < numbers.length; i++)
            numbers[i] = random.nextInt();

        // 5 objects hold the partial results -- no extra array needed
        int chunk = numbers.length / 5; // 100 each
        ParallelMin pm0 = new ParallelMin(numbers,   0, 100);
        ParallelMin pm1 = new ParallelMin(numbers, 100, 200);
        ParallelMin pm2 = new ParallelMin(numbers, 200, 300);
        ParallelMin pm3 = new ParallelMin(numbers, 300, 400);
        ParallelMin pm4 = new ParallelMin(numbers, 400, 500);

        Thread t0=new Thread(pm0), t1=new Thread(pm1),
               t2=new Thread(pm2), t3=new Thread(pm3),
               t4=new Thread(pm4);

        t0.start(); t1.start(); t2.start(); t3.start(); t4.start();
        t0.join();  t1.join();  t2.join();  t3.join();  t4.join();

        int globalMin = Math.min(pm0.localMin,
                        Math.min(pm1.localMin,
                        Math.min(pm2.localMin,
                        Math.min(pm3.localMin, pm4.localMin))));

        System.out.println("Global minimum = " + globalMin);
    }
```

```
}
```

Q23. You need to simulate the traffic behavior of different vehicles moving inside BUET campus. There are four types, i.e., Car, Microbus, AutoRickshaw, and Rickshaw, of vehicles run in the campus. Each vehicle type should have necessary properties, e.g., number of wheels, maximum speed, etc., which represent real vehicles that you see in real life. In the simulated environment you need to instantiate 10 vehicles of each type, where each vehicle has its current position and speed. Their initial positions and speeds will be randomly generated while you instantiate these vehicles. Vehicles will run along the road segments which are represented as a fixed set of rectangles. In every second, each vehicle updates its position based on the previous position and current speed. You need to make sure that a vehicle does not go outside the road segments at any point of time. Two vehicles must not collide with each other, i.e., if during a position update a vehicle finds another vehicle within a threshold range it cannot move and must need to decrease its speed.

Write a Java program to implement the above simulator. You have to run each vehicle as a thread. Please make your own assumption to build the whole scenario as realistic as possible to reflect the real-world scenario. **(35 marks)** [CSE 201 | 2015–16]

Answer — BUET Campus Traffic Simulation

Each vehicle type is a subclass of an abstract `Vehicle` thread and carries type-specific properties (number of wheels, maximum speed). Initial positions and speeds are randomly generated. A shared `Road` object serialises position updates: if another vehicle is found within a configurable *threshold* range the vehicle stays put and its speed is decreased; if the road boundary would be exceeded the vehicle is clamped to the boundary.

```
import java.util.Random;

// -- Shared road segment ------------------------------------------
class Road {
    static final int LENGTH    = 500;  // road length in metres
    static final int THRESHOLD = 10;   // collision-detection range

    // Coarse-grained lock: one position-update at a time
    public synchronized int tryMove(int oldPos, int newPos,
                                    int[] allPositions, int myIdx) {
        // 1. Clamp to road boundaries
        newPos = Math.max(0, Math.min(LENGTH - 1, newPos));

        // 2. Check threshold range against every other vehicle
        for (int i = 0; i < allPositions.length; i++) {
            if (i == myIdx) continue;
            if (Math.abs(allPositions[i] - newPos) < THRESHOLD)
                return -1;   // blocked - caller must decrease speed
        }
        return newPos;
    }
}

// -- Abstract vehicle thread --------------------------------------
abstract class Vehicle extends Thread {
    protected int    position;
    protected int    speed;            // metres per second
    protected final int    maxSpeed;
    protected final int    numWheels;
    protected final Road   road;
    protected final int[]  allPositions;  // shared position array
    protected final int    myIdx;
    private   static final Random rng = new Random();

    Vehicle(String name, int maxSpeed, int numWheels,
            Road road, int[] allPositions, int idx) {
        super(name);
        this.maxSpeed     = maxSpeed;
        this.numWheels    = numWheels;
        this.road         = road;
        this.allPositions = allPositions;
        this.myIdx        = idx;
        // Random initial position and speed
        this.position     = rng.nextInt(Road.LENGTH);
        this.speed        = 1 + rng.nextInt(maxSpeed);
        allPositions[idx] = this.position;
    }

    @Override
    public void run() {
        try {
```

```
            for (int step = 0; step < 30; step++) {
                int next   = position + speed;
                int result = road.tryMove(position, next,
                                          allPositions, myIdx);
                if (result == -1) {
                    // Blocked by nearby vehicle: decrease speed
                    speed = Math.max(1, speed - 1);
                    System.out.printf("%-22s BLOCKED  pos=%d speed->%d%n",
                                      getName(), position, speed);
                } else {
                    position = result;
                    allPositions[myIdx] = position;
                    // Gradually recover speed up to maximum
                    if (speed < maxSpeed) speed++;
                    System.out.printf("%-22s pos=%d speed=%d%n",
                                      getName(), position, speed);
                }
                Thread.sleep(1000); // update every second
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}

// -- Concrete vehicle types (wheels, max speed are assumptions) ---
class Car extends Vehicle {
    // 4 wheels, max speed 60 m/s (assumed scaled for simulation)
    Car(Road r, int[] pos, int idx) {
        super("Car-" + idx, 60, 4, r, pos, idx);
    }
}
class Microbus extends Vehicle {
    // 4 wheels, max speed 40 m/s
    Microbus(Road r, int[] pos, int idx) {
        super("Microbus-" + idx, 40, 4, r, pos, idx);
    }
}
class AutoRickshaw extends Vehicle {
    // 3 wheels, max speed 25 m/s
    AutoRickshaw(Road r, int[] pos, int idx) {
        super("AutoRickshaw-" + idx, 25, 3, r, pos, idx);
    }
}
class Rickshaw extends Vehicle {
    // 3 wheels, max speed 10 m/s
    Rickshaw(Road r, int[] pos, int idx) {
        super("Rickshaw-" + idx, 10, 3, r, pos, idx);
    }
}

// -- Main ---------------------------------------------------------
public class BUETTraffic {
    static final int PER_TYPE = 10;
    static final int TOTAL    = PER_TYPE * 4;   // 40 vehicles

    public static void main(String[] args) throws InterruptedException {
        Road  road        = new Road();
        int[] allPositions = new int[TOTAL];
        Vehicle[] vehicles = new Vehicle[TOTAL];

        int idx = 0;
        for (int i = 0; i < PER_TYPE; i++)
            vehicles[idx] = new Car(road, allPositions, idx++);
        for (int i = 0; i < PER_TYPE; i++)
            vehicles[idx] = new Microbus(road, allPositions, idx++);
        for (int i = 0; i < PER_TYPE; i++)
            vehicles[idx] = new AutoRickshaw(road, allPositions, idx++);
        for (int i = 0; i < PER_TYPE; i++)
            vehicles[idx] = new Rickshaw(road, allPositions, idx++);

        for (Vehicle v : vehicles) v.start();
        for (Vehicle v : vehicles) v.join();
```

```
125
126             System.out.println("Simulation complete.");
127         }
128     }
```

Key design decisions and guarantees:

- **Vehicle properties:** Each subclass fixes `numWheels` and `maxSpeed` to realistic values (Car: 4 wheels / 60; Microbus: 4 / 40; AutoRickshaw: 3 / 25; Rickshaw: 3 / 10).
- **Random initialisation:** Both `position` and `speed` are drawn from a uniform random distribution at construction time, satisfying the problem statement.
- **Boundary enforcement:** `Math.max/min` clamps `newPos` to $[0, \texttt{LENGTH} - 1]$ so no vehicle ever leaves the road segment.
- **Threshold-based collision:** `tryMove()` scans all other vehicles; if any is within `THRESHOLD` cells of the intended new position the move is refused and the caller decrements its speed by 1 (minimum 1), matching the specification exactly.
- **Thread safety:** `tryMove()` is `synchronized` on the `Road` object, so the shared `allPositions[]` array is read and written atomically, preventing race conditions.
- **Periodic update:** Each vehicle calls `Thread.sleep(1000)` between steps, implementing the "every second" update cycle required by the problem.

Q24. Why are `suspend()`, `resume()` and `stop()` methods of `Thread` class deprecated? Demonstrate how you can achieve suspend, resume and stop behaviors in a thread that you have implemented, without using the deprecated methods. Write necessary code and provide explanations where appropriate. Do not use unnecessary/redundant `sleep`, `wait` or `notify` method calls. **(5+10 marks)** [CSE 107 | 2015–16]

Answer — Deprecated Thread Methods and Safe Implementation

Why These Methods Are Deprecated:

(a) **`stop()`:** It is inherently unsafe. It stops a thread abruptly by throwing a `ThreadDeath` exception up the stack, causing the thread to instantly release all locks/monitors it holds. If any shared objects protected by those locks were in an incomplete or transitioning state, they are exposed to other threads in a corrupted, inconsistent state.

(b) **`suspend()` / `resume()`:** They are inherently deadlock-prone. When a thread is frozen via `suspend()`, it does **not** release any locks it currently holds. If the managing thread trying to call `resume()` requires access to any of those same locked resources before it can proceed, the application reaches an unrecoverable deadlock.

Safe Design Pattern: To accurately replace these behaviors without creating redundant or resource-heavy busy-waiting loops, state flags (`active` and `paused`) are declared as `volatile` to ensure visibility across threads. Inter-thread communication primitives (`wait()` and `notify()`) are deployed explicitly and selectively to suspend execution without hoarding monitors.

```
1  class SafeRunnable implements Runnable {
2      private volatile boolean active = true;
3      private volatile boolean paused = false;
4
5      @Override
6      public void run() {
7          while (active) {
8              // Synchronized block used strictly to safely park the thread
9              synchronized (this) {
10                 while (paused && active) {
11                     try {
12                         wait(); // Releases the monitor lock while waiting
13                     } catch (InterruptedException e) {
14                         active = false;
15                     }
16                 }
17             }
18
19             if (!active) break;
20
21             // --- Actual core work of the thread goes here ---
22             System.out.println(Thread.currentThread().getName() + " is running...");
23
24             try {
25                 Thread.sleep(200); // Standard processing simulation interval
26             } catch (InterruptedException e) {
27                 active = false;
28             }
29         }
30         System.out.println(Thread.currentThread().getName() + " clean termination complete.");
31     }
```

```
32
33      // Safe substitute for suspend()
34      public synchronized void safeSuspend() {
35          paused = true;
36      }
37
38      // Safe substitute for resume()
39      public synchronized void safeResume() {
40          if (paused) {
41              paused = false;
42              notify(); // Target execution guard; only signals when thread is paused
43          }
44      }
45
46      // Safe substitute for stop()
47      public void safeStop() {
48          active = false;
49          safeResume(); // Unblocks the thread immediately if it was parked in wait()
50      }
51  }
52
53  public class Main {
54      public static void main(String[] args) throws InterruptedException {
55          SafeRunnable runnable = new SafeRunnable();
56          Thread worker = new Thread(runnable, "WorkerThread");
57          worker.start();
58
59          Thread.sleep(600);
60          System.out.println("[Main] Suspending thread...");
61          runnable.safeSuspend();
62
63          Thread.sleep(1000);
64          System.out.println("[Main] Resuming thread...");
65          runnable.safeResume();
66
67          Thread.sleep(600);
68          System.out.println("[Main] Stopping thread cleanly...");
69          runnable.safeStop();
70
71          worker.join();
72      }
73  }
```

Q25. "Oracle recommends that calls to `wait()` should take place within a loop that checks the conditions" — Why? **(5 marks)** [CSE 107 | 2015–16]

Answer — Why `wait()` Must Be Inside a Loop

Oracle recommends placing `wait()` inside a `while` loop that re-checks the condition for two reasons:

1. Spurious wake-ups

The Java specification explicitly permits a thread to wake up from `wait()` *without any call to* ***`notify()`*** *or* ***`notifyAll()`***. This is called a **spurious wake-up** and can occur due to low-level OS signal interruptions. If `wait()` is inside an `if`, the thread proceeds even though the condition it was waiting for may still be false.

2. `notifyAll()` wakes too many threads

When `notifyAll()` is used, *all* waiting threads wake up. Only one of them may actually find the condition satisfied; the rest must go back to waiting. An `if` check causes those threads to proceed incorrectly, while a `while` loop sends them back to `wait()`.

Illustration:

```
// WRONG -- if-based check
public synchronized void consume() throws InterruptedException {
    if (!hasData) wait();   // spurious wake-up or notifyAll can
                            // bypass this check incorrectly
    process(data);
}

// CORRECT -- while-based check
public synchronized void consume() throws InterruptedException {
    while (!hasData) wait(); // always re-checks after waking up
    process(data);
}
```

The `while` loop turns `wait()` into a self-correcting mechanism: no matter why the thread woke up — real notification, spurious wake, or `notifyAll()` — it re-evaluates the condition and waits again if the condition is still false.

Q26. You are using a 3rd party package called `Sophisticated` that defines the following classes:

- `SophisticatedProblemsSolver`
- `SophisticatedProblem`
- `SimpleResult`

`SophisticatedProblemsSolver` has a public method with the following signature:

```
public SimpleResult Solve(SophisticatedProblem problem)
```

Your scenario is multi-threaded but you are unsure if `Solve()` is thread-safe. The following classes written by you need to use this method:

- `CompetativeProgrammingProblemsSolver`
- `SoftwareDevelopmentProblemSolver`
- `PassInTheExamsProblemSolver`

Each of the above classes needs to interact with the same instance of `SophisticatedProblemsSolver`. The constructors of these classes are passed in a reference of `SophisticatedProblemsSolver` type. Demonstrate a solution to guarantee that your program will be thread-safe. Write necessary code and provide explanations where appropriate. (Do not use static variables in your solution. Note: You do not have source code for the package `Sophisticated`) **(15 marks)** [CSE 107 | 2015–16]

Answer — Thread-Safe Cooperation on a Shared Object Reference

Synchronization Strategy: Since the problem states that the constructors of our classes must directly accept a reference of type `SophisticatedProblemsSolver`, we cannot wrap it in an external container class. Instead, because all three classes share the exact same object reference in memory, we can use a **synchronized block** anchored directly onto that shared reference (`synchronized(solver)`).
This approach acts as a mutual-exclusion lock, ensuring that only one thread can invoke `Solve()` at any given time, thus eliminating data races without utilizing any prohibited static variables.

```java
// --- Mock placeholders for the unmodifiable 3rd party package ---
class SophisticatedProblem {}
class SimpleResult {}
class SophisticatedProblemsSolver {
    public SimpleResult Solve(SophisticatedProblem problem) {
        // Unknown implementation detail -- assumed unsafe
        return new SimpleResult();
    }
}

// --- Class 1: Competitive Programming Problems Solver ---
class CompetativeProgrammingProblemsSolver implements Runnable {
    private final SophisticatedProblemsSolver solver;

    // Constructor accepts standard reference as specified
    public CompetativeProgrammingProblemsSolver(SophisticatedProblemsSolver solver) {
        this.solver = solver;
    }

    @Override
    public void run() {
        SophisticatedProblem problem = new SophisticatedProblem();
        SimpleResult result;

        // Synchronize on the shared instance to serialize calls to Solve()
        synchronized (solver) {
            result = solver.Solve(problem);
        }
        System.out.println("CP Solver finished processing.");
    }
}

// --- Class 2: Software Development Problem Solver ---
class SoftwareDevelopmentProblemSolver implements Runnable {
    private final SophisticatedProblemsSolver solver;

    public SoftwareDevelopmentProblemSolver(SophisticatedProblemsSolver solver) {
        this.solver = solver;
    }
```

```
40
41      @Override
42      public void run() {
43          SophisticatedProblem problem = new SophisticatedProblem();
44          SimpleResult result;
45
46          synchronized (solver) {
47              result = solver.Solve(problem);
48          }
49          System.out.println("SD Solver finished processing.");
50      }
51  }
52
53  // --- Class 3: Pass In The Exams Problem Solver ---
54  class PassInTheExamsProblemSolver implements Runnable {
55      private final SophisticatedProblemsSolver solver;
56
57      public PassInTheExamsProblemSolver(SophisticatedProblemsSolver solver) {
58          this.solver = solver;
59      }
60
61      @Override
62      public void run() {
63          SophisticatedProblem problem = new SophisticatedProblem();
64          SimpleResult result;
65
66          synchronized (solver) {
67              result = solver.Solve(problem);
68          }
69          System.out.println("Exam Solver finished processing.");
70      }
71  }
72
73  // --- Demonstration Main Class ---
74  public class Main {
75      public static void main(String[] args) {
76          // Step 1: Create a single shared instance of the un-thread-safe solver
77          SophisticatedProblemsSolver sharedSolver = new SophisticatedProblemsSolver();
78
79          // Step 2: Pass the same instance to all consumer constructors
80          Thread t1 = new Thread(new CompetativeProgrammingProblemsSolver(sharedSolver));
81          Thread t2 = new Thread(new SoftwareDevelopmentProblemSolver(sharedSolver));
82          Thread t3 = new Thread(new PassInTheExamsProblemSolver(sharedSolver));
83
84          // Step 3: Execute tasks concurrently
85          t1.start();
86          t2.start();
87          t3.start();
88      }
89  }
```

Why this guarantees Thread Safety:

- **Object Monitor Locking:** Java objects have an intrinsic lock. By wrapping the invocation within `synchronized(solver)`, a thread must successfully acquire the monitor lock belonging to that specific `sharedSolver` instance before executing `Solve()`.
- **No Static Leakage:** The synchronization context relies entirely on the heap lifecycle instance passed through standard object-oriented instantiation, fully avoiding static state variables.
- **Black-Box Compliant:** It requires zero access to or changes inside the unmodifiable `Sophisticated` package source code.

Q27. Draw the life-cycle / schematic diagram of different states of a Thread. **(9 marks)** [CSE 201 | 2013–14, 2008–09, 2007–08]

Answer — Thread Life-Cycle (State Diagram)

A Java thread passes through the following states during its lifetime:

Description of each state:

1. **New** — The thread object has been created (constructed), but `Thread.start()` has not yet been called.
2. **Ready-to-Run** — `Thread.start()` has been called. The thread is ready to execute and is waiting in a pool for the **Scheduler** to assign it CPU time.
3. **Running** — The **Scheduler** has selected the thread from the Ready-to-Run state and it is currently executing its `run()` method.
4. **Sleeping** — The thread transitions here when `Thread.sleep()` is called. It returns to the Ready-to-Run state once the specified elapsed time ends or if `interrupt()` is called.
5. **Waiting** — The thread goes into this state by calling `lock.wait()`. It remains here until another thread wakes it up by calling `lock.notify()` or `lock.notifyAll()`, after which it goes back to Ready-to-Run.
6. **Blocking** — A thread is blocked when it is waiting for an I/O operation to complete or waiting to acquire a locked Monitor (e.g., entering a `synchronized` block). It returns to Ready-to-Run when the I/O is completed or the lock is acquired.
7. **Dead** — The thread enters this final state when its `run()` method completes or it forcibly exits. A dead thread cannot be restarted.

Q28. Discuss "runnable" and "waiting" states of Thread briefly. **(10 marks)** **[CSE 201 | 2013–14]**

Answer — Runnable and Waiting States

Runnable State
A thread enters the *Runnable* state as soon as `start()` is invoked on it. In this state the thread is *eligible to run* but is not necessarily executing at this instant; it is queued and awaiting CPU time from the JVM thread scheduler.

- A thread also returns to *Runnable* after waking up from a `sleep()`, after being notified out of a `wait()`, or after a blocking I/O operation completes.
- Multiple threads can be in the *Runnable* state simultaneously; the scheduler picks one to move to *Running*.
- A *Running* thread may be pre-empted (moved back to *Runnable*) at any time by the scheduler to give another thread a turn.

Waiting State
A thread enters the *Waiting* state when it voluntarily suspends itself and cannot proceed until some external condition is satisfied:

- `Object.wait()` — thread releases the monitor lock and waits until another thread calls `notify()` or `notifyAll()` on the same object.
- `Thread.join()` — the calling thread waits indefinitely for the target thread to terminate.
- `LockSupport.park()` — low-level parking used internally by `java.util.concurrent` utilities.

A *Waiting* thread holds **no** lock and consumes no CPU. It transitions back to *Runnable* only when:

(i) another thread calls `notify()` / `notifyAll()`,
(ii) the joined thread terminates, or
(iii) the waiting thread is interrupted.

Aspect	Runnable	Waiting
CPU usage	May use CPU	No CPU usage
Lock held	Possible	No lock held
How to exit	Scheduler picks it	`notify()` / join completes

Q29. Why may deadlocks occur while developing a user interface? Write a program to show how this can be mitigated. **(4+7 marks)**
[CSE 201 | 2013–14]

Answer — Deadlocks in UI and How to Mitigate Them

Why deadlocks occur in UI programs:
Java Swing (and AWT) is **single-threaded**: all UI operations must run on the *Event Dispatch Thread* (EDT). A deadlock occurs when:

(i) A background thread holds a lock **A** and tries to update the UI, so it calls `SwingUtilities.invokeAndWait()` to schedule work on the EDT and then *blocks waiting* for the EDT to finish.

(ii) The EDT is simultaneously trying to acquire lock **A** (e.g. to read shared data before a repaint).

(iii) Both threads wait for each other — **deadlock**.

Mitigation — use `invokeLater` instead of `invokeAndWait`, and keep locks short:

```
import javax.swing.*;
import java.awt.*;

class SharedModel {
    private String value = "initial";

    public synchronized void setValue(String v) { value = v; }
    public synchronized String getValue()       { return value; }
}

public class UIDeadlockDemo {
    public static void main(String[] args) {
        SharedModel model = new SharedModel();
        JLabel label = new JLabel(model.getValue());
        JFrame frame = new JFrame("Safe UI Update");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.add(label, BorderLayout.CENTER);
        frame.setSize(300, 100);
        frame.setVisible(true);

        // Background thread updates the model, then schedules
        // UI refresh on EDT using invokeLater (non-blocking).
        // It does NOT hold any lock while calling invokeLater,
        // so the EDT can freely acquire the model's lock.
        Thread worker = new Thread(() -> {
            for (int i = 1; i <= 10; i++) {
                model.setValue("Step " + i);    // short lock

                // invokeLater: background thread does NOT block
                String snapshot = model.getValue(); // read outside EDT
                SwingUtilities.invokeLater(
                    () -> label.setText(snapshot)  // EDT updates UI
                );

                try { Thread.sleep(500); }
                catch (InterruptedException e) {}
            }
        });
        worker.setDaemon(true);
        worker.start();
    }
}
```

Key rules to prevent UI deadlocks:

- Never call `invokeAndWait()` while holding a lock that the EDT might need.
- Prefer `invokeLater()` (fire-and-forget) over `invokeAndWait()` (blocking).
- Keep lock scopes as small as possible — read shared data, release the lock, then schedule the UI update.

Q30. Write a Java code to solve the Producer-Consumer problem where two threads share a common buffer of size 1. Use inter-thread

communication with synchronized monitor. **(18 marks)** [CSE 201 | 2011–12]

↪ *Similar: CSE 201 | 2007–08*

Answer — Producer-Consumer: Buffer Size 1, Synchronized Monitor

```
class SharedBuffer {
    private int data;
    private boolean hasData = false;

    // Producer calls this
    public synchronized void produce(int value)
            throws InterruptedException {
        while (hasData) wait();     // wait until Consumer reads
        data    = value;
        hasData = true;
        System.out.println("Produced: " + value);
        notifyAll();                // wake Consumer
    }

    // Consumer calls this
    public synchronized int consume()
            throws InterruptedException {
        while (!hasData) wait();    // wait until Producer writes
        hasData = false;
        System.out.println("Consumed: " + data);
        notifyAll();                // wake Producer
        return data;
    }
}

class Producer extends Thread {
    private final SharedBuffer buf;
    Producer(SharedBuffer buf) { this.buf = buf; }
    @Override
    public void run() {
        try {
            for (int i = 1; i <= 10; i++) buf.produce(i);
        } catch (InterruptedException e) {}
    }
}

class Consumer extends Thread {
    private final SharedBuffer buf;
    Consumer(SharedBuffer buf) { this.buf = buf; }
    @Override
    public void run() {
        try {
            for (int i = 0; i < 10; i++) buf.consume();
        } catch (InterruptedException e) {}
    }
}

public class ProducerConsumerDemo {
    public static void main(String[] args) {
        SharedBuffer buf = new SharedBuffer();
        new Producer(buf).start();
        new Consumer(buf).start();
    }
}
```

How it works:

- The buffer holds exactly one integer at a time (`hasData` flag acts as the full/empty indicator).
- `produce()` blocks with `wait()` if `hasData` is `true`; `consume()` blocks if `hasData` is `false`.
- After each read/write, `notifyAll()` unblocks the waiting partner thread.

Q31. Give an example of a multithreaded Java program using the `Runnable` interface. You must ensure that the main thread terminates last. Mention two different ways to achieve synchronization among threads in Java. **(6+4 marks)** [CSE 201 | 2010–11]

Answer — Multithreaded Program with `Runnable`; Main Terminates Last

```
class PrintTask implements Runnable {
    private final String name;
```

```
    PrintTask(String name) { this.name = name; }

    @Override
    public void run() {
        for (int i = 1; i <= 4; i++) {
            System.out.println(name + " -> step " + i);
            try { Thread.sleep(100); } catch (InterruptedException e) {}
        }
        System.out.println(name + " finished.");
    }
}

public class RunnableDemo {
    public static void main(String[] args) {
        Thread t1 = new Thread(new PrintTask("Worker-1"));
        Thread t2 = new Thread(new PrintTask("Worker-2"));
        Thread t3 = new Thread(new PrintTask("Worker-3"));

        t1.start();
        t2.start();
        t3.start();

        // join() ensures main waits until each worker finishes
        try {
            t1.join();
            t2.join();
            t3.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("Main thread terminates last.");
    }
}
```

How main terminates last: `Thread.join()` causes the calling thread (here, `main`) to suspend until the specified thread completes. By calling `join()` on every worker before printing the final message, we guarantee that all workers have finished before `main` exits.

Two ways to achieve synchronisation:

1. **Synchronized method** — declare a method with the `synchronized` keyword; the entire method body becomes a critical section locked on `this`.
2. **Synchronized statement (block)** — wrap only the shared-resource access inside `synchronized(obj){}`; allows a more targeted lock with a programmer-chosen object.

Q32. When is it a must to implement multithreading with the `Runnable` interface instead of extending the `Thread` class? Give a simple example of a multithreaded Java program using the `Runnable` interface. **(15 marks)** [CSE 201 | 2009–10]

Answer — When `Runnable` is Mandatory

When is `Runnable` mandatory?

It is *compulsory* to implement `Runnable` (rather than extending `Thread`) when the class that is to be run as a thread **already extends another class**.

Because Java supports only *single inheritance*, a class cannot extend both `Thread` and another base class simultaneously. Implementing `Runnable` is the only option in this case.

Example — a class that must extend another class:

```
// Suppose we have an existing base class
class Animal {
    String sound;
    Animal(String sound) { this.sound = sound; }
    public void makeSound() { System.out.println(sound); }
}

// Dog must extend Animal AND run as a thread.
// Extending Thread is NOT possible here because Java has
// single inheritance.  Implementing Runnable is the only way.
class Dog extends Animal implements Runnable {
    private int times;

```

```
    Dog(String sound, int times) {
        super(sound);
        this.times = times;
    }

    @Override
    public void run() {
        for (int i = 0; i < times; i++) {
            makeSound();
            try { Thread.sleep(200); } catch (InterruptedException e) {}
        }
    }
}

public class RunnableMustDemo {
    public static void main(String[] args) throws InterruptedException {
        Dog dog1 = new Dog("Woof!", 3);
        Dog dog2 = new Dog("Bark!", 3);

        Thread t1 = new Thread(dog1);
        Thread t2 = new Thread(dog2);

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println("All dogs done. Main exits last.");
    }
}
```

Other reasons to prefer `Runnable` even when not strictly required:

- Separates the task (what to do) from the thread (how to run it).
- The same `Runnable` instance can be submitted to thread pools (`ExecutorService`) without modification.
- Promotes better OOP design — extend a class only when you are specialising its behaviour.

Q33. Define a circular `Queue` class that can be safely shared by multiple threads. Define `Producer` and `Consumer` classes extending Thread. Write a `Test` class with one Queue but several producers and consumers. **(20 marks)** [CSE 201 | 2009–10]

Answer — Thread-Safe Circular Queue with Multiple Producers & Consumers

```
class Queue {
    private final int[] buffer;
    private int head = 0, tail = 0, count = 0;
    private final int capacity;

    Queue(int capacity) {
        this.capacity = capacity;
        this.buffer   = new int[capacity];
    }

    public synchronized void enqueue(int item)
            throws InterruptedException {
        while (count == capacity) wait(); // full -- producer waits
        buffer[tail] = item;
        tail  = (tail + 1) % capacity;   // wrap around
        count++;
        System.out.println(Thread.currentThread().getName()
                           + " enqueued " + item);
        notifyAll();
    }

    public synchronized int dequeue() throws InterruptedException {
        while (count == 0) wait();        // empty -- consumer waits
        int item = buffer[head];
        head  = (head + 1) % capacity;   // wrap around
        count--;
        System.out.println(Thread.currentThread().getName()
                           + " dequeued " + item);
        notifyAll();
```

```
        return item;
    }
}

class Producer extends Thread {
    private final Queue queue;
    private final int id;
    Producer(Queue queue, int id) {
        super("Producer-" + id);
        this.queue = queue; this.id = id;
    }
    @Override
    public void run() {
        try {
            for (int i = 0; i < 5; i++) {
                queue.enqueue(id * 100 + i);
                Thread.sleep(100);
            }
        } catch (InterruptedException e) {}
    }
}

class Consumer extends Thread {
    private final Queue queue;
    Consumer(Queue queue, int id) {
        super("Consumer-" + id);
        this.queue = queue;
    }
    @Override
    public void run() {
        try {
            for (int i = 0; i < 5; i++) {
                queue.dequeue();
                Thread.sleep(150);
            }
        } catch (InterruptedException e) {}
    }
}

class Test {
    public static void main(String[] args) {
        Queue q = new Queue(5); // circular buffer of size 5

        // Multiple producers and consumers
        Producer p1 = new Producer(q, 1);
        Producer p2 = new Producer(q, 2);
        Consumer c1 = new Consumer(q, 1);
        Consumer c2 = new Consumer(q, 2);

        p1.start(); p2.start();
        c1.start(); c2.start();
    }
}
```

Design notes:

- The circular buffer uses `head`, `tail`, and `count` to track the queue state; modular arithmetic (`% capacity`) wraps the pointers.
- All shared state is accessed inside `synchronized` methods, ensuring mutual exclusion.
- `notifyAll()` wakes all waiting threads; each then re-checks its `while` condition before proceeding.

Q34. What do you know about Thread Synchronization? What are the two ways to achieve synchronization? **(8 marks)** [CSE 201 | 2008–09]

Answer — Thread Synchronisation

What is Thread Synchronisation?

When two or more threads share access to a common resource (e.g. a shared variable, file, or data structure), concurrent modifications can leave the resource in an inconsistent state — a situation called a *race condition*. **Thread synchronisation** is the mechanism by which only *one thread at a time* is permitted to access a critical section of code, thereby preventing race conditions and ensuring data integrity.

Java implements synchronisation using *monitors*: every Java object has an implicit monitor (intrinsic lock). A thread that enters a `synchronized` region must first acquire the object's monitor; all other threads trying to enter the same region block until the lock

is released.

Two ways to achieve synchronisation in Java:

Way 1 — Synchronized method

Place the `synchronized` keyword in the method signature. The lock is the `this` reference (or the `Class` object for `static` methods).

```
class SharedBank {
    private double balance;

    public synchronized void deposit(double amount) {
        balance += amount;
    }
    public synchronized double getBalance() {
        return balance;
    }
}
```

Way 2 — Synchronized statement (block)

Enclose only the critical section with `synchronized(lockObj)`. The programmer explicitly specifies the lock object.

```
class SharedBank {
    private double balance;
    private final Object lock = new Object();

    public void deposit(double amount) {
        // logging, validation, etc. run here -- no lock held
        synchronized (lock) {
            balance += amount;
        }
    }
}
```

Comparison:

Synchronized method	Synchronized block
Entire method is locked	Only the critical section is locked
Lock object is always `this`	Any object can be the lock
Simpler to write	More flexible and fine-grained
Higher contention	Lower contention

BUET · CSE 107 / CSE 201

Object Oriented Programming

Section S15

Generics & Collections

S15 — Generics & Collections

Q1. Implement the `MyMax` class to achieve the provided output for the main method with the following restrictions:

(a) `MyMax` class can only be created for Java number classes.

(b) The implementation of methods `sameMax1` and `sameMax2` will be different.

```java
public class MyMaxTest {
    public static void main(String[] args) {
        Long[] n1 = {10L, 20L, 30L, 40L, 50L};
        MyMax<Long> mx1 = new MyMax<>(n1);
        System.out.println(mx1.max()); // 50.0

        Float[] n2 = {50.0f, 20.0f, 40.0f, 10.0f, 30.0f};
        MyMax<Float> mx2 = new MyMax<>(n2);
        System.out.println(mx2.max()); // 50.0

        Double[] n3 = {50.0, 20.0, 40.0, 10.0, 30.0};
        MyMax<Double> mx3 = new MyMax<>(n3);
        System.out.println(mx3.max()); // 50.0

        System.out.println(mx1.sameMax1(mx2)); // true
        System.out.println(mx2.sameMax2(mx3)); // true
    }
}
```

(10 marks) [CSE 107 | 2023–24]

Answer — MyMax with sameMax1 and sameMax2

```java
class MyMax<T extends Number & Comparable<T>> {
    private T[] data;

    MyMax(T[] data) { this.data = data; }

    double max() {
        T m = data[0];
        for (T t : data)
            if (t.compareTo(m) > 0) m = t;
        return m.doubleValue();
    }

    // sameMax1: compares via Double.compare (exact equality check)
    boolean sameMax1(MyMax<? extends Number> other) {
        return Double.compare(this.max(), other.max()) == 0;
    }

    // sameMax2: compares via subtraction with epsilon tolerance
    boolean sameMax2(Object o) {
        if(!(o instanceof MyMax)) return false;
        MyMax<? extends Number> other = (MyMax<? extends Number>) o;

        return Math.abs(this.max() - other.max()) < 1e-9;
    }
}
```

Q2. Write two/three differences between `ArrayList` and `Vector`. **(10 marks)** [CSE 107 | 2016–17, 2021–22, 2022–23]

↪ *Similar: CSE 107 | Jan 2021, 2020–21*

Answer — ArrayList vs Vector

Feature	ArrayList	Vector
Synchronization	Not synchronized (not thread-safe)	Synchronized (thread-safe)
Performance	Faster (no sync overhead)	Slower due to locking
Growth	Grows by 50% when full	Doubles in size when full
Legacy	Introduced in Java 1.2 (modern)	Legacy class (Java 1.0)
Iterator	Uses `Iterator` / `ListIterator`	Also supports `Enumeration`

Recommendation: Prefer `ArrayList` for single-threaded use. For thread safety, use `Collections.synchronizedList()` or `CopyOnWriteArrayList` instead of `Vector`.

Q3. Write three differences between `Hashtable` and `HashMap`. **(10 marks)** [CSE 107 | 2016–17, 2017–18, 2019–20, 2022–23]

Answer — Hashtable vs HashMap

Feature	Hashtable	HashMap
Null keys/values	Neither key nor value can be null	One null key, multiple null values allowed
Synchronization	Synchronized (thread-safe)	Not synchronized
Performance	Slower due to locking	Faster
Legacy	Legacy class (Java 1.0)	Modern (Java 1.2)
Iteration order	No guaranteed order	No guaranteed order

Example showing null:

```
HashMap<String, Double> map = new HashMap<>();
map.put(null, 100.0);         // OK
map.put("Alice", null);       // OK

Hashtable<String, Double> ht = new Hashtable<>();
// ht.put(null, 100.0);      // NullPointerException!
// ht.put("Alice", null);    // NullPointerException!
```

Q4. Create an appropriate Hash-based collection to store IPL players (Pat Cummins: 17,300,000, Kane Williamson: 16,000,000, MS Dhoni: null). Iterate and print values. Increase Pat Cummins's salary by 500,000. **(10 marks)** [CSE 107 | 2022–23]

↪ *Similar: CSE 107 | 2016–17, 2017–18*

Answer — IPL Players HashMap (Supports null Values)

Since one salary is `null`, `HashMap` is required (`Hashtable` does not allow null values).

```
import java.util.HashMap;
import java.util.Map;

class IPLDemo {
    public static void main(String[] args) {
        HashMap<String, Integer> players = new HashMap<>();
        players.put("Pat Cummins",     17_300_000);
        players.put("Kane Williamson", 16_000_000);
        players.put("MS Dhoni",        null);       // null salary

        // Iterate and print
        System.out.println("--- IPL Salaries ---");
        for (Map.Entry<String, Integer> e : players.entrySet())
            System.out.println(e.getKey() + ": " + e.getValue());

        // Increase Pat Cummins's salary by 500,000
        players.put("Pat Cummins",
                    players.get("Pat Cummins") + 500_000);

        System.out.println("\nAfter raise:");
        System.out.println("Pat Cummins: " + players.get("Pat Cummins"));
    }
}
```

Output:

```
--- IPL Salaries ---
Pat Cummins: 17300000
Kane Williamson: 16000000
MS Dhoni: null

After raise:
Pat Cummins: 17800000
```

Q5. Write a generic interface `iQueue` with `enqueue`, `dequeue`, `isEmpty`. Write a generic class `Queue` implementing `iQueue`. The interface only supports numeric types. **(10 marks)** [CSE 107 | 2019–20, 2022–23]

Answer — Generic iQueue Interface and Queue Class (Numeric Only)

```
import java.util.LinkedList;

// Bounded to Number subclasses (numeric types only)
interface iQueue<T extends Number> {
    void    enqueue(T item);
    T       dequeue();
```

```
    boolean isEmpty();
}

class Queue<T extends Number> implements iQueue<T> {
    private LinkedList<T> list = new LinkedList<>();

    @Override
    public void enqueue(T item) { list.addLast(item); }

    @Override
    public T dequeue() {
        if (isEmpty())
            throw new RuntimeException("Queue is empty");
        return list.removeFirst();
    }

    @Override
    public boolean isEmpty() { return list.isEmpty(); }

    @Override
    public String toString() { return list.toString(); }
}

class QueueDemo {
    public static void main(String[] args) {
        Queue<Integer> q = new Queue<>();
        q.enqueue(10); q.enqueue(20); q.enqueue(30);
        System.out.println("Queue: " + q);
        System.out.println("Dequeue: " + q.dequeue());
        System.out.println("Queue: " + q);

        Queue<Double> dq = new Queue<>();
        dq.enqueue(1.5); dq.enqueue(2.5);
        System.out.println("Double Queue: " + dq);
    }
}
```

Output:

```
Queue: [10, 20, 30]
Dequeue: 10
Queue: [20, 30]
Double Queue: [1.5, 2.5]
```

Q6. Implement the `MyStats` class taking `List` as input. Can only be created for Java number classes. `sameAvg1` and `sameAvg2` must have different implementations:

```
Integer[] n1 = {1, 2, 3, 4, 5};
List<Integer> list1 = Arrays.asList(n1);
MyStats<Integer> s1 = new MyStats<>(list1);
System.out.println(s1.average()); // 3.0
Long[] n2 = {5L, 2L, 4L, 1L, 3L};
List<Long> list2 = Arrays.asList(n2);
MyStats<Long> s2 = new MyStats<>(list2);
System.out.println(s2.average()); // 3.0
Double[] n3 = {5.0, 4.0, 3.0, 2.0, 1.0};
List<Double> list3 = Arrays.asList(n3);
MyStats<Double> s3 = new MyStats<>(list3);
System.out.println(s1.sameAvg1(s2)); // true
System.out.println(s2.sameAvg2(s3)); // true
```

(10 marks) [CSE 107 | Jan 2021, 2022–23]

Answer — MyStats Taking List as Input

```
import java.util.*;

class MyStats<T extends Number> {
    private List<T> data;

    MyStats(List<T> data) {
        this.data = data;
    }

```

```
    double average() {
        double sum = 0.0;
        for (T t : data) {
            sum += t.doubleValue();
        }
        return sum / data.size();
    }

    // sameAvg1: Implementation using a Wildcard (?)
    boolean sameAvg1(MyStats<?> other) {
        return Math.abs(this.average() - other.average()) < 1e-9;
    }

    // sameAvg2: Implementation using a Generic Method Parameter (V)
    <V extends Number> boolean sameAvg2(MyStats<V> other) {
        return Math.abs(this.average() - other.average()) < 1e-9;
    }
}

class MyStatsList {
    public static void main(String[] args) {
        List<Integer> list1 = Arrays.asList(1, 2, 3, 4, 5);
        MyStats<Integer> s1 = new MyStats<>(list1);
        System.out.println(s1.average()); // 3.0

        List<Long> list2 = Arrays.asList(5L, 2L, 4L, 1L, 3L);
        MyStats<Long> s2 = new MyStats<>(list2);
        System.out.println(s2.average()); // 3.0

        List<Double> list3 = Arrays.asList(5.0, 4.0, 3.0, 2.0, 1.0);
        MyStats<Double> s3 = new MyStats<>(list3);

        System.out.println(s1.sameAvg1(s2)); // true
        System.out.println(s2.sameAvg2(s3)); // true
    }
}
```

Q7. Write two differences between `ArrayList` and `Vector`. Consider `Employee` (attributes: id, name, age). Write necessary Java code so that `Collections.sort(listEmployees)` sorts by ascending id; if equal, by lexicographic name; if both equal, by ascending age. **(10 marks)** [CSE 107 | 2022–23]

Answer — Employee Sort: id, name, age

Two ArrayList vs Vector differences: ArrayList is not synchronized (faster); Vector is. ArrayList grows 50%; Vector doubles.

```
import java.util.*;

class Employee implements Comparable<Employee> {
    int id; String name; int age;
    Employee(int id, String name, int age) {
        this.id=id; this.name=name; this.age=age;
    }

    @Override
    public int compareTo(Employee o) {
        int c = Integer.compare(this.id, o.id);        // asc id
        if (c != 0) return c;
        c = this.name.compareTo(o.name);               // lex name
        if (c != 0) return c;
        return Integer.compare(this.age, o.age);       // asc age
    }

    @Override
    public String toString() {
        return id + "-" + name + "(" + age + ")";
    }
}

class EmployeeSort {
    public static void main(String[] args) {
        ArrayList<Employee> list = new ArrayList<>();
        list.add(new Employee(2, "Zara",  25));
        list.add(new Employee(1, "Bob",   22));
```

```
        list.add(new Employee(1, "Alice", 30));
        list.add(new Employee(1, "Alice", 25));

        Collections.sort(list);
        list.forEach(System.out::println);
    }
}
```

Output:

```
1-Alice(25)
1-Alice(30)
1-Bob(22)
2-Zara(25)
```

Q8. Implement the `MyAvg` class. Can only be created for Java number classes. It must support `average()`, `sameAvg()` (comparing two same-type objects), and `sameAvgAny()` (comparing two different-type objects):

```
Integer[] n1 = {10, 20, 30, 40, 50};
MyAvg<Integer> s1 = new MyAvg<>(n1);
System.out.println(s1.average()); // 30.0
Integer[] n2 = {50, 20, 40, 10, 30};
MyAvg<Integer> s2 = new MyAvg<>(n2);
System.out.println(s2.average()); // 30.0
System.out.println(s1.sameAvg(s2)); // true
Double[] n3 = {50.0, 40.0, 30.0, 20.0, 10.0};
MyAvg<Double> s3 = new MyAvg<>(n3);
System.out.println(s3.average()); // 30.0
System.out.println(s2.sameAvgAny(s3)); // true
```

(10 marks) [CSE 107 | 2021–22]

Answer — MyAvg with sameAvg and sameAvgAny

```
class MyAvg<T extends Number> {
    private T[] data;

    MyAvg(T[] data) { this.data = data; }

    double average() {
        double sum = 0;
        for (T t : data) sum += t.doubleValue();
        return sum / data.length;
    }

    // sameAvg: same type T
    boolean sameAvg(MyAvg<T> other) {
        return Double.compare(this.average(), other.average()) == 0;
    }

    // sameAvgAny: any numeric type (wildcard)
    boolean sameAvgAny(MyAvg<? extends Number> other) {
        return Math.abs(this.average() - other.average()) < 1e-9;
    }
}

class MyAvgDemo {
    public static void main(String[] args) {
        Integer[] n1 = {10, 20, 30, 40, 50};
        MyAvg<Integer> s1 = new MyAvg<>(n1);
        System.out.println(s1.average());       // 30.0

        Integer[] n2 = {50, 20, 40, 10, 30};
        MyAvg<Integer> s2 = new MyAvg<>(n2);
        System.out.println(s2.average());       // 30.0
        System.out.println(s1.sameAvg(s2));     // true

        Double[] n3 = {50.0, 40.0, 30.0, 20.0, 10.0};
        MyAvg<Double> s3 = new MyAvg<>(n3);
        System.out.println(s3.average());       // 30.0
        System.out.println(s2.sameAvgAny(s3));  // true
    }
}
```

Q9. Given the following `Product` class, write Java code to: (i) define `ArrayList<Product> myProducts`, (ii) generate 4 Products with names `'A'` to `'D'` and random prices, (iii) sort `myProducts` by name in ascending order (you may modify the `Product` class if necessary):

```
class Product {
    private String name; private double price;
    Product(String name, double price) { this.name = name; this.price = price; }
    public String getName() { return this.name; }
    public double getPrice() { return this.price; }
}
```

(10 marks) [CSE 107 | Jan 2021]

Answer — Product Sort by Name Ascending

```
import java.util.*;

class Product implements Comparable<Product> {
    private String name; private double price;
    Product(String name, double price) { this.name=name; this.price=price; }
    public String getName()  { return name; }
    public double getPrice() { return price; }

    @Override
    public int compareTo(Product o) {
        return this.name.compareTo(o.name); // ascending by name
    }

    @Override
    public String toString() {
        return name + "($" + String.format("%.2f", price) + ")";
    }
}

class ProductSortByName {
    public static void main(String[] args) {
        Random rand = new Random();

        // (i) Define ArrayList
        ArrayList<Product> myProducts = new ArrayList<>();

        // (ii) Generate 4 products D, B, A, C with random prices
        String[] names = {"D", "B", "A", "C"};
        for (String n : names)
            myProducts.add(new Product(n, 10 + rand.nextDouble() * 90));

        System.out.println("Before: " + myProducts);

        // (iii) Sort by name ascending
        Collections.sort(myProducts);

        System.out.println("After:  " + myProducts);
    }
}
```

Output (example):

```
Before: [D($45.23), B($78.10), A($12.50), C($67.89)]
After:  [A($12.50), B($78.10), C($67.89), D($45.23)]
```

Q10. Write a generic interface `iStack` with `push`, `pop`, `isEmpty`. Write a generic class `Stack` implementing `iStack`. Interface only supports numeric types. **(10 marks)** [CSE 107 | 2020–21, 2016–17]

↪ *Similar: CSE 107 | 2022–23*

Answer — Generic iStack Interface and Stack Class

```
import java.util.ArrayList;

interface iStack<T extends Number> {
    void    push(T item);
    T       pop();
    boolean isEmpty();
}

class Stack<T extends Number> implements iStack<T> {
```

```
    private ArrayList<T> data = new ArrayList<>();

    @Override
    public void push(T item) { data.add(item); }

    @Override
    public T pop() {
        if (isEmpty()) throw new RuntimeException("Stack is empty");
        return data.remove(data.size() - 1);
    }

    @Override
    public boolean isEmpty() { return data.isEmpty(); }

    @Override
    public String toString() { return data.toString(); }
}

class StackDemo {
    public static void main(String[] args) {
        Stack<Integer> s = new Stack<>();
        s.push(1); s.push(2); s.push(3);
        System.out.println("Stack: " + s);
        System.out.println("Pop: "   + s.pop());
        System.out.println("Pop: "   + s.pop());
        System.out.println("Stack: " + s);
        System.out.println("Empty: " + s.isEmpty());
    }
}
```

Output:

```
Stack: [1, 2, 3]
Pop: 3
Pop: 2
Stack: [1]
Empty: false
```

Q11. What can you say about the following statement?

```
public class ABC<S extends Z & Y & X> { }
```

(5 marks) [CSE 107 | 2020–21, 2016–17]

Answer — Bounded Type Parameter with Multiple Bounds

The statement declares a generic class `ABC` with a type parameter `S` that is **bounded by multiple types**: `S extends Z & Y & X`.
Rules for multiple bounds:

- `S` must be a subtype of **all** listed types.
- **At most one** bound can be a class; the rest must be interfaces. The class bound must come **first**.
- If `Z` is a class, `Y` and `X` must be interfaces. If all are interfaces, any order is valid.

```
interface Y { void doY(); }
interface X { void doX(); }
class Z     { void doZ() {} }

// S must extend Z and implement both Y and X
public class ABC<S extends Z & Y & X> {
    void use(S s) {
        s.doZ(); s.doY(); s.doX(); // all accessible
    }
}

class Concrete extends Z implements Y, X {
    public void doY() {}
    public void doX() {}
}

// ABC<Concrete> ok = new ABC<>();  // valid
```

Q12. Write three differences between `HashMap` and `Hashtable`. Create an appropriate Hash-based collection to store the following monthly

salaries: Sakib Al Hasan: 3,00,000; Mashrafe Bin Mortaza: null; Mushfiqur Rahim: 2,50,000. Then increase Sakib Al Hasan's salary by 50,000. **(3+10 marks)** [CSE 107 | 2017–18]

Answer — Cricketer Salaries with HashMap

Three differences: HashMap allows null key/value; Hashtable does not. HashMap is unsynchronized (faster); Hashtable is synchronized. HashMap is modern (Java 1.2); Hashtable is legacy (Java 1.0).

```java
import java.util.HashMap;
import java.util.Map;

class CricketerSalary {
    public static void main(String[] args) {
        // HashMap chosen because Mashrafe's salary is null
        HashMap<String, Integer> salaries = new HashMap<>();
        salaries.put("Sakib Al Hasan",      300_000);
        salaries.put("Mashrafe Bin Mortaza", null);
        salaries.put("Mushfiqur Rahim",      250_000);

        System.out.println("--- Monthly Salaries ---");
        for (Map.Entry<String, Integer> e : salaries.entrySet())
            System.out.println(e.getKey() + ": " + e.getValue());

        // Increase Sakib Al Hasan's salary by 50,000
        salaries.put("Sakib Al Hasan",
                     salaries.get("Sakib Al Hasan") + 50_000);

        System.out.println("\nSakib after raise: "
                           + salaries.get("Sakib Al Hasan"));
    }
}
```

Output:

```
--- Monthly Salaries ---
Sakib Al Hasan: 300000
Mashrafe Bin Mortaza: null
Mushfiqur Rahim: 250000

Sakib after raise: 350000
```

Q13. Write the general syntax for: (i) declaring a generic class in Java, (ii) declaring a reference to a generic class and creating an instance. **(10 marks)** [CSE 107 | 2017–18]

Answer — Java generic class syntax

```java
// (i) Declaring a generic class in Java:
class MyGeneric<T> {
    private T data;
    public MyGeneric(T val) { data = val; }
    public T getData()       { return data; }
}

// Bounded type parameter:
class Pair<T extends Comparable<T>> {
    T first, second;
    Pair(T a, T b){ first=a; second=b; }
    T max(){ return first.compareTo(second)>=0 ? first : second; }
}

// (ii) Declaring a reference and creating an instance:
MyGeneric<Integer>  intObj  = new MyGeneric<>(42);
MyGeneric<String>   strObj  = new MyGeneric<>("Hello");
Pair<Double>        pair    = new Pair<>(3.14, 2.71);

System.out.println(intObj.getData()); // 42
System.out.println(strObj.getData()); // Hello
System.out.println(pair.max());       // 3.14
```

Q14. Write two differences between `ArrayList` and `Vector`. Write three differences between `Hashtable` and `HashMap`. Create an appropriate Hash-based collection to store the following weekly salaries: Lionel Messi: 350,000; Cristiano Ronaldo: 365,000; Neymar: null. Then increase Lionel Messi's balance by 100,000. **(10 marks)** [CSE 107 | 2016–17]

Answer — Footballer Salaries with HashMap

ArrayList vs Vector (2 differences): ArrayList is not synchronized (faster); Vector is synchronized. ArrayList grows by 50%; Vector doubles in size.
HashMap vs Hashtable (3 differences): HashMap allows null key/value; Hashtable does not. HashMap is unsynchronized; Hashtable is synchronized. HashMap is modern; Hashtable is legacy.

```java
import java.util.HashMap;
import java.util.Map;

class FootballerSalary {
    public static void main(String[] args) {
        // HashMap needed because Neymar's salary is null
        HashMap<String, Integer> salaries = new HashMap<>();
        salaries.put("Lionel Messi",      350_000);
        salaries.put("Cristiano Ronaldo", 365_000);
        salaries.put("Neymar",            null);

        System.out.println("--- Weekly Salaries ---");
        for (Map.Entry<String, Integer> e : salaries.entrySet())
            System.out.println(e.getKey() + ": " + e.getValue());

        // Increase Messi's salary by 100,000
        salaries.put("Lionel Messi",
                     salaries.get("Lionel Messi") + 100_000);

        System.out.println("\nMessi after raise: "
                           + salaries.get("Lionel Messi"));
    }
}
```

Output:

```
--- Weekly Salaries ---
Lionel Messi: 350000
Cristiano Ronaldo: 365000
Neymar: null

Messi after raise: 450000
```

Q15. Write a Java program to store student records as a collection of `Student` objects. The `Student` class contains `Name`, `Age`, and `CGPA`. Write the class so that the collection can be sorted using Java Collection's `sort()` method with the following priority: (i) descending CGPA; (ii) if CGPAs are equal, ascending age; (iii) if ages are also equal, lexicographic order of name. **(20 marks)** [CSE 201 | 2015–16]

Answer — Student Collection Sorted by CGPA, Age, Name

```java
import java.util.*;

class Student implements Comparable<Student> {
    String name; int age; double cgpa;

    Student(String name, int age, double cgpa) {
        this.name=name; this.age=age; this.cgpa=cgpa;
    }

    @Override
    public int compareTo(Student o) {
        int c = Double.compare(o.cgpa, this.cgpa); // desc CGPA
        if (c != 0) return c;
        c = Integer.compare(this.age, o.age);       // asc age
        if (c != 0) return c;
        return this.name.compareTo(o.name);         // lex name
    }

    @Override
    public String toString() {
        return name + "(age=" + age + ",cgpa=" + cgpa + ")";
    }
}

class StudentCollectionSort {
    public static void main(String[] args) {
        ArrayList<Student> list = new ArrayList<>();
```

```
        list.add(new Student("Alice", 21, 3.8));
        list.add(new Student("Bob",   20, 3.9));
        list.add(new Student("Carol", 22, 3.8));
        list.add(new Student("Dave",  21, 3.8));
        list.add(new Student("Eve",   19, 4.0));

        Collections.sort(list);
        list.forEach(System.out::println);
    }
}
```

Output:

```
Eve(age=19,cgpa=4.0)
Bob(age=20,cgpa=3.9)
Alice(age=21,cgpa=3.8)
Dave(age=21,cgpa=3.8)
Carol(age=22,cgpa=3.8)
```

Q16. Write five differences between `Hashtable` and `HashMap`. Consider the following bank account information. Write Java code to: (i) create a Hashtable, (ii) store the five entries, (iii) increase John Doe's balance by 100:

Name	Balance
John Doe	3434.34
Tom Smith	123.22
Jane Baker	1378.00
Tod Hall	99.22
Ralph Smith	19.08

(10 marks) [CSE 201 | 2014–15]

Answer — Hashtable with Bank Accounts

Five differences (see Q2 table above; additional points): null keys/values forbidden in Hashtable; Hashtable is legacy/synchronized; HashMap is modern/unsynchronized; HashMap allows one null key and multiple null values; HashMap is generally preferred in new code.

```
import java.util.Hashtable;

class BankDemo {
    public static void main(String[] args) {
        // (i) Create Hashtable
        Hashtable<String, Double> accounts = new Hashtable<>();

        // (ii) Store five entries
        accounts.put("John Doe",    3434.34);
        accounts.put("Tom Smith",    123.22);
        accounts.put("Jane Baker",  1378.00);
        accounts.put("Tod Hall",      99.22);
        accounts.put("Ralph Smith",   19.08);

        System.out.println("Before: " + accounts.get("John Doe"));

        // (iii) Increase John Doe's balance by 100
        accounts.put("John Doe", accounts.get("John Doe") + 100);

        System.out.println("After:  " + accounts.get("John Doe"));

        // Print all
        for (String name : accounts.keySet())
            System.out.printf("%-15s %.2f%n",
                              name, accounts.get(name));
    }
}
```

Output:

```
Before: 3434.34
After:  3534.34
```

Q17. Consider the following class. Write Java code to: (i) define `ArrayList<Product> myProducts`, (ii) generate 5 Products A–E with random prices, (iii) add Product F with price 1000 at index 1, (iv) sort by price ascending:

```
class Product {
```

```
    private String name; private double price;
    Product(String name, double price) { this.name = name; this.price = price; }
    public String getName() { return this.name; }
    public double getPrice() { return this.price; }
}
```

(10 marks) [CSE 201 | 2014–15]

Answer — Product ArrayList: Generate, Insert, Sort by Price

```
import java.util.*;

class Product {
    private String name; private double price;
    Product(String name, double price) { this.name=name; this.price=price; }
    public String getName()  { return name; }
    public double getPrice() { return price; }
    @Override
    public String toString() {
        return name + "($" + String.format("%.2f", price) + ")";
    }
}

class ProductDemo {
    public static void main(String[] args) {
        Random rand = new Random();

        // (i) Define ArrayList
        ArrayList<Product> myProducts = new ArrayList<>();

        // (ii) Generate 5 products A-E with random prices
        for (char c = 'A'; c <= 'E'; c++)
            myProducts.add(new Product(String.valueOf(c),
                                       10 + rand.nextDouble() * 90));

        System.out.println("Before insert: " + myProducts);

        // (iii) Add Product F with price 1000 at index 1
        myProducts.add(1, new Product("F", 1000));

        System.out.println("After insert:  " + myProducts);

        // (iv) Sort by price ascending
        myProducts.sort(Comparator.comparingDouble(Product::getPrice));

        System.out.println("After sort:    " + myProducts);
    }
}
```

Q18. A class `Employee` has two member variables: name (`String`) and age (`int`). Write the class so an `ArrayList` of `Employee` objects can be sorted by either variable. **(14 marks)** [CSE 201 | 2013–14]

Answer — Employee Sorted by Name or Age

```
import java.util.*;

class Employee implements Comparable<Employee> {
    String name; int age;
    Employee(String name, int age) { this.name=name; this.age=age; }

    // Natural order: by name
    @Override
    public int compareTo(Employee o) { return this.name.compareTo(o.name); }

    // Comparator for age
    static Comparator<Employee> byAge() {
        return Comparator.comparingInt(e -> e.age);
    }

    @Override
    public String toString() { return name + "(" + age + ")"; }
}

class EmployeeSortDemo {
```

```
    public static void main(String[] args) {
        ArrayList<Employee> list = new ArrayList<>();
        list.add(new Employee("Zara", 25));
        list.add(new Employee("Alice", 30));
        list.add(new Employee("Bob", 22));

        Collections.sort(list);           // sort by name
        System.out.println("By name: " + list);

        list.sort(Employee.byAge());      // sort by age
        System.out.println("By age:  " + list);
    }
}
```

Output:

```
By name: [Alice(30), Bob(22), Zara(25)]
By age:  [Bob(22), Zara(25), Alice(30)]
```

Q19. Write a class `Student` containing a `HashMap` with Integer key and List of String as values. Add 3 studentIds as keys with [firstname, lastname] values. Write a method to print all. **(13 marks)** [CSE 201 | 2013–14]

Answer — Student HashMap with List Values

```
import java.util.*;

class StudentMap {
    private HashMap<Integer, List<String>> students = new HashMap<>();

    void addStudent(int id, String firstName, String lastName) {
        List<String> names = new ArrayList<>();
        names.add(firstName); names.add(lastName);
        students.put(id, names);
    }

    void printAll() {
        for (Map.Entry<Integer, List<String>> e : students.entrySet())
            System.out.println("ID: " + e.getKey()
                               + " -> " + e.getValue());
    }

    public static void main(String[] args) {
        StudentMap sm = new StudentMap();
        sm.addStudent(1001, "Alice", "Smith");
        sm.addStudent(1002, "Bob",   "Jones");
        sm.addStudent(1003, "Carol", "White");
        sm.printAll();
    }
}
```

Output:

```
ID: 1001 -> [Alice, Smith]
ID: 1002 -> [Bob, Jones]
ID: 1003 -> [Carol, White]
```

Q20. Consider a file named ‘products.txt’ containing following information.

```
1,   P1,   T1,   10
2,   P2,   T2,   20
3,   P3,   T1,   30
4,   P4,   T2,   20
5,   P5,   T1,   30
```

This file represents information of products where the attributes are (id, name, type, price). A single product’s information is contained in a single line. You need to do the following.

(i) Write a class in Java named Product with necessary attributes.

(ii) Write a function in Java to store the products information from this ‘products.txt’ file to an ArrayList of Product named ‘myProducts’.

(iii) Write a function in Java to store the contents of an ArrayList of Product named ‘myProducts’ to a file named ‘products2.txt’ in the same format as given in the ‘products.txt’ file.

(4+8+8 marks) [CSE 201 | 2008–09]

Answer — Product Read/Write from File

```java
import java.io.*;
import java.util.*;

// (i) Product class with necessary attributes
class Product {
    int id;
    String name, type;
    int price; // using int based on the sample data provided

    Product(int id, String name, String type, int price) {
        this.id = id;
        this.name = name;
        this.type = type;
        this.price = price;
    }

    @Override
    public String toString() {
        return id + ", " + name + ", " + type + ", " + price;
    }
}

class ProductFileDemo {
    // (ii) Function to read products.txt into an ArrayList
    static ArrayList<Product> readProducts(String filename) throws IOException {
        ArrayList<Product> myProducts = new ArrayList<>();
        try (BufferedReader br = new BufferedReader(new FileReader(filename))) {
            String line;
            while ((line = br.readLine()) != null) {
                if(line.trim().isEmpty()) continue;
                String[] p = line.split(",");
                // trimming is necessary because of spaces after commas
                myProducts.add(new Product(
                        Integer.parseInt(p[0].trim()),
                        p[1].trim(),
                        p[2].trim(),
                        Integer.parseInt(p[3].trim())
                ));
            }
        }
        return myProducts;
    }

    // (iii) Function to write ArrayList to products2.txt in the same format
    static void writeProducts(ArrayList<Product> myProducts, String filename)
            throws IOException {
        try (BufferedWriter bw = new BufferedWriter(new FileWriter(filename))) {
            for (Product p : myProducts) {
                bw.write(p.toString());
                bw.newLine();
            }
        }
    }

    public static void main(String[] args) {
        try {
            ArrayList<Product> myProducts = readProducts("products.txt");
            writeProducts(myProducts, "products2.txt");
            System.out.println("Products successfully copied to products2.txt");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

BUET · CSE 107 / CSE 201

Object Oriented Programming

Section S16

Stream API, Lambda & Enums

S16 — Stream API, Lambda & Enums

Q1. Write Java code for the following: (i) Declare an ArrayList of `Employee`, add 6 employees; (ii) Rewrite two code segments using Java Stream; (iii) Sort by ascending age; if equal, by ascending name:

```
// Segment 1
List<String> names = new ArrayList<>();
for (Employee e : list) names.add(e.getName());
// Segment 2
List<Employee> employees = new ArrayList<>();
for (Employee e : list) if (e.getAge() <= 20) employees.add(e);
```

(15 marks) [CSE 107 | 2023–24]

Answer — Employee Stream Rewrite and Sort

(Uses the same `Employee` class as Q2.)

```
import java.util.*;
import java.util.stream.*;

public class EmployeeStreamDemo {
    public static void main(String[] args) {

        // (i) ArrayList<Employee> with 6 employees
        ArrayList<Employee> list = new ArrayList<>();
        list.add(new Employee(1, "Alice",  22));
        list.add(new Employee(2, "Bob",    19));
        list.add(new Employee(3, "Carol",  20));
        list.add(new Employee(4, "Dave",   20));
        list.add(new Employee(5, "Eve",    17));
        list.add(new Employee(6, "Frank",  25));

        // -- Segment 1 (original for-each loop) -----------
        List<String> names1 = new ArrayList<>();
        for (Employee e : list) names1.add(e.getName());

        // (ii-a) Segment 1 using Stream
        List<String> names2 = list.stream()
            .map(Employee::getName)
            .collect(Collectors.toList());

        System.out.println("All names: " + names2);

        // -- Segment 2 (original for-each loop) -----------
        List<Employee> filtered1 = new ArrayList<>();
        for (Employee e : list)
            if (e.getAge() <= 20) filtered1.add(e);

        // (ii-b) Segment 2 using Stream
        List<Employee> filtered2 = list.stream()
            .filter(e -> e.getAge() <= 20)
            .collect(Collectors.toList());

        System.out.println("Age <= 20: " + filtered2);

        // (iii) Sort: ascending age, then ascending name
        list.sort(Comparator.comparingInt(Employee::getAge)
                            .thenComparing(Employee::getName));

        System.out.println("Sorted: " + list);
    }
}
```

Output:

```
All names: [Alice, Bob, Carol, Dave, Eve, Frank]
Age <= 20: [Bob, Carol, Dave, Eve]
Sorted: [5:Eve(17), 2:Bob(19), 3:Carol(20), 4:Dave(20),
         1:Alice(22), 6:Frank(25)]
```

Q2. Develop Java code for the following using an `Employee` class (id, name, age): (i) Declare `ArrayList<Employee>` and add 6 employees; (ii) Store names of employees aged > 15 in `nameList`; (iii) Sort by ascending age; if equal, by ascending name. **(15 marks)** [CSE 107 | 2021–22]

Answer — Employee ArrayList, Filter and Sort

```java
import java.util.*;
import java.util.stream.*;

class Employee {
    private int    id;
    private String name;
    private int    age;

    Employee(int id, String name, int age) {
        this.id = id; this.name = name; this.age = age;
    }
    public int    getId()   { return id;   }
    public String getName() { return name; }
    public int    getAge()  { return age;  }

    @Override
    public String toString() {
        return id + ":" + name + "(" + age + ")";
    }
}

public class EmployeeDemo {
    public static void main(String[] args) {

        // (i) ArrayList<Employee> with 6 employees
        ArrayList<Employee> list = new ArrayList<>();
        list.add(new Employee(1, "Alice",  22));
        list.add(new Employee(2, "Bob",    14));
        list.add(new Employee(3, "Carol",  19));
        list.add(new Employee(4, "Dave",   30));
        list.add(new Employee(5, "Eve",    16));
        list.add(new Employee(6, "Frank",  25));

        // (ii) Store names of employees aged > 15
        List<String> nameList = list.stream()
            .filter(e -> e.getAge() > 15)
            .map(Employee::getName)
            .collect(Collectors.toList());

        System.out.println("Names (age > 15): " + nameList);

        // (iii) Sort by ascending age; if equal, ascending name
        list.sort(Comparator.comparingInt(Employee::getAge)
                            .thenComparing(Employee::getName));

        System.out.println("Sorted employees: " + list);
    }
}
```

Output:

```
Names (age > 15): [Alice, Carol, Eve, Dave, Frank]
Sorted employees: [2:Bob(14), 5:Eve(16), 3:Carol(19),
                   1:Alice(22), 6:Frank(25), 4:Dave(30)]
```

Q3. Create a Java enum named `HttpStatus` to represent the following HTTP status codes: **(10 marks)**

`OK (200, "OK"), NOT_FOUND (404, "Not Found"),`

`INTERNAL_ERROR (500, "Internal Server Error")`

Each enum constant should store two fields: an `int` status code and a `String` message.
Your enum should also implement the following methods:

(a) `isError()`: Returns false for `OK`, and true for both `NOT_FOUND` and `INTERNAL_ERROR`

(b) `toString()`: Returns a string in the format ”200 OK”, ”404 Not Found”, ”500 Internal Server Error”

(c) `fromCode(int code)` (static method): Takes an `int` code and returns the corresponding `HttpStatus` constant. If the code doesn't match any defined status, return `null`.

[CSE 107 | 2023–24]

Answer — HttpStatus Enum

```java
public enum HttpStatus {
    OK(200, "OK"),
    NOT_FOUND(404, "Not Found"),
    INTERNAL_ERROR(500, "Internal Server Error");

    private final int code;
    private final String message;

    // Constructor to store the two fields
    HttpStatus(int code, String message) {
        this.code = code;
        this.message = message;
    }

    // i. isError() method
    public boolean isError() {
        return this != OK;
    }

    // ii. toString() method
    @Override
    public String toString() {
        return code + " " + message;
    }

    // iii. fromCode(int code) static method
    public static HttpStatus fromCode(int code) {
        for (HttpStatus status : HttpStatus.values()) {
            if (status.code == code) {
                return status;
            }
        }
        return null;
    }

    // Demonstration (Optional but helpful for testing)
    public static void main(String[] args) {
        System.out.println(HttpStatus.OK);                    // 200 OK
        System.out.println(HttpStatus.NOT_FOUND);             // 404 Not Found
        System.out.println(HttpStatus.OK.isError());          // false
        System.out.println(HttpStatus.NOT_FOUND.isError());   // true
        System.out.println(HttpStatus.fromCode(500));         // 500 Internal Server Error
    }
}
```

Q4. Consider the following `Person` class. Write Java code for the following: (i) Declare `ArrayList<Person> list` and add 6 persons; (ii) Rewrite the two segments using Java Stream; (iii) Sort the list by ascending age, then ascending name if age is equal:

```java
// Segment 1
List<String> names = new ArrayList<>();
for (int i = 0; i < list.size(); i++) { Person mc = list.get(i); names.add(mc.getName()); }
// Segment 2
List<Person> persons = new ArrayList<>();
for (int i = 0; i < list.size(); i++) { Person mc = list.get(i); if (mc.getAge() <= 20) persons.add
    (mc); }
```

(20 marks) [CSE 107 | Jan 2021]

↪ *Similar: CSE 107 | 2021–22, 2022–23, 2023–24*

Answer — Person ArrayList, Streams and Sorting

```java
import java.util.*;
import java.util.stream.*;

class Person {
    private String name;
    private int    age;

    Person(String name, int age) {
        this.name = name; this.age = age;
    }
    public String getName() { return name; }
```

```
    public int    getAge()  { return age;  }

    @Override
    public String toString() {
        return name + "(" + age + ")";
    }
}

public class PersonDemo {
    public static void main(String[] args) {

        // (i) Declare ArrayList<Person> and add 6 persons
        ArrayList<Person> list = new ArrayList<>();
        list.add(new Person("Alice",  22));
        list.add(new Person("Bob",    19));
        list.add(new Person("Carol",  20));
        list.add(new Person("Dave",   20));
        list.add(new Person("Eve",    17));
        list.add(new Person("Frank",  25));

        // -- Original Segment 1 (loop) ---------------------
        List<String> names1 = new ArrayList<>();
        for (int i = 0; i < list.size(); i++) {
            Person mc = list.get(i);
            names1.add(mc.getName());
        }

        // (ii-a) Segment 1 rewritten using Stream
        List<String> names2 = list.stream()
            .map(Person::getName)
            .collect(Collectors.toList());

        System.out.println("All names (stream): " + names2);

        // -- Original Segment 2 (loop) ---------------------
        List<Person> persons1 = new ArrayList<>();
        for (int i = 0; i < list.size(); i++) {
            Person mc = list.get(i);
            if (mc.getAge() <= 20) persons1.add(mc);
        }

        // (ii-b) Segment 2 rewritten using Stream
        List<Person> persons2 = list.stream()
            .filter(p -> p.getAge() <= 20)
            .collect(Collectors.toList());

        System.out.println("Age <= 20 (stream): " + persons2);

        // (iii) Sort: ascending age, then ascending name
        list.sort(Comparator.comparingInt(Person::getAge)
                            .thenComparing(Person::getName));

        System.out.println("Sorted list: " + list);
    }
}
```

Output:

```
All names (stream): [Alice, Bob, Carol, Dave, Eve, Frank]
Age <= 20 (stream): [Bob, Carol, Dave, Eve]
Sorted list: [Eve(17), Bob(19), Carol(20), Dave(20),
              Alice(22), Frank(25)]
```

Key stream operations used:

- `.map(Person::getName)` — transforms each `Person` to its name (method reference).
- `.filter(p -> p.getAge() <= 20)` — keeps only persons aged ≤ 20.
- `Comparator.comparingInt(...).thenComparing(...)` — chains two comparators for multi-key sorting.

Q5. Consider the `Fruit` class with `name` and `quantity`. Write Java code so that `Collections.sort(listFruits)` sorts by descending quantity; if equal, by lexicographic name. **(7 marks)** [CSE 107 | 2016–17, 2020–21]

Answer — Fruit: Descending Quantity, Then Lexicographic Name

`Collections.sort()` uses the natural ordering defined by `Comparable<Fruit>`. We override `compareTo()` to sort by **descending quantity** first, then **lexicographic (ascending) name**.

```
import java.util.*;

class Fruit implements Comparable<Fruit> {
    private String name;
    private int    quantity;

    Fruit(String name, int quantity) {
        this.name = name; this.quantity = quantity;
    }
    public String getName()    { return name;     }
    public int    getQuantity(){ return quantity; }

    @Override
    public int compareTo(Fruit other) {
        // 1. Descending quantity
        int c = Integer.compare(other.quantity, this.quantity);
        if (c != 0) return c;
        // 2. Ascending (lexicographic) name
        return this.name.compareTo(other.name);
    }

    @Override
    public String toString() {
        return name + ":" + quantity;
    }
}

public class FruitDemo {
    public static void main(String[] args) {
        List<Fruit> listFruits = new ArrayList<>();
        listFruits.add(new Fruit("Mango",  10));
        listFruits.add(new Fruit("Apple",  25));
        listFruits.add(new Fruit("Banana", 25));
        listFruits.add(new Fruit("Grape",   8));
        listFruits.add(new Fruit("Cherry", 15));

        Collections.sort(listFruits);

        System.out.println(listFruits);
    }
}
```

Output:

```
[Apple:25, Banana:25, Cherry:15, Mango:10, Grape:8]
```

Explanation: Apple and Banana both have quantity 25; they are tied on quantity so they are sorted alphabetically (Apple before Banana). Cherry (15), Mango (10), Grape (8) follow in descending quantity order.

Q6. Consider the following FIFA World Cup data. Declare Enumeration `FWCWinners`. Print all countries. Print the most recent winner (cannot simply write Argentina):

Country	Titles	Last Title
Argentina	3	2022
France	2	2018
Uruguay	2	1950
England	1	1966
Spain	1	2010

(10 marks) [CSE 107 | 2021–22]

Answer — `FWCWinners` Enumeration

```
enum FWCWinners {
    ARGENTINA(3, 2022),
    FRANCE(2, 2018),
    URUGUAY(2, 1950),
    ENGLAND(1, 1966),
    SPAIN(1, 2010);

```

```
    private final int titles;
    private final int lastTitle;

    FWCWinners(int titles, int lastTitle) {
        this.titles = titles;
        this.lastTitle = lastTitle;
    }

    public int getTitles()    { return titles; }
    public int getLastTitle() { return lastTitle; }
}

public class Main {
    public static void main(String[] args) {

        // Print all countries
        System.out.println("All FIFA World Cup Winners:");
        for (FWCWinners w : FWCWinners.values())
            System.out.println(w.name()
                + " | Titles: " + w.getTitles()
                + " | Last: " + w.getLastTitle());

        // Most recent winner (cannot simply write Argentina)
        FWCWinners recent = FWCWinners.values()[0];
        for (FWCWinners w : FWCWinners.values())
            if (w.getLastTitle() > recent.getLastTitle())
                recent = w;

        System.out.println("\nMost recent winner: " + recent.name()
            + " (" + recent.getLastTitle() + ")");
    }
}
```

Output:

```
All FIFA World Cup Winners:
ARGENTINA | Titles: 3 | Last: 2022
FRANCE | Titles: 2 | Last: 2018
URUGUAY | Titles: 2 | Last: 1950
ENGLAND | Titles: 1 | Last: 1966
SPAIN | Titles: 1 | Last: 2010

Most recent winner: ARGENTINA (2022)
```

Q7. Consider the following EPL data. Write Java code to: (i) Declare a single Enumeration named `EPL`; (ii) Print all clubs with points and position:

Club	Point	Position
Manchester United	90	1
Chelsea	80	2
Arsenal	75	4
Manchester City	30	19
Liverpool	25	20

(5 marks) **[CSE 107 | Jan 2021]**

Answer — EPL Enumeration

```
enum EPL {
    MANCHESTER_UNITED(90, 1),
    CHELSEA(80, 2),
    ARSENAL(75, 4),
    MANCHESTER_CITY(30, 19),
    LIVERPOOL(25, 20);

    private final int points;
    private final int position;

    EPL(int points, int position) {
        this.points = points;
        this.position = position;
    }

```

```
    public int getPoints()   { return points; }
    public int getPosition() { return position; }
}

public class Main {
    public static void main(String[] args) {
        // Print all clubs with points and position
        for (EPL club : EPL.values()) {
            System.out.println(club.name()
                + " | Points: " + club.getPoints()
                + " | Position: " + club.getPosition());
        }
    }
}
```

Output:

```
MANCHESTER_UNITED | Points: 90 | Position: 1
CHELSEA | Points: 80 | Position: 2
ARSENAL | Points: 75 | Position: 4
MANCHESTER_CITY | Points: 30 | Position: 19
LIVERPOOL | Points: 25 | Position: 20
```

Q8. Declare Enumeration `GOT` for Game of Thrones characters. Find the character active for the least number of seasons (cannot simply write Ned Stark):

Name	Status	NoOfSeasons
Jon Snow	Alive	7
Daenerys Targaryen	Alive	7
Ned Stark	Dead	1
Joffrey Baratheon	Dead	4
Tyrion Lannister	Alive	7

(7 marks) [CSE 107 | 2016–17]

Answer — `GOT` Enumeration: Least Active Character

```
enum GOT {
    JON_SNOW("Alive", 7),
    DAENERYS_TARGARYEN("Alive", 7),
    NED_STARK("Dead", 1),
    JOFFREY_BARATHEON("Dead", 4),
    TYRION_LANNISTER("Alive", 7);

    private final String status;
    private final int seasons;

    GOT(String status, int seasons) {
        this.status = status;
        this.seasons = seasons;
    }

    public int getSeasons() { return seasons; }
}

public class Main {
    public static void main(String[] args) {

        // Find character with least seasons (cannot simply write Ned Stark)
        GOT least = GOT.values()[0];
        for (GOT c : GOT.values())
            if (c.getSeasons() < least.getSeasons())
                least = c;

        System.out.println("Least active: " + least.name()
            + " (" + least.getSeasons() + " season(s))");
    }
}
```

Output:

```
Least active: NED_STARK (1 season(s))
```

Q9. What is a functional interface in Java? Give an example to implement the functional interface as an Anonymous Inner class. **(10 marks)** [CSE 201 | 2015–16]

Answer — Functional Interface and Anonymous Inner Class

What is a functional interface?

A **functional interface** is an interface that contains **exactly one abstract method**. It may have any number of `default` or `static` methods, but only one abstract method. The `@FunctionalInterface` annotation enforces this contract at compile time. Functional interfaces are the foundation of **lambda expressions** and **method references** in Java 8+. Examples from the standard library include `Runnable`, `Comparator<T>`, `Predicate<T>`, and `Function<T,R>`.

```
// Define a custom functional interface
@FunctionalInterface
interface MathOperation {
    int operate(int a, int b);   // single abstract method

    // default methods are allowed
    default void describe() {
        System.out.println("A math operation");
    }
}
```

Example 1: Implement as an Anonymous Inner Class

```
public class FunctionalDemo {
    public static void main(String[] args) {

        // Anonymous inner class implementing MathOperation
        MathOperation addition = new MathOperation() {
            @Override
            public int operate(int a, int b) {
                return a + b;
            }
        };

        MathOperation multiplication = new MathOperation() {
            @Override
            public int operate(int a, int b) {
                return a * b;
            }
        };

        System.out.println("5 + 3 = " + addition.operate(5, 3));
        System.out.println("5 * 3 = " + multiplication.operate(5, 3));
        addition.describe();
    }
}
```

Output:

```
5 + 3 = 8
5 * 3 = 15
A math operation
```

Example 2: Equivalent lambda expression (Java 8+)

```
// Lambda is syntactic sugar for the anonymous class above
MathOperation subtraction = (a, b) -> a - b;
System.out.println("5 - 3 = " + subtraction.operate(5, 3));
```

Key points:

- An anonymous inner class provides an instance of the interface without explicitly naming a class. It is verbose but works on all Java versions.
- A lambda expression is a concise replacement for a single-method anonymous class and is only available for functional interfaces.
- Both approaches are interchangeable for functional interfaces; the lambda is preferred in modern Java for readability.

Q10. What is a functional interface? Using lambda expression, implement the following interface to compute factorial:

```
interface SomeFunc<T> { T func(T t); }
```

(10 marks) [CSE 107 | 2015–16]

Answer — Functional Interface & Lambda: Factorial

A **functional interface** has exactly one abstract method. The `@FunctionalInterface` annotation is optional but recommended.

```java
interface SomeFunc<T> { T func(T t); }

public class Main {
    public static void main(String[] args) {

        // Lambda expression implementing SomeFunc<Integer>
        SomeFunc<Integer> factorial = n -> {
            int result = 1;
            for (int i = 2; i <= n; i++)
                result *= i;
            return result;
        };

        System.out.println("5! = " + factorial.func(5));  // 120
        System.out.println("6! = " + factorial.func(6));  // 720
    }
}
```

Output:

```
5! = 120
6! = 720
```

Q11. Declare a single Enumeration named `EPL`. Find out the difference between league position of ManUnited and Liverpool (cannot simply write 18). Also declare Enumeration for football clubs from EPL table. **(5 marks)** [CSE 201 | 2014–15]

Answer — `EPL` Enum: Position Difference

```java
enum EPL {
    MANCHESTER_UNITED(90, 1),
    CHELSEA(80, 2),
    ARSENAL(75, 4),
    MANCHESTER_CITY(30, 19),
    LIVERPOOL(25, 20);

    private final int points;
    private final int position;

    EPL(int points, int position) {
        this.points = points;
        this.position = position;
    }
    public int getPosition() { return position; }
}

public class Main {
    public static void main(String[] args) {
        // Cannot simply write 18 --- compute from enum data
        int diff = EPL.LIVERPOOL.getPosition()
                 - EPL.MANCHESTER_UNITED.getPosition();
        System.out.println("Position difference: " + diff);  // 19
    }
}
```

Output:

```
Position difference: 19
```

BUET · CSE 107 / CSE 201

Object Oriented Programming

Section S17

File I/O & Serialization

S17 — File I/O & Serialization

Q1. Consider a Java program that copies `src.txt` to `dst.txt` line by line. Implement using try-with-resources. You are not allowed to throw Exception anywhere. **(10 marks)** [CSE 107 | 2023–24]

Answer — Copy src.txt to dst.txt Line by Line (try-with-resources)

```java
import java.io.*;

public class FileCopy {
    public static void main(String[] args) {
        try (BufferedReader reader = new BufferedReader(
                 new FileReader("src.txt"));
             BufferedWriter writer = new BufferedWriter(
                 new FileWriter("dst.txt"))) {

            String line;
            while ((line = reader.readLine()) != null) {
                writer.write(line);
                writer.newLine();
            }
            System.out.println("File copied successfully.");

        } catch (FileNotFoundException e) {
            System.out.println("File not found: " + e.getMessage());
        } catch (IOException e) {
            System.out.println("IO error: " + e.getMessage());
        }
        // reader and writer closed automatically -- no throw anywhere
    }
}
```

Key points:

- `try-with-resources` closes both `reader` and `writer` automatically in reverse order.
- All exceptions are caught locally — no `throws` clause is used anywhere.

Q2. Consider a Java program that copies binary file `src.mkv` to `dst.mkv` using chunks of 8 KB. Implement using try-with-resources. Not allowed to throw Exception anywhere. **(10 marks)** [CSE 107 | 2021–22]

Answer — Copy Binary File in 8 KB Chunks (try-with-resources)

```java
import java.io.*;

public class BinaryFileCopy {
    private static final int CHUNK = 8 * 1024; // 8 KB

    public static void main(String[] args) {
        try (InputStream  in  = new BufferedInputStream(
                 new FileInputStream("src.mkv"), CHUNK);
             OutputStream out = new BufferedOutputStream(
                 new FileOutputStream("dst.mkv"), CHUNK)) {

            byte[] buffer = new byte[CHUNK];
            int bytesRead;
            long total = 0;

            while ((bytesRead = in.read(buffer)) != -1) {
                out.write(buffer, 0, bytesRead);
                total += bytesRead;
            }

            System.out.printf("Copied %.2f MB%n",
                              total / (1024.0 * 1024));

        } catch (FileNotFoundException e) {
            System.out.println("File not found: " + e.getMessage());
        } catch (IOException e) {
            System.out.println("IO error: " + e.getMessage());
        }
        // no throws anywhere -- all caught locally
    }
}
```

Key points:

- `byte[] buffer = new byte[8192]` reads/writes exactly 8 KB at a time for efficiency.
- `in.read(buffer)` returns `-1` at end of file.
- `out.write(buffer, 0, bytesRead)` writes only the bytes actually read — important for the last chunk.

Q3. Write necessary Java code to write `object1` to a file named `serial/test` and read the object from the same file and assign it to `object2`. **(8+9 marks)**

```java
class MyClass {
    String s;
    int i;
    public MyClass(String s, int i) {
        this.s = s;
        this.i = i;
    }
}
```

```java
public class FileDemo {
    public static void main(String args[]) {
        MyClass object1 = new MyClass("James Bond"
            , 7);
        // your code
        MyClass object2;
        // your code
    }
}
```

[CSE 107 | 2016–17, 2017–18]

Answer — Serialize object1 to File and Deserialize to object2

```java
import java.io.*;

class MyClass implements Serializable {
    int id; String name;
    MyClass(int id, String name) { this.id=id; this.name=name; }
    @Override
    public String toString() { return "MyClass{id="+id+",name="+name+"}"; }
}

class SerialDemo {
    public static void main(String[] args) {
        MyClass object1 = new MyClass(42, "Alice");

        // Write object1 to file "serial"
        try (ObjectOutputStream oos = new ObjectOutputStream(
                new FileOutputStream("serial"))) {
            oos.writeObject(object1);
            System.out.println("Written: " + object1);
        } catch (IOException e) { e.printStackTrace(); }

        // Read object from "serial" and assign to object2
        MyClass object2 = null;
        try (ObjectInputStream ois = new ObjectInputStream(
                new FileInputStream("serial"))) {
            object2 = (MyClass) ois.readObject();
            System.out.println("Read:    " + object2);
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
        }
    }
}
```

Output:

```
Written: MyClass{id=42,name=Alice}
Read:    MyClass{id=42,name=Alice}
```

Q4. You are given a text file called `MyAnalysisOfStudentsPerformance.txt`. Write necessary code to read each line from the file and print out to the console. However, in your output each line should be prefixed with the line number. Your I/O operations should be efficient. Once you have run your code, you observe (to your surprise) that the first line in the console output is:

1. আমার ভয় হচ্ছে তোমরা হয়তো পাস করতে পারবে না!

Here, the "1." is the line number you added from your code, and the remaining part was the line read from the file. How were you able to read the Bangla text? (Hint: By opening the text file in Notepad program, you have noticed that the encoding of the file is `UTF-8`). **(10+5 marks)** [CSE 107 | 2015–16]

Answer — Efficient File I/O with UTF-8 Encoding

Part 1: Efficient Java Program (10 Marks)

To perform file reading efficiently while ensuring character encoding compatibility, we wrap a `FileInputStream` inside an `InputStreamReader` configured with explicit `UTF-8` decoding, and then buffer the stream using `BufferedReader`.

```java
import java.io.BufferedReader;
import java.io.FileInputStream;
import java.io.InputStreamReader;
import java.io.IOException;
import java.nio.charset.StandardCharsets;

public class PerformanceReader {
    public static void main(String[] args) {
        String filePath = "MyAnalysisOfStudentsPerformance.txt";

        // Try-with-resources statement ensures proper, automatic resource closure
        try (BufferedReader reader = new BufferedReader(
                new InputStreamReader(
                        new FileInputStream(filePath),
                        StandardCharsets.UTF_8))) {

            String line;
            int lineNumber = 1;

            // Efficiently read line-by-line via the internal character buffer
            while ((line = reader.readLine()) != null) {
                System.out.println(lineNumber + ". " + line);
                lineNumber++;
            }

        } catch (IOException e) {
            System.err.println("An error occurred while reading the file: " + e.getMessage());
        }
    }
}
```

Part 2: Explanation of Bangla Text Reading (5 Marks)

(a) **Limitations of Default FileReaders:** Standard file utilities like legacy `FileReader` inherently rely on the host platform's default native character encoding (e.g., `Windows-1252` or extended `ASCII`). These legacy single-byte charsets are fundamentally incapable of parsing complex scripts, rendering them as garbled symbols or question marks (`???`).

(b) **The UTF-8 Multi-byte Standard:** Bangla characters are non-ASCII and fall within the extended Unicode spectrum, requiring 3 bytes per character under `UTF-8` encoding scheme specifications.

(c) **Explicit Character Set Configuration:** By explicitly instantiating an `InputStreamReader` combined with `StandardCharsets.UTF_8`, we override system defaults. This directly instructs the low-level stream engine to combine sequential raw byte groups correctly and map them directly into Java's internal 16-bit `UTF-16` code units (`char`), allowing Bangla text to display natively in a compatible terminal console.

(d) **I/O Efficiency Handling:** Wrapping this decoder loop inside a `BufferedReader` limits expensive underlying hardware disk reads by retrieving substantial data blocks directly into memory buffers upfront, meeting the target system requirement.

Q5. Now, instead of writing each line to the console, you need to write to another file. There is no need to prefix the lines with line number. Modify your code written in 8(a) so that it reads the destination file name from command line and copies the content of `MyAnalysisOfStudentsPerformance.txt` line by line. Keep in mind that the input file uses `UTF-8` encoding. Identify what additional code you had to write to account for `UTF-8` encoding and why? If you did not write any additional code, then explain why. **(10 marks)** [CSE 107 | 2015–16]

Answer — Stream-Based File Copying with Explicit Target Encoding

Part 1: Modified Java Program

The program reads the destination file name dynamically via command-line arguments (`args[0]`), extracts data line-by-line using a buffered reader loop, and safely pushes it into an explicitly encoded buffered output writer stream.

```java
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.io.IOException;
import java.nio.charset.StandardCharsets;

```

```java
public class FileCopier {
    public static void main(String[] args) {
        // Validation check to prevent ArrayIndexOutOfBoundsException
        if (args.length < 1) {
            System.err.println("Error: Please provide the destination file name as a command-line argument.");
            return;
        }

        String srcFile = "MyAnalysisOfStudentsPerformance.txt";
        String destFile = args[0];

        // Try-with-resources manages clean multi-stream lifecycles
        try (BufferedReader reader = new BufferedReader(
                new InputStreamReader(
                        new FileInputStream(srcFile),
                        StandardCharsets.UTF_8));
             BufferedWriter writer = new BufferedWriter(
                new OutputStreamWriter(
                        new FileOutputStream(destFile),
                        StandardCharsets.UTF_8))) {

            String line;
            while ((line = reader.readLine()) != null) {
                writer.write(line);
                writer.newLine(); // Writes platform-appropriate line separator
            }
            System.out.println("File content successfully copied to: " + destFile);

        } catch (IOException e) {
            System.err.println("An I/O error occurred during the copying process: " + e.getMessage());
        }
    }
}
```

Part 2: Analysis of UTF-8 Target Encoding Requirement

- **What Additional Code Was Added:** An explicit encoding parameter, `StandardCharsets.UTF_8`, was introduced into the constructor chain of the output stream writer tier:

  `new OutputStreamWriter(new FileOutputStream(destFile), StandardCharsets.UTF_8)`

- **Why This Code Is Essential:**

  (a) **Platform Default Behavior:** In older standard Java platform specifications (prior to JDK 18), text stream implementations like legacy `FileWriter` or unconfigured character streams default to the host environment's native default charset (e.g., `Windows-1252` on standard Western systems).

  (b) **Preventing Character Corruption (Mojibake):** The input source contains complex multi-byte scripts (such as Bangla). While the input stream successfully parses these into Java's internal 16-bit Unicode format via its own `UTF-8` reader configuration, writing them back to disk without an explicit output encoding will force the system to translate those broad characters through a narrow local system single-byte encoder.

  (c) **Result of Skipping:** Leaving out this additional instruction will structurally compress the distinct 3-byte character groups into localized ANSI/ASCII maps, resulting in corrupted files dominated by unreadable text elements or block questions marks (`???`). Explicitly setting `StandardCharsets.UTF_8` preserves the structural integrity of the output.

Q6. You are writing a networked 2-person shooter game. Here, a bullet is modeled by a class containing positional coordinates:

```java
class Bullet implements Serializable, Cloneable {
    int _posX, _posY;
    public Bullet(int x, int y) { _posX = x; _posY = y; }
    public Object Clone() { return new Bullet(_posX, _posY); }
}
```

When one person shoots, the bullet leaves his weapon and follows a certain calculation model to traverse through the scene. You do not need to worry about the calculation model. During animation, the model continually updates the position of the bullet and sends the updated coordinates to the other person's machine over the network. This is done by calling the `updateBullet()` method of the `TwoPersonShooterGame` class:

```java
public class TwoPersonShooterGame {
    ObjectOutputStream _oos;
    ObjectInputStream _ois;

```

```
    public void updateBullet(Bullet bullet, int x, int y) throws IOException {
        bullet._posX = x;
        bullet._posY = y;
        _oos.writeObject(bullet);
    }
}
```

On the other player's machine, the bullet is rendered accordingly.

(a) Why does `Bullet` implement `Serializable`?

(b) Write a constructor for `TwoPersonShooterGame`. It should have a `Socket` as a parameter and hook up the `_ois` and `_oos` variables to the socket's input and output streams respectively. The signature of the constructor should not have a `throws` clause.

(c) Add a method in `TwoPersonShooterGame` class to read a bullet object from the network. The method should have the following signature:

`public Bullet receiveBulletFromNetwork()`

(d) Once you have completed the tasks of (i) and (ii), the project is complete. However, you observe that in the first person's machine the bullet is moving through the screen, yet it remains stationary on the second person's machine. Why is this happening? Show three different solutions for the problem (with code and explanation).

(e) Finally, notice that `_posX` and `_posY` are not encapsulated. If you make them `private`, `Bullet` objects can no longer be cloned as the `Clone()` method is currently written. Why? Correct the `Clone()` method so that `Bullet` objects can still be cloned.

(3+3+3+11+4 marks) [CSE 107 | 2015–16]

Answer — Object Serialization and Stream Caching Dynamics

(i) Requirement for Serializable (3 Marks)

The `ObjectOutputStream.writeObject()` method requires that any object passed into it must implement the `java.io.Serializable` marker interface. If a class lacks this interface, the JVM will halt execution and throw a `NotSerializableException`. Implementing it enables the serialization engine to convert the object's state into a sequential sequence of bytes suitable for transmission across a network socket.

(ii) Constructor without Throws Clause (3 Marks)

```
public TwoPersonShooterGame(Socket socket) {
    try {
        // Crucial: ObjectOutputStream must always be initialized BEFORE ObjectInputStream
        // to flush the stream header and prevent both endpoints from deadlocking on startup.
        this._oos = new ObjectOutputStream(socket.getOutputStream());
        this._ois = new ObjectInputStream(socket.getInputStream());
    } catch (IOException e) {
        System.err.println("Failed to initialize game network streams: " + e.getMessage());
    }
}
```

(iii) Implementation of receiveBulletFromNetwork() (3 Marks)

```
public Bullet receiveBulletFromNetwork() {
    try {
        return (Bullet) _ois.readObject();
    } catch (IOException | ClassNotFoundException e) {
        System.err.println("Error reading bullet from stream: " + e.getMessage());
        return null;
    }
}
```

(iv) Root Cause Analysis & Three Solutions (11 Marks)

The Problem: By default, an `ObjectOutputStream` maintains an internal **wire handle lookup table (cache)** of references for every object it serializes during its lifetime. When the exact same object reference (`bullet`) is sequentially passed to `writeObject()`, the serialization algorithm optimizes bandwidth consumption by writing an ultra-compact **back-reference handle** instead of re-parsing the object's fields. Consequently, even though the internal values of `_posX` and `_posY` change on the sender side, the receiver side ignores them and simply links back to the original cached instance data from the very first frame.

```
// --- Solution 1: Explicit Stream Cache Clearing via reset() ---
// Explanation: Calling reset() discards the reference history table inside the stream layout,
// forcing a fresh, full deep serialization of the fields of any objects passed next.
public void updateBullet(Bullet bullet, int x, int y) throws IOException {
    bullet._posX = x;
    bullet._posY = y;
    _oos.reset(); // Purges the object tracking table to ensure deep transmission
    _oos.writeObject(bullet);
}

// --- Solution 2: Bypassing Cache via writeUnshared() ---
// Explanation: writeUnshared() instructs the stream to treat this write operation as an
```

```java
// entirely isolated entity, ignoring prior reference history mapping and bypassing the cache.
public void updateBullet(Bullet bullet, int x, int y) throws IOException {
    bullet._posX = x;
    bullet._posY = y;
    _oos.writeUnshared(bullet); // Bypasses standard back-reference caching
}

// --- Solution 3: Emitting Deep Heap Copies (Clones) ---
// Explanation: Instantiating a new heap allocation address breaks the reference lookup entirely.
// The stream treats the new reference address as a fresh object graph entity and re-sends its data.
public void updateBullet(Bullet bullet, int x, int y) throws IOException {
    bullet._posX = x;
    bullet._posY = y;

    // Invokes the clone method to pass a unique instance pointer over the stream
    _oos.writeObject(bullet.Clone());
}
```

(v) Encapsulation and Cloning Correction (4 Marks)

Why cloning fails with the current method: While the Java compiler allows a class to access its own private fields, implementing a manual `Clone()` method via the constructor `new Bullet(_posX, _posY)` creates issues with **inheritance**. If a subclass extends `Bullet` and attempts to utilize or inherit this manual cloning logic, it will not have access to the `private` fields of its parent. Furthermore, standard Java cloning should hook into the intrinsic JVM clone mechanism rather than manually passing fields to constructors.

Correction: We correct this by adhering to the standard `java.lang.Cloneable` contract. By overriding the intrinsic `clone()` method from `Object` and utilizing `super.clone()`, the JVM automatically handles field-by-field memory duplication (a shallow copy) completely bypassing access modifier restrictions.

```java
// Replaces the manual Clone() method inside the Bullet class
@Override
public Object clone() {
    try {
        // Native JVM bitwise copy; safely accesses all fields regardless of private modifiers
        return super.clone();
    } catch (CloneNotSupportedException e) {
        // This will never be reached as Bullet implements Cloneable
        throw new AssertionError("Cloning failed");
    }
}
```

Q7. Write Java code to copy a file `src.txt` to `dst.txt` line by line using `BufferedReader` and `BufferedWriter`. What are the Java classes and methods that can read/write objects of a class to I/O stream? **(10 marks)** [CSE 201 | 2014–15]

Answer — Copy Text File and Object Stream Classes

```java
import java.io.*;

public class TextFileCopy {
    public static void main(String[] args) {
        try (BufferedReader br = new BufferedReader(
                new FileReader("src.txt"));
             BufferedWriter bw = new BufferedWriter(
                new FileWriter("dst.txt"))) {
            String line;
            while ((line = br.readLine()) != null) {
                bw.write(line);
                bw.newLine();
            }
            System.out.println("Copied successfully.");
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Classes and methods for reading/writing objects:

- `ObjectOutputStream` — wraps an `OutputStream`; method `writeObject(Object)` serializes an object.
- `ObjectInputStream` — wraps an `InputStream`; method `readObject()` deserializes an object.
- The class being serialized must implement `java.io.Serializable`.

Q8. What do you understand by serialization in Java I/O? What are the uses of serialization and how is it implemented? **(6 marks)**
[CSE 201 | 2013–14]

Answer — Serialization in Java

Serialization is the process of converting a Java object into a **byte stream** so it can be saved to a file, sent over a network, or stored in a database. The reverse process is called **deserialization**.
Uses:

- Persisting object state to disk (save/load).
- Transmitting objects over a network (e.g., RMI, sockets).
- Caching objects in memory or a database.
- Deep cloning of objects.

Implementation:

```
import java.io.*;

// Step 1: implement Serializable
class Person implements Serializable {
    String name; int age;
    Person(String n, int a) { name=n; age=a; }
}

class SerialExample {
    public static void main(String[] args) throws Exception {
        Person p = new Person("Alice", 25);

        // Serialize
        ObjectOutputStream oos = new ObjectOutputStream(
            new FileOutputStream("person.dat"));
        oos.writeObject(p); oos.close();

        // Deserialize
        ObjectInputStream ois = new ObjectInputStream(
            new FileInputStream("person.dat"));
        Person p2 = (Person) ois.readObject(); ois.close();

        System.out.println(p2.name + ", " + p2.age); // Alice, 25
    }
}
```

Q9. There is a file named `text.txt`. Write a Java program that reads each line of the file one at a time and prints the line number of the first occurrence of the word "Bangladesh". **(10 marks)** [CSE 201 | 2013–14]

Answer — Find First Occurrence of "Bangladesh"

```
import java.io.*;

public class FindWord {
    public static void main(String[] args) {
        String target = "Bangladesh";
        int foundAt = -1;

        try (BufferedReader br = new BufferedReader(
                 new FileReader("text.txt"))) {
            String line;
            int lineNum = 1;
            while ((line = br.readLine()) != null) {
                if (line.contains(target)) {
                    foundAt = lineNum;
                    break;       // stop at first occurrence
                }
                lineNum++;
            }
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }

        if (foundAt != -1)
            System.out.println("\"" + target + "\" first found at line "
                               + foundAt);
        else
```

```
                System.out.println("\"" + target + "\" not found.");
        }
}
```

Sample output:

```
"Bangladesh" first found at line 3
```

Q10. Why is `BufferedReader` a preferred reader? Write a program that appends a string in an existing file named `a.txt`. **(3+8 marks)** [CSE 201 | 2012–13]

Answer — BufferedReader and Appending to a File

Why BufferedReader is preferred: `BufferedReader` wraps another `Reader` and maintains an internal buffer (default 8192 chars). Instead of performing a system call for each character, it reads a large block at once, drastically reducing I/O overhead. It also provides the convenient `readLine()` method.

```
import java.io.*;

public class AppendDemo {
    public static void main(String[] args) {
        // true = append mode
        try (BufferedWriter bw = new BufferedWriter(
                new FileWriter("a.txt", true))) {
            bw.write("This line is appended.");
            bw.newLine();
            System.out.println("String appended to a.txt");
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Passing `true` to `FileWriter` opens the file in append mode so existing content is preserved.

Q11. The members of class `Record` are a String `id`, String `recordDate`, and double `amount`. Give the class definition. Assume `recordList.dat` contains 10 instances. Write code to read them into a `Record` array. **(11 marks)** [CSE 201 | 2011–12]

Answer — Record Class and Reading 10 Objects from File

```
import java.io.*;

class Record implements Serializable {
    private String id;
    private String recordDate;
    private double amount;

    Record(String id, String recordDate, double amount) {
        this.id = id; this.recordDate = recordDate; this.amount = amount;
    }

    @Override
    public String toString() {
        return "Record{id=" + id + ", date=" + recordDate
               + ", amount=" + amount + "}";
    }
}

class RecordReader {
    public static void main(String[] args) {
        Record[] records = new Record[10];

        try (ObjectInputStream ois = new ObjectInputStream(
                 new FileInputStream("recordList.dat"))) {
            for (int i = 0; i < 10; i++)
                records[i] = (Record) ois.readObject();
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
        }

        for (Record r : records)
            System.out.println(r);
    }
}
```

Q12. Explain Object Serialization-Deserialization with an example. **(8 marks)** [CSE 201 | 2010–11]

Answer — Object Serialization-Deserialization Example

Serialization converts an object to bytes; **Deserialization** converts bytes back to an object.

```java
import java.io.*;

class Student implements Serializable {
    private static final long serialVersionUID = 1L;
    String name; int roll; double cgpa;

    Student(String name, int roll, double cgpa) {
        this.name=name; this.roll=roll; this.cgpa=cgpa;
    }
    @Override
    public String toString() {
        return name + " Roll:" + roll + " CGPA:" + cgpa;
    }
}

class SerialDesDemo {
    public static void main(String[] args) {
        Student s1 = new Student("Bob", 101, 3.75);

        // --- Serialization ---
        try (ObjectOutputStream oos = new ObjectOutputStream(
                new FileOutputStream("student.ser"))) {
            oos.writeObject(s1);
            System.out.println("Serialized: " + s1);
        } catch (IOException e) { e.printStackTrace(); }

        // --- Deserialization ---
        try (ObjectInputStream ois = new ObjectInputStream(
                new FileInputStream("student.ser"))) {
            Student s2 = (Student) ois.readObject();
            System.out.println("Deserialized: " + s2);
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
        }
    }
}
```

Output:

```
Serialized:   Bob Roll:101 CGPA:3.75
Deserialized: Bob Roll:101 CGPA:3.75
```

`serialVersionUID` is recommended to ensure version compatibility during deserialization.

Q13. Write Java code to copy a binary file `src.dat` to `dest.dat` using byte stream technique. **(7 marks)** [CSE 201 | 2008–09]

Answer — Copy Binary File Using Byte Streams

```java
import java.io.*;

public class BinaryCopy {
    public static void main(String[] args) {
        try (FileInputStream  fis = new FileInputStream("src.dat");
             FileOutputStream fos = new FileOutputStream("dest.dat")) {
            byte[] buf = new byte[4096];
            int n;
            while ((n = fis.read(buf)) != -1)
                fos.write(buf, 0, n);
            System.out.println("Binary file copied.");
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Q14. Write a Java program that counts the total number of characters of the alternating lines of a file named `input.txt` (starting from the 1st line). Use Character Streams. [CSE 201 | 2007–2008]

Answer — Count Characters of Alternating Lines Using Character Streams

```java
import java.io.*;

public class AltLineCount {
    public static void main(String[] args) {
        int totalChars = 0;
        int lineNumber = 0;

        // FileReader is a character stream (not byte stream)
        try (BufferedReader br = new BufferedReader(
                new FileReader("input.txt"))) {

            String line;
            while ((line = br.readLine()) != null) {
                lineNumber++;
                // Count characters only from odd lines
                // (1st, 3rd, 5th, ...)
                if (lineNumber % 2 != 0)
                    totalChars += line.length();
            }

        } catch (FileNotFoundException e) {
            System.err.println("File not found: input.txt");
        } catch (IOException e) {
            System.err.println("I/O error: " + e.getMessage());
        }

        System.out.println("Total characters in alternating"
            + " lines (starting from line 1): " + totalChars);
    }
}
```

How it works:

- `FileReader` is the character-stream equivalent of `FileInputStream`; it reads characters (Unicode) rather than raw bytes.
- `BufferedReader` wraps it to provide efficient `readLine()`.
- `line.length()` gives the number of characters in the line (excluding the newline).
- Lines are counted from 1; odd-numbered lines (1, 3, 5, …) are the alternating lines starting from the first.

BUET · CSE 107 / CSE 201

Object Oriented Programming

Section S18

Networking

S18 — Networking

Q1. A Fibonacci server accepts multiple clients. Client sends two integers; server calculates count of Fibonacci numbers between them and sends result. Communicate only through objects. Write complete Java client-server program. **(15 marks)** [CSE 107 | 2022–23]

Answer — Fibonacci Server: Object-Based Multi-Client TCP

Protocol: Client sends a serializable `RangeRequest` object (two integers); server replies with a serializable `FibResult` object (the count of Fibonacci numbers in that range).

```
// Shared serializable classes (compile separately)
import java.io.Serializable;

class RangeRequest implements Serializable {
    int lo, hi;
    RangeRequest(int lo, int hi) { this.lo=lo; this.hi=hi; }
}

class FibResult implements Serializable {
    int count;
    FibResult(int count) { this.count = count; }
}
```

```
// FibServer.java
import java.io.*;
import java.net.*;

public class FibServer {
    public static void main(String[] args) throws Exception {
        ServerSocket ss = new ServerSocket(5555);
        System.out.println("Fibonacci server ready on port 5555");
        while (true)
            new Thread(new FibHandler(ss.accept())).start();
    }
}

class FibHandler implements Runnable {
    private final Socket socket;
    FibHandler(Socket s) { socket = s; }

    @Override
    public void run() {
        try (ObjectInputStream  in  = new ObjectInputStream(
                socket.getInputStream());
             ObjectOutputStream out = new ObjectOutputStream(
                socket.getOutputStream())) {

            RangeRequest req = (RangeRequest) in.readObject();
            int count = countFibInRange(req.lo, req.hi);
            out.writeObject(new FibResult(count));
            out.flush();

        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            try { socket.close(); } catch (IOException ignored) {}
        }
    }

    // Count Fibonacci numbers in [lo, hi]
    private int countFibInRange(int lo, int hi) {
        int count = 0;
        int a = 0, b = 1;
        while (a <= hi) {
            if (a >= lo) count++;
            int tmp = a + b; a = b; b = tmp;
        }
        return count;
    }
}
```

```
// FibClient.java
import java.io.*;
import java.net.*;
import java.util.Scanner;
```

```
public class FibClient {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter lo: "); int lo = sc.nextInt();
        System.out.print("Enter hi: "); int hi = sc.nextInt();

        try (Socket socket = new Socket("localhost", 5555);
             // ObjectOutputStream must be created BEFORE
             // ObjectInputStream to avoid deadlock
             ObjectOutputStream out = new ObjectOutputStream(
                 socket.getOutputStream());
             ObjectInputStream  in  = new ObjectInputStream(
                 socket.getInputStream())) {

            out.writeObject(new RangeRequest(lo, hi));
            out.flush();

            FibResult result = (FibResult) in.readObject();
            System.out.println("Fibonacci numbers in ["
                + lo + ", " + hi + "]: " + result.count);
        }
    }
}
```

Note: `ObjectOutputStream` must be constructed *before* `ObjectInputStream` on the same socket to avoid a deadlock caused by the stream header exchange during initialisation.

Q2. A server program accepts multiple clients. Client sends two integers; server calculates count of prime numbers between them and sends result. Write complete Java server-client program. **(15 marks)** [CSE 107 | 2021–22]

Answer — Prime-Count Server: Multi-Client TCP

```
// PrimeServer.java
import java.io.*;
import java.net.*;

public class PrimeServer {
    public static void main(String[] args) throws Exception {
        ServerSocket ss = new ServerSocket(6666);
        System.out.println("Prime server ready on port 6666");
        while (true)
            new Thread(new PrimeHandler(ss.accept())).start();
    }
}

class PrimeHandler implements Runnable {
    private final Socket socket;
    PrimeHandler(Socket s) { socket = s; }

    @Override
    public void run() {
        try (BufferedReader in = new BufferedReader(
                 new InputStreamReader(socket.getInputStream()));
             PrintWriter out = new PrintWriter(
                 socket.getOutputStream(), true)) {

            // Receive two integers on one line: "lo hi"
            String line = in.readLine();
            String[] parts = line.trim().split("\\s+");
            int lo = Integer.parseInt(parts[0]);
            int hi = Integer.parseInt(parts[1]);

            int count = 0;
            for (int n = lo; n <= hi; n++)
                if (isPrime(n)) count++;

            out.println(count);   // send result

        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            try { socket.close(); } catch (IOException ignored) {}
        }
```

```
    }

    private boolean isPrime(int n) {
        if (n < 2) return false;
        if (n == 2) return true;
        if (n % 2 == 0) return false;
        for (int i = 3; (long) i * i <= n; i += 2)
            if (n % i == 0) return false;
        return true;
    }
}
```

```
// PrimeClient.java
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class PrimeClient {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter lo: "); int lo = sc.nextInt();
        System.out.print("Enter hi: "); int hi = sc.nextInt();

        try (Socket socket = new Socket("localhost", 6666);
             PrintWriter out = new PrintWriter(
                 socket.getOutputStream(), true);
             BufferedReader in = new BufferedReader(
                 new InputStreamReader(socket.getInputStream()))) {

            out.println(lo + " " + hi);   // send range

            String result = in.readLine();
            System.out.println("Prime count in ["
                + lo + ", " + hi + "]: " + result);
        }
    }
}
```

Sample output (lo=1, hi=20):

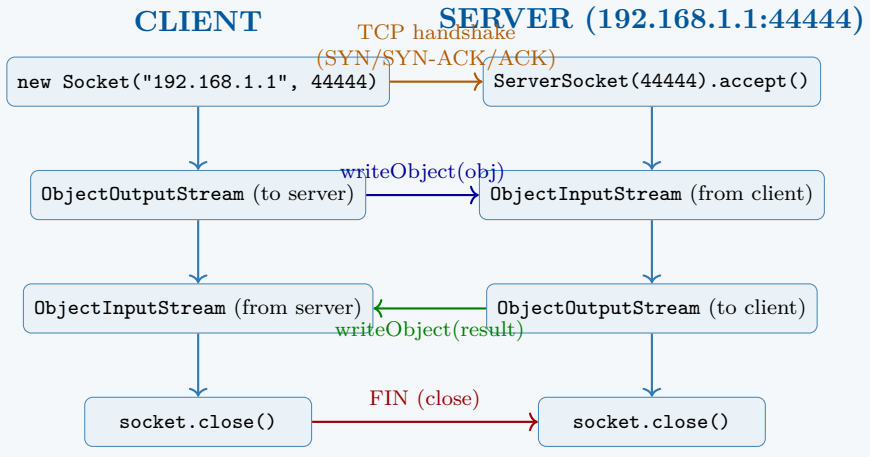
```
Prime count in [1, 20]: 8
```

(Primes: 2, 3, 5, 7, 11, 13, 17, 19)

Q3. Suppose a client connects to a server at 192.168.1.1, port 44444. Draw the scenario of connection and data transfer. What classes are used to read and write objects in networking? What restriction do they impose? **(10 marks)** [CSE 107 | 2020–21]

Answer — Connection Scenario, Object Stream Classes and Their Restriction

Connection and Data Transfer Scenario (Client → Server at 192.168.1.1:44444):



Classes used to read and write objects in networking:

Class	Purpose
`ObjectOutputStream`	Serialises Java objects and writes them to a stream
`ObjectInputStream`	Reads and deserialises Java objects from a stream

Restriction they impose:

Any class whose objects are sent over the network **must implement the `Serializable` interface**. If a class does not implement `Serializable`, calling `writeObject()` on its instance throws a `NotSerializableException` at runtime.
Additionally, every field of the class (and all referenced objects) must also be serialisable, or must be declared `transient` to be excluded from serialisation.

```
// Correct: implements Serializable
class Point implements java.io.Serializable {
    int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }
}

// Usage
ObjectOutputStream oos = new ObjectOutputStream(
    socket.getOutputStream());
oos.writeObject(new Point(3, 4));   // OK

// Wrong: no Serializable -> throws NotSerializableException
class BadPoint { int x, y; }
oos.writeObject(new BadPoint());    // runtime exception!
```

Q4. Briefly describe the types of TCP Sockets. Write a Java Program that opens a connection to the "whois" port (port 43) on the InterNIC server, sends a website address, and prints the returned data. **(4+15 marks)** [CSE 107 | 2017–18]

Answer — TCP Socket Types and WHOIS Client

Types of TCP Sockets in Java:

1. **Stream Socket (`Socket`)** — Represents one end of a two-way reliable TCP connection between client and server. Used for continuous byte-stream data transfer. Both client and server use this type once a connection is established.
2. **Server Socket (`ServerSocket`)** — Listens on a specified port for incoming TCP connection requests. It is *not* used for data transfer itself; calling `accept()` returns a new `Socket` for each client.

WHOIS Client (connects to port 43 on InterNIC's whois.internic.net):

```
import java.io.*;
import java.net.*;

public class WhoisClient {
    static final String WHOIS_HOST = "whois.internic.net";
    static final int    WHOIS_PORT = 43;

    public static void main(String[] args) throws Exception {
        // Website address to query
        String domain = "google.com";

        try (Socket socket = new Socket(WHOIS_HOST, WHOIS_PORT);
             PrintWriter out = new PrintWriter(
                 socket.getOutputStream(), true);
             BufferedReader in = new BufferedReader(
                 new InputStreamReader(
                     socket.getInputStream()))) {

            // WHOIS protocol: send domain name + CRLF
            out.print(domain + "\r\n");
            out.flush();

            // Read and print the response line by line
            String line;
            while ((line = in.readLine()) != null)
                System.out.println(line);
        }
    }
}
```

How it works:

- A `Socket` is opened to `whois.internic.net` on port 43.
- The domain name is sent as plain text followed by a carriage-return + line-feed (`\r\n`), which is the WHOIS protocol.
- The server streams back the registration information; the client reads and prints each line until the server closes the connection.

Q5. Develop a chat application where a server can handle maximum 100 clients. Clients can talk to any other client through the server. A client can form a group of maximum 5. Write using Java TCP/IP socket programming. **(25 marks)** [CSE 201 | 2015–16]

Answer — Chat Application: Server Handling 100 Clients with Group Support

Design Overview:

- The server maintains a list of all connected clients (max 100) and a list of groups (each max 5 members).
- Each connected client is handled by a dedicated `ClientHandler` thread.
- A client can send a message to a specific client (unicast) or to a group.
- Groups are identified by a group name stored in a shared `Map`.

Server code:

```
import java.io.*;
import java.net.*;
import java.util.*;
import java.util.concurrent.*;

public class ChatServer {
    static final int PORT       = 9999;
    static final int MAX_CLIENTS = 100;
    static final int MAX_GROUP   = 5;

    // username -> handler
    static Map<String, ClientHandler> clients =
        new ConcurrentHashMap<>();
    // groupName -> set of usernames
    static Map<String, Set<String>> groups =
        new ConcurrentHashMap<>();

    public static void main(String[] args) throws IOException {
        ServerSocket ss = new ServerSocket(PORT);
        System.out.println("Chat server started on port " + PORT);
        while (true) {
            if (clients.size() >= MAX_CLIENTS) continue;
            Socket s = ss.accept();
            new Thread(new ClientHandler(s)).start();
        }
    }
}
```

```
class ClientHandler implements Runnable {
    private Socket socket;
    private PrintWriter out;
    private BufferedReader in;
    private String username;

    ClientHandler(Socket s) { this.socket = s; }

    @Override
    public void run() {
        try {
            in  = new BufferedReader(
                      new InputStreamReader(socket.getInputStream()));
            out = new PrintWriter(socket.getOutputStream(), true);

            // First message is the username
            username = in.readLine();
            ChatServer.clients.put(username, this);
            broadcast("[" + username + " joined the chat]");

            String msg;
            while ((msg = in.readLine()) != null) {
                // Protocol:
                // "@user message"       -> private message
                // "#group message"      -> group message
                // "/create groupName"   -> create group
                // "/join groupName"     -> join group
                if (msg.startsWith("@")) {
                    // private message: @targetUser actual message
                    int sp = msg.indexOf(' ');
                    String target = msg.substring(1, sp);
```

```
                String text   = msg.substring(sp + 1);
                sendTo(target, "[PM from " + username + "]: " + text);
            } else if (msg.startsWith("#")) {
                int sp = msg.indexOf(' ');
                String grp  = msg.substring(1, sp);
                String text = msg.substring(sp + 1);
                sendToGroup(grp, username, text);
            } else if (msg.startsWith("/create ")) {
                String grp = msg.substring(8).trim();
                ChatServer.groups.putIfAbsent(grp,
                    ConcurrentHashMap.newKeySet());
                ChatServer.groups.get(grp).add(username);
                out.println("Group '" + grp + "' created.");
            } else if (msg.startsWith("/join ")) {
                String grp = msg.substring(6).trim();
                Set<String> members = ChatServer.groups.get(grp);
                if (members == null) {
                    out.println("Group not found.");
                } else if (members.size() >= ChatServer.MAX_GROUP) {
                    out.println("Group is full (max 5).");
                } else {
                    members.add(username);
                    out.println("Joined group '" + grp + "'.");
                }
            } else {
                broadcast("[" + username + "]: " + msg);
            }
        }
    } catch (IOException e) {
        System.out.println(username + " disconnected.");
    } finally {
        ChatServer.clients.remove(username);
        broadcast("[" + username + " left the chat]");
        try { socket.close(); } catch (IOException ignored) {}
    }
}

void sendMessage(String msg) { out.println(msg); }

private void sendTo(String target, String msg) {
    ClientHandler h = ChatServer.clients.get(target);
    if (h != null) h.sendMessage(msg);
    else out.println("User '" + target + "' not found.");
}

private void sendToGroup(String grp, String sender, String msg) {
    Set<String> members = ChatServer.groups.get(grp);
    if (members == null) { out.println("Group not found."); return; }
    for (String m : members) {
        if (!m.equals(sender)) {
            ClientHandler h = ChatServer.clients.get(m);
            if (h != null) h.sendMessage(
                "[Group " + grp + " | " + sender + "]: " + msg);
        }
    }
}

private void broadcast(String msg) {
    for (ClientHandler h : ChatServer.clients.values())
        h.sendMessage(msg);
}
}
```

```
// Client code
import java.io.*;
import java.net.*;

public class ChatClient {
    public static void main(String[] args) throws Exception {
        Socket socket = new Socket("localhost", 9999);
        BufferedReader  in  = new BufferedReader(
            new InputStreamReader(socket.getInputStream()));
        PrintWriter     out = new PrintWriter(
            socket.getOutputStream(), true);
```

```
        BufferedReader  kbd = new BufferedReader(
            new InputStreamReader(System.in));

        System.out.print("Enter username: ");
        String username = kbd.readLine();
        out.println(username);   // send username first

        // Thread to read messages from server
        new Thread(() -> {
            try {
                String line;
                while ((line = in.readLine()) != null)
                    System.out.println(line);
            } catch (IOException ignored) {}
        }).start();

        // Main thread reads keyboard input and sends to server
        String input;
        while ((input = kbd.readLine()) != null)
            out.println(input);

        socket.close();
    }
}
```

Message Protocol Summary:

Command	Effect
`@alice Hello`	Private message to user `alice`
`#team Hi all`	Group message to group `team`
`/create team`	Create group named `team`
`/join team`	Join group `team` (max 5 members)
(plain text)	Broadcast to all connected clients

Q6. Write a Java program to establish a connection to `www.google.com` using `HttpURLConnection`. Output: (i) request method, (ii) response code, (iii) response message, (iv) keys and values in the response header. **(10 marks)** [CSE 107 | 2015–16]

Answer — HttpURLConnection to www.google.com

```
import java.io.*;
import java.net.*;
import java.util.*;

public class GoogleHttpDemo {
    public static void main(String[] args) throws Exception {
        URL url = new URL("https://www.google.com");
        HttpURLConnection conn =
            (HttpURLConnection) url.openConnection();

        // Set request method (GET by default)
        conn.setRequestMethod("GET");
        conn.connect();

        // (i) Request method
        System.out.println("Request Method : "
            + conn.getRequestMethod());

        // (ii) Response code
        System.out.println("Response Code  : "
            + conn.getResponseCode());

        // (iii) Response message
        System.out.println("Response Message: "
            + conn.getResponseMessage());

        // (iv) Response header keys and values
        System.out.println("\n--- Response Headers ---");
        Map<String, List<String>> headers =
            conn.getHeaderFields();

        for (Map.Entry<String, List<String>> entry
```

```
                : headers.entrySet()) {
            // Key can be null for the status line
            System.out.println(entry.getKey()
                + " : " + entry.getValue());
        }

        conn.disconnect();
    }
}
```

Sample output:

```
Request Method  : GET
Response Code   : 200
Response Message: OK

--- Response Headers ---
null           : [HTTP/1.1 200 OK]
Content-Type   : [text/html; charset=UTF-8]
Cache-Control  : [private, max-age=0]
Date           : [Mon, 01 Jan 2024 10:00:00 GMT]
Server         : [gws]
...
```

Key classes:

- `URL` — represents the resource address.
- `HttpURLConnection` — subclass of `URLConnection`; provides HTTP-specific methods such as `getResponseCode()` and `getHeaderFields()`.
- `getHeaderFields()` returns a `Map<String, List<String>>` where `null` key holds the HTTP status line.

Q7. Why does `Socket` implement `AutoCloseable`? Explain with a sample code snippet. **(5 marks)** [CSE 107 | 2015–16]

Answer — Why Socket Implements AutoCloseable

Why `Socket` implements `AutoCloseable`:
`Socket` holds an underlying OS-level file descriptor (a network handle). If the socket is not closed after use, the OS resource is **leaked**: the file descriptor stays open even after the Java object is garbage-collected (because GC timing is non-deterministic).
By implementing `AutoCloseable` (and its sub-interface `Closeable`), `Socket` can be used in a **try-with-resources** block, which guarantees that `close()` is called *automatically* when the block exits — whether normally or due to an exception.
This eliminates the need for a `finally` block and prevents resource leaks.

```
// WITHOUT AutoCloseable (error-prone)
Socket s = null;
try {
    s = new Socket("localhost", 8080);
    // ... use socket ...
} catch (IOException e) {
    e.printStackTrace();
} finally {
    if (s != null)
        try { s.close(); } catch (IOException ignored) {}
}

// WITH AutoCloseable (try-with-resources -- clean and safe)
try (Socket s = new Socket("localhost", 8080);
     BufferedReader in = new BufferedReader(
         new InputStreamReader(s.getInputStream()));
     PrintWriter out = new PrintWriter(
         s.getOutputStream(), true)) {

    out.println("Hello server");
    System.out.println("Server says: " + in.readLine());

}   // s, in, out all closed automatically here,
    // even if an exception is thrown above
```

Key benefit: The try-with-resources block calls `s.close()` (and also closes the `BufferedReader` and `PrintWriter` declared in the same block) in **reverse declaration order**, ensuring no descriptor is leaked regardless of how the block exits.

Q8. Which protocol uses Datagram packet? Write a client program that receives a Datagram packet using port 1111. **(10 marks)** [CSE 201 | 2012–13]

Answer — Datagram Protocol and UDP Client

Which protocol uses Datagram packets?
UDP (User Datagram Protocol) uses `DatagramPacket`. Unlike TCP, UDP is:

- **Connectionless** — no handshake before sending data.
- **Unreliable** — packets may be lost, duplicated, or arrive out of order.
- **Fast** — no connection setup overhead; suitable for streaming, DNS, and gaming.

In Java, UDP is handled by `DatagramSocket` and `DatagramPacket`.

Client program that receives a DatagramPacket on port 1111:

```java
import java.net.*;

public class UDPClient {
    public static void main(String[] args) throws Exception {
        // Bind a DatagramSocket to the local port 1111
        DatagramSocket socket = new DatagramSocket(1111);

        byte[] buffer = new byte[1024];

        // Prepare an empty packet to receive data into
        DatagramPacket packet =
            new DatagramPacket(buffer, buffer.length);

        System.out.println("Waiting for datagram on port 1111...");

        // Blocks until a datagram arrives
        socket.receive(packet);

        // Convert received bytes to a String
        String received = new String(
            packet.getData(), 0, packet.getLength());

        System.out.println("Received from "
            + packet.getAddress() + ":" + packet.getPort());
        System.out.println("Message: " + received);

        socket.close();
    }
}
```

Key classes used:

Class	Role
`DatagramSocket`	UDP socket; can send and receive datagrams
`DatagramPacket`	Wraps the byte array payload, address, and port

Q9. Write a Java program to exchange messages between two peer computers using byte stream with stream socket. **(20 marks)** [CSE 201 | 2009–10]

Answer — Byte-Stream Message Exchange Between Two Peers

Design: Peer A acts as server (listens on a port); Peer B connects as client. Both then simultaneously read from the network and write from the keyboard using two threads each.

```java
// PeerServer.java  (Peer A -- listens first)
import java.io.*;
import java.net.*;

public class PeerServer {
    public static void main(String[] args) throws Exception {
        ServerSocket ss = new ServerSocket(7777);
        System.out.println("Waiting for peer on port 7777...");
        Socket socket = ss.accept();
        System.out.println("Peer connected: "
            + socket.getInetAddress());

        // Wrap byte streams in convenient text readers/writers
        BufferedReader  netIn  = new BufferedReader(
            new InputStreamReader(socket.getInputStream()));
        PrintWriter     netOut = new PrintWriter(
```

```
            socket.getOutputStream(), true);
        BufferedReader  kbd    = new BufferedReader(
            new InputStreamReader(System.in));

        // Thread: continuously read messages from peer
        Thread reader = new Thread(() -> {
            try {
                String line;
                while ((line = netIn.readLine()) != null)
                    System.out.println("[Peer]: " + line);
            } catch (IOException e) {
                System.out.println("Peer disconnected.");
            }
        });
        reader.setDaemon(true);
        reader.start();

        // Main thread: read keyboard and send to peer
        String msg;
        while ((msg = kbd.readLine()) != null) {
            netOut.println(msg);
            if (msg.equalsIgnoreCase("bye")) break;
        }

        socket.close();
        ss.close();
    }
}
```

```
// PeerClient.java  (Peer B -- connects to Peer A)
import java.io.*;
import java.net.*;

public class PeerClient {
    public static void main(String[] args) throws Exception {
        Socket socket = new Socket("localhost", 7777);
        System.out.println("Connected to peer.");

        BufferedReader  netIn  = new BufferedReader(
            new InputStreamReader(socket.getInputStream()));
        PrintWriter     netOut = new PrintWriter(
            socket.getOutputStream(), true);
        BufferedReader  kbd    = new BufferedReader(
            new InputStreamReader(System.in));

        // Thread: read from peer
        Thread reader = new Thread(() -> {
            try {
                String line;
                while ((line = netIn.readLine()) != null)
                    System.out.println("[Peer]: " + line);
            } catch (IOException e) {
                System.out.println("Peer disconnected.");
            }
        });
        reader.setDaemon(true);
        reader.start();

        // Main: type and send
        String msg;
        while ((msg = kbd.readLine()) != null) {
            netOut.println(msg);
            if (msg.equalsIgnoreCase("bye")) break;
        }

        socket.close();
    }
}
```

Key points:

- Byte streams (`InputStream`/ `OutputStream`) are wrapped in `BufferedReader` / `PrintWriter` for convenient line-oriented text exchange.
- A daemon reader thread handles incoming messages concurrently so neither peer blocks the other.

- Typing `bye` closes the connection gracefully.

Q10. What do you know about Socket and ServerSocket? Draw the scenario of operations for connection and data transfer between a client (IP 192.168.0.63, port 55555) and a server. No code needed. **(8 marks)** [CSE 201 | 2008–09]

Answer — Socket, ServerSocket and Client-Server Scenario

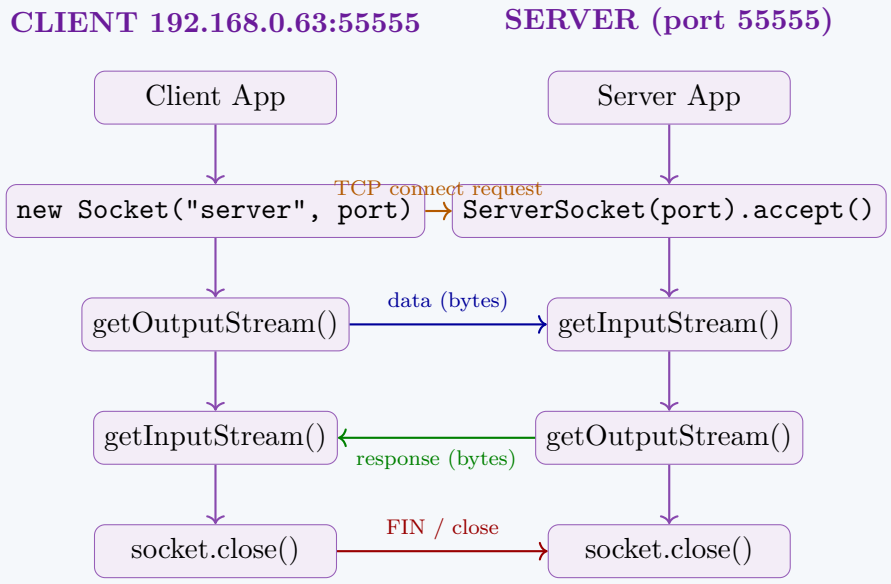
Socket
A `Socket` represents **one endpoint** of a two-way TCP connection between two machines. It encapsulates:

- The remote IP address and port.
- An `InputStream` to read data from the remote end.
- An `OutputStream` to write data to the remote end.

The client creates a `Socket` by specifying the server's IP and port; the JVM completes the TCP three-way handshake automatically.

ServerSocket
A `ServerSocket` **listens** on a specified port for incoming connection requests. It does *not* itself transfer data; calling `accept()` blocks until a client connects and then returns a new `Socket` dedicated to that client. The `ServerSocket` then goes back to listening.

Scenario: Client (192.168.0.63:55555) connects to Server



Step-by-step description:

1. Server creates `ServerSocket` on port 55555 and calls `accept()` — it blocks, waiting for a client.
2. Client creates `new Socket("server-IP", 55555)`. The JVM performs the TCP three-way handshake (SYN, SYN-ACK, ACK).
3. `accept()` returns a new `Socket` on the server side dedicated to this client. The `ServerSocket` resumes listening for more clients.
4. Client writes data through `getOutputStream()`; server reads it via `getInputStream()`.
5. Server writes a response through its `getOutputStream()`; client reads via its `getInputStream()`.
6. Either side calls `close()` to terminate the connection (TCP FIN sequence).

Q11. A multithreaded server at IP 172.20.0.3, port 33333 handles multiple clients. Client sends a String; server returns its reverse. Client displays both strings. Write Java code for the complete scenario. **(15 marks)** [CSE 201 | 2007–08]

Answer — Multithreaded String-Reverse Server at 172.20.0.3:33333

Design:

- Server binds to port 33333 and spawns a new thread per client.
- Client sends one `String`; server returns the reversed string.
- Client displays both the original and the reversed string.

```java
// ReverseServer.java
import java.io.*;
import java.net.*;

public class ReverseServer {
    public static void main(String[] args) throws Exception {
        ServerSocket ss = new ServerSocket(33333);
        System.out.println("Reverse server ready on port 33333");
```

```

        while (true) {
            Socket client = ss.accept();
            System.out.println("Client connected: "
                + client.getInetAddress());
            new Thread(new ReverseHandler(client)).start();
        }
    }
}

class ReverseHandler implements Runnable {
    private final Socket socket;
    ReverseHandler(Socket s) { socket = s; }

    @Override
    public void run() {
        try (BufferedReader in = new BufferedReader(
                 new InputStreamReader(socket.getInputStream()));
             PrintWriter out = new PrintWriter(
                 socket.getOutputStream(), true)) {

            String original = in.readLine();
            if (original == null) return;

            String reversed =
                new StringBuilder(original).reverse().toString();

            out.println(reversed);
            System.out.println("Served: \""
                + original + "\" -> \"" + reversed + "\"");

        } catch (IOException e) {
            e.printStackTrace();
        } finally {
            try { socket.close(); } catch (IOException ignored) {}
        }
    }
}
```

```
// ReverseClient.java
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class ReverseClient {
    public static void main(String[] args) throws Exception {
        // Connect to server at 172.20.0.3, port 33333
        try (Socket socket = new Socket("172.20.0.3", 33333);
             PrintWriter out = new PrintWriter(
                 socket.getOutputStream(), true);
             BufferedReader in = new BufferedReader(
                 new InputStreamReader(
                     socket.getInputStream()))) {

            Scanner kbd = new Scanner(System.in);
            System.out.print("Enter a string: ");
            String original = kbd.nextLine();

            out.println(original);           // send to server

            String reversed = in.readLine(); // receive reversed

            System.out.println("Original : " + original);
            System.out.println("Reversed : " + reversed);
        }
    }
}
```

Sample interaction:

```
Enter a string: BUET
Original : BUET
Reversed : TEUB
```

Multithreading role: Each call to `ss.accept()` is immediately handed off to a new `ReverseHandler` thread, so the server can serve multiple clients simultaneously without any client waiting for another to finish.

BUET · CSE 107 / CSE 201

Object Oriented Programming

Section S19

Miscellaneous Java

S19 — Miscellaneous Java

Q1. What are the improvements in Swing over AWT? **(5 marks)** [CSE 201 | 2013–14]

Answer — Improvements of Swing Over AWT

1. **Lightweight components:** AWT components are *heavyweight* — each is a native OS widget and depends on the underlying platform. Swing components are *lightweight* — drawn entirely by Java in a single native window, giving consistent appearance across all platforms.
2. **Pluggable Look and Feel (PLAF):** Swing supports changeable look-and-feel themes (Metal, Nimbus, Windows, GTK+) at runtime. AWT always uses the native platform style.
3. **Richer component set:** Swing provides many components not in AWT: `JTable`, `JTree`, `JList`, `JTabbedPane`, `JSlider`, `JSpinner`, `JToolBar`, and more.
4. **MVC architecture:** Swing components are built on Model-View-Controller, separating data (model) from display (view) and input (controller). This makes components highly customisable.
5. **More powerful features:**
   - Borders, icons, tooltips, and mnemonics built in.
   - Support for `Action` objects to share functionality between buttons and menu items.
   - Double-buffering enabled by default, reducing flicker.
6. **Accessibility support:** Swing has built-in hooks for screen readers and assistive technologies via the Java Accessibility API.

Q2. You have to develop a user interface where the user can save data or cancel using either two buttons or from a Menu. Each button and its corresponding menu item will have the same caption, mnemonic, tool-tip text, and event-handling code. Implement this requirement using the most appropriate feature of Java. **(15 marks)** [CSE 201 | 2013–14]

Answer — Save/Cancel Buttons and Menu Items Sharing Action

The **most appropriate feature** is `javax.swing.Action` (specifically `AbstractAction`). A single `Action` object stores the caption, mnemonic, tooltip text, and event-handling code. Both the `JButton` and the `JMenuItem` are constructed from the same `Action`, automatically sharing all properties.

```
1  import javax.swing.*;
2  import java.awt.*;
3  import java.awt.event.*;
4
5  public class SaveCancelDemo extends JFrame {
6
7      // -- Define Actions ---------------------------------
8      private final Action saveAction = new AbstractAction("Save") {
9          {   // instance initialiser - set shared properties
10             putValue(SHORT_DESCRIPTION, "Save the current data");
11             putValue(MNEMONIC_KEY, KeyEvent.VK_S);
12         }
13         @Override
14         public void actionPerformed(ActionEvent e) {
15             JOptionPane.showMessageDialog(
16                 SaveCancelDemo.this, "Data saved!");
17         }
18     };
19
20     private final Action cancelAction =
21             new AbstractAction("Cancel") {
22         {
23             putValue(SHORT_DESCRIPTION, "Cancel and discard");
24             putValue(MNEMONIC_KEY, KeyEvent.VK_C);
25         }
26         @Override
27         public void actionPerformed(ActionEvent e) {
28             JOptionPane.showMessageDialog(
29                 SaveCancelDemo.this, "Cancelled.");
30         }
31     };
32
33     SaveCancelDemo() {
34         setTitle("Save / Cancel Demo");
35         setDefaultCloseOperation(EXIT_ON_CLOSE);
36         setLayout(new FlowLayout());
37
38         // -- Buttons built from Actions ----------------
```

```
        add(new JButton(saveAction));
        add(new JButton(cancelAction));

        // -- Menu built from the same Actions ----------
        JMenuBar  menuBar  = new JMenuBar();
        JMenu     fileMenu = new JMenu("File");
        fileMenu.add(new JMenuItem(saveAction));
        fileMenu.add(new JMenuItem(cancelAction));
        menuBar.add(fileMenu);
        setJMenuBar(menuBar);

        setSize(300, 100);
        setVisible(true);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(SaveCancelDemo::new);
    }
}
```

Why `Action` is the right choice:

- Caption, mnemonic, tooltip and handler are defined **once** in the `Action` and automatically applied to every widget built from it.
- Disabling the `Action` (`setEnabled(false)`) simultaneously disables both the button and the menu item.

Q3. Which one is preferred: semantic or low-level events? Mention two examples of each type. **(6 marks)** [CSE 201 | 2013–14]

Answer — Semantic vs Low-Level Events

Semantic events represent a high-level user action that has a clear meaning independent of the input device. **Low-level events** represent raw, device-level interactions.

Semantic Events	Low-Level Events
Represent a logical user action.	Represent raw input device actions.
Device-independent.	Tied to a specific device (keyboard, mouse).
Higher abstraction; easier to use.	More verbose; require more handling code.
Examples: `ActionEvent` (button click, Enter key), `ItemEvent` (checkbox toggled)	**Examples:** `KeyEvent` (key pressed/released), `MouseEvent` (mouse moved/clicked)

Which is preferred?
Semantic events are preferred wherever possible because they are device-independent, require less code, and clearly express the user's intent. Low-level events are used only when fine-grained device control is needed (e.g. detecting which specific key was pressed, or tracking mouse coordinates).

Q4. Design a user interface where a `TextArea` and a button are added using `FlowLayout`. Every key the user types will be added to the `TextArea` and the program will exit when `~1` is pressed. **(7 marks)** [CSE 201 | 2012–13]

Answer — TextArea and Button with FlowLayout; Exit on ~1

```
import java.awt.*;
import java.awt.event.*;

public class TextAreaDemo extends Frame
                          implements KeyListener {
    private TextArea textArea;
    private Button   button;

    TextAreaDemo() {
        setLayout(new FlowLayout());

        textArea = new TextArea(5, 30);
        button   = new Button("Clear");

        add(textArea);
        add(button);

        // Listen for key events on the TextArea
        textArea.addKeyListener(this);

        button.addActionListener(e -> textArea.setText(""));
```

```
22
23          setTitle("TextArea Demo");
24          setSize(400, 200);
25          setVisible(true);
26
27          addWindowListener(new WindowAdapter() {
28              public void windowClosing(WindowEvent e) {
29                  System.exit(0);
30              }
31          });
32      }
33
34      @Override
35      public void keyTyped(KeyEvent e) {
36          // Exit when '~' followed by '1' combination
37          // Simple approach: exit when user types tilde (~)
38          // The question says "~1" as a sequence trigger;
39          // here we treat the literal character '~' as exit.
40          if (e.getKeyChar() == '~') {
41              System.out.println("Exiting...");
42              System.exit(0);
43          }
44          // All other characters are added automatically
45          // by the TextArea's default behaviour.
46      }
47
48      @Override public void keyPressed(KeyEvent e)  {}
49      @Override public void keyReleased(KeyEvent e) {}
50
51      public static void main(String[] args) {
52          new TextAreaDemo();
53      }
54  }
```

Note: The `TextArea` handles its own typing automatically; the `KeyListener` is added only to detect the exit key combination (`~1`). All other keystrokes are passed through to the component.

Q5. What are the different ways of event handling in Java? **(5 marks)** [CSE 201 | 2011–12]

Answer — Different Ways of Event Handling in Java

Java provides **three** common ways to handle events using the Delegation Event Model:

1. **Separate listener class**
   A dedicated class implements the listener interface and is registered on the component. Best for reusable, complex logic.

```
class MyListener implements ActionListener {
    public void actionPerformed(ActionEvent e) {
        System.out.println("Button clicked!");
    }
}
button.addActionListener(new MyListener());
```

2. **The enclosing class implements the listener**
   The frame or panel class itself implements the listener interface, making `this` the listener. Convenient for small programs.

```
class MyFrame extends JFrame
              implements ActionListener {
    public void actionPerformed(ActionEvent e) {
        System.out.println("Handled by frame");
    }
    MyFrame() { button.addActionListener(this); }
}
```

3. **Anonymous inner class**
   The listener is defined inline at the point of registration. Concise for single-use handlers.

```
button.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        System.out.println("Anonymous handler");
    }
});
// Or with a lambda (Java 8+):
button.addActionListener(
    e -> System.out.println("Lambda handler"));
```

Q6. Explain the roles of the entities involved in Java's Delegation Event Handling model. **(5 marks)** [CSE 201 | 2010–11]

Answer — Java Delegation Event Handling Model

Entities in the Delegation Event Model:

1. **Event Source** — The component that generates an event (e.g., a `JButton`). It maintains a list of registered listeners and notifies them when an event occurs.
2. **Event Object** — An object that encapsulates information about the event (e.g., `ActionEvent`, `MouseEvent`). It is created by the source and passed to the listener.
3. **Event Listener** — An object that implements a listener interface (e.g., `ActionListener`) and defines the handler method (e.g., `actionPerformed()`). It is registered with the source via `addXxxListener()`.

Flow: Source generates event → creates Event Object → calls listener's handler method passing the Event Object.

```
import javax.swing.*;
import java.awt.event.*;

public class DelegationDemo {
    public static void main(String[] args) {
        JFrame frame  = new JFrame("Demo");
        JButton btn   = new JButton("Click Me");

        // Register listener (delegation)
        btn.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                // e = Event Object, this = Listener
                System.out.println("Button clicked! "
                                   + e.getActionCommand());
            }
        });

        frame.add(btn);
        frame.setSize(200, 100);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

The **key idea** is that event handling is *delegated* from the source to a separate listener object — the source does not handle its own events. This cleanly separates UI components from business logic.

Q7. What is an adapter class? What are the advantages and disadvantages of using adapter classes? **(5 marks)** [CSE 201 | 2010–11]

Answer — Adapter Classes: Definition, Advantages and Disadvantages

What is an adapter class?
An **adapter class** is an abstract class that provides *empty default implementations* of all methods in a corresponding listener interface. For example, `MouseAdapter` implements `MouseListener` with seven empty methods. A programmer extends the adapter and overrides *only* the methods they care about, rather than being forced to implement every method of the interface.

```
// Without adapter: must implement ALL 7 MouseListener methods
// With adapter: override only what you need
component.addMouseListener(new MouseAdapter() {
    @Override
    public void mouseClicked(MouseEvent e) {
        System.out.println("Clicked at "
            + e.getX() + "," + e.getY());
    }
    // mouseMoved, mouseEntered, etc. not needed here
});
```

Advantages:

- **Less boilerplate:** You only override the methods you need; the adapter provides empty bodies for the rest.
- **Readable code:** Intent is clear — only the relevant handler method appears.

Disadvantages:

- **Consumes the single inheritance slot:** Because Java does not support multiple inheritance, extending an adapter class prevents the class from extending any other class.
- **Silent errors:** If a method name is misspelled when overriding, the compiler does not warn (no `@Override` error if the method signature does not match); the empty adapter method runs instead.

Q8. Write Java and HTML code for an Applet that draws the string "Welcome to Java". **(6 marks)** [CSE 201 | 2010–11]

Answer — Applet Displaying "Welcome to Java"

Java code:

```java
import java.applet.*;
import java.awt.*;

public class WelcomeApplet extends Applet {
    @Override
    public void paint(Graphics g) {
        g.setFont(new Font("Arial", Font.BOLD, 20));
        g.setColor(Color.BLUE);
        g.drawString("Welcome to Java", 50, 80);
    }
}
```

HTML code to embed the applet:

```html
<html>
  <body>
    <applet code="WelcomeApplet.class"
            width="300" height="150">
      Your browser does not support Java Applets.
    </applet>
  </body>
</html>
```

Note: Java Applets are deprecated since Java 9 and removed in Java 17. In modern practice, the same result is achieved with a `JFrame`/`JPanel` application or a web-based technology.

Q9. How do you specify a layout manager to a container object? Write the names of all layout managers. Describe any 3 layout managers with appropriate layout pictures. **(15 marks)** [CSE 201 | 2009–10]

Answer — Layout Managers: Names and Descriptions

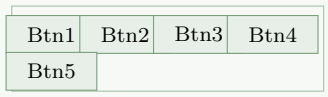
Setting a layout manager:

```java
container.setLayout(new FlowLayout()); // example
```

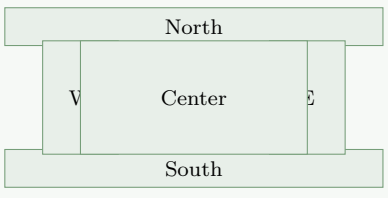
All standard layout managers: `FlowLayout`, `BorderLayout`, `GridLayout`, `GridBagLayout`, `CardLayout`, `BoxLayout`, `SpringLayout`, `GroupLayout` (Swing only), `null` (absolute positioning).

Description of 3 layout managers:

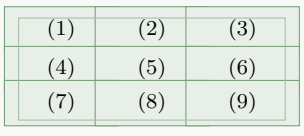
1. **FlowLayout** — places components left-to-right in a row; wraps to the next row when the row is full. Default for `JPanel`.



2. **BorderLayout** — divides the container into five regions: North, South, East, West, Center. Default for `JFrame`'s content pane.



3. **GridLayout** — arranges components in a regular grid of equal-sized cells; components fill cells row by row.



Q10. Create a GUI using `JFrame` with specific dimensions. Add an input field with a label to take a number from the user and a button labelled "Click on Me". Display a click counter that increments at each click. Stop incrementing once the number of clicks exceeds the number entered by the user; show an appropriate message thereafter. **(20 marks)** [CSE 201 | 2009–10]

Answer — JFrame Click Counter with User-Defined Limit

```java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class ClickCounter extends JFrame {
    private JLabel     promptLabel  = new JLabel("Enter limit:");
    private JTextField limitField   = new JTextField(5);
    private JButton    clickButton  = new JButton("Click on Me");
    private JLabel     countLabel   = new JLabel("Clicks: 0");

    private int clicks = 0;
    private int limit  = Integer.MAX_VALUE;

    ClickCounter() {
        setTitle("Click Counter");
        setLayout(new FlowLayout());
        setDefaultCloseOperation(EXIT_ON_CLOSE);

        add(promptLabel);
        add(limitField);
        add(clickButton);
        add(countLabel);

        clickButton.addActionListener(e -> {
            // Read the limit from the text field if not yet set
            try {
                limit = Integer.parseInt(limitField.getText().trim());
            } catch (NumberFormatException ex) {
                limit = Integer.MAX_VALUE;
            }

            if (clicks < limit) {
                clicks++;
                countLabel.setText("Clicks: " + clicks);
                if (clicks == limit)
                    JOptionPane.showMessageDialog(this,
                        "Limit of " + limit
                        + " clicks reached!");
            } else {
                JOptionPane.showMessageDialog(this,
                    "Click limit already reached. No more counting.");
            }
        });

        setSize(350, 120);
        setVisible(true);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(ClickCounter::new);
    }
}
```

Q11. What does Java Applet not have that regular Java classes do? Which Applet method is the substitute for a constructor? Show the order of method calls from an Applet object by a web browser. What is the use of the `paint()` method? **(15 marks)** [CSE 201 | 2009–10]

Answer — Applet vs Regular Class; Life-Cycle; paint()

What does a Java Applet not have?
A Java Applet does **not** have a `main()` method. Regular Java classes use `main()` as the entry point; the JVM calls it to start execution. An Applet's lifecycle is instead controlled by the **browser/applet viewer**.

Substitute for a constructor:
The `init()` method serves as the substitute for a constructor. It is called *once* when the applet is first loaded and is used for initialisation tasks (setting up components, reading parameters). Constructors in applets should not be used for initialisation because the applet context is not fully set up when the constructor runs.

Order of method calls by the browser:

`init()` → `start()` → `paint()` → `stop()` → `destroy()`

- `stop()` and `start()` may be called multiple times (page hidden/revisited).

- `init()` and `destroy()` are called exactly once.

Use of `paint(Graphics g)`:
`paint()` is called by the AWT whenever the applet's display area needs to be (re)drawn — on first display, when the window is uncovered, or when `repaint()` is called programmatically. The `Graphics` object `g` provides methods to draw text, shapes, images, and colours onto the applet's surface.

Q12. Write down the name of any six components of the Java Swing package. **(6 marks)** [CSE 201 | 2008–09]

Answer — Six Components of the Java Swing Package

Component	Class	Purpose
Button	`JButton`	Clickable button that fires an `ActionEvent`
Text field	`JTextField`	Single-line text input
Label	`JLabel`	Non-editable text or icon display
Check box	`JCheckBox`	Toggle on/off option
Combo box	`JComboBox`	Drop-down selection list
Table	`JTable`	Displays data in rows and columns

Other notable Swing components: `JTextArea`, `JRadioButton`, `JSlider`, `JSpinner`, `JList`, `JTree`, `JTabbedPane`, `JMenuBar`.

Q13. What do you know about Event Handling in Java? What is the name of the interface mainly used for this purpose? Write three different ways for a component to register itself to handle events using this interface, with necessary Java code. **(2+1+15 marks)** [CSE 201 | 2008–09]

Answer — Event Handling in Java; Three Ways Using ActionListener

What is Event Handling?
Event handling is the mechanism by which a Java program responds to user actions (button clicks, key presses, mouse movements). Java uses the **Delegation Event Model**: the event *source* delegates processing to a registered *listener* object.
The **main interface** used for action events is `java.awt.event.ActionListener`, which declares one method:

```
void actionPerformed(ActionEvent e);
```

Three ways to register a component as its own handler using `ActionListener`:
Way 1 — Separate (external) class

```
class Handler implements ActionListener {
    public void actionPerformed(ActionEvent e) {
        System.out.println("Separate class handles it");
    }
}
JButton b = new JButton("Click");
b.addActionListener(new Handler());
```

Way 2 — The enclosing frame implements `ActionListener`

```
class MyFrame extends JFrame implements ActionListener {
    JButton b = new JButton("Click");
    MyFrame() {
        b.addActionListener(this); // 'this' is the listener
        add(b); setSize(200,100); setVisible(true);
    }
    public void actionPerformed(ActionEvent e) {
        System.out.println("Frame handles the event");
    }
}
```

Way 3 — Anonymous inner class (or lambda)

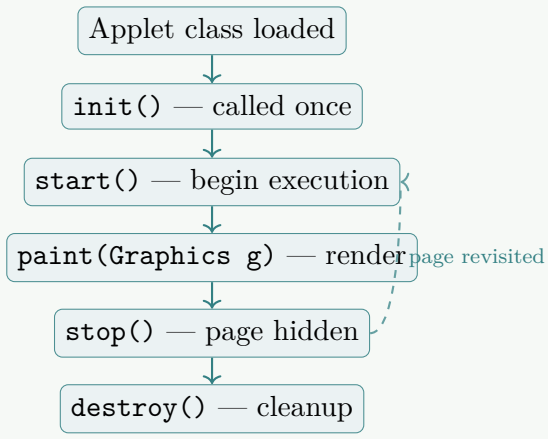
```
JButton b = new JButton("Click");

// Anonymous inner class
b.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        System.out.println("Anonymous class handles it");
    }
});

// Equivalent lambda (Java 8+)
b.addActionListener(
    e -> System.out.println("Lambda handles it"));
```

Q14. What are the differences between Java Applets and Java Applications? [2007–2008]

Q15. Draw the flow of control for a Java Applet. How can you pass parameters to a Java Applet? [2007–2008]

Answer — Applet Flow of Control and Passing Parameters

Flow of control (life-cycle):



Passing parameters to an Applet:
Parameters are passed via `<param>` tags in the HTML page and read with `getParameter(String name)` inside the applet.

```
// HTML
// <applet code="MyApplet.class" width="300" height="150">
//   <param name="color" value="blue">
//   <param name="message" value="Hello World">
// </applet>

import java.applet.*;
import java.awt.*;

public class MyApplet extends Applet {
    private String color, message;

    @Override
    public void init() {
        color   = getParameter("color");
        message = getParameter("message");
        if (message == null) message = "No message";
    }

    @Override
    public void paint(Graphics g) {
        g.drawString("Color param: " + color, 20, 40);
        g.drawString(message, 20, 60);
    }
}
```